

The Zcash Arboretum

Foundations, core protocol, and frontier designs

m@rek.onl

Contents

Introduction	6
I Math Guide: Arithmetic, randomness, and algebra for shielded payments	7
1 Language, integers, and encodings	7
2 Prime fields and modular arithmetic	9
3 Groups and scalar multiplication	11
4 Probability and computation	12
5 Linear algebra and uniform masks	19
6 Polynomials and evaluation	21
7 Evaluation domains and fast transforms	25
8 Pasta curves and their encodings	27
9 Byte fields for AES	32
II Crypto Guide: Mechanisms and security contracts for shielded payments	34
1 Security contracts and reductions	34
2 Hash functions and key derivation	36
3 Groups, commitments, and Sinsemilla	42
4 Knowledge, simulation, and Schnorr	45
5 RedPallas signatures	48
6 Key agreement and note encryption	51
7 Authenticated trees and append-only histories	53
8 Wallet permutations and address encodings	55
III Halo 2 Guide: From a circuit to a checked proof	60
1 From a relation to a circuit	61
2 Columns as polynomials	63
3 The permutation argument	65
4 The lookup argument	67
5 Polynomial commitments and the masked inner-product argument	68

6	Reducing multiple openings	70
7	The transcript and the verifier	72
8	Knowledge soundness and zero knowledge	75
IV	Consensus Guide: Ledger state, block validation, and the meaning of confirmation	79
1	Ledger state and transaction acceptance	79
2	Blocks and branch-local validation	81
3	Work, difficulty, and chain selection	83
4	Supply and pool accounting	85
5	Authenticated block history	86
6	Reorganisation and confirmation confidence	88
V	Ironwood Guide: Constructing and validating a shielded payment	90
1	The shielded payment contract	90
2	Keys and viewing capabilities	91
3	Notes and their commitments	93
4	Nullifiers and authenticated existence	93
5	Note delivery and recovery	96
6	Actions and the proof relation	97
7	Balance, digests, and authorisation	99
8	Pool rules and complete validation	102
9	One payment, end to end	103
10	Security boundaries	104
VI	Wallet Guide: From a seed to an authorised payment	106
1	From seed to account	106
2	Addresses and receiver selection	108
3	Fees, inputs, and change	109
4	Constructing and authorising a transaction	111
5	Tracking payment state and recovery	113

VII Sync Guide: Discovering notes and maintaining spendable state	116
1 What a wallet must reconstruct	116
2 Compact discovery	117
3 Commitment positions and witnesses	118
4 Atomic scanning and rollback	120
5 Server trust and observable metadata	122
VIII FlyClient Guide: Sampling an authenticated proof-of-work history	124
1 The chain-verification contract	124
2 Work intervals and authenticated samples	125
3 Choosing the sampling distribution	127
4 The interactive verifier	130
5 Fiat-Shamir and persistent verification	131
6 Connecting the construction to Zcash	132
IX ZSA Guide: Issuance, private transfer, and public supply	134
1 The asset ledger contract	134
2 Asset identity and issuance authority	135
3 Issuance state and issue notes	138
4 Asset-bearing notes and conservation	141
5 Split inputs and the counterfeiting boundary	143
6 The Action relation and transaction validation	145
X Crosslink Guide: Two confirmation guarantees from one transaction chain	148
1 Two ledgers and their guarantees	148
2 Chain and agreement interfaces	149
3 Cross-links and the state machine	152
4 Ledger extraction and safety	153
5 Progress, stake, and settlement	157
6 The implementation boundary	159

XI	FROST Guide: Threshold custody for randomised RedPallas keys	161
1	Threshold custody and its contract	161
2	Shares and verifiable key generation	162
3	Two-round signing	164
4	Security and nonce state	166
5	Randomised RedPallas signing	168
6	Transaction construction with separated signers	169

Introduction

The Zcash Arboretum is a non-normative guide to the mathematics, cryptography, and engineering of the Zcash protocol. It covers the foundations of Halo 2 and Orchard, deployed consensus and wallet protocols, and separately identified frontier designs. The protocol specification, applicable ZIPs, and consensus rules remain authoritative.

Read the parts in dependency order. The *Math Guide* constructs the required mathematical language; the *Crypto Guide* constructs the primitives; the *Halo 2 Guide* connects circuit constraints to a proof and its explicitly stated security contract.

The *Consensus*, *Ironwood*, *Wallet*, and *Sync* Guides then follow one shielded payment through ledger validation, cryptographic construction, authorisation and recovery. Specified upgrade profiles are distinguished from activated network rules. The four remaining parts are independent destinations: FlyClient, shielded assets, Crosslink and threshold signing. None is required for the core payment.

Each technical term must be explained before use, either locally or in an identified prerequisite section. Every part distinguishes mathematical identities, computational assumptions, imported results and unresolved interfaces. The checked small-field examples are not production proofs. Each part restarts its section numbering to agree with the separate volume.

Part I

Math Guide: Arithmetic, randomness, and algebra for shielded payments

Abstract

This volume of *The Zcash Arboretum* constructs the mathematical objects used by its cryptographic volumes. It assumes elementary integer arithmetic and the ability to follow an algorithm. We build ways to represent data, calculate with it and reason about random choices, introducing each required concept before applying it. Proofs establish the stated elementary results; facts imported without proof are identified explicitly. Knowing how a calculation works does not by itself establish that an attacker cannot reverse it.

1 Language, integers, and encodings

A calculation on numbers and a message sent between computers are different objects. We first specify how messages represent numbers, which messages to reject, and whether different messages can denote the same number.

1.1 Objects, functions and assertions

A *set* is a collection whose members are distinct. We write $x \in S$ for membership, $S \subseteq T$ when every member of S belongs to T , and $\emptyset$ for the empty set. The symbol $S \cup T$ denotes the union: members in either set. The intersection $S \cap T$ contains members in both; $S \setminus T$ contains those in S but not T . The integer $|S|$ counts the elements of a finite set. An ordered pair (x, y) records both its entries and their order. The product $S \times T$ is the set of such pairs with $x \in S$ and $y \in T$. The notation S^n means ordered sequences, or tuples, of exactly n elements of S , including one empty sequence when $n = 0$.

A *function* $f : S \rightarrow T$ assigns exactly one output $f(x) \in T$ to each input $x \in S$. The sets S, T are its domain and codomain, respectively. Its image is $\text{im } f = \{f(x) : x \in S\}$. It is injective when equal outputs require equal inputs, surjective when $\text{im } f = T$, and bijective when both conditions hold. A bijection has an inverse function, which returns x from $f(x)$. The composition $g \circ f$ means first apply f , then g . A function that may reject has codomain $T \cup \{\perp\}$, where $\perp$ is a distinct failure symbol, not the number zero or an empty sequence.

An assertion has a truth value. The expressions $A \wedge B$, $A \vee B$, and $\neg A$ mean “both”, “at least one”, and “not”. The implication $A \Rightarrow B$ prohibits the case in which A is true and B false. Its contrapositive $\neg B \Rightarrow \neg A$ has exactly the same truth value. An equivalence $A \iff B$ asserts both implications. The quantifiers $\forall x \in S$ and $\exists x \in S$ mean “for every member” and “for at least one member”. Their order matters: $\forall x \exists y$ permits a different y for each x . An existence proof does not, by itself, give an efficient way to find the object it proves exists.

The natural numbers are $\mathbb{N} = \{0, 1, 2, \dots\}$; the integers $\mathbb{Z}$ include their negatives. We use induction as follows. To prove an assertion for every $n \in \mathbb{N}$, prove it at zero and prove that its truth at n implies its truth at $n + 1$. Equivalently, a nonempty set of natural numbers has a least member: a least counterexample could be neither zero nor the successor of an earlier non-counterexample. These elementary properties of integers are our starting mathematical assumptions, not consequences of cryptography.

1.2 Remainders and positional representations

Theorem 1.1 (Integer division). *For an integer a and positive integer b , unique integers q, r satisfy $a = qb + r$ and $0 \leq r < b$.*

Proof. Choose the least nonnegative integer in the set of integers $a - kb$. Such integers exist by taking k sufficiently negative. Its value r cannot be at least b , because $r - b$ would be a smaller nonnegative member. For uniqueness, two decompositions give $(q - q')b = r' - r$. The right side has absolute value less than b ; the only multiple of b with that property is zero. $\square$

We write $a \bmod b$ for this nonnegative remainder, including when a is negative. The floor $\lfloor x \rfloor$ is the greatest integer not exceeding x ; the ceiling $\lceil x \rceil$ is the least not smaller. For positive integer b , the quotient above is $\lfloor a/b \rfloor$.

Definition 1.2 (Digits and their value). For a base, or radix, $B \geq 2$, an n -digit little-endian sequence $(d_0, \dots, d_{n-1})$, with $0 \leq d_i < B$, represents

$$\text{val}_B(d_0, \dots, d_{n-1}) = \sum_{i=0}^{n-1} d_i B^i.$$

The sum sign adds one term for each index from zero through $n - 1$. The highest possible value is $B^n - 1$. Big-endian order reverses the digits, so its first digit has weight B^{n-1} .

Here $[0, B^n)$ denotes the integers from zero up to, but excluding, B^n . Repeated division by B constructs the digits of every integer in this interval: take the remainder as the next digit and continue with the quotient. Uniqueness of division proves uniqueness of every digit, starting with the least significant. Thus length- n digit strings are in bijection with the integers in that interval. A leading zero does not change the value, but it does change the length; lengths belong to the encoding specification.

A *bit* has radix two. The notation $\text{LE}_k(a)$ means the k -bit little-endian encoding of an integer $0 \leq a < 2^k$. A *byte* is an integer between zero and 255; a byte string is a sequence of bytes. We write $\text{LE}_n^{\text{byte}}(a)$ and $\text{BE}_n^{\text{byte}}(a)$ when an n -byte order must be explicit. A hexadecimal digit has radix 16; two such digits specify a byte. For instance $0x53$ denotes the integer $5 \cdot 16 + 3$, not a decimal string containing “53”. Concatenation $s \parallel t$ appends the sequence t to s ; the empty sequence is ϵ .

1.3 Canonical encodings and unambiguous tuples

Definition 1.3 (Canonical encoding). An encoding of a set S is an injective function $\text{enc} : S \rightarrow \{0, \dots, 255\}^*$, together with a decoder that returns x exactly on $\text{enc}(x)$ and returns $\perp$ on every other string. The star denotes all finite lengths, not just one fixed length.

Suppose an integer is required to lie in $[0, p)$ and occupies 32 bytes. The decoder must first recover the integer and check that bound. Reducing an arbitrary 32-byte integer modulo p defines a different operation: it accepts multiple encodings of the same remainder. Rejection and reduction cannot be interchanged merely because subsequent arithmetic produces the same value.

Concatenation of variable-length strings is not an injective tuple encoding: (a, bc) and (ab, c) produce the same concatenation. Fixed component lengths resolve the ambiguity. Alternatively, encode each component’s length in a specified format before its bytes. The format must itself be unambiguous and enforce its bounds. A type or domain tag distinguishes different constructions; it does not recover component boundaries that were never encoded.

Bitwise exclusive-or, written $\oplus$, adds corresponding bits modulo two. It is not ordinary integer addition: carries are discarded. A word is a fixed-length sequence of bits. For a w -bit word, rotation by j positions moves each bit j places, wrapping around the ends. A logical shift inserts zero bits and discards bits that pass the boundary; a rotation does neither. These operations, together with addition modulo 2^w , specify the integer operations used later by byte-oriented algorithms.

2 Prime fields and modular arithmetic

Finite arithmetic lets a protocol store every number in a bounded space. It also changes the meaning of equality. We first construct the arithmetic, then prove the bounds needed to recover statements about ordinary integers from statements about residues.

2.1 Divisibility, greatest common divisors and inversion

An integer d divides a , written $d \mid a$, if $a = dk$ for some integer k . A common divisor of a, b divides every integer combination $ua + vb$. For integers not both zero, $\gcd(a, b)$ is their greatest positive common divisor.

Theorem 2.1 (Euclid and Bézout). *The repeated replacement $(a, b) \leftarrow (b, a \bmod b)$, starting with nonnegative integers and $b > 0$, terminates. Its last nonzero remainder is $g = \gcd(a, b)$. Integers u, v can be computed with $ua + vb = g$.*

Proof. Writing $a = qb + r$ shows that the common divisors of a, b are exactly those of b, r . Positive second entries strictly decrease, so a zero remainder is reached. At that point the positive first entry is the greatest common divisor. Initially, the two entries have coefficient pairs $(1, 0)$ and $(0, 1)$ as combinations of the original inputs. If the current coefficient pairs are (u, v) and (u', v') , the new remainder has pair $(u - qu', v - qv')$. Updating these pairs alongside the remainders gives the claimed identity constructively. $\square$

A positive integer $p > 1$ is prime when its only positive divisors are 1 and itself. If $p \nmid a$, then $\gcd(a, p) = 1$, so Theorem 2.1 gives $ua + vp = 1$. Consequently, if $p \mid ab$ and $p \nmid a$, multiplying the identity by b shows $p \mid b$. This is the prime-divisor property used below. Repeatedly factoring a composite integer proves existence of a product of primes, because each factor is smaller. Applying the prime-divisor property to two products and cancelling matching factors proves uniqueness, apart from ordering. This also justifies describing a positive integer by its prime-power factors.

2.2 Constructing a field

We write $a \equiv b \pmod{m}$ when $m \mid (a - b)$. Congruence is an equivalence relation: $a \equiv a$; $a \equiv b$ implies $b \equiv a$; and $a \equiv b, b \equiv c$ imply $a \equiv c$, all with the same modulus. These three properties are called reflexivity, symmetry and transitivity, respectively. Replacing either operand by a congruent integer leaves its sum and product congruent to the original result. We may therefore compute with representatives $0, \dots, m - 1$, reducing after each operation. Addition, multiplication and subtraction obey the usual associative, commutative and distributive laws because their integer counterparts do.

Definition 2.2 (Field). A field is a set with distinct elements $0, 1$, associative and commutative addition and multiplication, additive identity zero and multiplicative identity one, an additive inverse for every element, and a multiplicative inverse for every nonzero element. Multiplication distributes over addition.

Proposition 2.3 (Prime fields). *For prime p , the residues with arithmetic modulo p form a field $\mathbb{F}_p$. Its inverses are computed by extended Euclid. For composite $m > 1$, the residues modulo m do not form a field.*

Proof. All laws except inversion have just been inherited from the integers. For $0 < a < p$, Bézout gives $ua + vp = 1$, so $u \bmod p$ is an inverse of a . If $m = ab$ with $1 < a, b < m$, the two nonzero residues a, b have product zero. An inverse of a would imply $b = 0$, a contradiction. $\square$

In every field, $ab = 0$ implies $a = 0$ or $b = 0$: multiply by a^{-1} when $a \neq 0$. We may cancel a common nonzero factor for the same reason. Division a/b means ab^{-1} and is undefined when $b = 0$. An algorithm that uses a division must prove the denominator nonzero or specify a failure case.

Example 2.4 (A computed inverse). The Euclidean chain is

$$97 = 5 \cdot 17 + 12, \quad 17 = 12 + 5, \quad 12 = 2 \cdot 5 + 2, \quad 5 = 2 \cdot 2 + 1.$$

Back substitution gives $1 = 40 \cdot 17 - 7 \cdot 97$. Thus the inverse of 17 in $\mathbb{F}_{97}$ is 40. The example checker verifies the final identity independently of the displayed chain.

2.3 Efficient powers and simultaneous inverses

To compute a^e for a nonnegative integer e , initialise $r = 1, b = a, k = e$. While $k > 0$, multiply r by b if k is odd, replace b by b^2 , and replace k by $\lfloor k/2 \rfloor$. The invariant $rb^k = a^e$, interpreted before each iteration, proves the result. At most twice the bit length of e multiplications are needed. A secret-dependent branch in this description is mathematical pseudocode, not permission for a timing-dependent implementation.

There is also a useful way to invert many nonzero elements with one inversion. For $a_1, \dots, a_n \neq 0$, form prefix products $P_0 = 1, P_i = P_{i-1}a_i$. Invert P_n , obtaining $u = P_n^{-1}$. In reverse order, output $a_i^{-1} = uP_{i-1}$ and update $u \leftarrow ua_i$. The update makes $u = P_{i-1}^{-1}$, proving each output. Zero inputs must be removed and handled separately; the product method does not invert zero.

2.4 When a field equality is an integer equality

The notation $[a]_p$ denotes the residue of an integer a . The map $a \mapsto [a]_p$ preserves sums and products, but it is not injective on all integers. In particular, there is no order relation on $\mathbb{F}_p$ compatible with the usual ordered-field laws. The representative 96 is larger than 1 as an integer, but in $\mathbb{F}_{97}$ it is also -1 .

Lemma 2.5 (No wraparound). *If integers a, b satisfy $[a]_p = [b]_p$ and $|a - b| < p$, then $a = b$. More generally, reduction modulo p is injective on any interval of integers whose largest and smallest members differ by less than p .*

Proof. The difference is a multiple of p . Every nonzero such multiple has absolute value at least p . $\square$

Corollary 2.6 (A bounded balance equation). *Let $0 \leq u_i < U_i, 0 \leq v_j < V_j$, and let b be a signed integer with $|b| \leq B$. If*

$$\sum_i (U_i - 1) + \sum_j (V_j - 1) + B < p,$$

then the field equation $\sum_i [u_i]_p - \sum_j [v_j]_p = [b]_p$ implies the corresponding integer equation.

Proof. The absolute value of the difference between the two sides is bounded by the displayed sum. Apply Lemma 2.5. Tighter one-sided bounds are possible; this sufficient bound does not assume that either side is nonnegative. $\square$

A field constraint $b(b - 1) = 0$ forces $b \in \{0, 1\}$. Therefore constraints of this form together with $x = \sum_{i=0}^{k-1} 2^i b_i$ describe a k -bit integer in the field. When $2^k \leq p$, the represented integer is unique. When $2^k > p$, bit constraints alone do not prove that the representation is canonical modulo p ; an additional less-than- p condition is necessary. A comparison can be checked by a binary subtraction with explicit borrow bits, or by a range-constrained difference, provided its own field-to-integer bound is established.

Example 2.7 (A false integer balance). The equality $80 + 80 = 63$ holds in $\mathbb{F}_{97}$, but not in $\mathbb{Z}$. All three numbers are themselves canonical field elements. Canonical inputs do not prevent their sum from wrapping. A proof of a monetary balance must bound the complete signed difference, not merely each encoded input.

A field homomorphism is a function between fields preserving one, addition and multiplication. It necessarily preserves zero, negatives and inverses: apply it to $0 + 0 = 0$, $a + (-a) = 0$ and $aa^{-1} = 1$. It is injective, since a nonzero element mapped to zero would have its inverse product mapped to zero instead of one. Such an inclusion is also called a field embedding. A bijective field homomorphism is a field isomorphism; its inverse preserves the same operations by applying the original map and using injectivity. These names concern preservation of both operations, not merely a bijection of the underlying sets.

For distinct primes p, r , conversion from a field element to a scalar means a specified sequence of operations: take its canonical integer representative, optionally check an additional bound, then reduce or canonically encode for the destination. This conversion is not a field homomorphism. Indeed, a map preserving one and addition would send $p \cdot 1 = 0$ in $\mathbb{F}_p$ to $p \cdot 1 \neq 0$ in $\mathbb{F}_r$. Coordinates and scalars must consequently retain their types even when their moduli have nearly the same size.

3 Groups and scalar multiplication

Field elements support two operations. For the objects called points we need only one operation and its inverse. The group laws organise that operation; they do not say that recovering a multiplier from its result is difficult.

3.1 The operation and its order

Definition 3.1 (Abelian group). An abelian group $(\mathbb{G}, +)$ has an associative and commutative operation, an identity $\mathcal{O}$ satisfying $P + \mathcal{O} = P$, and for each P an inverse $-P$ with $P + (-P) = \mathcal{O}$. Its order is its number of elements when finite. Subtraction means adding an inverse.

The identity and inverses are unique. If both I, J are identities, then $I = I + J = J$; if both Q, R invert P , then $Q = Q + (P + R) = (Q + P) + R = R$. Cancellation follows by adding the inverse to both sides of an equality. A subgroup is a subset containing the identity and closed under addition and inverses; the group laws restrict to it.

For an integer $k \geq 0$, define $[k]P$ to be the sum of k copies of P , with $[0]P = \mathcal{O}$. For negative k , set $[k]P = -[-k]P$. Associativity and induction give

$$[a + b]P = [a]P + [b]P, \quad [ab]P = [a]([b]P), \quad [a](P + Q) = [a]P + [a]Q.$$

The last identity uses commutativity. Double-and-add computes $[k]P$ by the same binary method as field exponentiation, replacing multiplication by point addition and squaring by doubling. In a finite group, some positive multiple of P is the identity: among more multiples than group elements, two coincide. The least such positive integer is the order $\text{ord}(P)$.

Proposition 3.2 (Scalar reduction). *If P has order r , then $[a]P = [b]P$ exactly when $a \equiv b \pmod{r}$. The subgroup generated by P has exactly r elements.*

Proof. Divide $a - b$ by r . Its remainder s satisfies $[a - b]P = [s]P$. An identity with $0 < s < r$ would contradict minimality of r . Conversely, a multiple of r acts as zero. The multiples indexed by $0, \dots, r-1$ are therefore distinct and include all multiples. $\square$

3.2 Counting subgroups and prime-order groups

For a subgroup H , a coset is the set $P+H = \{P+Q : Q \in H\}$. Translation is a bijection from H onto this set. Two cosets that share an element are equal: from $P+h = Q+h'$ one obtains $P-Q = h'-h \in H$, which translates one description into the other. Cosets thus partition a finite group into equally sized pieces.

Theorem 3.3 (Lagrange’s counting theorem). *The order of a subgroup divides the order of a finite group. In particular, the order of each element divides the group order.*

Proof. Count the partition into cosets, then apply the result to the subgroup generated by the element. $\square$

If the group order is prime r , every nonidentity point has order r ; hence every such point generates the whole group. A group generated by one element is called cyclic. After choosing a generator G , the function $a \mapsto [a]G$ is a bijection from $\mathbb{F}_r$ to the group. Multiplication by a nonzero scalar is also a bijection, with inverse multiplication by its field inverse. These are algebraic statements even when computing the inverse of the first bijection from the encoded point is not known to be efficient.

The nonzero elements $\mathbb{F}_p^\times$ form a multiplicative group of order $p-1$. Lagrange therefore proves Fermat’s identity $a^{p-1} = 1$ for $a \neq 0$. The formula $a^{-1} = a^{p-2}$ is an alternative inversion algorithm. Unlike $\mathbb{F}_p^\times$, a prime-order point group has no nontrivial proper subgroups. When a larger point group contains a prime-order subgroup of order r , the ratio of the group order to r is the cofactor. Multiplying by a cofactor and checking membership are different operations; a protocol must specify which it requires.

3.3 Linear combinations of points

A map $f : \mathbb{G} \rightarrow \mathbb{G}'$ preserving addition is a homomorphism. It preserves the identity, inverses and integer scalar multiplication by the laws already proved. A bijective homomorphism is an isomorphism; it preserves the abstract operation, not necessarily the representation or the cost of computing its inverse.

For points $G_0, \dots, G_{n-1}$ in a prime-order group and scalars $a_i \in \mathbb{F}_r$, the multiscalar multiplication is

$$\langle \mathbf{a}, \mathbf{G} \rangle = \sum_{i=0}^{n-1} [a_i]G_i.$$

It is linear in each argument where addition and scalar multiplication are defined. Evaluating each term by double-and-add is a complete algorithm; faster implementations share work among the terms but do not change the result. Relative to any fixed generator, every point has a scalar representation. Thus a list of group generators always satisfies some relation $\sum_i [a_i]G_i = \mathcal{O}$ with coefficients not all zero when at least two bases are present. Cryptographic uses of several generators rely instead on a claim that certain relations between their scalars are computationally unknown.

4 Probability and computation

A random choice is specified by a distribution, not by the word “random”. In particular, a value can be uniform before an observation and strongly biased after it. We develop the counting and conditioning rules needed to make such distinctions explicit.

4.1 Real quantities and approximation

The rational numbers $\mathbb{Q}$ are fractions a/b of integers, with $b \neq 0$, identifying a/b and c/d when $ad = bc$. Some limits of increasingly accurate rational approximations, such as the positive square root

of two, are not rational. We use the real numbers $\mathbb{R}$ to include such limits. Concretely, an exact real can be specified by nested closed intervals with rational endpoints and lengths tending to zero. The intervals describe approximations at arbitrary accuracy, not a promise that every real has a finite decimal expansion.

The foundational property used here is completeness: every nonempty set of reals bounded above has a least upper bound, its supremum. This is an explicit property of the real number system, alongside its ordered-field laws; we do not require the reader to know calculus. The infimum is the corresponding greatest lower bound. Nested nonempty closed bounded intervals whose lengths tend to zero have exactly one common point: take the supremum of their left endpoints, and use the shrinking lengths for uniqueness. This explains the interval representation and the convergence of binary search.

The notation $[a, b]$ includes both endpoints; (a, b) excludes both, with mixed brackets for one included endpoint. A sequence x_n tends to x if, for every accuracy $\epsilon > 0$, eventually $|x_n - x| < \epsilon$. A function is continuous at x if arbitrarily small changes in its input near x make its output arbitrarily close to $f(x)$: for every $\epsilon > 0$ there is $\delta > 0$ such that $|u - x| < \delta$ implies $|f(u) - f(x)| < \epsilon$. Finite sums and products preserve limits, as follows by bounding the difference of sums or expanding a difference of products. Quotients preserve them when the limiting denominator is nonzero. These elementary rules justify the approximations used below.

4.2 Experiments, conditioning and expectation

A finite probability space consists of a set Ω of outcomes and nonnegative weights $w(\omega)$ summing to one. An event is a subset $A \subseteq \Omega$, with probability $\Pr[A] = \sum_{\omega \in A} w(\omega)$. In a uniform space all weights equal $1/|\Omega|$; this definition requires $\Omega \neq \emptyset$. The notation $X \leftarrow S$ means choose uniformly from a nonempty finite set S , using fresh independent randomness unless another distribution is specified. For a finite distribution, its support is the set of outcomes with positive probability.

For $\Pr[B] > 0$, conditional probability is

$$\Pr[A \mid B] = \frac{\Pr[A \cap B]}{\Pr[B]}.$$

Events are independent when $\Pr[A \cap B] = \Pr[A] \Pr[B]$. For mutually independent trials this factorisation extends to every selection of events from the different trials; pairwise independence alone does not establish it. A partition B_i of the outcome space gives the total-probability identity $\Pr[A] = \sum_i \Pr[B_i] \Pr[A \mid B_i]$, omitting zero-probability parts. Rearranging the definition gives Bayes' identity $\Pr[B \mid A] = \Pr[A \mid B] \Pr[B] / \Pr[A]$.

A random variable is a function on outcomes. For a real-valued finite random variable, its expectation is the weighted average $\mathbb{E}[X] = \sum_{\omega} w(\omega) X(\omega)$. Rearranging finite sums proves linearity $\mathbb{E}[aX + bY] = a\mathbb{E}[X] + b\mathbb{E}[Y]$, without an independence requirement. An indicator 1_A is one on A and zero elsewhere; its expectation is $\Pr[A]$. The variance $\text{Var}(X) = \mathbb{E}[(X - \mathbb{E}X)^2]$ measures squared deviation from the mean. Expanding the square gives $\text{Var}(X) = \mathbb{E}[X^2] - (\mathbb{E}X)^2$.

Lemma 4.1 (Basic probability bounds). *For events $A_1, \dots, A_n$, $\Pr[\bigcup_i A_i] \leq \sum_i \Pr[A_i]$. For nonnegative X and $t > 0$, $\Pr[X \geq t] \leq \mathbb{E}[X]/t$. Consequently, $\Pr[|X - \mathbb{E}X| \geq t] \leq \text{Var}(X)/t^2$.*

Proof. Pointwise, $1_{\bigcup_i A_i} \leq \sum_i 1_{A_i}$ and $X \geq t 1_{X \geq t}$. Taking expectations gives the first two bounds. Apply the second to $(X - \mathbb{E}X)^2$ for the third. $\square$

These are the union, Markov and Chebyshev bounds, respectively. None permits replacing a conditional distribution by an unconditional one. For instance, if a hidden integer is uniform on a set of size N , then after one incorrect guess is revealed, the next distinct guess hits with probability $1/(N-1)$, not $1/N$.

4.3 Sampling, collisions and concentrated counts

For q independent uniform choices from N objects, a fixed pair collides with probability $1/N$. The union bound over pairs gives probability at most $q(q-1)/(2N)$ for any collision. For $q \leq N$, multiplying the probabilities of avoiding all earlier choices gives the exact no-collision probability

$$\prod_{j=0}^{q-1} \left(1 - \frac{j}{N}\right).$$

The pair count is useful even when it exceeds one, but then its upper bound is uninformative and should be capped at one.

If n independent trials succeed with probability θ , the count X has binomial distribution

$$\Pr[X = k] = \binom{n}{k} \theta^k (1 - \theta)^{n-k}, \quad \binom{n}{k} = \frac{n!}{k!(n-k)!}.$$

Here $n! = 1 \cdot 2 \cdots n$, with $0! = 1$. The coefficient counts the choices of the k successful positions. Summing over all subsets proves that the probabilities sum to one. Writing the count as a sum of indicators gives $\mathbb{E}X = n\theta$; independence makes the cross terms in its variance vanish, giving $\text{Var}(X) = n\theta(1 - \theta)$.

Without replacement, sampling n distinct objects uniformly from N objects, of which K are marked, instead gives

$$\Pr[X = k] = \frac{\binom{K}{k} \binom{N-K}{n-k}}{\binom{N}{n}}.$$

Impossible choices have coefficient zero. Count the marked and unmarked subsets to obtain this formula. These two distributions cannot be exchanged simply because the sample size is small relative to the population; an approximation needs its own error bound.

For sharper upper tails, apply Markov to t^X , with $t > 1$. Let X be a sum of independent zero/one variables, where variable i equals one with probability θ_i . For an integer threshold a , they satisfy the finite bound

$$\Pr[X \geq a] \leq t^{-a} \prod_i (1 - \theta_i + \theta_i t). \quad (1)$$

Indeed, the expectation of a product of independent variables factors, and $\mathbb{E}[t^{1_A}] = 1 - \theta + \theta t$. In the equal-probability case, for $0 < \theta < 1$ and $n\theta < a < n$, substituting $t = a(1 - \theta)/(\theta(n - a)) > 1$ yields

$$\Pr[X \geq a] \leq \left(\frac{n\theta}{a}\right)^a \left(\frac{n(1 - \theta)}{n - a}\right)^{n-a}.$$

This explicit bound needs no replacement distribution and no unstated limit. If $\theta = 0$, then $X = 0$ certainly and every positive threshold has probability zero; if $\theta = 1$, then $X = n$ certainly. Related sampling models must justify their own expectation factorisation or another bound replacing it.

4.4 Countable experiments and waiting times

An algorithm may retry indefinitely. A finite probability space no longer describes all its possible running times. For a countable set of outcomes, meaning outcomes that can be listed by natural-number indices, we assign nonnegative weights with sum one; an infinite sum of nonnegative terms is the supremum of its finite partial sums. This definition permits rearranging such terms, since both arrangements have the same finite subsums. Probabilities of increasing events tend to the probability of their union by the same reasoning. We need no general integration theory for these uses.

For an integer-valued $T \geq 0$, write $T = \sum_{j \geq 0} 1_{T > j}$ pointwise. Nonnegative rearrangement gives the tail-sum formula

$$\mathbb{E}T = \sum_{j \geq 0} \Pr[T > j], \quad (2)$$

allowing an infinite expectation. With independent success probability $\theta > 0$ on every trial, the number $T \geq 1$ of trials up to and including the first success has $\Pr[T > j] = (1 - \theta)^j$ and $\mathbb{E}T = 1/\theta$. The geometric series identity follows by summing its first m terms and taking the limit. Success occurs eventually with probability one, but that statement by itself would not imply finite expected time.

Waiting for $r \geq 1$ successes requires a number of failures K with

$$\Pr[K = k] = \binom{k+r-1}{k} \theta^r (1-\theta)^k.$$

The final trial must succeed; choose the k failures among the preceding $k+r-1$ positions. Equivalently, the total waiting time is a sum of r geometric waiting times, so it has expectation r/θ . This is the negative-binomial waiting model. It differs from fixing an elapsed time first and counting successes during it.

4.5 Integrals and continuous distributions

For a bounded function f on $[a, b]$, divide the interval at finitely many points $a = x_0 < \dots < x_n = b$. Multiplying each subinterval's width by the infimum of f there gives a lower sum; using its supremum gives an upper sum. Refining a partition cannot decrease the lower sum or increase the upper sum. If the supremum of all lower sums equals the infimum of all upper sums, their common value is the integral $\int_a^b f(x) dx$. It is thus a quantity constructed by progressively tighter finite arithmetic bounds. Choosing one point in each subinterval instead gives a Riemann sum, the sum of its function value times that subinterval's width. It lies between the lower and upper sums. The mesh is the largest width in the partition.

Continuous functions on a closed bounded interval have such an integral. Here are the needed details. Repeated interval bisection shows that any sequence in $[a, b]$ has a subsequence converging to a point in that interval: retain a half containing infinitely many sequence entries at each stage and choose entries with increasing indices. If a continuous function were unbounded, a sequence on which its absolute value grows beyond every bound would have such a subsequence, contradicting continuity at its limiting point. Likewise, if there were no uniform input tolerance for a fixed output accuracy, there would be pairs u_n, v_n with $|u_n - v_n| \rightarrow 0$ but a fixed positive separation of their outputs. A converging subsequence of u_n , and hence of v_n , contradicts continuity. Thus for any $\epsilon > 0$ we can choose a mesh small enough that the upper and lower values in each subinterval differ by less than ϵ . The upper and lower sums differ by at most $\epsilon(b-a)$, proving integrability.

The finite sums prove positivity, additivity over adjoining intervals, and linearity in the integrand. They also prove $|\int_a^b f| \leq (b-a) \sup |f|$. Define an integral with reversed endpoints by $\int_b^a f = -\int_a^b f$. A shift and rescaling of the variable, $t = c + du$, rescales every partition width by $|d|$; accounting for endpoint order gives

$$\int_\alpha^\beta f(c+du) du = \int_{c+d\alpha}^{c+d\beta} f(t) dt.$$

This substitution identity is proved directly from the partition sums, without assuming a differentiation rule.

A continuous probability density on an interval is a nonnegative function f whose integral over its domain is one. An interval probability is its integral on that interval. For an unbounded domain, define the total integral as the increasing limit over bounded intervals. A cumulative distribution function, abbreviated CDF, records $F(x) = \Pr[X \leq x]$. For a density on $[a, b]$, it is $F(x) = \int_a^x f(t) dt$ inside the interval, zero below it and one above it. Individual points have probability zero; probability is spread over intervals, not assigned a positive weight to each real point. For a density f , the expectation of a function $g(X)$ is $\int g(x)f(x) dx$ when the required integrals exist and the positive and negative parts are not both infinite. Our applications use continuous or piecewise-continuous integrands and their stated nonnegative limits. Piecewise-continuous means continuous on finitely many adjoining

pieces of each bounded integration interval, with finite one-sided endpoint limits; the integral construction does not make every arbitrary bounded function integrable. For nonnegative quantities, rearrangement of nonnegative partition sums gives the continuous counterpart of (2):

$$\mathbb{E}X = \int_0^\infty \Pr[X > t] dt \quad (X \geq 0).$$

Indeed, approximate each value x from below by a sum of equally spaced threshold indicators, apply linearity to the finite sum, and take increasingly fine meshes and increasing cutoffs. Both sides are limits of the same nonnegative finite sums.

An ideal uniform real U on $[0, 1]$ can be represented by an infinite sequence of independent fair bits: $U = \sum_{i \geq 1} b_i 2^{-i}$. Fixing the first k bits selects an interval of length 2^{-k} with that probability. Approximating arbitrary interval endpoints by such binary intervals gives probability equal to interval length. Endpoints with two binary expansions have probability zero and do not change the distribution. This is an ideal continuous experiment, not an algorithm that can return infinitely many bits in finite time.

Lemma 4.2 (Sampling by the inverse CDF). *If F is continuous and strictly increasing from zero to one on an interval, then $X = F^{-1}(U)$ has CDF F , where U is uniform on $[0, 1]$. Endpoints or infinite endpoints can be defined by limits.*

Proof. Monotonicity gives $F^{-1}(U) \leq x$ exactly when $U \leq F(x)$. The latter event has probability $F(x)$. Existence of the inverse follows by binary search and continuity, and uniqueness follows by strict monotonicity. $\square$

A finite implementation may replace U by a uniform grid point $j/2^k$, $0 \leq j < 2^k$. For every threshold, the resulting interval count differs from its uniform-real probability by at most 2^{-k} . This is a bound on CDF error, not closeness for every possible event. The event that the output lies among the finitely many values the discrete sampler can return has probability one in the discrete experiment and zero in the continuous experiment. A protocol requiring a discrete distribution must specify its actual rounding or rejection algorithm rather than claiming that floating-point inversion is exact continuous sampling.

4.6 Natural logarithms, exponentials and their identities

For $x > 0$, define the natural logarithm constructively by

$$\ln x = \int_1^x \frac{dt}{t}.$$

The integrand is continuous on every positive closed bounded interval, so the preceding construction applies. Positivity of $1/t$ makes $\ln$ strictly increasing. The integral bound on a short interval proves its continuity. Splitting the interval at $a > 0$ and substituting $t = au$ proves

$$\ln(ab) = \ln a + \ln b, \quad \ln(1/a) = -\ln a, \quad \ln(a/b) = \ln a - \ln b.$$

For example, $\ln 2 \geq 1/2 > 0$, so $\ln(2^n) = n \ln 2$ grows without bound and $\ln(2^{-n}) = -n \ln 2$ decreases without bound. Continuity and binary search therefore give, for every real y , a unique positive real x with $\ln x = y$. Define $\exp(y)$ to be that real, and write $e = \exp(1)$. The product law above gives

$$\exp(u + v) = \exp(u) \exp(v), \quad \exp(-u) = 1/\exp(u).$$

For $a > 0$ and real u , define $a^u = \exp(u \ln a)$. This agrees with integer powers and supplies the exact meaning of fractional powers in sampling formulas. Logarithm to base $a \neq 1$ is $\log_a x = \ln x / \ln a$, the inverse of $u \mapsto a^u$.

The exponential also has the convergent series

$$\exp(x) = \sum_{k=0}^{\infty} \frac{x^k}{k!}. \quad (3)$$

To justify the identity without presupposing calculus, call the series $E(x)$. Absolute convergence follows because successive absolute terms eventually have ratio at most one half. Multiplying two absolutely convergent series and grouping by total degree, then using the finite binomial identity, gives $E(x+y) = E(x)E(y)$. These regroupings are legitimate: absolute tail sums bound the discarded terms. In particular $E(x)E(-x) = 1$. The series is positive for $x \geq 0$, so it is positive everywhere. For $|h| \leq 1$, its terms of degree at least two have total absolute value at most a fixed constant times h^2 . Hence $(E(h) - 1)/h \rightarrow 1$, and the product identity makes E continuous everywhere.

The continuous function $L(x) = \ln E(x)$ is additive. Every continuous additive function equals cx for a fixed constant c : prove it first at integers, then rational fractions, then pass to real limits. For $v \rightarrow 0$, the integral bounds for $1/t$ between 1 and $1+v$ give $\ln(1+v)/v \rightarrow 1$. Substituting $v = E(h) - 1$ therefore gives $L(h)/h \rightarrow 1$, forcing $c = 1$. Thus $\ln E(x) = x$, which proves (3) from the inverse definition.

The shift and rescaling $t = 1 - x$ now yield the exact integral identity needed for inverse-distance sampling:

$$\int_a^b \frac{dx}{1-x} = \ln(1-a) - \ln(1-b), \quad a < b < 1. \quad (4)$$

The equivalent notation $\int dx/(1-x) = -\ln(1-x) + C$ means that differences of the expression at two endpoints give the definite integral; the arbitrary constant cancels. No knowledge of integration notation is assumed before this construction.

An exponential waiting time of rate $\lambda > 0$ has CDF $F(t) = 1 - \exp(-\lambda t)$ for $t \geq 0$, and zero for $t < 0$. It is sampled by $-\ln(1-U)/\lambda$; the event $U = 1$ has probability zero and is handled by a limiting endpoint. Its survival probability is $\exp(-\lambda t)$, so

$$\Pr[T > s + t \mid T > s] = \exp(-\lambda t) = \Pr[T > t].$$

This is the memoryless property, derived from the exponential product identity, not assumed from the phrase “waiting time”. Its density is $\lambda \exp(-\lambda t)$. One can verify the corresponding integral directly: the series gives $\exp(-\lambda h) = 1 - \lambda h + R(h)$, where $|R(h)| \leq Ch^2$ for a fixed constant on bounded intervals. Summing the increments of $1 - \exp(-\lambda t)$ along a partition differs from the density’s Riemann sum by at most a constant times the largest partition width, which tends to zero. The tail-integral formula and this density calculation give $\mathbb{E}T = 1/\lambda$.

For completeness, a Poisson distribution with parameter $\lambda > 0$ assigns probability $e^{-\lambda} \lambda^k / k!$ to $k \geq 0$. One may define its normalising constant directly by $e^\lambda = \sum_{k \geq 0} \lambda^k / k!$; the series converges because the ratio of consecutive terms eventually becomes less than one half. Reindexing nonnegative sums gives mean and variance λ . A counting process records the number $N(t)$ of events up to each time $t \geq 0$, starting from zero. An increment $N(t+h) - N(t)$ counts events in the subsequent interval. Describing this as a Poisson process additionally assumes that increments on disjoint intervals are independent and have this distribution with λ proportional to the interval length. That is a modelling hypothesis about event arrivals, not a consequence of the binomial formula or of a hash function’s definition.

4.7 Statistical distance and coupled executions

Definition 4.3 (Statistical distance). For distributions P, Q on the same finite or countable set,

$$\text{SD}(P, Q) = \frac{1}{2} \sum_x |P(x) - Q(x)|.$$

Let $D = \{x : P(x) > Q(x)\}$. Since both distributions sum to one, the total positive excess equals the total negative deficit, so $SD(P, Q) = P(D) - Q(D)$. For any event A , the difference $P(A) - Q(A)$ cannot exceed that positive excess; reversing the roles gives $|P(A) - Q(A)| \leq SD(P, Q)$. Thus distance is exactly the largest difference in probability of an observable event.

The triangle inequality for absolute values proves $SD(P, R) \leq SD(P, Q) + SD(Q, R)$. Applying the same deterministic or randomised computation to both inputs cannot increase their distance: for each output event, its input-dependent probability is between zero and one, and summing against $P - Q$ is bounded by the total positive excess. This is the data-processing property.

Lemma 4.4 (Identical until a bad event). *Suppose two executions are realised on one probability space and their observable outputs agree except on an event B . The statistical distance between their output distributions is at most $\Pr[B]$.*

Proof. For any output event A , its two indicators differ only when the outputs differ, and their absolute difference is at most one. Their expectations therefore differ by at most $\Pr[B]$. Use the event characterisation of distance. $\square$

A shared realisation of two distributions is a coupling. Its usefulness is that it can preserve a complete adaptive transcript until a precise failure occurs. The lemma does not assert that the two executions have their original distributions *conditioned on* avoiding failure. Conditioning changes distributions and must be analysed separately.

4.8 Uniform maps, rejection and reduction bias

A bijection carries the uniform distribution to uniform. More generally, a function with exactly c preimages for every output carries uniform input to uniform output: the probability of each output is c divided by the input-set size. These facts need no computational assumption.

To sample uniformly from $\{0, \dots, m-1\}$, choose k with $2^k \geq m$, sample k independent bits, and reject values at least m . Each accepted value had the same probability before conditioning, so the result is exactly uniform. The expected number of trials is $2^k/m$. If instead one reduces the k -bit integer modulo m , write $2^k = qm + s$, $0 \leq s < m$. The first s residues have $q+1$ preimages, and the others have q . Direct substitution in the distance formula gives

$$SD(U_k \bmod m, U_m) = \frac{s(m-s)}{m2^k} \leq \frac{m}{4 \cdot 2^k}. \quad (5)$$

The inequality uses $s(m-s) \leq m^2/4$, obtained by completing the square. A long reduction input makes the bias small; it does not make it identically zero.

Example 4.5 (Visible modular bias). Reducing a uniform five-bit integer modulo seven produces preimage counts (5, 5, 5, 5, 4, 4, 4). Its distance from uniform modulo seven is $3/56$. The checker counts the preimages and evaluates the distance using integers, so this example does not rely on floating-point rounding.

4.9 Algorithms, costs and security parameters

An algorithm is a finite description of operations on finite inputs, with a specified result or failure behaviour. A deterministic algorithm has no random choices; a randomised one also reads random bits. Its input length counts bits, not the numerical value of an encoded integer. Testing every candidate below a k -bit modulus takes on the order of 2^k iterations, even though the modulus occupies only k bits. Counting a field multiplication as one operation is an algebraic cost model; a bit-cost claim must additionally account for the size and implementation of the multiplication.

For nonnegative functions on positive integers, $f = O(g)$ means there are constants C, n_0 with $f(n) \leq Cg(n)$ for $n \geq n_0$. The notation $\Omega(g)$ reverses the bound, and $\Theta(g)$ asserts both. Logarithms

$\log_2 n$ measure binary length up to rounding. Polynomial time means a bound Cn^d for fixed constants C, d , not constants chosen separately for each input. A probabilistic polynomial-time algorithm, abbreviated PPT, uses random bits and obeys such a time bound on every execution. Expected polynomial time is a different condition and must be named when used.

A security parameter λ indexes a family of instances and algorithms. A nonnegative function $\epsilon(\lambda)$ is negligible if, for every positive integer d , eventually $\epsilon(\lambda) < \lambda^{-d}$. A fixed inverse polynomial is not negligible. Exponential decay such as $2^{-\lambda}$ is negligible: for a fixed d , the ratio of consecutive terms of $\lambda^d/2^\lambda$ tends to one half, so eventually the sequence decreases geometrically to zero. Finite sums of negligible functions, and their products by fixed polynomials, are negligible by choosing a stronger exponent in the definition.

An asymptotic claim does not supply a concrete attack cost for one fixed curve. A concrete statement records running time, query counts and success probability at the chosen parameters. Likewise, expected time bounded by T can be truncated after T/η steps with abort probability at most η , by Markov; truncation is a change to the algorithm whose failure probability belongs in the final bound.

5 Linear algebra and uniform masks

Linear algebra studies operations that preserve sums and multiplication by field elements. It supplies two different tools: efficient identities for rearranging long computations, and exact counting statements about which observations remain hidden by random masks.

5.1 Vectors, subspaces and bases

For a field $\mathbb{F}$, a vector in $\mathbb{F}^n$ is an ordered tuple $\mathbf{v} = (v_0, \dots, v_{n-1})$. Addition and scalar multiplication operate componentwise. A subspace $V \subseteq \mathbb{F}^n$ contains zero and is closed under those operations. A linear combination of vectors $\mathbf{v}_i$ is a sum $\sum_i a_i \mathbf{v}_i$. Their span is the subspace of all their linear combinations. The vectors are linearly independent if $\sum_i a_i \mathbf{v}_i = 0$ forces every coefficient to be zero. A basis is an independent spanning list.

The standard coordinate vectors $\mathbf{e}_i$, with a single one in position i , form a basis of $\mathbb{F}^n$. Relative to any basis, the coordinates of a vector are unique: subtracting two expressions would give a nonzero dependence. If a list is dependent, one vector whose coefficient in a dependence is nonzero can be expressed using the others and removed without changing the span. Removing such vectors from a finite spanning list constructs a basis.

Lemma 5.1 (Basis exchange). *If an independent list has k vectors and a spanning list has m vectors in the same space, then $k \leq m$. All bases of a space with a finite basis consequently have the same length.*

Proof. Express the first independent vector using the spanning list. Some coefficient is nonzero, so solve for that spanning vector and replace it by the independent vector, preserving the span. Repeat with the next independent vector. Its expression must use at least one not-yet-replaced vector with a nonzero coefficient; otherwise it would lie in the span of earlier independent vectors. Each step consumes a different member of the original spanning list, so there can be no more than m steps. Apply this in both directions to two bases. $\square$

The common basis length is the dimension. An independent list can be extended to a basis by adding vectors from a known spanning list when they increase the span. Thus every subspace of $\mathbb{F}^n$ has a basis of at most n vectors: start with an empty list and add a vector outside its span until none remains. More than n additions would contradict Lemma 5.1 applied in $\mathbb{F}^n$.

5.2 Linear maps, matrices and elimination

A linear map $L : V \rightarrow W$ satisfies $L(a\mathbf{v} + b\mathbf{w}) = aL(\mathbf{v}) + bL(\mathbf{w})$. It is determined by its values on a basis. In coordinate spaces these values become the columns of a matrix A ; the output coordinates are

$$(A\mathbf{x})_i = \sum_j A_{ij}x_j.$$

Matrix multiplication represents composition: $(AB)_{ij} = \sum_k A_{ik}B_{kj}$. Interchanging finite sums verifies $(AB)\mathbf{x} = A(B\mathbf{x})$. The identity matrix has ones on its diagonal and zeros elsewhere.

The kernel $\ker L$ is the subspace mapped to zero. The image is a subspace of the codomain; its dimension is the rank. A square matrix is invertible when its linear map is bijective. Its inverse represents the inverse map, so multiplication in either order gives the identity.

One can solve $A\mathbf{x} = \mathbf{b}$ constructively by row operations on the augmented matrix $(A \mid \mathbf{b})$. Swap two rows, multiply a row by a nonzero field element, or add a multiple of one row to another. Each operation has an inverse of the same form and preserves the set of solutions. Working from left to right, choose a nonzero entry as a pivot, move its row into position, scale it to one, and eliminate entries below it. A row with zero coefficients and a nonzero right side proves inconsistency. Otherwise, choose arbitrary values for the nonpivot variables and recover the pivot variables by back substitution. Eliminating above pivots as well gives an explicit description without back substitution.

Theorem 5.2 (Rank and nullity). *For a linear map $L : V \rightarrow W$ on a finite-dimensional domain,*

$$\dim V = \dim \ker L + \dim \operatorname{im} L.$$

Proof. Take a basis $\mathbf{k}_1, \dots, \mathbf{k}_s$ of the kernel and extend it to a basis of V by $\mathbf{v}_1, \dots, \mathbf{v}_t$. The images of these additional vectors span the image because the kernel basis maps to zero. They are independent: a combination with zero image would lie in the kernel and therefore express a dependence in the extended basis. Their number is the rank, giving the equality. $\square$

In particular, a square matrix is invertible exactly when its kernel is zero. Elimination tests this and constructs the inverse by applying the row operations to $(A \mid I)$. For an n -dimensional domain over a field with q elements, a rank- r linear map has a kernel with q^{n-r} elements, since each basis coordinate has q choices. Every nonempty fibre $L^{-1}(\mathbf{y})$ is a translate of that kernel: if $L(\mathbf{x}_0) = \mathbf{y}$, then $L(\mathbf{x}) = \mathbf{y}$ precisely when $\mathbf{x} - \mathbf{x}_0 \in \ker L$.

5.3 Inner products and point-valued linear maps

The field inner product is $\langle \mathbf{a}, \mathbf{b} \rangle = \sum_i a_i b_i$. It is bilinear, meaning linear in either argument with the other fixed. It is also nondegenerate: if its value is zero for every $\mathbf{b}$, test $\mathbf{b} = \mathbf{e}_i$ to obtain $a_i = 0$ for every i . Unlike a sum of squares over the real numbers, this field-valued expression need not be nonzero on a nonzero vector paired with itself. For instance, a nonzero vector can have $\langle \mathbf{a}, \mathbf{a} \rangle = 0$ over a finite field. No argument may infer that a sum of field squares vanishes only when every term vanishes.

The mixed inner product $\langle \mathbf{a}, \mathbf{G} \rangle$ of §3 has a point as output. The same distribution laws permit splitting it into halves or changing coordinates. For an invertible field matrix M , transposition exchanges row and column indices, and

$$\langle M\mathbf{a}, M^{-T}\mathbf{b} \rangle = \langle \mathbf{a}, \mathbf{b} \rangle, \quad M^{-T} = (M^{-1})^T.$$

Expanding the products and using $M^T M^{-T} = I$ proves this identity. Point-valued versions are proved by the same finite sum expansion, using the scalar laws in the group. An implementation may use such identities to combine many checks, but the identities alone say nothing about which coefficients an adversary knows.

5.4 What a random linear mask hides

An affine map is a linear map followed by a translation: $\mathbf{x} \mapsto \mathbf{c} + L(\mathbf{x})$. A translate $\mathbf{c} + V = \{\mathbf{c} + \mathbf{v} : \mathbf{v} \in V\}$ of a subspace is an affine set. It need not contain zero and need not be a subspace. These definitions describe the exact output sets whose probabilities we now count.

Theorem 5.3 (Uniform image of an affine map). *Let R be uniform on $\mathbb{F}_q^n$, let $L : \mathbb{F}_q^n \rightarrow \mathbb{F}_q^k$ have rank r , and fix $\mathbf{c} \in \mathbb{F}_q^k$. The vector $\mathbf{c} + L(R)$ is uniform on the affine set $\mathbf{c} + \text{im } L$, which has q^r elements. It is uniform on the entire codomain exactly when $r = k$.*

Proof. Every output in the image has q^{n-r} preimages by the fibre count above. Its probability is $q^{n-r}/q^n = q^{-r}$. Translation by $\mathbf{c}$ is a bijection. $\square$

If a secret random variable W is independent of R and L is surjective, then $C(W) + L(R)$ has the same uniform distribution for every possible value of W . Hence the output is independent of the secret. If the rank is smaller, the affine support may depend on the secret and reveal information even though every individual coordinate is uniform.

Example 5.4 (Marginal hiding is not joint hiding). Over $\mathbb{F}_5$, let u, v be independent uniform values. The map $(u, v) \mapsto (u + v, u + 2v)$ is bijective: subtracting the coordinates recovers v , then u . Adding any secret pair therefore leaves the output uniform on all 25 pairs. In contrast, $(u, v) \mapsto (u + v, 2u + 2v)$ has only five outputs. After adding a secret pair (c_1, c_2) , the observations satisfy $y_2 - 2y_1 = c_2 - 2c_1$. Each observation separately is uniform, yet together they reveal this secret linear combination. The checker enumerates both distributions exactly.

Previously disclosed linear information consumes masking dimensions. This can be quantified without an independence guess.

Lemma 5.5 (Masks after linear observations). *Let R be uniform on $\mathbb{F}_q^n$. Condition on $BR = \mathbf{b}$, for a value in the image of B . Then AR is uniform on a translate of $A(\ker B)$, and*

$$\dim A(\ker B) = \text{rank} \begin{pmatrix} B \\ A \end{pmatrix} - \text{rank } B.$$

In particular, k output coordinates of AR remain jointly uniform exactly when the displayed rank difference equals k .

Proof. Conditioning a uniform input on a fibre makes it uniform on that fibre, namely $\mathbf{r}_0 + \ker B$. Apply Theorem 5.3 to the restriction of A to $\ker B$. Rank-nullity identifies its rank as $\dim \ker B - \dim(\ker B \cap \ker A)$. Applying rank-nullity again to B and the stacked map proves the formula. $\square$

This lemma has fixed linear maps. If challenges or maps depend adaptively on previous observations, one must justify the conditional distribution at each step, including which fresh mask coordinates remain. Merely counting random coefficients in the original witness does not prove that a complete adaptive transcript is hidden.

6 Polynomials and evaluation

A polynomial turns a list of coefficients into a rule for obtaining field values. Its crucial property is that a nonzero polynomial of small degree cannot vanish at many different points. That fact supports both reconstruction from evaluations and probabilistic equality checks.

6.1 Formal polynomials and division

A polynomial over $\mathbb{F}$ is a finite formal sum $f(X) = \sum_{i=0}^d a_i X^i$. The symbol X is an indeterminate, not a sampled field element. Equality of polynomials means equality of all coefficients after appending zero coefficients as necessary. Addition is coefficientwise and multiplication is convolution:

$$(fg)_k = \sum_{i+j=k} a_i b_j.$$

Associativity, commutativity and distributivity follow by rearranging finite sums. The set of polynomials is written $\mathbb{F}[X]$. Evaluation substitutes $x \in \mathbb{F}$ for X , giving $f(x) = \sum_i a_i x^i$. Evaluation preserves addition and multiplication, again by expansion.

For nonzero f , its degree is the highest index with nonzero coefficient. The zero polynomial has degree $-\infty$, a convention that makes inequalities about upper degree bounds include zero. We never use an expression such as a negative number of roots for this special degree. For nonzero polynomials, multiplying the leading coefficients gives a nonzero leading coefficient, hence $\deg(fg) = \deg f + \deg g$. Also $\deg(f + g) \leq \max(\deg f, \deg g)$, with possible strict inequality when leading terms cancel.

Theorem 6.1 (Polynomial division). *For $f, g \in \mathbb{F}[X]$ with $g \neq 0$, unique polynomials q, r satisfy $f = qg + r$ and either $r = 0$ or $\deg r < \deg g$.*

Proof. If the current remainder has degree at least $\deg g$, subtract the unique scalar multiple of a power of X times g that cancels its leading term. This uses the inverse of the leading coefficient of g . Its degree strictly decreases, so the process terminates, and the subtracted multiples give q . For uniqueness, two outputs would give $(q - q')g = r' - r$. A nonzero left side has degree at least $\deg g$; a nonzero right side has smaller degree. They can therefore only both be zero. $\square$

Evaluation can be performed by Horner's rule. Starting from the highest coefficient, repeatedly multiply the accumulated value by x and add the next coefficient. Expanding the nested expression proves it equals $f(x)$, using d multiplications and d additions for a degree- d polynomial. Polynomial division, likewise, is an explicit finite algorithm, not a claim that a quotient happens to exist.

6.2 Roots, multiplicity and equality tests

Theorem 6.2 (Factor and root theorems). *For $a \in \mathbb{F}$, the equality $f(a) = 0$ holds exactly when $X - a$ divides f . A nonzero degree- d polynomial has at most d distinct roots in $\mathbb{F}$.*

Proof. Division by $X - a$ gives a constant remainder; evaluating at a identifies it with $f(a)$. For the root bound, a nonzero constant has no roots. If a positive-degree polynomial has a root a , write $f = (X - a)g$. Every other root is a root of g , since its $X - a$ factor is nonzero there. Induction on the degree proves the bound. $\square$

A root has multiplicity m if $(X - a)^m$ divides f but $(X - a)^{m+1}$ does not. We use this definition only for nonzero polynomials. Removing factors successively shows that the sum of multiplicities of distinct roots is at most the degree. The formal derivative is $f'(X) = \sum_{i \geq 1} i a_i X^{i-1}$, where the integer coefficient is interpreted in the field. Direct multiplication proves the product rule $(fg)' = f'g + fg'$. Thus a root of multiplicity at least two is also a root of f' . Conversely, if $f = (X - a)g$, then $f'(a) = g(a)$; if this vanishes, $X - a$ divides g , giving multiplicity at least two. These statements use a formal operation and require no calculus.

Over $\mathbb{F}_p$, the nonzero polynomial $X^p - X$ evaluates to zero at every field element, by Fermat and the separate zero case. Therefore polynomials and the functions they induce on a finite field are not the same objects. Degree restrictions are essential to uniqueness arguments.

Corollary 6.3 (A random evaluation test). *Fix different polynomials f, g of degree at most d , and then sample x uniformly from a nonempty finite set $S \subseteq \mathbb{F}$. The probability that $f(x) = g(x)$ is at most $\min(1, d/|S|)$.*

Proof. Apply the root bound to the nonzero difference. $\square$

The word “then” is a hypothesis. If the purported polynomial can be chosen after observing x , the adversary can choose $X - x$, which passes at that point with certainty. Any use of the corollary in a protocol must explain how its polynomial is fixed before the sampled challenge, or invoke a separate argument that supplies that property.

6.3 Interpolation and vanishing polynomials

Let $H = \{x_0, \dots, x_{n-1}\}$ contain n distinct field elements, with $n \geq 1$. Define

$$Z_H(X) = \prod_{i=0}^{n-1} (X - x_i), \quad L_i(X) = \prod_{j \neq i} \frac{X - x_j}{x_i - x_j}.$$

All denominators are nonzero. The polynomials L_i have degree less than n and satisfy $L_i(x_j) = 1$ when $i = j$, zero otherwise.

Theorem 6.4 (Interpolation). *For any $y_0, \dots, y_{n-1} \in \mathbb{F}$, exactly one polynomial of degree less than n takes those values on the corresponding points. It is $f(X) = \sum_i y_i L_i(X)$.*

Proof. The evaluation property of the L_i proves existence. The difference of two candidates would have n distinct roots and degree less than n , so the root bound forces it to be zero. $\square$

The space of degree-less-than- n polynomials has coefficient basis $1, X, \dots, X^{n-1}$. Interpolation proves it also has basis $L_0, \dots, L_{n-1}$: these polynomials span by the formula and are independent by evaluation at each x_i . Evaluation is therefore an invertible linear change of coordinates. Its matrix is $V_{ij} = x_i^j$, often called a Vandermonde matrix; its invertibility follows from the interpolation argument just given.

Proposition 6.5 (Vanishing on a domain). *A polynomial f is zero at every point of H exactly when $Z_H \mid f$. Moreover,*

$$L_i(X) = \frac{Z_H(X)}{(X - x_i)Z'_H(x_i)}.$$

The quotient by $X - x_i$ in this expression is a polynomial.

Proof. Remove the factor $X - x_0$. At any other root, this factor is nonzero, so the remaining polynomial still vanishes there. Repeat to remove all factors. The reverse implication follows by evaluation. The product rule gives $Z'_H(x_i) = \prod_{j \neq i} (x_i - x_j) \neq 0$, proving the formula. $\square$

Polynomial division now has an evaluation interpretation. The unique remainder of f modulo Z_H is its degree-less-than- n interpolant on H , because subtraction of a multiple of Z_H does not change those values. This is also the precise source of aliasing: values on H determine a remainder, not an unrestricted high-degree polynomial.

6.4 Random polynomial masks and threshold observations

Choose the d coefficients of a polynomial of degree less than d independently and uniformly in $\mathbb{F}_q$. At any $k \leq d$ distinct points its evaluations are jointly uniform in $\mathbb{F}_q^k$. Indeed, restricting the evaluation map to the first k coefficients already gives an invertible interpolation matrix, so the full map is surjective. Apply Theorem 5.3.

Suppose instead that a polynomial must vanish on a fixed set H . Write it as $Z_H(X)r(X)$, where r has d uniform coefficients. At $k \leq d$ distinct points outside H , its evaluations remain jointly uniform: multiplication by the nonzero values $Z_H(x_i)$ is an invertible diagonal map. At a point inside H , the output is necessarily zero. Repeated observations at the same point are not additional independent masks; their evaluation rows coincide. Linear observations already made must be accounted for using Lemma 5.5.

Lemma 6.6 (A hidden constant coefficient). *Fix a value $s \in \mathbb{F}_q$ and let $f(X) = s + \sum_{i=1}^{t-1} a_i X^i$, with independent uniform coefficients. For any $k < t$ distinct nonzero evaluation points, the k evaluations have a uniform joint distribution independent of s . Conversely, evaluations at t distinct points determine the complete polynomial and hence s .*

Proof. For the first claim, the columns corresponding to $X, \dots, X^k$ give a matrix equal to the invertible Vandermonde matrix for $1, \dots, X^{k-1}$, with each row multiplied by its nonzero evaluation point. It has full row rank. The affine-mask theorem removes dependence on the constant shift. The second claim is interpolation. $\square$

When reconstruction needs only the constant coefficient, interpolation gives the explicit formula

$$f(0) = \sum_i \lambda_i f(x_i), \quad \lambda_i = \prod_{j \neq i} \frac{-x_j}{x_i - x_j}.$$

The index set in these products is the exact set of observations being used. Changing that set changes the coefficients. The nonzero-point condition in the hiding lemma is essential: an observation at zero reveals s immediately.

6.5 Several variables and random combinations

A polynomial in m variables is a finite sum of monomials $cX_1^{e_1} \dots X_m^{e_m}$. Its total degree is the largest sum $e_1 + \dots + e_m$ with nonzero coefficient. We may regard it as a polynomial in the last variable whose coefficients are polynomials in the others.

Theorem 6.7 (Schwartz–Zippel). *For a fixed nonzero polynomial of total degree at most d , and independent uniform $x_1, \dots, x_m$ in a finite nonempty set $S \subseteq \mathbb{F}$, the probability of a zero evaluation is at most $\min(1, d/|S|)$.*

Proof. The case $m = 1$ is the root bound. For the induction step, let k be the degree in X_m and $g \neq 0$ its leading coefficient polynomial. Its total degree is at most $d - k$, so the probability that it vanishes on the first $m - 1$ choices is at most $(d - k)/|S|$. Otherwise the remaining univariate polynomial is nonzero of degree k , and its conditional root probability is at most $k/|S|$, because x_m is still independent uniform. Add the two bounds and cap at one. $\square$

To combine fixed scalar equations $e_0 = 0, \dots, e_{k-1} = 0$, choose a fresh scalar η and test $\sum_i \eta^i e_i = 0$. If at least one equation is false, this is a nonzero polynomial of degree at most $k - 1$ in η . Independent uniform coefficients would instead give error exactly $1/q$ over $\mathbb{F}_q$, by solving for the coefficient of a nonzero e_i . The two methods have different bounds.

For polynomial equations, first apply the same reasoning to a coefficient that is nonzero in at least one equation; then account for the later random evaluation of the resulting polynomial. These are separate bad events. Random aggregation does not turn a single root-bound calculation into a security proof for the complete interactive protocol in which the equations arose.

7 Evaluation domains and fast transforms

Interpolation works on any distinct points, but a geometric sequence of points makes both evaluation and interpolation much faster. We construct those sequences and the transforms rather than treating the fast Fourier transform, abbreviated FFT, as an unexplained operation.

7.1 Roots of unity and their existence

An element $\omega \in \mathbb{F}^\times$ is a primitive n -th root of unity if its multiplicative order is exactly n . Its powers $1, \omega, \dots, \omega^{n-1}$ are then distinct, form a subgroup, and are roots of $X^n - 1$. The root bound proves that there are no further roots and hence $Z_H(X) = X^n - 1$ for this set H .

Theorem 7.1 (Cyclicity of a finite multiplicative subgroup). *Every finite subgroup of a field's multiplicative group is cyclic. In particular, $\mathbb{F}_q^\times$ has an element of order $q - 1$.*

Proof. Let M be the least common multiple of the orders of all elements. Factor M into its maximal prime-power factors. For each such factor ℓ^a , some element has order divisible by ℓ^a . Raising that element to the part of its order not divisible by ℓ gives an element of order ℓ^a . This follows directly from scalar reduction: for an element of order r , the order of its k -th power is $r/\gcd(r, k)$.

Two integers are coprime when their greatest common divisor is one. The product of commuting elements of coprime orders has order the product. If $(ab)^t = 1$, then $a^t = b^{-t}$ lies in the intersection of the two cyclic subgroups. Its order divides both coprime orders by Lagrange, so it is the identity; each order therefore divides t . Multiplying the elements constructed for all prime powers yields an element of order M .

Every member of the original subgroup is a root of $X^M - 1$, so its size is at most M by the root bound. The element just constructed already contributes M distinct powers. Thus the subgroup consists exactly of those powers. $\square$

There is a primitive n -th root in $\mathbb{F}_q$ exactly when $n \mid (q - 1)$: necessity follows from Lagrange, and for sufficiency raise a multiplicative generator to $(q - 1)/n$. The largest s for which $2^s \mid (q - 1)$ is the two-adicity of the field. An advertised root of order 2^s can be checked without finding a full generator: verify $\omega^{2^s} = 1$ and $\omega^{2^{s-1}} \neq 1$ when $s \geq 1$. When $s = 0$, the unique element of order one is simply 1. For general n , verify $\omega^n = 1$ and $\omega^{n/\ell} \neq 1$ for each prime divisor ℓ of n . Every proper divisor of n divides some n/ℓ , which proves the test.

7.2 The transform and its inverse

The characteristic of a finite field is the least positive integer ℓ for which $\ell \cdot 1 = 0$. It exists because the additive group is finite. It is prime: a factorisation $\ell = ab$ with $1 < a, b < \ell$ would produce two nonzero elements $a \cdot 1, b \cdot 1$ with zero product. Its additive subgroup generated by one has order ℓ , so Lagrange shows $\ell \mid q$ for a field of size q .

Let $n \mid (q - 1)$ and let ω have order n . The field element represented by n is nonzero and invertible: the characteristic prime divides q , not $q - 1$. For an integer k , a geometric sum gives

$$\sum_{j=0}^{n-1} \omega^{jk} = \begin{cases} n, & n \mid k, \\ 0, & n \nmid k. \end{cases} \quad (6)$$

In the second case multiply the sum by $1 - \omega^k \neq 0$; the product is $1 - \omega^{nk} = 0$. This proves the identity. For coefficients $a_0, \dots, a_{n-1}$, the transform is $y_j = \sum_i a_i \omega^{ij}$, evaluation at ω^j . Its inverse is

$$a_i = n^{-1} \sum_{j=0}^{n-1} y_j \omega^{-ij}. \quad (7)$$

Substitute the expression for y_j , interchange the two sums, and use (6). Only the term with the matching coefficient index remains. In particular, the Lagrange basis here is

$$L_j(X) = \frac{1}{n} \sum_{i=0}^{n-1} \omega^{-ij} X^i = \frac{\omega^j}{n} \frac{X^n - 1}{X - \omega^j}.$$

The second expression follows from the derivative $(X^n - 1)' = nX^{n-1}$ and Proposition 6.5. At $X = \omega^j$ it denotes the polynomial quotient, not a numerical division by zero.

7.3 The radix-two algorithm

For even n , separate the even and odd coefficients:

$$f(X) = f_{\text{even}}(X^2) + X f_{\text{odd}}(X^2).$$

The root ω^2 has order $n/2$. Compute the two length- $n/2$ transforms to obtain values E_j, O_j at ω^{2j} . For $0 \leq j < n/2$, combine them as

$$y_j = E_j + \omega^j O_j, \quad y_{j+n/2} = E_j - \omega^j O_j.$$

The identity $\omega^{n/2} = -1$ follows because it has order two, and the only roots of $X^2 - 1$ in a field of odd characteristic are $1, -1$. These formulas compute the two required evaluations. When $n = 2^k$, repeat the split down to length one, where a constant evaluates to itself.

At each of k levels the total number of field operations is bounded by a fixed multiple of n ; the result costs $O(n \log n)$, instead of evaluating one degree- $n - 1$ polynomial at n points by separate quadratic total work. The inverse uses the same algorithm with root ω^{-1} , followed by multiplication of each result by n^{-1} . An in-place implementation may reorder entries; the output contract remains the explicit indexing in (7).

Example 7.2 (A complete transform). In $\mathbb{F}_{97}$, the element 22 has order four, with powers $(1, 22, 96, 75)$. The polynomial $3 + 5X + 7X^2 + 11X^3$ evaluates on this domain to $(26, 58, 91, 31)$. Applying the inverse transform recovers $(3, 5, 7, 11)$. Both directions are checked independently in the accompanying script.

7.4 Products, rotations and cosets

To multiply polynomials of degrees at most a, b , choose an available transform length $N > a + b$, pad both coefficient lists with zeros, evaluate them, multiply corresponding values, and interpolate. The true product has degree less than N , so interpolation uniquely recovers it. If $N \leq a + b$, the same procedure instead returns the remainder modulo $X^N - 1$. Calling it the full product would be incorrect.

For a polynomial f , the rotation $f(\omega X)$ evaluates on H to the cyclic shift $f(\omega^{j+1})$. Its coefficient of X^i is $a_i \omega^i$. The wrap at the last row is an algebraic property of the domain, not a decision about which circuit rows should participate in a constraint.

A multiplicative coset is $gH = \{g\omega^j\}$, with $g \neq 0$. Evaluating f on it is the ordinary transform of coefficients $a_i g^i$. Interpolation reverses this scaling. Its vanishing polynomial is $X^n - g^n$. If $g \notin H$, then $g^n \neq 1$, and the coset is disjoint from H .

Suppose a quotient $t = f/(X^n - 1)$ exists and has degree less than N , where $n \mid N$ and a size- N subgroup H_N is available. Choose $g \notin H_N$. Every point of gH_N lies outside H , because otherwise g would belong to H_N . Thus

$$t(x) = \frac{f(x)}{x^n - 1} \quad (x \in gH_N)$$

has no zero denominator and can be interpolated on that coset. The bound needed for this interpolation is on $\deg t$, not on $\deg f$. To obtain the numerator values one may evaluate its constituent low-degree polynomials on the coset and perform the specified pointwise expression; one need not interpolate the high-degree numerator itself in a space that is too small for it.

There is a converse trap. Pointwise division on a coset followed by interpolation always returns some polynomial, even when f is not divisible by $X^n - 1$. That does not prove the polynomial identity $f = (X^n - 1)t$. Divisibility must be established separately or tested with the appropriate degree and challenge conditions.

7.5 Chunks and recombination

Every polynomial of degree less than kn decomposes uniquely as

$$t(X) = \sum_{j=0}^{k-1} X^{jn} t_j(X), \quad \deg t_j < n,$$

by grouping consecutive blocks of n coefficients. At a point z , its value is $\sum_j z^{jn} t_j(z)$. This is a deterministic identity, with no randomness or probability loss. It allows a large quotient to be represented by several polynomials with a common smaller degree bound. A later protocol may commit to the chunks separately; such a commitment does not alter the recombination formula or the need for the degree bounds.

8 Pasta curves and their encodings

We now construct the point arithmetic used throughout the cryptographic volumes. Two distinctions remain visible: coordinates are not scalars, and an algebraic formula valid away from exceptional inputs is not a total group operation. The finite-field algorithms already developed make every ordinary arithmetic step explicit.

8.1 The curve and the complete affine operation

An affine coordinate pair means two ordinary field coordinates (x, y) , without identifying different pairs by a scaling rule. We first specify the curve and its operation in those coordinates, treating its extra identity as a separate object.

For a field $\mathbb{F}_p$ with prime $p > 3$, choose $A, B \in \mathbb{F}_p$ with $4A^3 + 27B^2 \neq 0$. Consider the finite coordinate pairs satisfying

$$y^2 = x^3 + Ax + B, \tag{8}$$

and add a separate point $\mathcal{O}$, called the point at infinity. This equation is called short-Weierstrass form. The nonvanishing condition is a hypothesis of the group theorem below. To see its algebraic meaning, a repeated root u of $X^3 + AX + B$ would satisfy $3u^2 + A = 0$ and $u^3 + Au + B = 0$, hence $A = -3u^2, B = 2u^3$, giving $4A^3 + 27B^2 = 0$.

Negate a finite point by $-(x, y) = (x, -y)$, and set $-\mathcal{O} = \mathcal{O}$. Define addition with the identity by $P + \mathcal{O} = \mathcal{O} + P = P$. For finite points $P = (x_1, y_1)$, $Q = (x_2, y_2)$, use the following cases:

1. If $x_1 = x_2$ and $y_1 = -y_2$, return $\mathcal{O}$. This includes doubling a point with $y_1 = 0$.
2. Otherwise, if $P = Q$, set $\lambda = (3x_1^2 + A)/(2y_1)$.

3. Otherwise, set $\lambda = (y_2 - y_1)/(x_2 - x_1)$.

In the last two cases return

$$x_3 = \lambda^2 - x_1 - x_2, \quad y_3 = \lambda(x_1 - x_3) - y_1. \quad (9)$$

The cases cover all inputs. With equal x -coordinates, the curve equation gives $y_1^2 = y_2^2$; factoring their difference shows the points are equal or opposite. The excluded cases ensure every displayed denominator is nonzero.

These formulas come from intersecting a line with the cubic. Substitute the line $y = \lambda(x - x_1) + y_1$ into (8). After moving all terms to one side, the coefficient of x^2 is $-\lambda^2$. Two roots are x_1, x_2 , so the third is $\lambda^2 - x_1 - x_2$, by expanding the product of the three linear factors. Reflection in the x -axis gives y_3 . For equal inputs, the slope is chosen so that x_1 is a repeated root of the substituted polynomial; the formal derivative condition gives $2y_1\lambda = 3x_1^2 + A$. This proves that the returned finite point is on the curve and explains both ordinary addition and doubling without division by a vanishing denominator.

Theorem 8.1 (Imported elliptic-curve group theorem). *For $p > 3$ and $4A^3 + 27B^2 \neq 0$, the points of (8), together with $\mathcal{O}$, form an abelian group under the complete operation just defined.*

Closure was established above; the identity, inverses and symmetry of the construction are explicit. Associativity, including all exceptional cases, is the imported part of this theorem. The line-intersection picture does not prove it, and checking a few triples numerically does not replace it. We use the standard short-Weierstrass group theorem also assumed by the protocol specification's section "Pallas and Vesta". This is an explicit mathematical boundary of this volume, not a computational hardness assumption and not a claim that the preceding coefficient calculation proves the entire theorem.

8.2 Incomplete addition and circuit equations

The incomplete operation $P \boxplus Q$ is defined only for finite points with different x -coordinates. It uses the last slope formula and (9); on any other input it returns $\perp$. Thus it agrees with complete addition wherever defined, but it rejects $P = Q$, $P = -Q$, and any identity input. Equality of two finite x -coordinates is equivalent to equality up to negation, as proved above.

One can replace the division in its slope by the equation

$$(x_2 - x_1)\lambda = y_2 - y_1. \quad (10)$$

With $x_2 \neq x_1$, this equation uniquely determines the correct slope. With $P = Q$, it becomes $0 = 0$ and leaves λ unconstrained. Consequently, polynomial equations obtained by clearing denominators do not automatically enforce the domain of an incomplete operation. A circuit may need a nonzero check, or its surrounding security argument may need to account explicitly for exceptional witnesses allowed by the implemented relation. A description of a partial function and a description of its circuit constraints must not silently be identified.

A nonzero check can introduce z and enforce $xz = 1$: an inverse exists precisely when $x \neq 0$. A conditional nonzero check with a Boolean flag b , meaning an element constrained to zero or one, instead needs a correctly specified conditional relation, since $xz = b$ alone does not force $x = 0$ when $b = 0$. Such details are statements about equations, not minor implementation conventions.

8.3 Projective representation and scalar evaluation

Affine formulas require inversions. A projective coordinate system introduces a shared denominator and identifies scaled representatives. In ordinary projective coordinates, a point is an equivalence class of nonzero triples (X, Y, Z) , where (X, Y, Z) and (uX, uY, uZ) represent the same point for every

nonzero u . For $Z \neq 0$, the corresponding affine coordinates are $(X/Z, Y/Z)$. The homogeneous curve equation, meaning that every term has the same total degree, is

$$Y^2Z = X^3 + AXZ^2 + BZ^3.$$

It is unchanged by the scaling because every term acquires a factor u^3 . With $Z = 0$, the equation forces $X = 0$, and the nonzero-triple condition leaves the single equivalence class of $(0, 1, 0)$. This constructs the extra point at infinity in the same coordinate language as the finite points.

Jacobian coordinates use a different scaling rule, chosen to reduce arithmetic costs. A finite point is represented by (X, Y, Z) , with $Z \neq 0$, meaning $x = X/Z^2$, $y = Y/Z^3$. Its equation becomes

$$Y^2 = X^3 + AXZ^4 + BZ^6.$$

Scaling the triple to (u^2X, u^3Y, uZ) , with $u \neq 0$, does not change the represented point. This is a weighted scaling: the powers of u are two, three and one rather than all one. For the algorithms here the identity is a separate tagged value. An implementation may instead reserve particular zero- Z triples for it; that internal convention is not an ordinary affine point and must be handled before division.

To see how denominator removal works, consider two finite triples. Set

$$U_1 = X_1Z_2^2, \quad U_2 = X_2Z_1^2, \quad S_1 = Y_1Z_2^3, \quad S_2 = Y_2Z_1^3,$$

and $H = U_2 - U_1$, $R = S_2 - S_1$. When $H \neq 0$, return

$$\begin{aligned} X_3 &= R^2 - H^3 - 2U_1H^2, \\ Y_3 &= R(U_1H^2 - X_3) - S_1H^3, \\ Z_3 &= HZ_1Z_2. \end{aligned}$$

Dividing these expressions by Z_2^2, Z_3^3 respectively recovers (9), because $x_2 - x_1 = H/(Z_1^2Z_2^2)$ and $y_2 - y_1 = R/(Z_1^3Z_2^3)$. This is an explicit inversion-free ordinary-addition formula, not a complete formula for $H = 0$. Doubling, opposites and identities require their own cases or a separately verified complete projective formula.

For correctness, one may always convert to affine and use the complete operation already specified. For performance, an implementation retains projective coordinates through a sequence of additions and normalises once at the end. Batch normalisation can use the simultaneous inversion algorithm in §2. No choice of coordinates alters the distinction between the coordinate field and the scalar modulus.

8.4 Quadratic residues and square-root extraction

A nonzero element $a \in \mathbb{F}_p$ is a square if $a = y^2$ for some y . A nonzero square has exactly two square roots $y, -y$, because $X^2 - a$ has at most two roots and the two are distinct when p is odd. The squaring map on $\mathbb{F}_p^\times$ is therefore two-to-one onto a set of size $(p-1)/2$. By cyclicity, writing $a = g^k$ for a generator g , the squares are exactly the even powers. The element $g^{(p-1)/2}$ is -1 , so

$$a^{(p-1)/2} = \begin{cases} 1, & a \text{ is a nonzero square,} \\ -1, & a \text{ is a nonsquare.} \end{cases} \quad (11)$$

This is Euler's criterion, including an efficient test by exponentiation. The zero input has the unique square root zero and is handled separately.

Here is a constructive algorithm valid for every odd prime. Write $p-1 = 2^s d$, with d odd, and choose a nonsquare z , verified with (11). For a nonzero input a , first reject unless Euler's test gives one. Initialise

$$x = a^{(d+1)/2}, \quad t = a^d, \quad c = z^d, \quad m = s.$$

While $t \neq 1$, find the least $i \geq 1$ with $t^{2^i} = 1$, set $b = c^{2^{m-i-1}}$, and update

$$x \leftarrow xb, \quad t \leftarrow tb^2, \quad c \leftarrow b^2, \quad m \leftarrow i.$$

Return x . This is the Tonelli–Shanks algorithm.

Proposition 8.2 (Square-root algorithm correctness). *For the accepted inputs above, the loop is defined, terminates and returns a square root of a . It uses $O(\log p + s^2)$ field multiplications after a verified nonsquare has been supplied. A search for that nonsquare is additional work; independent uniform trials need two trials on average, with an $O(\log p)$ Euler test per trial. This is an expected bound, not a fixed worst-case bound for deterministic sequential search.*

Proof. Maintain $x^2 = at$, $\text{ord}(c) = 2^m$, and $\text{ord}(t) \mid 2^{m-1}$. Initially the first identity is a direct power calculation. Euler’s criterion gives $c^{2^{s-1}} = -1$, so c has order 2^s , and $t^{2^{s-1}} = 1$ since a is a square. Thus all invariants hold.

When $t \neq 1$, its order is 2^i with $1 \leq i < m$, so the exponent defining b is nonnegative. The element b has order 2^{i+1} , and b^2 has order 2^i . Both t and b^2 , raised to 2^{i-1} , equal the unique order-two element -1 . Their product consequently has order dividing 2^{i-1} . The updated variables satisfy all three invariants with the strictly smaller value $m = i$. Termination follows, and the first invariant gives $x^2 = a$ when $t = 1$. Initial exponentiations cost $O(\log p)$; each of at most s iterations uses at most $O(s)$ squarings or multiplications. $\square$

A nonsquare can be found by testing candidates, or by independent uniform sampling from $\mathbb{F}_p^\times$, which succeeds with probability one half per trial. The variable-time algorithm above specifies an arithmetic result; an implementation handling secrets must separately prevent execution time or memory accesses from revealing sensitive choices. A final check $x^2 = a$ is a useful implementation assertion but does not replace input validation.

8.5 Concrete parameters and the two field roles

Define

$$\begin{aligned} p_{\text{Pallas}} &= 2^{254} + 45560315531419706090280762371685220353, \\ p_{\text{Vesta}} &= 2^{254} + 45560315531506369815346746415080538113. \end{aligned}$$

The curve E_{Pallas} has equation $y^2 = x^3 + 5$ over $\mathbb{F}_{p_{\text{Pallas}}}$; the curve E_{Vesta} has the same equation over $\mathbb{F}_{p_{\text{Vesta}}}$. The short-Weierstrass nonvanishing condition is satisfied because $27 \cdot 25 \neq 0$ in either field.

Assumption 8.3 (Imported, verifiable parameter facts). Both displayed integers are prime, and the complete point groups have orders

$$|E_{\text{Pallas}}(\mathbb{F}_{p_{\text{Pallas}}})| = p_{\text{Vesta}}, \quad |E_{\text{Vesta}}(\mathbb{F}_{p_{\text{Vesta}}})| = p_{\text{Pallas}}.$$

These are mathematical parameter facts imported from the Zcash protocol specification, “Pallas and Vesta”, not assumptions of computational intractability.

The specification is available at the primary protocol source. The inspected implementation is `pasta_curves` version 0.5.2, with moduli in `src/fields/fp.rs` and `src/fields/fq.rs`, and group/type definitions in `src/curves.rs`. These identify the rule set being explained; they do not by themselves establish the activation of any network upgrade.

Under these parameter facts, every nonidentity point generates its group and both cofactors are one. The Pallas base field, used for coordinates, is $\mathbb{F}_{p_{\text{Pallas}}}$, but its scalar field is $\mathbb{F}_{p_{\text{Vesta}}}$. Vesta reverses these roles. This is the Pasta cycle: the base field of each curve is the scalar field of the other. The

cycle makes Pallas coordinate operations expressible directly in arithmetic over Vesta’s scalar field. It does not identify Pallas scalars with Pallas coordinates, and it does not make all operations in an algorithm using both curves become operations in a single field.

Both fields have two-adicity exactly 32: divide the corresponding $p - 1$ by 2^{32} , and the remaining integer is odd. The checker verifies this, as well as Euler’s criterion showing that -13 and 5 are nonsquares in both fields. The point $(-1, 2)$ satisfies $4 = (-1)^3 + 5$, so it is a nonidentity point in each group. The checker also verifies that multiplication by the published opposite-field order returns the identity. That check confirms a scalar-multiplication identity; it is not a primality certificate or a proof that there are no additional curve points. The imported order facts remain explicit.

8.6 Compressed points and coordinate extraction

Let p be the relevant base modulus. For a finite point $P = (x, y)$, write $\tilde{y}$ for the low bit of the canonical integer representative of y . The star encoding is

$$P^* = \text{LE}_{256}(\text{val}(x) + 2^{255}\tilde{y}), \quad \mathcal{O}^* = \text{LE}_{256}(0). \quad (12)$$

Here $\text{val}(x)$ denotes the canonical integer in $[0, p)$. Packing each consecutive eight bits into a byte gives the 32-byte representation. The top bit carries the sign selector; the lower 255 bits carry the x -coordinate.

Decoding performs the following exact checks:

1. Require exactly 32 bytes. Extract the top bit s , and interpret the remaining 255 bits as an integer u .
2. Reject if $u \geq p$. Otherwise set $x = [u]_p$.
3. If $u = 0$ and $s = 0$, return $\mathcal{O}$.
4. Compute a square root y of $x^3 + 5$. Reject if no root exists. Select y or $-y$ so that the canonical integer’s low bit equals s , then return (x, y) .

This is the decoding behaviour in `pasta_curves 0.5.2, src/curves.rs`, implementing the specification’s “Pallas and Vesta” encoding. In particular, reduction of a noncanonical $u \geq p$ would be incorrect.

There are no finite points with $y = 0$, because such a point would be a nonidentity element of order two in a group of odd prime order. The two square roots in step four are therefore distinct and have opposite parity, since their canonical representatives sum to the odd integer p . There are also no finite points with $x = 0$, because 5 is a nonsquare in these fields. Consequently, the identity encoding does not collide with a finite point, and the input with only the top bit set is invalid rather than another identity encoding. Every valid group point has exactly one valid byte representation.

Define coordinate extraction by $\text{Extract}(\mathcal{O}) = 0$ and $\text{Extract}(x, y) = \text{val}(x)$. Extending it to a partial operation preserves failure: $\text{Extract}(\perp) = \perp$, not zero. Equality of extracted coordinates for two finite points means the points are equal or opposite; it does not generally mean the full points are equal. For Pasta, extracted zero identifies the identity, because finite $x = 0$ points do not exist. These exact facts are required whenever a commitment is shortened to one coordinate.

8.7 A small curve and the limits of examples

Over $\mathbb{F}_{19}$, the equation $y^2 = x^3 + 5$, together with the identity, has 27 points. Direct enumeration of all 361 coordinate pairs establishes this count in the checker. The point $(0, 9)$ doubles to $(0, 10)$, its negative, and its third multiple is the identity. This illustrates the complete operation and a non-prime curve order. It does not use Pasta’s compressed-identity convention, whose disjointness relied on the absence of finite $x = 0$ points.

No small example establishes a security level. Even the exact Pasta group-order facts would not prove that discrete logarithms are hard. They establish the algebraic domains on which a later security assumption can be stated. The imported group theorem and parameter facts are the only substantial unproved mathematical inputs to this curve construction; all encoding operations and ordinary arithmetic algorithms used here have been given explicitly.

9 Byte fields for AES

A byte-oriented algorithm may use a field that is not a prime field. For AES, bytes represent polynomials with binary coefficients, not integers modulo 256. This final construction supplies exactly that field and its invertible operations.

9.1 Binary polynomial arithmetic

Use $\mathbb{F}_2 = \{0, 1\}$, where addition and subtraction coincide with exclusive-or. Represent a byte a by

$$a(X) = a_0 + a_1X + \cdots + a_7X^7,$$

where a_i are the bits of its integer encoding. Define $M(X) = X^8 + X^4 + X^3 + X + 1$. Add polynomials coefficientwise and multiply them by convolution, then reduce the result modulo M using polynomial division. The result again has degree less than eight and so fits in one byte. Replacing either input by a congruent polynomial does not change the result, because multiples of M remain multiples under addition and multiplication. Associativity and distributivity therefore descend from $\mathbb{F}_2[X]$.

A polynomial of positive degree is irreducible if it is not a product of two polynomials of smaller positive degree. A reducible degree-eight polynomial has a factor of degree at most four: in any factorisation, at least one of the two degrees is at most four. Since the only nonzero leading coefficient in $\mathbb{F}_2$ is one, all candidate factors are monic, meaning that their leading coefficient is one. There are $2 + 4 + 8 + 16 = 30$ monic polynomials of degrees one through four. Dividing M by each is a complete finite irreducibility test. The accompanying checker performs all 30 divisions and verifies that none has zero remainder. The hexadecimal representation of the full modulus is `0x11b`; the ninth bit records the X^8 term and is not itself an eight-bit field element.

Proposition 9.1 (The AES field). *The 256 degree-less-than-eight binary polynomials, with the operations above, form a field.*

Proof. We have established the arithmetic laws inherited from polynomials. To invert a nonzero polynomial a of degree less than eight, run Euclid's algorithm on a, M , using polynomial division. The last nonzero remainder is a common divisor and a polynomial combination $ua + vM$, by exactly the coefficient bookkeeping in the integer algorithm. A nonconstant common divisor would factor the irreducible polynomial M ; its degree cannot be eight because it also divides a . Thus the greatest common divisor is the constant one. Reducing the identity $ua + vM = 1$ modulo M gives the inverse $u \bmod M$. $\square$

The field contains a copy of $\mathbb{F}_2$: send zero and one to their constant-polynomial representatives. This map preserves one, addition and multiplication, and is injective. Its image is therefore a subfield, a subset that is itself a field under the inherited operations. We may identify that image with the original $\mathbb{F}_2$.

A field containing a specified copy of another field is a field extension of that smaller field. Thus the byte field is an extension of $\mathbb{F}_2$, not merely a larger set carrying the same names for some elements. Multiplication by a constant polynomial makes it a vector space over $\mathbb{F}_2$. Its eight representatives $1, X, \dots, X^7$ form a basis: every reduced polynomial is their linear combination, and a nonzero combination of degree less than eight cannot be a multiple of the degree-eight modulus. The dimension

of this vector space is the degree of the field extension, here eight. Consequently it has exactly 2^8 elements, one for each choice of eight binary coefficients. This derives both the extension terminology and the size from the constructed operations; no unspecified extension arithmetic is being used.

We denote this field by $\mathbb{F}_{2^8}$. Its concrete byte representation depends on the chosen modulus and basis. A field isomorphism preserves its mathematical operations, as defined in §2, but need not preserve the integer labels of the bytes. Consequently another byte basis can give a different encoded multiplication table. The symbol $0x02$ represents X , not the prime-field integer two, which is zero in characteristic two.

9.2 Constructive multiplication and inversion

Multiplication by X shifts the byte left by one bit. If the old highest bit was one, the shift introduced X^8 , which reduces to $X^4 + X^3 + X + 1$. Therefore the byte operation is

$$\text{xtime}(a) = ((a \ll 1) \bmod 256) \oplus \begin{cases} 0x1b, & a \geq 128, \\ 0, & a < 128. \end{cases}$$

To multiply a by b , initialise a zero accumulator and scan the eight bits of b from low to high. Exclusive-or the current value of a into the accumulator for each set bit, and apply xtime to a after each step. This is polynomial convolution followed by reduction, so it proves the operation implements the field product.

The nonzero multiplicative group has order 255. Lagrange gives $a^{255} = 1$, so $a^{-1} = a^{254}$ for nonzero a . Exponentiation using the byte multiplication just specified is a second complete inversion algorithm. A later substitution operation may define an extended inverse that maps zero to zero. That is a separate total function, not a claim that zero has a multiplicative inverse.

Example 9.2 (Byte arithmetic). The field product $0x57 \cdot 0x83$ is $0xc1$, and the inverse of $0x53$ is $0xca$. The checker evaluates both through the explicit polynomial arithmetic and checks $aa^{254} = 1$ for all 255 nonzero byte values.

An affine map on the eight bits means a binary linear transformation followed by exclusive-or with a fixed byte. To test its invertibility, form the eight-by-eight matrix of its effect on the coordinate bits and apply elimination over $\mathbb{F}_2$. A byte substitution formed by the extended inverse followed by an invertible affine map is a permutation: the extended inverse exchanges each nonzero element with its inverse and fixes zero; composition preserves bijectivity. This establishes the algebra needed to construct AES's substitution step, without identifying byte-field arithmetic with ordinary integer arithmetic or assuming a cipher's security from its invertibility.

Verification boundary

The numerical examples in this volume are produced and checked by `check_mathcrypto.py` alongside the rewrite sources. The checks include field inverses and reduction bias, rank-one and rank-two mask distributions, both directions of the four-point transform, a complete small-curve point count, published Pasta scalar identities, and exhaustive byte-field inverses and modulus irreducibility. They do not certify Pasta primality or point counts, prove the imported elliptic-curve associativity theorem, establish cryptographic hardness, or certify constant-time implementations. Those limits are part of the mathematical contract, not omissions to be filled by trusting a worked example.

Part II

Crypto Guide: Mechanisms and security contracts for shielded payments

Abstract

This volume of *The Zcash Arboretum* constructs the cryptographic mechanisms needed to understand private payments: how information is concealed, who can approve a transfer, and how its recipient recovers it. It assumes the arithmetic, encodings, probability and algebra developed in the *Math Guide*. Each security claim states what an attacker sees, what it may attempt, and which assumptions support the guarantee. Specifying a reproducible calculation is distinct from showing that it protects the information or authority entrusted to it.

1 Security contracts and reductions

An algorithm specifies outputs. A security contract specifies what an adversary sees, what it may ask, and which event counts as a violation. The same algorithm can satisfy one contract and fail another. We use the probability, conditioning and computational conventions of the *Math Guide*, “Probability and computation”, throughout.

1.1 Experiments and advantage

A digital signature scheme has three algorithms. Key generation samples a secret signing key and a corresponding public verifying key. Signing takes the secret key and a message and returns a signature, possibly using fresh randomness. Verification takes the public key, message and signature and returns accept or reject. Correctness requires accepting signatures generated honestly under that key. A signature authenticates bytes relative to a public key; it does not establish the identity of that key’s owner by itself.

A security experiment is an algorithm that samples keys and other randomness, interacts with an adversary, and returns a bit. An oracle is a callable procedure within that experiment; it may retain state between calls. A signing oracle, for example, accepts chosen messages and returns signatures under a key held by the experiment. The adversary’s next request may depend on every earlier response. Its view includes all public inputs and observed messages, together with its own internal randomness. A hidden key is not part of that view unless the experiment explicitly reveals it.

Existential unforgeability under chosen-message attack, abbreviated EUF-CMA, means that an efficient adversary given an honestly generated verifying key and a signing oracle cannot produce an accepted signature on a message it never asked the oracle to sign, except with negligible probability. Strong unforgeability, SUF-CMA, instead forbids any new accepted message–signature pair, including a new signature for an already signed message. Replaying an exact oracle response wins neither game. These definitions concern honestly generated keys; related-key and adversarial-key variants must separately specify how keys may be selected or modified.

For two experiments G_0, G_1 , the distinguishing advantage of an adversary A is

$$|\Pr[G_0^A = 1] - \Pr[G_1^A = 1]|.$$

If a uniformly sampled hidden bit chooses the experiment and the adversary guesses that bit, the absolute excess of its success probability over one half is half this distinguishing advantage, after choosing the output orientation appropriately. We keep these normalisations distinct. For search problems, such as finding a collision, advantage usually means the success probability itself.

An asymptotic statement quantifies over efficient adversaries and requires negligible advantage as the security parameter grows. A concrete statement instead gives a bound in terms of time, query

counts and the chosen modulus or output length. Neither format removes the need to state the experiment. In particular, a theorem for a uniformly generated key does not automatically cover an adversarially chosen key, a key revealed after signing, or several related keys used simultaneously.

1.2 Reductions and hybrid changes

A reduction turns an adversary for one task into an algorithm for another. It receives a challenge for the second task, constructs an environment for the adversary, and transforms a successful attack into a solution to its challenge. A proved inequality of the form

$$\Pr[B \text{ solves its challenge}] \geq \frac{\text{Adv}(A) - \delta}{L}$$

records both a loss factor $L > 0$ and additive simulation error δ . The reduction’s running time and oracle access also belong to the statement. Large losses can make an asymptotic theorem weak at fixed parameters.

Often a proof replaces one part of an experiment at a time. For experiments $G_0, \dots, G_k$, the triangle inequality gives

$$|\Pr[G_0 = 1] - \Pr[G_k = 1]| \leq \sum_{i=0}^{k-1} |\Pr[G_i = 1] - \Pr[G_{i+1} = 1]|.$$

Each difference must be justified: perhaps by a computational assumption, perhaps by exact equality of distributions, or by a coupling that fails only on a bounded bad event. We use the *Math Guide*, “Probability and computation”, for these coupling and data-processing facts. Conditioning on no failure is not a substitute for proving the coupling.

1.3 Group hardness contracts

Let $\mathbb{G}$ be a prime-order group of order r with generator G , using the operations of the *Math Guide*, “Groups and scalar multiplication”. The following experiments sample scalars uniformly from $\mathbb{F}_r$; any restriction to nonzero scalars must be stated as a variant.

Assumption 1.1 (Discrete logarithm). Given G and $X = [x]G$ for uniform x , an efficient adversary has only negligible probability of returning x .

Assumption 1.2 (Computational Diffie–Hellman). Given $G, [a]G, [b]G$ for independent uniform a, b , an efficient adversary has only negligible probability of returning $[ab]G$.

Assumption 1.3 (Decisional Diffie–Hellman). The distributions $(G, [a]G, [b]G, [ab]G)$ and $(G, [a]G, [b]G, [c]G)$, with independent uniform a, b, c , are computationally indistinguishable.

These assumptions have different content. A logarithm solver computes a and hence solves the computational problem. A computational solver distinguishes the two decisional distributions by comparing its output with the last point, losing at most the accidental equality probability $1/r$ in the random case. Therefore decisional hardness is the stronger assumption; logarithm hardness alone does not imply it. Scalar zero creates explicitly easy cases of probability on the order of $1/r$, not a contradiction of an asymptotic assumption.

The curve arithmetic and parameter facts in the *Math Guide*, “Pasta curves and their encodings”, define concrete instances for these assumptions. They do not prove them. A generic search can

split the unknown scalar as $x = i + jm$, with $m = \lfloor \sqrt{r} \rfloor$, store the m points $[i]G$, and compare against $X - [jm]G$. Some pair matches, recovering $x \bmod r$ in $O(\sqrt{r})$ group work and storage. This is an upper bound on an attack's cost, not a lower bound for every algorithm exploiting a curve's representation.

1.4 Hash and function contracts

A hash function has arbitrary or specified finite-length input and a fixed-length output. Collision resistance means that, after any public parameters are given, it is difficult to find distinct allowed inputs m, m' with $H(m) = H(m')$. Preimage resistance instead asks for an input matching a supplied target output. Neither property says that a secret-keyed use of the hash is pseudorandom, nor that outputs on related inputs are independent.

A pseudorandom function, or PRF, is a keyed family $F_k : D \rightarrow R$. For a randomly sampled secret key, oracle access to F_k must be indistinguishable from access to a uniformly random function $D \rightarrow R$. Repeated inputs to either oracle return the same output. The adversary may choose its inputs adaptively, subject to the contract's query bound. If keys have a specified nonuniform distribution or are accompanied by correlated public information, that distribution and information belong in the assumption.

A pseudorandom permutation, or PRP, is a keyed family of bijections on a finite set, compared against a uniformly random bijection. The contract specifies whether the adversary may query only the forward direction or both directions. A random permutation never collides on distinct inputs; a random function may. The distinction is material when query counts approach the square root of the domain size. Bijectivity alone is not a PRP proof.

1.5 Random oracles and domain separation

In the random-oracle model, all parties can query the same ideal random function. A lazy implementation maintains a table. It answers an old input from the table; for a new input it samples a uniform output, stores it, and returns it. This describes exactly the finite transcript distribution of a random function without having to sample an infinite table in advance.

A proof may simulate that oracle, including programming a chosen output at a fresh input. Programming an already answered input would make the transcript inconsistent. Even at a fresh input, the programmed value and the rest of the simulated transcript must have the required joint distribution. A real hash algorithm is a fixed public computation, not a programmable oracle; a random-oracle theorem is a model-dependent security statement about its idealisation.

If domains are encoded injectively as $\text{enc}(D, m)$, then disjoint domain labels query disjoint inputs to the ideal oracle. Their restrictions are therefore independent random functions. Ambiguous concatenation can destroy this reasoning. We use the tuple and byte conventions of the *Math Guide*, “Language, integers, and encodings”. A personalisation parameter inside a concrete hash is likewise part of its algorithm; modelling different personalisations as independent functions is an additional security idealisation, not an algebraic consequence of changing a few initial-state bits.

2 Hash functions and key derivation

The constructions in this section are executable specifications. Their security contracts are identified separately. Byte-level hashes operate on word arithmetic and bit operations from the *Math Guide*, “Language, integers, and encodings”; the group maps use “Pasta curves and their encodings”; the field permutation uses “Prime fields and modular arithmetic” and “Linear algebra and uniform masks”.

2.1 SHA-256: words, padding and compression

SHA-256 maps byte strings shorter than 2^{61} bytes to 32 bytes. All its additions are modulo 2^{32} . Let $\text{rotr}_j(x)$ be right rotation of a 32-bit word, and $\text{shr}_j(x) = \lfloor x/2^j \rfloor$. Define

$$\begin{aligned} \text{Ch}(x, y, z) &= (x \& y) \oplus ((\neg x) \& z), \\ \text{Maj}(x, y, z) &= (x \& y) \oplus (x \& z) \oplus (y \& z), \\ \Sigma_0(x) &= \text{rotr}_2(x) \oplus \text{rotr}_{13}(x) \oplus \text{rotr}_{22}(x), \\ \Sigma_1(x) &= \text{rotr}_6(x) \oplus \text{rotr}_{11}(x) \oplus \text{rotr}_{25}(x), \\ \sigma_0(x) &= \text{rotr}_7(x) \oplus \text{rotr}_{18}(x) \oplus \text{shr}_3(x), \\ \sigma_1(x) &= \text{rotr}_{17}(x) \oplus \text{rotr}_{19}(x) \oplus \text{shr}_{10}(x). \end{aligned}$$

The operations $\&$ and $\neg$ are bitwise and and 32-bit complement. Initialise eight state words to $I_j = \lfloor 2^{32} \sqrt[p_j]{p_j} \rfloor \bmod 2^{32}$, where p_j , indexed from zero, are the first eight primes. Define 64 round constants $K_j = \lfloor 2^{32} \sqrt[p_j]{p_j} \rfloor \bmod 2^{32}$ using the first 64 primes. These are exact integer constructions: compute the integer square root of $p_j 2^{64}$, or integer cube root of $p_j 2^{96}$, respectively, then retain the low 32 bits. An integer d -th root is the largest integer u with $u^d \leq n$, found by binary search; no floating-point rounding is required. Trial division generates the required finite prime list, beginning 2, 3, 5, 7, 11, 13, 17, 19.

For a message of bit length L , append one bit, then enough zero bits to make the length 448 modulo 512, then append the 64-bit big-endian encoding of L . Parse each resulting 64-byte block into 16 big-endian words $W_0, \dots, W_{15}$. Extend them for $16 \leq j < 64$ by

$$W_j = \sigma_1(W_{j-2}) + W_{j-7} + \sigma_0(W_{j-15}) + W_{j-16}.$$

Copy the current state into (a, b, c, d, e, f, g, h) . For each $j = 0, \dots, 63$, calculate

$$T_1 = h + \Sigma_1(e) + \text{Ch}(e, f, g) + K_j + W_j, \quad T_2 = \Sigma_0(a) + \text{Maj}(a, b, c),$$

then simultaneously replace the working words by $(T_1 + T_2, a, b, c, d + T_1, e, f, g)$. Add the eight final working words to the eight chaining-state words. Repeat for every block, then concatenate the eight state words in big-endian order.

This constructs the SHA-256 algorithm in FIPS 180-4, sections 5 and 6.2. Applying it twice defines $\text{SHA256d}(m) = \text{SHA256}(\text{SHA256}(m))$. Neither double application nor the padding structure is, by itself, a proof of collision resistance or a random-oracle property. Where a protocol uses a tagged wrapper, its precise prefix is an additional construction, not a different SHA-256 compression function.

2.2 Personalised BLAKE2b

We need the sequential, unkeyed BLAKE2b function with an output length d between 1 and 64 bytes and a 16-byte personalisation P . The adjective unkeyed refers to BLAKE2b's native key parameter; a secret may still occur in the message of a higher-level PRF construction. The following fixed initialisation specifies the unkeyed sequential mode; no optional modes are used.

All additions now use 64-bit words. Define $I_j = \lfloor 2^{64} \sqrt[p_j]{p_j} \rfloor \bmod 2^{64}$ for the same first eight primes, constructible as integer square roots of $p_j 2^{128}$. Initialise $h_j = I_j$, then apply

$$h_0 \leftarrow h_0 \oplus (0x01010000 \oplus d), \quad h_6 \leftarrow h_6 \oplus \text{val}_{256}^{\text{LE}}(P[0..8]), \quad h_7 \leftarrow h_7 \oplus \text{val}_{256}^{\text{LE}}(P[8..16]).$$

Slices use an inclusive lower and exclusive upper byte index. The notation $\text{val}_{256}^{\text{LE}}$ interprets a byte slice little-endian. A shorter application label is padded with zero bytes to 16 bytes before becoming P ; longer labels require a different construction and are not silently truncated.

Split the message into 128-byte blocks, padding the final block with zeros if necessary. An empty message has one all-zero final block. A nonempty exact multiple of 128 bytes has its last existing

block marked final; no extra block is appended. For each block, parse 16 little-endian 64-bit message words m_i . Let t be the number of actual message bytes processed through this block, excluding padding, represented as a 128-bit integer; the total message length must be less than 2^{128} bytes. Initialise $v_0, \dots, v_7 = h_0, \dots, h_7$ and $v_8, \dots, v_{15} = I_0, \dots, I_7$; exclusive-or the low and high 64 bits of t into v_{12}, v_{13} , and complement v_{14} for the final block.

The mixing operation $G(a, b, c, d, x, y)$ updates four selected working words in this order. All additions now use 64-bit words, and rotr_j rotates a 64-bit word right, not the 32-bit word used in SHA-256:

$$\begin{aligned} a &\leftarrow a + b + x, & d &\leftarrow \text{rotr}_{32}(d \oplus a), \\ c &\leftarrow c + d, & b &\leftarrow \text{rotr}_{24}(b \oplus c), \\ a &\leftarrow a + b + y, & d &\leftarrow \text{rotr}_{16}(d \oplus a), \\ c &\leftarrow c + d, & b &\leftarrow \text{rotr}_{63}(b \oplus c). \end{aligned}$$

For round j , use the row $\sigma_{j \bmod 10}$ from the following table. Each row lists a permutation of message-word indices, written in hexadecimal so every index occupies one digit.

0	0123456789abcdef
1	ea489fd61c02b753
2	b8c052fdae367194
3	7931dcbe265a40f8
4	905724afe1bc683d
5	2c6a0b834d75fe19
6	c51fed4a0763928b
7	db7ec13950f4862a
8	6fe9b308c2d714a5
9	a2847615fb9e3cd0

Spaces in a row only aid reading. Apply G to working-word index quadruples

$$(0, 4, 8, 12), (1, 5, 9, 13), (2, 6, 10, 14), (3, 7, 11, 15),$$

then

$$(0, 5, 10, 15), (1, 6, 11, 12), (2, 7, 8, 13), (3, 4, 9, 14).$$

The k -th application, indexed from zero, receives message arguments $m_{\sigma(2k)}, m_{\sigma(2k+1)}$. Perform 12 rounds, then set $h_i \leftarrow h_i \oplus v_i \oplus v_{i+8}$ for $0 \leq i < 8$. After the final block, serialize the eight state words little-endian and take the first d bytes.

Write the result as $\text{BLAKE2b}_{8d}(P, m)$, where the subscript counts bits and the internal parameter d counts bytes. The output length is part of initialisation: a 32-byte output is not the first half of the 64-byte-output function. The compression is described in RFC 7693; personalisation follows the parameter block in the protocol specification’s “BLAKE2 Hash Functions” and the `blake2b_simd` implementation. All later calls fix d, P explicitly.

2.3 Expansion from a secret seed

For a 32-byte seed k and byte string t , define

$$\text{PRFexpand}_k(t) = \text{BLAKE2b}_{512}(\text{Zcash_ExpandSeed}, k || t). \quad (1)$$

The 16-byte label is exact. This is an unkeyed BLAKE2b invocation whose message begins with the secret seed, not BLAKE2b with its native key parameter set to k . The subscript on PRFexpand identifies the secret input in the PRF contract. Distinct application purposes use their specified tag bytes in t , together with any additional length-fixed fields.

For a 64-byte string b , define

$$\text{ToBase}(b) = [\text{val}_{256}^{\text{LE}}(b)]_{p_{\text{Pallas}}}, \quad \text{ToScalar}(b) = [\text{val}_{256}^{\text{LE}}(b)]_{p_{\text{Vesta}}}.$$

These are reduction functions, not canonical field decoders. A uniform 512-bit input yields the small, precisely bounded reduction bias from the *Math Guide*, “Probability and computation”. The PRF assumption for (1), including the actual seed distribution and public leakage of its outputs, is separate from that arithmetic calculation. Revealing one derived key may change which other secrecy claims a key hierarchy can support; domain separation alone does not prove every such claim.

2.4 Poseidon’s exact three-word permutation

Let $p = p_{\text{Pallas}}$. The permutation acts on $\mathbf{s} \in \mathbb{F}_p^3$. It has 64 rounds with three round constants per round and one fixed three-by-three matrix M . In rounds 0, 1, 2, 3, 60, 61, 62, 63, first add the three constants and raise all three words to the fifth power. In rounds 4 through 59, add all three constants but raise only word zero to the fifth power. End every round by replacing the state with $M\mathbf{s}$. There are thus eight full rounds and 56 partial rounds; the latter do not omit the additions to the other two words.

The following generator fixes every constant, avoiding an undefined external table. Initialise an 80-bit register by concatenating, in most-significant-bit-first order,

$$\text{BE}_2(1) \parallel \text{BE}_4(0) \parallel \text{BE}_{12}(255) \parallel \text{BE}_{12}(3) \parallel \text{BE}_{10}(8) \parallel \text{BE}_{10}(56) \parallel 1^{30}.$$

Here BE_k denotes exactly k bits, and 1^{30} means 30 one bits. For register bits $s_0, \dots, s_{79}$, one raw step computes $b = s_0 \oplus s_{13} \oplus s_{23} \oplus s_{38} \oplus s_{51} \oplus s_{62}$, shifts the register left by one position, appends b , and outputs b . Discard 160 raw outputs. Thereafter read raw outputs in pairs: emit the second bit exactly when the first is one, otherwise discard the pair and repeat. This is a deterministic parameter generator, not a source of secret random bits.

To generate one round constant, read 255 emitted bits as a big-endian integer; reject and read another 255 bits if it is at least p . Generate 192 accepted constants and arrange them row-by-row into the 64 rounds. Continue from the resulting register state to generate six further 255-bit integers, this time reducing each modulo p without rejection. If any of the six residues coincide, discard the whole six-value candidate and continue. Call the first three x_i and the last three y_j , and set

$$M_{ij} = (x_i + y_j)^{-1}.$$

For these pinned parameters the first distinct candidate is used, and every displayed denominator is nonzero; the checker verifies this and the generated constants against the implementation. The algorithm is the parameter generation in `halo2_poseidon 0.1.0`, `src/grain.rs`, `src/mds.rs` and `src/p128pow5t3.rs`, corresponding to the protocol specification’s “PoseidonHash Function”.

For distinct x_i , distinct y_j , and nonzero sums, every square submatrix of this matrix is invertible. To prove it for a k -by- k submatrix, suppose coefficients a_j give $\sum_j a_j/(x_i + y_j) = 0$ at its k distinct row values. Multiply by $\prod_j (X + y_j)$. The resulting polynomial has degree less than k and k roots, hence is zero. Evaluating at $X = -y_j$ forces $a_j = 0$ because all other factors are nonzero. This proves the matrix property called maximum distance separable, or MDS, in this construction. It is a diffusion property, not a security proof for the permutation.

The fifth-power map is a bijection because $\gcd(5, p - 1) = 1$: an inverse exponent exists modulo $p - 1$ on nonzero elements, and zero is fixed. Constant addition and matrix multiplication are also bijections, so every round and the complete function π are permutations. The two-input hash is exactly

$$\text{PoseidonHash}(x, y) = \pi(x, y, 2^{65})_0. \quad (2)$$

The last word fixes the constant two-element input length and one-element output length; no extra input padding or permutation round is performed. The nullifier PRF contract treats its first input as

a secret key and its second as the chosen argument. Permutation invertibility, the MDS proof, and reproducibility of the round constants do not establish that PRF contract. We assume the required pseudorandomness for the specified key distribution and leakage, rather than inferring it from collision resistance.

2.5 Hashing to the Pasta groups

A generator derived from a public message must be a group point, not an unchecked coordinate or a known scalar times a fixed base. We define $\text{GroupHash}_C(D, m)$ for target curve C equal to Pallas or Vesta. The base modulus p is the corresponding one from the *Math Guide*, “Pasta curves and their encodings”. The domain D and message m are bytes. Set

$$\text{DST} = D \parallel \text{name}(C) \parallel \text{XMD:BLAKE2b_SSWU_RO_},$$

where the names are the lowercase ASCII strings `pallas` and `vesta`. Reject if this string is longer than 255 bytes. Append its one-byte length to obtain DST' . With all-zero BLAKE2b personalisation, calculate

$$\begin{aligned} b_0 &= \text{BLAKE2b}_{512}(0^{16}, 0^{128} \parallel m \parallel [0, 128, 0] \parallel \text{DST}'), \\ b_1 &= \text{BLAKE2b}_{512}(0^{16}, b_0 \parallel [1] \parallel \text{DST}'), \\ b_2 &= \text{BLAKE2b}_{512}(0^{16}, (b_0 \oplus b_1) \parallel [2] \parallel \text{DST}'). \end{aligned}$$

In these expressions 0^n denotes n zero bytes, and a bracketed list denotes literal byte values. Interpret b_1, b_2 as *big-endian* 512-bit integers and reduce modulo p , giving u_0, u_1 . This is intentionally a different byte order from ToBase. The bytes `[0, 128]` encode the 128-byte expansion length, and the following zero is a separate separator.

The intermediate curve has equation $y^2 = x^3 + Ax + B$, with $B = 1265$, $Z = -13$, and

C	A as a hexadecimal integer modulo p
Pallas	18354a2eb0ea8c9c49be2d7258370742b74134581a27a59f92bb4b0b657a014b
Vesta	267f9b2ee592271a81639c4d96f787739673928c7d01b212c515ad7242eaa6b1

For each u , put $t = Zu^2$. If $t^2 + t \neq 0$, set $x_1 = -(B/A)(1 + (t^2 + t)^{-1})$; otherwise set $x_1 = B/(ZA)$. Compute $g_1 = x_1^3 + Ax_1 + B$. If g_1 is square, choose $x = x_1$ and a square root y of g_1 . Otherwise choose $x = tx_1$ and a square root y of $x^3 + Ax + B$. Finally negate y if necessary so that its canonical integer has the same low bit as u . Call the resulting intermediate point $Q(u)$.

The square root in the second branch exists. When $u \neq 0$ and $t^2 + t \neq 0$, substitution gives

$$(tx_1)^3 + A(tx_1) + B = t^3 g_1.$$

The value t is a nonsquare because Z is a nonsquare and u^2 is a nonzero square. Multiplication by t^3 therefore exchanges square and nonsquare values. The exceptional case is also completely specified: since -1 is square in both Pasta fields whereas Z is nonsquare, $Zu^2 = -1$ is impossible. Thus $t^2 + t = 0$ requires $u = 0$, and for the stated constants $g(B/(ZA))$ is a nonzero square, as verified by Euler’s criterion in the checker. The parity selection makes the output independent of the square-root algorithm’s choice. This affine form is equivalent to the denominator-cleared SWU implementation in `pasta_curves`; it exposes the divisions and exceptional case explicitly.

Add $Q(u_0) + Q(u_1)$ using the complete operation on the intermediate curve. Map the identity to the target identity. For a finite intermediate point (x, y) , the target map is

$$\begin{aligned} N_x &= c_0 x^3 + c_1 x^2 + c_2 x + c_3, & D_x &= x^2 + c_4 x + c_5, \\ N_y &= (c_6 x^3 + c_7 x^2 + c_8 x + c_9) y, & D_y &= x^3 + c_{10} x^2 + c_{11} x + c_{12}. \end{aligned}$$

Pallas:

Vesta:

The primary definition is the protocol specification’s “Group Hash into Pallas and Vesta”, implemented in `pasta_curves 0.5.2, src/hashtocurve.rs` and `src/curves.rs`. Its exact XMD framing takes precedence over a differently parameterised generic hash-to-curve suite. A caller that requires a nonidentity point must check or otherwise establish that requirement. For an 11-byte diversifier d , the concrete diversification wrapper is

41

Then

$$\text{DiversifyHash}(d) = \begin{cases} \text{GroupHash}_{\text{Pallas}}(\text{z.cash:Orchard-gd}, \epsilon), & P = \mathcal{O}, \\ P, & P \neq \mathcal{O}. \end{cases}$$

The fallback is a fixed separately checked nonidentity point; it is not a retry with an unspecified counter. Application-level address unlinkability still needs its own adversary and key distribution, in addition to this deterministic construction.

3 Groups, commitments, and Sinsemilla

A commitment lets a party fix a value without publishing it. We require two different properties: hiding the value from observers, and preventing the committer from later opening the same object as two different values. The point operations and scalar domains are those of the *Math Guide*, “Groups and scalar multiplication” and “Pasta curves and their encodings”.

3.1 Commitment syntax and security games

A commitment algorithm takes public parameters, a message m and randomness r , and returns C . An opening consists of the message and the auxiliary information needed to verify the commitment; for the constructions here it is (m, r) . Correctness means an honestly computed opening verifies.

In the hiding experiment, the adversary chooses two allowed messages of the same permitted type, a hidden random bit chooses one, and the adversary sees its commitment with freshly sampled randomness. Perfect hiding means the two distributions are identical; statistical hiding bounds their statistical distance; computational hiding bounds the advantage of efficient distinguishers. In the binding experiment, the adversary returns one commitment and two accepted openings with different messages. Computational binding requires its success probability to be negligible. Neither game requires an arbitrary commitment to come with an efficiently extractable opening.

3.2 Pedersen commitments and their reductions

In a prime-order group of order r , choose nonidentity points G, H . For $m, \rho \in \mathbb{F}_r$, define

$$\text{Com}(m; \rho) = [m]G + [\rho]H.$$

Verification recomputes the right side. For each fixed message, uniform ρ makes the commitment uniform on the group: multiplication by H is a bijection, and addition of $[m]G$ is a translation. This proves perfect hiding exactly, provided ρ is uniform and independent of the message.

Theorem 3.1 (Pedersen binding reduction). *Two openings $(m, \rho), (m', \rho')$ of the same commitment, with $m \neq m'$, yield the discrete logarithm of H relative to G .*

Proof. Subtract the opening equations to obtain $[m - m']G + [\rho - \rho']H = \mathcal{O}$. The scalar $\rho - \rho'$ cannot be zero, because G has order r and $m - m' \neq 0$. Writing $H = [h]G$, scalar injectivity gives $h = -(m - m')/(\rho - \rho')$ in $\mathbb{F}_r$. $\square$

The reduction assumes the public parameters have the promised distribution and that the adversary does not already know that logarithm. Choosing $H = [h]G$ with a publicly known h would destroy binding immediately, despite leaving perfect hiding intact. A public hash-to-group procedure supplies reproducible bases; its unknown-relation property uses the security model stated in §2.

The vector construction is

$$\text{Com}(\mathbf{m}; \rho) = \sum_{i=0}^{n-1} [m_i]G_i + [\rho]H.$$

The same translation argument proves perfect hiding. A pair of distinct vector openings yields a nonzero coefficient relation among the public bases. The following lemma connects that event to ordinary discrete logarithms without calling the bases linearly independent.

Lemma 3.2 (Unknown relations from discrete logarithms). *For independent uniform group bases $H_1, \dots, H_N$, an algorithm returning a nonzero coefficient vector with $\sum_i [c_i]H_i = \mathcal{O}$, with probability ϵ , gives a discrete-log solver with success probability at least $\epsilon(1 - 1/r)$. The statement also permits independently uniform nonidentity bases.*

Proof. Given $X = [x]G$, sample independent uniform scalars a_i, b_i and supply $H_i = [a_i]G + [b_i]X$. For the fixed hidden scalar x , the change of variables $(a_i, b_i) \mapsto (a_i + xb_i, b_i)$ is a bijection. Hence the bases have the required independent uniform distribution, and the hidden b_i remain uniform independent of the entire adversary view and returned relation. For nonidentity bases, resample any identity result; conditioning on the first coordinate being nonzero leaves the second uniform.

For a nonzero returned vector, the scalar $B = \sum_i c_i b_i$ is uniform, by solving for any b_i whose coefficient is nonzero. It is zero with probability $1/r$. Otherwise the relation gives $x = -(\sum_i c_i a_i)/B$, which the reduction can compute. The argument conditions on the bases and the returned relation, not on hidden coefficients unavailable to the adversary. $\square$

Random-oracle-derived bases can be simulated by this construction at each distinct domain/input pair, reusing the same sampled point on repeat queries. A concrete hash-to-curve implementation is covered only under the corresponding idealisation or an explicit imported composition result. Fixed bases that happen to be distinct are not enough for the lemma’s hypothesis.

3.3 Adding commitments and retaining integer meaning

The scalar identities imply

$$\text{Com}(m; \rho) + \text{Com}(m'; \rho') = \text{Com}(m + m'; \rho + \rho').$$

Subtraction and multiplication by public scalars obey analogous identities. This property is called additive homomorphism. It holds in $\mathbb{F}_r$, not automatically for unbounded integers. To infer an integer balance, use the *Math Guide*, “Prime fields and modular arithmetic”, with bounds on the complete signed sum. Knowing that all individual amounts fit their encodings is insufficient without a bound preventing wraparound in the aggregate.

There is also a knowledge boundary. A point always has some discrete logarithm relative to a generator; existence of that scalar does not mean a prover knows it. An application that combines a commitment-opening proof with a signature must state how the relevant witnesses can be extracted together. The binding theorem only turns two *given* inconsistent openings into a hard-problem solution.

3.4 The exact Sinsemilla construction

Use the Pallas group, with scalar modulus $r = p_{\text{Vesta}}$. The input is a domain byte string D and at most 2530 bits M . Split the input into pieces of ten bits after appending zero bits to a multiple of ten. For the empty input use no pieces. Interpret each piece little-endian as an integer $m_i \in \{0, \dots, 1023\}$. The number of pieces is $n = \lceil M/10 \rceil \leq 253$. Define the bases

$$\begin{aligned} Q_D &= \text{GroupHash}_{\text{Pallas}}(\text{z.cash:SinsemillaQ}, D), \\ S_j &= \text{GroupHash}_{\text{Pallas}}(\text{z.cash:SinsemillaS}, \text{LE}_4^{\text{byte}}(j)). \end{aligned}$$

The table has 1024 entries indexed by piece *value*, shared by every position. It is not a new generator for each message position. Required fixed generators are checked to be nonidentity; a general

adversarially selected domain needs the corresponding exceptional-base event included in its security analysis.

Let $\boxplus$ be the incomplete addition defined in the *Math Guide*, “Pasta curves and their encodings”, extended to propagate $\perp$. Set $A_0 = Q_D$ and for $1 \leq i \leq n$ compute

$$A_i = (A_{i-1} \boxplus S_{m_i}) \boxplus A_{i-1}. \quad (3)$$

The result $\text{HashToPoint}(D, M)$ is A_n , possibly $\perp$. The coordinate hash applies the extended Extract function, preserving failure. These algorithms are specified in “Sinsemilla Hash Function” of the protocol specification and implemented by the `sinsemilla` crate’s hash-domain and incomplete-addition operations.

3.5 Why the reused table still binds a fixed-length message

When no exceptional addition occurs, the complete group law gives $A_i = [2]A_{i-1} + S_{m_i}$. Induction therefore yields

$$A_n = [2^n]Q_D + \sum_{i=1}^n [2^{n-i}]S_{m_i} = [2^n]Q_D + \sum_{j=0}^{1023} [\chi_j(m)]S_j,$$

where $\chi_j(m) = \sum_{i:m_i=j} 2^{n-i}$. The binary digits of χ_j identify exactly the positions occupied by piece value j . Since $0 \leq \chi_j < 2^n < r$, scalar reduction loses none of these digits. Thus the complete coefficient vector determines every piece. Padding is injective on inputs of any one fixed bit length, so it also determines the original message under that restriction.

Theorem 3.3 (Fixed-length Sinsemilla collision reduction). *For one domain and one fixed input length, two distinct messages with equal successful point hashes give a nonzero relation among the table bases. Equal successful coordinate hashes also give a nonzero relation, allowing the initial base Q_D .*

Proof. For point equality subtract the two coefficient expressions. The initial-base coefficients cancel, while the table coefficient difference is nonzero by injectivity. For equal coordinates, the *Math Guide*, “Pasta curves and their encodings”, shows that the points are equal or opposite, including the identity case. Equality was handled already. For opposites, add the two expressions. The coefficient of Q_D is 2^{n+1} , which is nonzero modulo r because $2^{n+1} \leq 2^{254} < r$. The relation is therefore nontrivial regardless of the table coefficients. $\square$

Apply Lemma 3.2 in the independently distributed-base model to obtain the discrete-log reduction. The fixed-length hypothesis is essential: appending zero bits without changing the number of pieces can leave the padded message unchanged. A domain label is not a length encoding unless the application assigns one fixed length to it or encodes lengths injectively itself.

3.6 Exceptional inputs and the relation they expose

Suppose the initial and table bases are nonidentity, and consider the first failing iteration. All preceding incomplete additions agree with complete addition, so $A = A_{i-1}$ has the known coefficient expression above and is nonidentity. Write $S = S_{m_i}$. The first addition fails precisely when $A = S$ or $A = -S$. If it succeeds, its output $A + S$ is nonidentity. The second addition fails when $A + S = A$ or $A + S = -A$. The first would require $S = \mathcal{O}$, excluded by hypothesis; the second gives $[2]A + S = \mathcal{O}$. Thus every first failure exposes

$$[\alpha]A + S = \mathcal{O}, \quad \alpha \in \{-1, 1, 2\}.$$

Substituting the known expression for A gives a relation whose initial-base coefficient is $\alpha 2^{i-1} \neq 0$ modulo r . Hence finding a failure efficiently also finds a nontrivial generator relation. An identity

initial or table base is itself such a relation and must not be silently ignored when the model allows arbitrary domain queries.

This is a computational exceptional-input argument, not an unconditional statement that $\boxplus$ never fails. It also does not say that denominator-cleared circuit equations reject every exceptional assignment. Their exact relation must be compared with the first-exception argument; otherwise the circuit and the specified partial function could describe different behaviours on precisely the inputs omitted by the ordinary formula.

3.7 Sinsemilla commitments and shortening

For a commitment domain D , use

$$H_D = \text{GroupHash}_{\text{Pallas}}(D||-r, \epsilon), \quad T = \text{HashToPoint}(D||-M, M).$$

If $T = \perp$, return $\perp$. Otherwise define

$$\text{SinsemillaCommit}_D(M; \rho) = T + [\rho]H_D, \quad \text{ShortCommit}_D(M; \rho) = \text{Extract}(T + [\rho]H_D).$$

The final addition is *complete*. Replacing it with incomplete addition changes the function and its valid domain. For every message whose point hash succeeds, uniform independent ρ gives a uniform group commitment when $H_D \neq \mathcal{O}$; deterministic coordinate extraction preserves perfect hiding, although it need not produce a uniform field element. This is hiding over the admitted message domain, not a claim that the public failure bit hides which invalid message was attempted.

For fixed-length messages, two inconsistent point openings produce the table-coefficient relation above with an additional coefficient of H_D . Equal shortened openings either reduce to this case or have opposite underlying points, whose initial-base coefficient 2^{n+1} again prevents the relation from being trivial. Thus shortening retains computational binding under the same base model. This does not promote coordinate extraction to an injective map.

The application domain `z.cash:Orchard-NoteCommit` instantiates note commitments. The short commitment domain `z.cash:Orchard-CommitIvk` instantiates `CommitIvk` on the fixed 510-bit concatenation of two canonical 255-bit field encodings. These are applications of the exact commitment construction in the protocol specification’s “Sinsemilla Commitments”, not new group operations. The upper-level protocol supplies which values occupy those encodings, which randomness is permitted, and which exceptional or zero outputs invalidate a key or note.

4 Knowledge, simulation, and Schnorr

A relation $R \subseteq X \times W$ specifies allowed pairs of public statements x and private witnesses w , with an efficient algorithm deciding whether $R(x, w) = 1$. The statement is true if at least one such witness exists. A prover has a witness and sends messages intended to convince a verifier that the statement is true. We now distinguish the properties that such an exchange may satisfy.

4.1 Completeness, soundness and knowledge

An interactive protocol specifies the messages, random choices, state transitions and final verification rule. Completeness means that an honest prover with a valid witness is accepted, perfectly or except with a stated error probability. Soundness means that a false statement is accepted with at most a stated probability even when the prover deviates. When this restriction is proved only against computationally bounded provers, the protocol is called an argument rather than an unconditional proof. This distinction concerns the guarantee, not whether the transmitted object is colloquially called a proof.

Knowledge soundness is stronger than ordinary soundness. It requires an extractor: an algorithm that, from suitably successful interaction with the alleged prover, obtains a witness satisfying the same

relation. The definition must specify the extractor's access. It may receive the prover's code, invoke it only through calls without inspecting its code, or restore a saved execution state while keeping every random choice fixed. The last operation is called rewinding; the sequence of fixed random choices is called its random tape. These are different models. A knowledge error κ allows success up to κ without requiring extraction; above it, the extractor's running time is bounded in terms of the prover's cost and its excess success probability. Existence of a witness in the mathematical relation is not knowledge of it.

Several proofs about one object may also need joint extraction. Separate statements that each accepting proof has some witness do not necessarily provide one extractor producing all consistent witnesses in the same adversarial execution. The surrounding application must state its relation and composition theorem precisely enough to make that requirement meaningful.

4.2 Simulation and what remains public

A simulator produces the verifier's view without receiving the witness. A protocol is zero knowledge when, for true statements, that simulated view matches the real view in the specified sense: exactly for perfect zero knowledge, within a small statistical distance for statistical zero knowledge, or against efficient distinguishers for computational zero knowledge. The simulator receives the public statement and whatever public or auxiliary input the definition permits. It is not required to hide information already determined by those inputs.

The view includes correlated messages, not just each message's marginal distribution. The *Math Guide*, "Linear algebra and uniform masks", exhibits two individually uniform observations that together reveal a secret combination. A simulation proof must preserve such correlations and the adversary's permitted interactions. Honest-verifier zero knowledge considers the prescribed verifier's challenge distribution; general zero knowledge permits a deviating verifier. General zero knowledge includes the honest-verifier case; the converse requires an additional argument.

4.3 A three-message proof of a discrete logarithm

Fix the group and generator of §1. The public statement is Y , and the witness is $x \in \mathbb{F}_r$ with $Y = [x]G$. The Schnorr exchange is:

1. The prover samples $k \leftarrow \mathbb{F}_r$ and sends $A = [k]G$.
2. The verifier samples $c \leftarrow \mathbb{F}_r$ and sends it.
3. The prover returns $z = k + cx$. The verifier accepts exactly when $[z]G = A + [c]Y$.

All encodings and point/scalar validity checks are part of the concrete implementation of this abstract exchange. Completeness follows directly by distributing scalar multiplication over $k + cx$.

Lemma 4.1 (Special soundness). *From two accepting transcripts (A, c, z) , (A, c', z') with $c \neq c'$, one recovers the witness $x = (z - z')/(c - c')$.*

Proof. Subtract the verification equations to obtain $[z - z']G = [c - c']Y$. Multiplying by the inverse of $c - c'$ proves that the displayed scalar has public point Y . $\square$

For any fixed challenge c , a simulator can sample $z \leftarrow \mathbb{F}_r$ and set $A = [z]G - [c]Y$. The transcript accepts. In a real exchange with this challenge, $z = k + cx$ is uniform because translation is a bijection; the same equation determines A . Thus the simulated and real joint distributions are identical. This is special honest-verifier zero knowledge: simulation works for a supplied challenge, not merely for one sampled by the simulator. It is not yet a proof for an arbitrary verifier choosing a challenge after seeing A .

4.4 The general three-message extraction calculation

A Sigma protocol has the same first-message, challenge, response shape, perfect completeness, special soundness from distinct challenges, and special honest-verifier zero knowledge in a stated perfect, statistical or computational sense. Its challenge set C is finite; write $N = |C|$, with honest challenges uniform.

Special soundness gives an actual extractor, not just an identity conditional on somehow obtaining two transcripts. Fix the prover's complete random tape and state just before the challenge; call this a row. Let δ be the fraction of challenges for which this row accepts. Over freshly sampled rows, suppose $\mathbb{E}\delta = \epsilon$. For each extraction attempt, generate a fresh row and run it on two independent uniform challenges, restoring the saved state and random tape for the second run. The probability that both accept with distinct challenges, conditional on the row, is $\delta^2 - \delta/N$. Averaging and using nonnegative variance gives

$$\Pr[\text{successful extraction attempt}] = \mathbb{E}[\delta^2] - \epsilon/N \geq \epsilon(\epsilon - 1/N). \quad (4)$$

When $\epsilon > 1/N$, independent attempts therefore extract in expected at most $1/(\epsilon(\epsilon - 1/N))$ attempts. Each uses two prover runs and one application of special soundness. This is polynomial in the inverse excess success probability; it suffices for knowledge error $1/N$. It is not claimed to be the sharpest expected-time bound. Sampling a fresh row on every attempt prevents the extractor from remaining trapped forever on a row that accepts only one challenge.

The calculation uses classical state restoration and a fixed complete random tape. It is not automatically valid for a quantum prover, for an unreplayable external device, or for a model in which the extractor cannot restore the required state. Those are different extraction contracts.

4.5 Fiat–Shamir and the forking lemma

The Fiat–Shamir transform replaces the verifier's random challenge by a hash of an injective encoding of the statement, first message and required context. The proof contains the first message and response; the verifier recomputes the challenge. This turns interaction into a noninteractive object, but its security is not proved merely by writing down the hash expression.

The following calculation describes the classical random-oracle tool used for extraction. Suppose an algorithm makes at most $Q \geq 1$ first-time oracle queries and on success identifies one of those queries as decisive. Repeated queries reuse their earlier answer and do not receive a new index in this accounting. Let its success probability be ϵ . Run it once; if it succeeds at query index j , rewind to just before that query, keeping the algorithm's random tape and all earlier oracle answers fixed, and resample the answer at j and all later fresh answers.

Lemma 4.2 (General forking bound). *For independent uniform oracle outputs in a set of size N , the probability that both runs succeed with the same decisive index and different answers there is at least*

$$\epsilon \left(\frac{\epsilon}{Q} - \frac{1}{N} \right).$$

A negative right side is interpreted only as a vacuous lower bound.

Proof. For each index j , fix the algorithm's tape and the oracle prefix before j , and let a_j be the conditional probability of success with decisive index j . Unused query positions can be padded with dummy random answers and assigned success probability zero. The original suffix and resampled suffix are independent given this prefix, so the probability that both succeed at j , summed over j , is $\sum_j \mathbb{E}[a_j^2]$. Nonnegative variance gives $\mathbb{E}[a_j^2] \geq (\mathbb{E}a_j)^2$; applying the same inequality to the finite list of Q means gives $\sum_j (\mathbb{E}a_j)^2 \geq \epsilon^2/Q$. On an initially successful run the new answer at its decisive index equals the original with probability $1/N$. Discarding all such events costs at most ϵ/N , proving the bound. $\square$

This is the classical random-oracle lemma underlying Bellare and Neven’s general forking argument. Applying it requires that the two successful outputs determine the same statement and first message at that decisive query, so that special soundness is applicable. Unqueried guesses, simulation aborts, reduction bias and additional oracle interactions have to be bounded separately.

For zero knowledge of a transformed Sigma protocol, the simulator may select a challenge, run the special honest-verifier simulator, and program that challenge at the resulting hash input. This is valid only when the point is fresh with suitably high probability and the joint distribution has the required simulation property. For example, if every first-message value has conditional probability at most 2^{-h} , then a fixed pre-existing table of at most q matching query inputs is hit with probability at most $q2^{-h}$. Here the bound is conditioned on all prior public information and the chosen challenge; an unconditional entropy statement would not suffice. The resulting simulation matches the real proof in the same perfect, statistical or computational sense, with the programming-bad probability accounted for.

4.6 Proof-system interfaces and explicit model boundaries

A noninteractive argument specifies parameter generation, proving and verification algorithms for a relation. A publicly reproducible parameter set is called transparent when no secret setup information is required. This does not prove that arbitrary public parameters are safe or that an adversary chose their encoding correctly. Succinctness is a bound on proof size and verifier work relative to the represented computation; a logarithmic proof can still have linear verification work. The actual bounds should replace the adjective whenever they matter.

In the algebraic group model, an algorithm producing a group output must also provide its expression as a linear combination of the group elements made available to it, under a specified model of oracle access. Such coefficients are additional information for an extractor. Ordinary access to an encoded point does not supply them. A security theorem proved in this model restricts the adversary accordingly; it is not a consequence of the curve equation. Likewise, composing a polynomial argument, a commitment-opening argument and Fiat–Shamir requires a theorem for that complete composition. The component definitions in this section do not establish that theorem for every construction using them.

5 RedPallas signatures

A digital signature scheme consists of key generation, a signing algorithm and a public verification algorithm. The private key authorises signing; possession of the public key permits verification but is not intended to permit creating fresh accepted signatures. We construct the exact RedPallas algorithms and then identify the security statement supplied by the Schnorr reduction.

5.1 Exact keys, signatures and verification

Use the Pallas group with scalar modulus $r = p_{\text{Vesta}}$ and a specified nonidentity base G . A private scalar x has verifying key $Y = [x]G$. Define

$$h(m) = \text{ToScalar}(\text{BLAKE2b}_{512}(\text{Zcash_RedPallasH}, m)).$$

The personalisation is exactly 16 bytes. For a message byte string M , signing performs:

1. Sample 80 independent uniform bytes T .
2. Set $k = h(T \| Y^* \| M)$, and $R = [k]G$.
3. Set $c = h(R^* \| Y^* \| M)$ and $z = k + cx$ in $\mathbb{F}_r$.
4. Return the 64 bytes $\sigma = R^* \| \text{LE}_{32}^{\text{byte}}(\text{val}(z))$.

Star encodings are packed as the 32-byte encodings in the *Math Guide*, “Pasta curves and their encodings”. The nonce is k ; its 80-byte random input must not be replaced by an undocumented deterministic or shorter construction.

Verification requires a canonically decoded public key and exactly 64 signature bytes. Decode the first 32 as a Pallas point R ; reject failure or a noncanonical encoding. Interpret the last 32 bytes little-endian as an integer z ; reject unless $z < r$. Compute $c = h(R^* || Y^* || M)$ using exactly the signature’s encoded first component, then accept exactly when $[z]G = R + [c]Y$. The group cofactor is one, so there is no additional cofactor multiplication. The primitive does not reject identity R merely because it is the identity; applications may impose additional conditions on their own keys.

The algorithms implement the protocol specification’s “RedDSA, RedJubjub, and RedPallas” definition, specialised to Pallas, and the `reddsa` crate’s signing and verification-key operations. Their completeness is the Schnorr scalar identity in §4, independent of whether a security proof models h ideally.

5.2 Unforgeability and the simulation it needs

For the following single-key theorem, key generation samples the signing scalar uniformly from the nonzero scalars. This is a hypothesis of the unforgeability experiment, not a rule rejecting identity keys throughout the general signature interface.

In existential unforgeability under chosen-message attack, an adversary receives an honestly generated public key and access to signatures under it. A forgery is an accepted signature on a message not submitted to that oracle. Strong unforgeability also counts a different accepted signature on a previously submitted message, unless that exact signature was returned before.

In the ideal scalar-random-oracle model, a signing simulator samples independent uniform c, z , sets $R = [z]G - [c]Y$, and programs the hash at $R^* || Y^* || M$ to equal c . Before a conflicting pre-existing query occurs, this is coupled to a real Schnorr signature with a uniform nonce: the transformation $k = z - cx$ is bijective. The uniform point R hits at most q relevant old table entries with probability at most q/r . This establishes an identical-until-bad coupling, not a claim that conditioning on no conflict leaves every original marginal distribution unchanged.

For an unqueried challenge input and a nonidentity Y , a fixed purported signature satisfies the verification equation for at most one scalar challenge, so blind success has probability at most $1/r$. Otherwise a successful new signature identifies the decisive query required by Lemma 4.2. Two forks with the same encoded R, Y, M and different challenges recover x . If that input instead came from a previously simulated signature, the verification equation uniquely determines z ; canonical encodings then make the signature identical to the old one. Thus the same argument covers the strong-forgery case.

This proves a reduction with the forking loss and explicit signing-simulation error. To apply it to RedPallas’s exact nonce and hash-to-scalar mechanism, make two further hybrid changes. A uniform 512-bit hash output reduced modulo r differs from a uniform scalar by at most $r/(4 \cdot 2^{512})$, from the *Math Guide*, “Probability and computation”. Sum that distance over the bounded number of replaced oracle answers. The 80-byte hidden nonce prefixes are fresh except on prefix collisions or adversarial guesses; a union bound over the relevant signing and hash-query pairs gives their explicit 2^{-640} -scaled bad-event bound. The signing simulation additionally has its $1/r$ -scaled programming-conflict bound. Concretely, for at most q_s signing requests and q_h adversarial hash queries, one conservative single-execution bound is

$$\delta \leq (q_h + 2q_s + 1) \frac{r}{4 \cdot 2^{512}} + \frac{q_s(q_h + 2q_s)}{2^{640}} + \frac{q_s(q_h + q_s)}{r}.$$

The terms count scalar-distribution changes, hidden prefix coincidences, and challenge-programming conflicts, respectively. If real forgery success is ϵ , the simulated queried-forgery probability is at least $\epsilon_* = \max(0, \epsilon - \delta - 1/r)$. With $Q = \max(1, q_h)$, the resulting logarithm solver has success at least $\max(0, \epsilon_*(\epsilon_*/Q - 1/r))$, under the forking and ideal-oracle hypotheses just stated. These are distinct

losses. The theorem remains a random-oracle reduction, not an unconditional proof about the fixed BLAKE2b function.

5.3 Randomised signing keys

For a scalar α , define

$$x' = x + \alpha, \quad Y' = Y + [\alpha]G.$$

The public and private transformations agree because $[x']G = [x]G + [\alpha]G$. Uniform α independent of a fixed key makes Y' uniform, proving an exact statement about the randomised public point alone. If α is revealed, the original point is recovered by subtraction; message context or other correlated data may also permit linking. Point randomisation alone is not a complete privacy theorem.

Ordinary single-key unforgeability cannot simply be renamed randomised-key security. The latter experiment allows signatures under keys $Y + [\alpha]G$, with specified adversarial choice of α , and asks for a fresh accepted signature under such a key. For the Schnorr construction above, simulate each requested effective key using its actual public point in the programmed challenge. A fork at the same decisive query fixes Y' , hence fixes α by scalar injectivity. Extraction returns

$$x = \frac{z - z'}{c - c'} - \alpha.$$

If the adversary produces an α with $Y + [\alpha]G = \mathcal{O}$, then $x = -\alpha$ already solves the original logarithm. This handles the identity effective-key case without incorrectly applying a unique-challenge argument to it. The same simulation and freshness errors still have to be included. The key prefix in the challenge is essential to fixing the effective key across forks.

The spend-authorisation base is

$$G = \text{GroupHash}_{\text{Pallas}}(z.\text{cash}:\text{Orchard}, G).$$

The binding-signature base is instead the randomness base of the relevant value commitment. These are separate generator instantiations in the protocol specification’s “Spend Authorisation Signature” and “Binding Signature” sections. Using the same curve and signature equation does not make their application-level authorisation roles interchangeable.

5.4 What a binding signature contributes to balance

To expose the logical boundary, suppose commitments $C_i = [v_i]V + [\rho_i]R$ use nonidentity bases V, R , and a public signed balance is b . The point

$$B = \sum_i C_i - [b]V = \left[\sum_i v_i - b \right] V + \left[\sum_i \rho_i \right] R$$

is a public candidate verifying key under base R . When $\sum_i v_i = b$ as integers, the sum of randomness scalars is a signing key for it. Conversely, suppose a suitable composed knowledge theorem extracts both all commitment openings and a signing witness k with $B = [k]R$. Subtraction yields

$$\left[\sum_i v_i - b \right] V + \left[\sum_i \rho_i - k \right] R = \mathcal{O}.$$

If the first coefficient is nonzero modulo the group order, this is a nontrivial generator relation and breaks the binding assumption. If an independently proved range bound excludes wraparound, a nonzero integer imbalance has a nonzero field coefficient. These two steps explain why the composition can enforce integer conservation.

The hypothesis concerning joint extraction is not discharged by commitment binding, by existence of some logarithm for B , or by ordinary unforgeability for an independently generated key. An

adversary may have chosen the commitments and therefore influenced B . The application’s actual relation, transcript and key-generation conditions require a matching knowledge and composition theorem. The algebra above is the reduction once its witnesses are available; it does not claim that every accepted signature in every surrounding protocol yields them.

6 Key agreement and note encryption

An encryption algorithm takes a key and a plaintext byte string and produces a ciphertext. Decryption returns the original plaintext or the failure symbol $\perp$. Correctness concerns ciphertexts made honestly; confidentiality concerns what the ciphertext reveals; integrity concerns which altered ciphertexts are accepted. These are different requirements.

6.1 Diffie–Hellman and diversified bases

Fix a nonidentity point B in the prime-order Pallas group. For nonzero scalars x, e , a receiver publishes $P = [x]B$, and a sender publishes an ephemeral point $E = [e]B$. Both compute

$$S = [e]P = [ex]B = [x]E.$$

“Ephemeral” means intended for this particular encryption, not a second kind of group element. The equality proves agreement, not secrecy: computing S from B, P, E is the CDH problem defined in “Security contracts and reductions”. Nor does agreement alone authenticate a sender: anyone can choose e and encrypt to P .

The Orchard interface obtains B from $\text{DiversifyHash}(d)$ and uses the same nonzero incoming-viewing scalar across diversified bases. Public keys and ephemeral keys must decode canonically to nonidentity points. No cofactor multiplication is needed because the group already has prime order. Multiplication by a nonzero scalar is a bijection of its nonidentity points.

A different address base does not create a different secret scalar. Whether pairs $(B_1, [x]B_1)$ and $(B_2, [x]B_2)$ expose their common scalar is a decisional question, distinct from recovering x . Its security requires a DDH-style assumption for the actual distribution and information supplied about these bases. For example, revealing t with $B_2 = [t]B_1$ makes linkage easy: test whether the second public key equals $[t]$ times the first. The hash-to-group construction does not reveal such logarithms. This explains the role of its security model without claiming that arbitrary collections of publicly related bases are unlinkable.

Orchard derives the 32-byte encryption key by

$$K = \text{BLAKE2b}_{256}(\text{Zcash_OrchardKDF}, S^* \parallel E^*),$$

where stars denote the canonical 32-byte point encodings from the *Math Guide*, “Pasta curves and their encodings”. The ephemeral key is part of the input even though knowledge of S is the source of secrecy. This binds the derived key to the advertised ephemeral point. The concrete specification is the protocol’s Orchard key-agreement and key-derivation sections.

6.2 Authenticated encryption and nonce discipline

Authenticated encryption with associated data, abbreviated AEAD, takes a secret key K , a nonce N , associated data A and plaintext M . A nonce is a public input whose required uniqueness is specified by the scheme; it need not be secret. Associated data is authenticated but not encrypted. Encryption returns ciphertext with an authentication tag. Decryption receives the same K, N, A and either returns M or rejects.

For confidentiality, a nonce-respecting adversary supplies pairs of equal-length plaintexts and receives encryptions of one fixed hidden choice. It never repeats a nonce under the same key. Its distinguishing advantage is the difference between its output probabilities for the two choices. For

ciphertext integrity, an adversary with encryption access wins by producing an accepted (N, A, C) not previously returned for that same tuple of public inputs. Variants that additionally expose decryption need to forbid trivial replay of a challenge; we do not silently import such a stronger experiment into a proof about encryption access.

Orchard uses IETF ChaCha20–Poly1305 with empty associated data and the all-zero 96-bit nonce. Consequently nonce safety rests on the derived key not being reused for different plaintexts. “The nonce is fixed by the protocol” is not a proof of this condition. Repeating the effective DH/KDF inputs repeats the stream and authentication key. The note-generation rules and their randomness or derivation conditions must establish the necessary freshness in their actual application.

6.3 The ChaCha20 stream, word by word

All words here are unsigned 32-bit integers. Addition is modulo 2^{32} ; $\oplus$ is bitwise XOR, and rotl_j rotates a 32-bit word left by j places, moving high bits back into low positions. These are operations on fixed-width representations, not multiplication in the Pasta field. The quarter-round transforms four words in this order, always using the latest values:

$$\begin{aligned} a &\leftarrow a + b, & d &\leftarrow \text{rotl}_{16}(d \oplus a), \\ c &\leftarrow c + d, & b &\leftarrow \text{rotl}_{12}(b \oplus c), \\ a &\leftarrow a + b, & d &\leftarrow \text{rotl}_8(d \oplus a), \\ c &\leftarrow c + d, & b &\leftarrow \text{rotl}_7(b \oplus c). \end{aligned}$$

The 16-word initial state is, in order, the four little-endian words of ASCII expand 32-byte k , eight little-endian words of the 32-byte key, one counter word, and three little-endian words of the 12-byte nonce.

Copy this state to a working state. A double round applies quarter-rounds to index quadruples

$$\begin{aligned} (0, 4, 8, 12), & \quad (1, 5, 9, 13), & (2, 6, 10, 14), & \quad (3, 7, 11, 15), \\ (0, 5, 10, 15), & \quad (1, 6, 11, 12), & (2, 7, 8, 13), & \quad (3, 4, 9, 14). \end{aligned}$$

Perform ten double rounds. Add the initial state word by word, then encode the 16 resulting words little-endian to obtain a 64-byte block. Counter zero supplies the first 32 bytes of the one-time Poly1305 key. Counters 1, 2, ... supply the stream XORed with plaintext, truncating the final block. The counter must not wrap: at most $(2^{32} - 1)64$ plaintext bytes use this construction under one nonce.

Applying the same stream XOR again recovers the plaintext, because $(m \oplus s) \oplus s = m$. But if two messages reuse the stream, $C_1 \oplus C_2 = M_1 \oplus M_2$, which reveals a relation without recovering the key. Correct invertibility therefore gives no protection against nonce misuse.

6.4 Poly1305 and the authenticated byte string

A message authentication code, abbreviated MAC, is a secret-key tag used to reject unauthorised changes to a message.

Split the one-time 32-byte key into two 16-byte strings. Let s be the little-endian integer of the second. Obtain r from the first by bitwise AND with the integer mask

$$0x0fffffff c0fffffff c0fffffff c0fffffff.$$

This clearing of designated bits is called clamping. It is not reduction modulo the group order, and its output is not uniform over all residues modulo $2^{130} - 5$.

For an input byte string, split it into chunks of at most 16 bytes. For a chunk D of length ℓ , set $n = \text{val}_{\text{LE}}(D) + 2^{8\ell}$. Starting from $a = 0$, update for each chunk

$$a \leftarrow (a + n)r \pmod{2^{130} - 5}.$$

The tag is the 16-byte little-endian encoding of $(a + s) \bmod 2^{128}$. Appending the $2^{8\ell}$ bit distinguishes a chunk from a shorter chunk extended by zero bytes. Expanding the recurrence shows a polynomial in r , which is the algebra behind Poly1305's authentication bounds. Those bounds must account for clamping and the final reduction; the unrestricted uniform-field root bound alone is not an exact forgery probability for this concrete MAC.

Write $\text{pad}_{16}(X)$ for the shortest string of zero bytes that makes $|X| + |\text{pad}_{16}(X)|$ a multiple of 16. The authenticated string is exactly

$$A \parallel \text{pad}_{16}(A) \parallel C \parallel \text{pad}_{16}(C) \parallel \text{LE}_8^{\text{byte}}(|A|) \parallel \text{LE}_8^{\text{byte}}(|C|),$$

where lengths count bytes and must fit in 64 bits. The superscript specifies eight-byte encodings, as in the *Math Guide*, “Language, integers, and encodings”. The ciphertext C is the plaintext XORed with the counter-one stream. Encryption appends the resulting 16-byte tag to C .

Decryption recomputes and checks the tag before accepting any plaintext. A constant-time comparison processes the entire fixed-length tag without timing differences that depend on the first mismatching position; otherwise the implementation can expose information beyond the experiment. Empty associated data does not remove the ciphertext padding or the two length fields. All these details follow RFC 8439. We take the concrete nonce-respecting confidentiality and integrity bounds for ChaCha20–Poly1305 as cryptographic assumptions or imported analyses with their stated message limits; reproducing the permutation does not prove them.

6.5 What a hashed-DH confidentiality reduction proves

Consider a single-recipient, single-challenge experiment with independent uniform nonzero x, e , public $B, [x]B, E = [e]B$, and a random oracle H returning the encryption key on input $S^* \parallel E^*$. The adversary chooses two equal-length plaintexts; it has oracle access but no secret-key or challenge-decryption access. Replace the challenge key by an independent uniform key. The two executions can be coupled until the adversary queries $S^* \parallel E^*$.

Let the probability of that event in the independent-key experiment be α , and bound the number of oracle queries by $q_H \geq 1$. A CDH algorithm can run this experiment without knowing S , choosing its encryption key independently. It records the query candidates and outputs the first point component of a uniformly selected query slot, returning failure on a malformed or unused slot. If the critical query occurred, this outputs S with probability at least $1/q_H$. Thus $\alpha \leq q_H \text{Adv}_{\text{CDH}}$ for this constructed algorithm. Nonzero challenge scalars are part of this version of the CDH experiment.

For each of the two plaintext choices, changing the key changes an output probability by at most α for its corresponding reduction. The independent-key distinguishing advantage is exactly the confidentiality advantage of an AEAD adversary using that key. With a common upper bound on the two CDH algorithms, we obtain

$$\text{Adv}_{\text{hybrid}} \leq 2q_H \text{Adv}_{\text{CDH}} + \text{Adv}_{\text{AEAD}}.$$

The factor two belongs to switching both distinguishing experiments; confusing distinguishing advantage with excess guessing probability loses it. This theorem assumes a random-oracle KDF and fresh key conditions. It does not by itself prove recipient anonymity, chosen-ciphertext security of every surrounding note format, or security after revealing the viewing key. Authenticating ciphertext bytes likewise does not show that the decrypted fields satisfy a ledger relation.

7 Authenticated trees and append-only histories

A hash tree compresses a structured collection to a short root. An authentication path lets a verifier recompute that root from one item and a small set of siblings. The root authenticates the item only if the verifier already trusts the root or obtains it through a separate authenticated channel. Sending an item, path and arbitrary root together establishes no external truth.

7.1 Binary trees with unambiguous node encodings

A binary tree has either a leaf or an internal node with an ordered left and right child. Its height is the maximum number of edges from its root to a leaf. A perfect tree of height h has 2^h leaves at the same depth. Encode a leaf containing bytes m by a length-delimited tuple, and hash it under a leaf tag. Encode an internal node by a different tag, its level where required, and the two fixed-length child digests in left-to-right order:

$$L(m) = H(\text{enc}(\text{leaf}, m)), \quad N_j(a, b) = H(\text{enc}(\text{node}, j, a, b)).$$

Here enc is an injective tuple encoding as constructed in the *Math Guide*, “Language, integers, and encodings”. These are generic illustrative tags, not the literal Zcash domain strings. A concrete protocol may instead use distinct personalised functions, fixed input widths, or a different level convention; the proof requires the same lack of ambiguity.

For a leaf position $i \in \{0, \dots, 2^h - 1\}$, write $i = \sum_{j=0}^{h-1} b_j 2^j$. An authentication path contains sibling digests $s_0, \dots, s_{h-1}$, ordered from leaf to root. Set $v_0 = L(m)$ and compute

$$v_{j+1} = \begin{cases} N_j(v_j, s_j) & b_j = 0, \\ N_j(s_j, v_j) & b_j = 1. \end{cases}$$

Accept if v_h equals the trusted root and all encodings, path lengths and positions are valid. There are h siblings and h internal hashes. A path proves occurrence at a specified position; it does not prove uniqueness of the underlying value.

Theorem 7.1 (Tree binding reduces to node collisions). *Fix a tree shape, node encodings and leaf positions. Two different leaf collections with the same root yield a collision in one of the tagged hash inputs. Two accepted paths for different leaf values at the same position and root do likewise.*

Proof. For full collections, compare the root inputs. If they differ while the roots agree, this is a collision. Otherwise both child digests agree; descend into a child containing a differing leaf. The finite descent either finds differing internal inputs with equal outputs or reaches differing leaf encodings with equal leaf hashes. Injective encodings ensure the last inputs really differ. For two paths, begin with the different leaf encodings. If their digests already agree there is a collision. Otherwise ascend to the first level where unequal computed values become equal. Its ordered inputs differ at least in the position occupied by the computed value, so it supplies a node collision. Direction bits must describe the same position for this argument. $\square$

A level-specific compression function may be partial, returning $\perp$ on some inputs. If a protocol maps failure to a field value, binding of the resulting total function requires analysing that failure branch too: the generic theorem does not turn two failures into a collision of a compression function whose security game excludes failing inputs. Similarly, a zero leaf is a designated value, not an absence proof unless occupancy rules make it one.

7.2 Fixed-depth append frontiers

An append operation adds the next leaf without changing earlier leaves. For capacity 2^h , define empty subtree digests recursively by $e_0 = L(\text{empty})$ and $e_{j+1} = N_j(e_j, e_j)$, with the concrete empty leaf fixed by the protocol. A frontier stores one completed subtree at each height where the binary expansion of the current leaf count n has a one. The stored subtrees partition the occupied prefix in left-to-right order of decreasing heights.

To append a new leaf, set a carry to its digest at height zero. While a frontier entry exists at the carry’s height, hash that old entry on the left and the carry on the right, erase the entry, and raise the carry’s height by one. Store the carry at the first empty height, then increment n . This is exactly

binary addition with hash operations in place of numerical carries. The invariant about the occupied prefix follows by induction: two adjacent completed subtrees of equal size are replaced by their parent and nothing earlier changes.

When $n < 2^h$, the full padded root can be recovered with $v = e_0$. For $j = 0, \dots, h - 1$, set

$$v \leftarrow \begin{cases} N_j(f_j, v) & \text{bit } j \text{ of } n \text{ is one,} \\ N_j(v, e_j) & \text{bit } j \text{ of } n \text{ is zero.} \end{cases}$$

Here f_j is the occupied frontier entry at height j . At each step, v is the partially occupied or empty right-hand region needed at that height. For $n = 2^h$, the frontier entry at height h is already the full root; appending again is invalid. The count is needed: a frontier is not simply an unordered collection of digests.

An individual append may merge h times, but over n appends fewer than n merges occur. At height $j \geq 1$, there are at most $\lfloor n/2^j \rfloor$ newly formed parents; their sum is less than n . This amortised bound means a bound on total work divided by operation count, not a worst-case bound for every operation. Recovering a padded root separately still costs h hashes in this simple construction.

7.3 Peaks, history commitments and structural binding

An unbounded append history can retain the same completed subtrees without padding to a fixed capacity. Its current forest is an ordered list of peaks, one for each nonzero bit in the leaf count; a peak is the root of one completed subtree. This construction is commonly called a Merkle mountain range. The binary carry rule determines its shape exactly from the leaf count. For example, thirteen leaves partition as $8 + 4 + 1$, so the peak heights are 3, 2, 0 in that order.

To obtain a single commitment, specify a function that combines the ordered peaks and authenticates the information needed to recover their shape. One simple generic choice is a separate tagged hash of the count and a length-delimited list of (height, digest) pairs. Other protocols combine peaks recursively. These choices are not interchangeable on the wire. An inclusion proof consists of a path inside one peak and the data needed to authenticate that peak in the committed ordered forest.

A history node can also authenticate metadata, such as a covered interval or aggregate work. For example, an encoding may include the two child digests and their claimed aggregate values. Hashing this tuple binds the claim as bytes; it does not prove that the aggregate equals the sum of valid child work. The verifier must check the combination rule and the leaf rules where its security theorem requires them. This distinction is essential when a proof opens only sampled parts of a large history.

An append-consistency proof establishes that one committed sequence is a prefix of another. For a direct construction, supply the earlier frontier and the appended leaves, authenticate the earlier root, then replay the carry rule and compare the later root. Tree binding makes two inconsistent reconstructions a hash-collision event, provided counts and shapes are bound. This direct proof can be large. A smaller proof may replace groups of new leaves by completed subtree roots, but then its exact decomposition, ordering and endpoint checks must be specified. Membership alone is not append consistency.

The generic constructions here discharge the hash-tree and append invariants. Exact ledger node encodings, history metadata, peak-combining rules and block commitment composition are separate protocol choices; their security inherits these lemmas only after those choices satisfy the hypotheses. No network activation claim follows from the name of a tree algorithm.

8 Wallet permutations and address encodings

A wallet needs more than a secure scalar generator. It must map indices to diversifiers without repetition, encode structured receiver data, and detect damaged text. This section constructs the lower-level transformations. A secret-key permutation, a public reversible mixing function, and an error-detecting checksum have different contracts even if all three change bytes.

8.1 AES-256 from the byte field

A block cipher is a keyed family of permutations of fixed-length blocks. AES-256 has a 32-byte key and 16-byte blocks. The number 256 describes the key length, not the block length. Its byte arithmetic uses exactly the field constructed in the *Math Guide*, “Byte fields for AES”: XOR addition and polynomial multiplication reduced by $X^8 + X^4 + X^3 + X + 1$. We denote multiplication in this field by a centred dot.

AES begins with a substitution of each byte. For input a , let $u = a^{-1}$ in the byte field when $a \neq 0$, and let $u = 0$ when $a = 0$. Write u_i for bit i , counting from the least significant bit. Output bit i is

$$u_i \oplus u_{(i+4) \bmod 8} \oplus u_{(i+5) \bmod 8} \oplus u_{(i+6) \bmod 8} \oplus u_{(i+7) \bmod 8} \oplus c_i,$$

where $c = 0x63$. This defines the entire substitution table, conventionally called the S-box, without importing 256 unexplained entries. The inversion map, with zero fixed, is a permutation. The following inverse for the affine bit map shows that the S-box is a permutation too: from output bits v_i , recover

$$u_i = v_{(i+2) \bmod 8} \oplus v_{(i+5) \bmod 8} \oplus v_{(i+7) \bmod 8} \oplus d_i, \quad d = 0x05.$$

Substitute the preceding formula and cancel each duplicated bit to verify this identity. For example, $0x53^{-1} = 0xca$ and its substituted value is $0xed$; zero maps to $0x63$.

Arrange an input block as a 4-by-4 byte array $s_{r,c}$ with $s_{r,c} = \text{input}[4c + r]$. The following transformations operate on that array:

- SubBytes replaces every byte by its S-box value.
- ShiftRows sends the old $s_{r,(c+r) \bmod 4}$ to position (r, c) : row r rotates left by r .
- MixColumns replaces each column vector by its product with the byte-field matrix

$$\begin{pmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{pmatrix}.$$

The entries are hexadecimal byte values, not decimal scalars acting by repeated addition in characteristic two.

- AddRoundKey XORs a 16-byte round key into the state in the same column-major order.

The inverse MixColumns matrix is

$$\begin{pmatrix} 0e & 0b & 0d & 09 \\ 09 & 0e & 0b & 0d \\ 0d & 09 & 0e & 0b \\ 0b & 0d & 09 & 0e \end{pmatrix}.$$

Multiplying the two matrices in the byte field gives the identity, a direct verification rather than an assumption. ShiftRows is reversed by rotating right, and AddRoundKey is its own inverse. Every round is therefore invertible.

To derive round keys, divide the 32-byte key in order into eight four-byte words $w_0, \dots, w_7$. A word here is an ordered four-byte tuple; no host integer endianness is implicit. Define $\text{RotWord}(a, b, c, d) = (b, c, d, a)$ and apply the S-box to each byte for SubWord. Generate w_i for $i = 8, \dots, 59$ as follows. Begin with $t = w_{i-1}$. If $i \bmod 8 = 0$, replace

$$t \leftarrow \text{SubWord}(\text{RotWord}(t)) \oplus (\rho_{i/8}, 0, 0, 0),$$

where $\rho_1 = 01$ and $\rho_{j+1} = 02 \cdot \rho_j$ in the byte field. If $i \bmod 8 = 4$, replace $t \leftarrow \text{SubWord}(t)$ instead. In all cases set $w_i = w_{i-8} \oplus t$. Round key j , for $0 \leq j \leq 14$, is the concatenation $w_{4j} \parallel \dots \parallel w_{4j+3}$.

Encryption first applies `AddRoundKey` with key zero. Rounds 1 through 13 apply `SubBytes`, `ShiftRows`, `MixColumns`, and `AddRoundKey` with that round's key, in that order. Round 14 omits `MixColumns`. Serialise the resulting array in its original column-major order. Decryption is well-defined by reversing this complete sequence and applying each inverse transformation. FF1 below only calls AES encryption, even when decrypting its own format-preserving permutation.

These are the AES-256 algorithms of FIPS 197. Their invertibility is proved by the component inverses. Their resemblance to a random permutation under an unknown uniformly sampled key is the AES PRP assumption, not a consequence of invertibility or of having 14 rounds.

8.2 The exact 88-bit FF1 permutation

A format-preserving permutation maps a prescribed finite set bijectively to itself. The diversifier permutation uses AES-256-based FF1 on bit strings of length 88, with radix two and an empty tweak. Radix means the number of possible numeral values; here each numeral is a bit. A tweak is an additional public parameter selecting a permutation under the same secret key; this protocol fixes it to the empty byte string. We specialise the algorithm to these parameters, so no unused decimal or variable-length cases are needed.

For a bit string $B = (b_0, \dots, b_{43})$, define

$$\text{NUM}_2(B) = \sum_{j=0}^{43} b_j 2^{43-j}.$$

Its inverse $\text{STR}_{2,44}$ writes a number in $[0, 2^{44})$ as exactly 44 bits, most significant first. This numeral interpretation is distinct from the wallet's little-endian encoding of an 88-bit index. If the index is j , the FF1 input sequence is its 88 little-endian bits; the first bit of each 44-bit half then becomes the most significant numeral of that half inside FF1. There is no contradiction: these are two explicitly different maps applied consecutively.

Let the secret key be K , and split the 88-bit input into consecutive halves A, B of 44 bits. Construct the fixed 16-byte block

$$P = [1, 2, 1, 0, 0, 2, 10, 44] \parallel \text{BE}_4^{\text{byte}}(88) \parallel \text{BE}_4^{\text{byte}}(0).$$

For $i = 0, \dots, 9$, form

$$Q_i = 0^9 \parallel [i] \parallel \text{BE}_6^{\text{byte}}(\text{NUM}_2(B)).$$

Here 0^9 means nine zero bytes, and $[i]$ one byte; Q_i has exactly 16 bytes. The two-block chaining computation is

$$R_i = \text{AES}_K(\text{AES}_K(P) \oplus Q_i).$$

Interpret the first 12 bytes of R_i as a big-endian integer y_i , and update

$$C = \text{STR}_{2,44}((\text{NUM}_2(A) + y_i) \bmod 2^{44}), \quad (A, B) \leftarrow (B, C).$$

After ten rounds, output $A \parallel B$. To represent this output as 11 bytes, put sequence bit $8j + k$ in bit k of byte j , matching the initial little-endian bit-sequence convention.

For connection with the general FF1 specification, the half-lengths are $u = v = 44$, the byte length of a half-numeral is $b = \lceil 44/8 \rceil = 6$, and the output expansion length is $d = 4\lceil b/4 \rceil + 4 = 12$. Since $d < 16$, the expansion uses only the first block's prefix; there are no omitted extra AES blocks in this case. The fixed prefix encodes these parameters and the zero tweak length. This is the specialisation of NIST SP 800-38G used by the protocol's diversifier PRP.

Proposition 8.1 (FF1 is a permutation). *For every fixed key, the ten-round function above is a bijection of the 2^{88} input bit strings.*

Proof. Write the round's derived integer as $F_i(B)$. From output (A', B') , the prior right half is $B = A'$. Recompute $F_i(A')$ and recover the prior left integer as $(\text{NUM}_2(B') - F_i(A')) \bmod 2^{44}$. Thus every round has a unique inverse. Apply these inverses for $i = 9, 8, \dots, 0$ to invert the whole construction. This argument does not assume that the round function itself is invertible. $\square$

Distinct indices therefore produce distinct 88-bit diversifier strings under one key. Their distribution as a secret-key pseudorandom permutation is a separate FF1 security contract, using its analysis and AES assumption for this domain and query count. A permutation over 2^{88} elements cannot provide an unlimited 256-bit domain-size security claim merely because its AES key has 256 bits. Nor does permutation injectivity assert that a subsequent hash-to-group map is injective.

8.3 F4Jumble: a public reversible mixing function

F4Jumble accepts a byte string of length $48 \leq n \leq 4\,194\,368$. Split it into $L||R$ with $\ell = \min(64, \lfloor n/2 \rfloor)$ bytes on the left and $r = n - \ell$ on the right. For $i \in \{0, 1\}$, define

$$H_i(R) = \text{BLAKE2b}_{8\ell}(\text{UA_F4Jumble_H}||[i, 0, 0], R),$$

$$G_i(L) = \left(\bigparallel_{j=0}^{\lceil r/64 \rceil - 1} \text{BLAKE2b}_{512}(\text{UA_F4Jumble_G}||[i]||\text{LE}_2^{\text{byte}}(j), L) \right) [0 : r].$$

The strings before the appended bytes have length 13, so each personalisation is exactly 16 bytes. The maximum right length is $2^{16}64$, so its last block index fits in two bytes. The notation $[0 : r]$ means the first r bytes, not a field reduction. Perform four updates in order:

$$R \leftarrow R \oplus G_0(L), \quad L \leftarrow L \oplus H_0(R), \quad R \leftarrow R \oplus G_1(L), \quad L \leftarrow L \oplus H_1(R).$$

Concatenate the final L, R . To invert, perform the same updates in reverse order, beginning with $L \leftarrow L \oplus H_1(R)$. Each update changes one half by XOR with a function of the unchanged half, and so is its own inverse at that stage. This proves bijectivity at every valid length.

There is no secret key. Anyone can invert the function or deliberately construct a chosen output. It therefore supplies neither encryption nor message authentication. Its role is to spread changes across an encoded structured payload before text encoding and checksum computation. Claims about resisting specially structured mutations need the corresponding analysis, not just the four-round invertibility proof. The construction and address application are specified in ZIP 316.

8.4 Bech32m: symbols, checksum and canonical decoding

An error-detecting checksum is redundant data computed publicly from a message to help detect accidental damage. Because anyone can recompute it, a valid checksum is not evidence that an adversary did not replace the message. Bech32m represents values 0 through 31 using this ordered alphabet:

qpzry9x8gf2tvdw0s3jn54khce6mua7l.

To convert a byte payload to these five-bit symbols, read each byte most significant bit first, concatenate the bits, split into groups of five, and pad the final group with zero bits on the right. There are $\lceil 8n/5 \rceil$ symbols for n bytes. Inverting this operation must reject five or more leftover bits and must reject nonzero leftover bits. Otherwise multiple symbol strings could encode the same bytes. This bit convention differs from the little-endian diversifier-bit packing above.

The human-readable part, abbreviated HRP, is a nonempty string of at most 83 printable ASCII characters with codes 33 through 126. It identifies the application or network namespace. For checksum calculation use lowercase and expand it into the integer sequence

$$(\lfloor c_0/32 \rfloor, \dots, \lfloor c_{h-1}/32 \rfloor, 0, c_0 \bmod 32, \dots, c_{h-1} \bmod 32),$$

where c_i are its ASCII codes. Define five 30-bit constants, written in hexadecimal, by

$$\begin{aligned} g_0 &= 3b6a57b2, & g_1 &= 26508e6d, & g_2 &= 1ea119fa, \\ g_3 &= 3d4233dd, & g_4 &= 2a1462b3. \end{aligned}$$

Starting with $z = 1$, process each input symbol $v \in [0, 31]$ by first setting $b = \lfloor z/2^{25} \rfloor$ and then

$$z \leftarrow ((z \bmod 2^{25})2^5) \oplus v \oplus \bigoplus_{i: \text{bit } i \text{ of } b=1} g_i.$$

Call the final value *polymod* of the symbol sequence. This recurrence is a complete algorithm; using the name does not require any unstated coding theory.

For expanded HRP E and payload symbols D , compute

$$z = \text{polymod}(E\|D\|[0, 0, 0, 0, 0, 0]) \oplus 0x2bc830a3.$$

Append the six five-bit digits of z , most significant first, to D . The encoded text is HRP, the character 1, and the alphabet characters for these data and checksum symbols. The verification condition is

$$\text{polymod}(E\|D\|\text{checksum}) = 0x2bc830a3.$$

For example, `a11qfn3a` is the valid empty-payload Bech32m string with HRP `a`. The older Bech32 checksum uses constant one instead; accepting that constant would accept a different format, not a second spelling of Bech32m.

A decoder rejects mixed letter case, invalid ASCII or alphabet characters, an empty HRP, missing separator or fewer than six checksum symbols. The separator is the last 1, since this character can occur in the HRP but not in the data alphabet. Uniform uppercase can be normalised to lowercase for checksum calculation when the application permits it; canonical encoders choose a single case. After checksum verification, remove the six checksum symbols, invert the bit conversion with its padding checks, then enforce the application's exact HRP, payload length and structural decoding rules.

The original generic Bech32 format limits the whole text to 90 characters. Unified address encodings explicitly remove that limit, so a generic library's 90-character rejection cannot be imported unchanged. Conversely, length-specific error-detection guarantees cannot automatically be extended to every longer unified address. The checksum and generic format are defined by BIP 350 and BIP 173; the Zcash length and payload rules are in ZIP 316. The recurrence is fully specified here; the exhaustive classification of detected error patterns is an imported coding result, not a cryptographic authenticity theorem.

8.5 The composition boundary

The roles now separate cleanly. AES supplies a keyed block permutation. FF1 uses it to permute the index domain without repetitions. Hash-to-group turns the resulting diversifier into a public curve base. F4Jumble reversibly mixes a structured byte payload. Bech32m converts those bytes into checksummed text. Each transformation has explicit input sizes, bit order and failure conditions.

No individual primitive proves the validity of a receiver tuple or secret-key derivation. That requires the application's tuple encoding, domain tags, key relationships and validation rules. In particular, neither a valid Bech32m checksum nor successful F4Jumble inversion establishes ownership of the advertised receiver. Their job is faithful representation, not authority.

Part III

Halo 2 Guide: From a circuit to a checked proof

Abstract

This volume of *The Zcash Arboretum* constructs the Halo 2 proof system used to prove shielded transaction relations. It assumes the *Math Guide*'s fields, polynomials, linear algebra and Pasta curves, and the *Crypto Guide*'s commitments, security contracts and interactive proofs. We construct a way to convince a verifier that private data satisfies public rules without disclosing that data. The verifier's complete calculation makes explicit what it checks, what the application must supply, and which security conclusions require more than the algebraic identities.

1 From a relation to a circuit

1.1 The statement being proved

The mathematical prerequisites are the *Math Guide*, “Prime fields and modular arithmetic”, “Linear algebra and uniform masks”, “Polynomials and evaluation”, “Evaluation domains and fast transforms”, and “Pasta curves and their encodings”. We use their fields, vectors, polynomials and curve groups with the same definitions.

A proof establishes a precisely selected relation, not an informal claim that a transaction is good. Let u be a public input, w a private witness, and $\mathcal{R}(u, w)$ a deterministic predicate. The prover receives both inputs; the verifier receives u and a proof. Completeness asks that an honestly constructed proof be accepted when $\mathcal{R}(u, w) = 1$. Knowledge soundness asks for a witness-extraction guarantee against an efficient prover that succeeds sufficiently often. Zero knowledge asks that the verifier’s view be simulatable from the public input alone. These distinct contracts are defined in the *Crypto Guide*, “Security contracts and reductions” and “Knowledge, simulation, and Schnorr”.

The application chooses $\mathcal{R}$. Halo 2 represents that relation by an *arithmetic circuit*: field-valued cells connected by polynomial equations which must hold simultaneously. Its soundness cannot repair an omitted bound on a value, a wrong hash input, or a public value left unconnected to the witness. There are therefore two obligations:

$$\begin{aligned} \text{circuit satisfaction} &\implies \mathcal{R}(u, w) = 1, \\ \text{accepted proof} &\implies \text{the proof system’s knowledge guarantee.} \end{aligned}$$

The first is about the circuit. The second is about the argument system. Neither implication follows merely from the other.

We use the Zcash Halo 2 implementation, specifically `halo2_proofs` at commit

261faaccd5a30c19bb8468600841a0dac305adf5.

The pinned source is the authority for the folding and transcript conventions below. Other systems named Halo 2 can use different polynomial commitments or transcripts. The implementation sources are linked in §8.7. This construction does not verify earlier proofs inside another proof.

1.2 The field and the table

Let $\mathbb{F}$ be the field in which the circuit computes. For the relevant Pasta instantiation it is the Pallas base field, equivalently the Vesta scalar field. The commitment group used below is Vesta, not Pallas. The *Math Guide*, “Pasta curves and their encodings”, gives both fields and their encodings.

A circuit is a fixed arrangement of columns and rows. An *advice* column holds witness-dependent values. A *fixed* column holds circuit constants. An *instance* column holds public inputs, padded with zeroes to the circuit size. A *gate* is a polynomial expression in selected cells which must equal zero. A query can select the current row or a fixed rotation, such as the next row. Rows are indexed modulo the row count; a next-row query at the last row wraps to row zero.

A *verification key* is public data fixing this circuit’s columns, equations, constants and wiring. The verifier must use the intended key, not a different circuit selected by the prover. A fixed selector is a column which turns a gate on at its intended rows. For example,

$$q_{\text{mul}}(ab - c) = 0$$

imposes multiplication where the selector is one and imposes nothing where it is zero. In the implementation, selectors may be compressed into fixed columns and replaced by polynomial expressions. The verification key fixes the resulting expressions; they are not choices the prover can change after seeing a challenge.

Assignments are not constraints. Computing a value in the witness generator only tells the honest prover what to put in a cell. A malicious prover need not run that generator. Two cells assigned the

same host-language variable must still be joined by an equality constraint if their equality matters. Similarly, a witness calculated from a public input must be constrained to that input.

1.3 A complete small computation

Consider the relation

$$\mathcal{R}(t, s) = 1 \iff s^3 + s + 5 = t \text{ in } \mathbb{F}_{97}.$$

The public input is $t = 35$; one witness is $s = 3$. We use four rows and the gate

$$q_M ab + q_L a + q_R b + q_O c + q_C + \pi = 0. \quad (1)$$

The column π is public. All five q columns are fixed.

i	a	b	c	q_M	q_L	q_R	q_O	q_C	π
0	3	3	9	1	0	0	-1	0	0
1	9	3	27	1	0	0	-1	0	0
2	27	3	30	0	1	1	-1	0	0
3	30	0	0	0	1	0	0	5	-35

The negative entries denote field elements: for example $-1 = 96$. The equalities are

$$a_0 = b_0 = b_1 = b_2, \quad c_0 = a_1, \quad c_1 = a_2, \quad c_2 = a_3.$$

These make the four local equations one computation: s^2 , then s^3 , then $s^3 + s$, then comparison with $t - 5$. Without the copy equalities, the rows could describe unrelated numbers.

This four-row table isolates the arithmetic. It does not reserve blinding rows—unused rows filled with randomness to conceal witness values—or instantiate a secure curve, so it is not a production Halo 2 proof. Its purpose is to make the polynomial calculations independently reproducible. The verifier and masking rules for an actual circuit are introduced below.

1.4 Integers inside a field

The circuit sees residues, not unbounded integers. A field equality $a + b = c$ also holds when the intended integer sum differs by the field modulus. To interpret a field element as a k -bit nonnegative integer, one can constrain bits b_i by

$$b_i(b_i - 1) = 0, \quad a = \sum_{i=0}^{k-1} 2^i b_i,$$

and require 2^k not to exceed the modulus. A *lookup* checks that a value belongs to a specified table. Such table-membership checks can replace individual bit constraints when enforcing a range of integers. Bounds on intermediate sums must also rule out wraparound. The *Math Guide*, “Language, integers, and encodings” and “Prime fields and modular arithmetic”, proves the required integer-to-field statements. This obligation is essential whenever a circuit represents money or encoded byte strings.

1.5 The commitment interface

The *Crypto Guide*, “Groups, commitments, and Sinsemilla”, constructs commitments to vectors of scalars. Committing to a polynomial of degree less than n means applying that construction to its coefficient vector, padded to length n :

$$C = \sum_{i=0}^{n-1} [a_i]G_i + [r]W \quad \text{for } p(X) = \sum_{i=0}^{n-1} a_i X^i.$$

The public points G_i, W belong to a group with scalar field $\mathbb{F}$. The scalar r is the commitment blind. The lower volume's hiding and binding contracts apply to these vectors. The concrete parameter derivation and the algorithm for proving an evaluation are constructed in §5; until then, the arithmetic reductions describe the identities that such evaluations must authenticate.

The *Crypto Guide*, “Knowledge, simulation, and Schnorr”, also introduces the Fiat–Shamir transformation: deterministic hash-derived challenges replace an interactive verifier’s random challenges. We first explain the arithmetic with random challenges. A *transcript* is the ordered record of public inputs, messages and challenges. Its concrete byte construction is given in §7; its security is not assumed to follow from a single random-point calculation.

2 Columns as polynomials

2.1 A row is an evaluation point

Choose $n = 2^k$ dividing $|\mathbb{F}| - 1$ and a primitive n th root of unity ω . The evaluation domain is

$$H = \{1, \omega, \dots, \omega^{n-1}\}, \quad Z_H(X) = X^n - 1.$$

For each column with entries a_i , there is exactly one polynomial $a(X)$ of degree less than n satisfying $a(\omega^i) = a_i$. The *Math Guide*, “Polynomials and evaluation”, constructs it as $\sum_i a_i L_i(X)$, where

$$L_i(X) = \frac{\omega^i X^n - 1}{n X - \omega^i}.$$

The quotient is a polynomial; the displayed rational expression is evaluated by cancellation at a domain point. A rotation by r rows is the polynomial $a(\omega^r X)$. It has the same degree as a .

Substituting the column polynomials into a gate produces a polynomial $g(X)$. Its row constraints hold exactly when g vanishes on H . Since the elements of H are distinct,

$$\forall h \in H, g(h) = 0 \iff \exists q(X) : g(X) = Z_H(X)q(X). \quad (2)$$

This is polynomial divisibility, not pointwise division on H , where the denominator would be zero.

2.2 The four-row table in coefficient form

In the toy example, choose $\omega = 22$. Its successive powers modulo 97 are 1, 22, 96, 75. Interpolation gives

$$\begin{aligned} a(X) &= 90 + 61X + 22X^2 + 24X^3, \\ b(X) &= 75 + 32X + 25X^2 + 65X^3, \\ c(X) &= 65 + 16X + 3X^2 + 22X^3. \end{aligned}$$

Interpolate the fixed and public columns in exactly the same way and substitute them into (1). The resulting gate polynomial is

$$g(X) = (X^4 - 1)(61 + 70X + 43X^2 + 47X^3 + 81X^4 + 46X^5).$$

Every coefficient and the following evaluations are checked by `research/check_foundations.py`. At the off-domain point $x = 20$,

$$Z_H(20) = 46, \quad q(20) = 67, \quad g(20) = 75, \quad 46 \cdot 67 = 75 \pmod{97}.$$

Now change only c_0 from 9 to 10. Division of the new gate polynomial by $X^4 - 1$ leaves remainder

$$r(X) = 24(1 + X + X^2 + X^3).$$

Its roots in $\mathbb{F}_{97}$ are 22, 75, 96: exactly the three unchanged rows. At $x = 20$, the altered numerator is 20, while its Euclidean quotient times $Z_H(20)$ is 64. The test rejects. The changed table also breaks a copy equality, but this calculation concerns the gate check alone.

For this particular remainder, a uniform field point is a root with probability $3/97$. That is not a security estimate for Halo 2. The field is deliberately tiny, the commitment argument has not yet been applied, and the production verifier has exceptional cases such as $x \in H$ which must be accounted for separately.

2.3 Combining identities

Suppose the circuit and its auxiliary arguments require $g_0, \dots, g_{M-1}$ to vanish on H . After their defining commitments are fixed, sample a challenge y and combine them in Horner order:

$$N_y(X) = y^{M-1}g_0(X) + y^{M-2}g_1(X) + \dots + g_{M-1}(X).$$

The order is the verification key's gate order, followed by permutation and lookup constraints, repeated for each circuit instance in a joint proof.

If one row violates any of these identities, at that row the combined expression is a nonzero polynomial in y of degree at most $M - 1$. For fixed constraints and an independent uniform challenge, accidental cancellation has probability at most $(M - 1)/|\mathbb{F}|$, by the root bound in the *Math Guide*, “Polynomials and evaluation”. This is a local interactive argument. An actual Fiat–Shamir security bound must also account for how the adversary obtains and retries challenges.

After receiving y , the prover commits to a quotient h intended to satisfy

$$N_y(X) = (X^n - 1)h(X).$$

The verifier subsequently samples x and checks the equality at x . If the difference polynomial is nonzero and has degree at most D , an independent uniform x hides that difference with probability at most $D/|\mathbb{F}|$. The timing matters: choosing h after learning x would allow a prover to fit one point without satisfying the identity.

2.4 Degree, cosets and quotient chunks

Let d be the configured constraint-degree bound. It is the maximum of the permutation and lookup requirements, all gate degrees, and any explicitly configured minimum degree. In counting expression degrees, every queried column counts as degree one; selectors and boundary polynomials count too. Thus d need not be attained by an expression. Substituting degree-less-than- n column polynomials gives

$$\deg N_y \leq d(n - 1).$$

For a satisfying witness, division by the degree- n polynomial Z_H therefore gives $\deg h < (d - 1)n$. Write

$$h(X) = \sum_{j=0}^{d-2} X^{jn} h_j(X), \quad \deg h_j < n. \quad (3)$$

Each chunk now fits the same commitment parameters as a column. There are always $d - 1$ chunk commitments, including any zero high chunks; proof shape does not reveal the actual degree.

The prover evaluates the constraints on an extended coset of size

$$N = 2^{\lceil \log_2((d-1)n) \rceil}.$$

The implementation uses $\zeta \langle \omega_{\text{ext}} \rangle$, where ζ has order three and ω_{ext} has order N . The coset is disjoint from H , since a nontrivial element of order three cannot belong to a group of power-of-two order. Pointwise division by $X^n - 1$ is therefore defined there. An inverse transform recovers the quotient coefficients. The *Math Guide*, “Evaluation domains and fast transforms”, supplies these transforms.

The extended size accommodates the *quotient*, not necessarily the numerator. For $d = 9$, it is $8n$, although the numerator can have degree close to $9n$. Evaluating the constraint expression directly on the coset and dividing is sufficient for an honest witness. For an invalid witness, the interpolated ratios need not be its Euclidean quotient; acceptance still requires the independently challenged identity above.

Once x is known, the verifier forms the commitment to

$$h_x(X) = \sum_{j=0}^{d-2} (x^n)^j h_j(X).$$

The coefficients $(x^n)^j$ are now scalars. This new polynomial has degree less than n , and $h_x(x) = h(x)$. Its expected value is $N_y(x)/(x^n - 1)$, calculated from the other claimed evaluations. The proof does not separately transmit each $h_j(x)$.

2.5 Reserved rows

Let b be the number of blinding factors, and put $\ell = n - b - 1$. Rows $0, \dots, \ell - 1$ are usable. Row ℓ holds the final value of an auxiliary running product. Rows $\ell + 1, \dots, n - 1$ are product blinding rows. Define

$$L_{\text{blind}}(X) = \sum_{i=\ell+1}^{n-1} L_i(X), \quad A_{\text{act}}(X) = 1 - L_{\ell}(X) - L_{\text{blind}}(X).$$

On H , the active-row polynomial is one exactly at usable rows.

Advice columns are randomised at *all* $b + 1$ unusable rows, including row ℓ . Product polynomials retain their constrained value at row ℓ and randomise only the last b rows. Gates must be disabled where they would impose unintended constraints on these random cells. A rotation must likewise not make an active gate depend on a random reserved cell unless that dependence is intended.

3 The permutation argument

3.1 Copy constraints as a permutation

Copy equalities partition selected cells into classes whose values must agree. Their endpoints must lie in the usable rows $0, \dots, \ell - 1$; key generation rejects wiring to any other row. Number the equality-enabled columns $j = 0, \dots, m - 1$. Label the cell in row i , column j , by

$$\text{id}_{j,i} = \delta^j \omega^i.$$

Choose δ so that these column cosets are disjoint. Halo 2 uses the field's designated DELTA; the circuit must fit the supply of disjoint cosets. Distinct labels identify distinct cells even when their witness values happen to coincide.

For each equality class, choose one cycle through its cells. Cells outside a nontrivial class are fixed points. Together these cycles define a permutation σ . The fixed polynomial s_j contains the label of the successor:

$$s_j(\omega^i) = \text{id}_{\sigma(j),i}.$$

Both the wiring and the commitments to these polynomials belong to the verification key.

Lemma 3.1 (Why a labelled product checks equality). *Let v_c be the value at cell c , and let λ_c be its distinct label. The formal identity*

$$\prod_c (v_c + \beta \lambda_c + \gamma) = \prod_c (v_c + \beta \lambda_{\sigma(c)} + \gamma)$$

holds as a polynomial in β, γ if and only if values are constant on every cycle of σ .

Proof. Fix a cell c and substitute $\gamma = -v_c - \beta\lambda_c$. The left product is zero. The right becomes a product of polynomials in β , so one of its factors must be the zero polynomial: the product of nonzero polynomials over a field is nonzero. Its coefficient of β requires $\lambda_{\sigma(d)} = \lambda_c$ for that factor's cell d , and its constant term requires $v_d = v_c$. Distinct labels therefore give $d = \sigma^{-1}(c)$. Doing this for every c proves constancy on each cycle. Conversely, these equalities merely reorder the factors. The polynomial facts used here are proved in the *Math Guide*, “Polynomials and evaluation”. $\square$

For actual random field values β, γ , a nonzero difference can vanish accidentally. The polynomial root bound controls that event; the product identity is not an unconditional test at one challenge pair. The advice commitments precede these challenges.

3.2 The running product

For one set of equality columns, an honest prover starts with $Z(1) = 1$ and, for usable rows, computes

$$Z(\omega^{i+1}) = Z(\omega^i) \frac{\prod_j (a_j(\omega^i) + \beta \delta^j \omega^i + \gamma)}{\prod_j (a_j(\omega^i) + \beta s_j(\omega^i) + \gamma)}.$$

This formula describes the ordinary case in which all denominator factors are nonzero. It telescopes: the final product equals one precisely when the two labelled products agree.

The verifier does not receive all these intermediate values. It checks a polynomial recurrence, multiplied by A_{act} :

$$A_{\text{act}}(X) \left[Z(\omega X) \prod_j (a_j(X) + \beta s_j(X) + \gamma) - Z(X) \prod_j (a_j(X) + \beta \delta^j X + \gamma) \right] = 0. \quad (4)$$

Boundary constraints fix the initial and final values. These expressions join the combined numerator N_γ ; checking them uses the same quotient machinery as checking an arithmetic gate.

3.3 Chunking the columns

A product over m columns would make (4) have expression degree $m + 2$: one for the active selector, one for Z , and m factors. For a circuit degree d , the implementation therefore puts at most $d - 2$ columns in a chunk. Let J_t be the columns of chunk t , and let Z_t be its product polynomial.

The first chunk starts at one. Each subsequent chunk starts at the previous chunk's final usable product. The exact boundary constraints are

$$L_0(X)(1 - Z_0(X)) = 0, \quad (5)$$

$$L_0(X)(Z_t(X) - Z_{t-1}(\omega^\ell X)) = 0 \quad (t > 0), \quad (6)$$

$$L_\ell(X)(Z_{T-1}(X)^2 - Z_{T-1}(X)) = 0. \quad (7)$$

Each chunk also has recurrence (4) with $j \in J_t$. Here T is the number of chunks. At row zero, $\omega^\ell X$ addresses row ℓ , which explains the rotated query in (6).

The final constraint is *Boolean*: it permits zero or one. It is not written as $Z_{T-1} = 1$. Off the exceptional set where a numerator or denominator factor vanishes, every running product is nonzero, so this Boolean constraint forces one. A soundness argument must include the exceptional event rather than silently replace the implemented constraint by a stronger one.

The verifier needs each Z_t at x and ωx , and all but the last at $\omega^\ell x$. The latter is also the rotation $-(b + 1)$, because $\omega^n = 1$. It needs every fixed s_j at x . At degree nine, fifteen equality-enabled columns require three seven-column-capacity chunks, not two. Columns can be advice, instance or fixed; counting advice columns alone is insufficient.

4 The lookup argument

4.1 Membership, not equal multiplicity

A lookup requires a row's tuple to occur among the rows of a table. For example, a table containing $0, \dots, 255$ constrains a value to one byte without eight separate bit equations. A table row can be used repeatedly. A *multiset* records values together with their multiplicities, meaning how many times each occurs. The input multiset therefore need not have the same multiplicities as the table.

Let the input tuple be $(a_0, \dots, a_{r-1})$ and the table tuple $(s_0, \dots, s_{r-1})$. After committing to the advice columns, sample θ and compress each tuple in Horner order:

$$A = \sum_{j=0}^{r-1} \theta^{r-1-j} a_j, \quad S = \sum_{j=0}^{r-1} \theta^{r-1-j} s_j.$$

A pair of distinct, fixed tuples collides under this compression with probability at most $(r-1)/|\mathbb{F}|$. To protect all relevant pairs, the analysis must count them and combine their bounds. The single-pair bound is not the error of the whole lookup argument.

Lookup expressions exist at every usable row. Disabling a conditional lookup must map the inactive input to a tuple actually present in the table, often an all-zero tuple. A selector which merely sets a value to zero is not enough if zero is absent from the table.

4.2 Arranging equal runs

The prover permutes the compressed input values into A' with equal values adjacent. It permutes the table values into S' so that the first row of each equal run in A' matches the table value on that row. Unused table entries fill the remaining positions.

For example, with input and table values

$$A = (3, 1, 3, 3), \quad S = (1, 2, 3, 4),$$

a valid arrangement is

$$A' = (1, 3, 3, 3), \quad S' = (1, 3, 2, 4).$$

The value three appears in three input rows but only one table row. At the first three, the table matches. At the next two, equality to the preceding input suffices.

The conditions are

$$A'_0 = S'_0, \quad (A'_i - S'_i)(A'_i - A'_{i-1}) = 0 \quad (0 \leq i < \ell).$$

At row zero the separate first-row equation supplies the match, regardless of the wrapped previous-row value. At a later row, either the current table entry matches or the preceding input is the same. Induction from row zero shows that every input belongs to the table, provided A' and S' really are permutations of their originals. Sorting is an honest-prover construction; the verifier does not need to prove a numerical ordering.

4.3 Proving the two permutations

The prover commits to A' and S' after θ but before the independent challenges β, γ . It forms a product Z whose ordinary-case recurrence is

$$Z(\omega^{i+1}) = Z(\omega^i) \frac{(A(\omega^i) + \beta)(S(\omega^i) + \gamma)}{(A'(\omega^i) + \beta)(S'(\omega^i) + \gamma)}.$$

The two challenges keep the input and table multiset tests separate. As a formal polynomial identity, equality of the products in β, γ implies equality of each respective multiset: compare the coefficient

of the highest power of γ to isolate the input product, and the highest power of β to isolate the table product. The one-variable fact follows by matching the monic linear factors.

The exact constraint list is

$$L_0(1 - Z) = 0, \quad (8)$$

$$L_\ell(Z^2 - Z) = 0, \quad (9)$$

$$A_{\text{act}}[Z(\omega X)(A' + \beta)(S' + \gamma) - Z(X)(A + \beta)(S + \gamma)] = 0, \quad (10)$$

$$L_0(A' - S') = 0, \quad (11)$$

$$A_{\text{act}}(A' - S')(A' - A'(\omega^{-1}X)) = 0. \quad (12)$$

Unmarked polynomials in these equations are evaluated at X . As in the permutation argument, the Boolean endpoint forces one only off the zero-factor exceptional set.

The prover randomises A' and S' in the last $b + 1$ rows. It randomises Z in the last b rows while retaining its endpoint at row ℓ . The verifier needs five evaluations per lookup:

$$Z(x), \quad Z(\omega x), \quad A'(x), \quad A'(\omega^{-1}x), \quad S'(x).$$

Together with evaluations of the original input and table expressions, these suffice to compute all five constraints at x . Authentication of those evaluations is the remaining task.

5 Polynomial commitments and the masked inner-product argument

5.1 Committing to a bounded-degree polynomial

Let $\mathbb{G}$ be a prime-order group whose scalar field is $\mathbb{F}$. Its identity is $\mathcal{O}$. Choose public generators $G_0, \dots, G_{n-1}, W, U \in \mathbb{G}$ without known efficiently computable discrete logarithm relations. For $p(X) = \sum_{i=0}^{n-1} a_i X^i$ and blind $r \in \mathbb{F}$, define

$$\text{Commit}(p; r) = \sum_{i=0}^{n-1} [a_i]G_i + [r]W.$$

This is the vector commitment from the *Crypto Guide*, “Groups, commitments, and Sinsemilla”. We use $\langle a, G \rangle = \sum_i [a_i]G_i$ for a scalar–point inner product, and $\langle a, b \rangle = \sum_i a_i b_i$ for a scalar–scalar inner product.

A uniform independent r makes the commitment perfectly hiding: multiplication by nonidentity W is a bijection from $\mathbb{F}$ to $\mathbb{G}$. Binding is computational, not information-theoretic. The map from $n + 1$ scalars to one group element necessarily has many collisions; the assumption is that a bounded adversary cannot find the relevant distinct openings. Binding alone does not supply an extractor for a prover that sends group elements without their representations.

Halo 2 derives its parameters deterministically. With the Pasta hash-to-curve construction from the *Crypto Guide*, “Hash functions and key derivation”, using domain `Hal02-Parameters`, it hashes

point	message bytes
G_i	<code>0x00</code> <code>LE32_{bytes}(i)</code>
W	<code>0x01</code>
U	<code>0x02</code>

where `LE32bytes` encodes an integer in four little-endian bytes. This needs no secret setup ceremony. It does not remove the assumptions about the resulting points or their derivation.

For a column already stored as evaluations $p(\omega^i)$, precompute Lagrange-basis generators

$$G_i^{\text{Lag}} = \frac{1}{n} \sum_{j=0}^{n-1} [\omega^{-ij}]G_j.$$

Here the factor $1/n$ also acts by scalar multiplication. Inverse Fourier interpolation gives

$$\text{Commit}(p; r) = \sum_i [p(\omega^i)] G_i^{\text{Lag}} + [r]W.$$

Both bases therefore commit to the same polynomial. Fixed and instance commitments use the implementation's known default blind, which is *one*; it is not a fresh secret mask and not zero.

5.2 The opening claim and its mask

An opening claim consists of a commitment P , a point $x \in \mathbb{F}$ and a value $v \in \mathbb{F}$. The honest prover knows (p, r) with

$$P = \text{Commit}(p; r), \quad p(x) = v, \quad \deg p < n.$$

The desired verification statement combines commitment consistency with evaluation correctness. Sending all coefficients would prove the identity but disclose the polynomial and cost n scalars.

The implemented opening starts by sampling a uniform degree-less-than- n polynomial s_0 and setting

$$s(X) = s_0(X) - s_0(x).$$

Thus $s(x) = 0$. This is a root at the opening point x , not necessarily at zero. The map $s_0 \mapsto s$ is onto the kernel of evaluation at x , with exactly $|\mathbb{F}|$ preimages per output, so s is uniform in that $(n - 1)$ -dimensional subspace. The linear-map facts are proved in the *Math Guide*, “Linear algebra and uniform masks”.

The prover samples an independent blind r_s and sends $S = \text{Commit}(s; r_s)$. The transcript then produces challenges ξ and z , in that order. Define

$$\hat{p} = p - v + \xi s, \quad \hat{r} = r + \xi r_s, \quad \hat{P} = P - [v]G_0 + [\xi]S.$$

The honest claim now says $\hat{P} = \text{Commit}(\hat{p}; \hat{r})$ and $\hat{p}(x) = 0$. For a fixed nonzero ξ independent of s , the polynomial $p - v + \xi s$ is uniform in the zero-evaluation subspace, regardless of which p with evaluation v was used. This is a local masking fact, not yet a simulator for an adaptively hashed transcript.

5.3 One folding step

Let a be the coefficients of $\hat{p}$, and let

$$b = (1, x, \dots, x^{n-1}), \quad f = \hat{r}.$$

Initially the verifier's group expression is $M = \hat{P}$. The invariant to preserve is

$$M = \langle a, G \rangle + [z\langle a, b \rangle]U + [f]W. \tag{13}$$

The inner product is initially zero, so this invariant holds for an honest opening.

Split each vector into equal left and right halves, denoted L, R . Sample fresh independent blinds λ, ρ , and send

$$\begin{aligned} L_j &= \langle a_R, G_L \rangle + [z\langle a_R, b_L \rangle]U + [\lambda]W, \\ R_j &= \langle a_L, G_R \rangle + [z\langle a_L, b_R \rangle]U + [\rho]W. \end{aligned}$$

The transcript supplies challenge u_j . For $u_j \neq 0$, fold by

$$\begin{aligned} a' &= a_L + u_j^{-1}a_R, & b' &= b_L + u_j b_R, \\ G' &= G_L + [u_j]G_R, & f' &= f + u_j^{-1}\lambda + u_j\rho, \\ M' &= M + [u_j^{-1}]L_j + [u_j]R_j. \end{aligned}$$

Vector operations are componentwise.

Lemma 5.1 (The fold preserves the invariant). *If (13) holds, then the same identity holds for the primed vectors and scalars.*

Proof. Expand the folded scalar–point inner product:

$$\begin{aligned}\langle a', G' \rangle &= \langle a_L, G_L \rangle + \langle a_R, G_R \rangle \\ &\quad + [u_j] \langle a_L, G_R \rangle + [u_j^{-1}] \langle a_R, G_L \rangle.\end{aligned}$$

The scalar inner product has the identical expansion with b in place of G . The two cross terms are exactly those added by $[u_j^{-1}]L_j + [u_j]R_j$, including their blind contributions. The remaining terms are the old invariant. $\square$

This asymmetric convention uses u_j and u_j^{-1} , not their squares. Changing to a symmetric convention without changing every equation would produce a different protocol.

5.4 The final check

After $k = \log_2 n$ rounds, each vector has length one. The prover sends $c = a_{\text{final}}$ and $f = f_{\text{final}}$. The verifier checks

$$\hat{P} + \sum_{j=0}^{k-1} ([u_j^{-1}]L_j + [u_j]R_j) = [c]G_{\text{final}} + [czb_{\text{final}}]U + [f]W. \quad (14)$$

It calculates the folded public quantities without receiving them. Define the degree- $n-1$ polynomial

$$B(X) = \prod_{j=0}^{k-1} (1 + u_j X^{n/2^{j+1}}) = \sum_{i=0}^{n-1} B_i X^i.$$

Then

$$b_{\text{final}} = B(x), \quad G_{\text{final}} = \sum_{i=0}^{n-1} [B_i]G_i.$$

The first split separates coefficient indices by $n/2$, the next by $n/4$, and so on, which gives this factor order. For $n = 4$, for example, the coefficient weights are $(1, u_1, u_0, u_0u_1)$.

The opening proof contains S , two group elements per round, and the two final scalars. Its communication is logarithmic in n , but the ordinary verifier still computes a linear-size multiscalar multiplication to obtain the final generator contribution. A multiscalar multiplication is a sum of specified scalar multiples of points; batching such sums does not itself verify a proof inside another proof.

The implementation returns a deferred verification object called a Guard. Its challenge-dependent generator contribution must still be supplied and the resulting multiscalar multiplication evaluated to the identity. The single-proof verification strategy does both. Constructing the deferred object alone is not proof acceptance.

The algebra above excludes zero challenges where an inverse is needed; a zero z or ξ also removes a needed security contribution. These are exceptional events in the analysis. The pinned transcript does not implement an implicit retry-until-nonzero convention. An implementation failure or rejection on an exceptional transcript must not be described as resampling.

6 Reducing multiple openings

6.1 The list to be authenticated

The gate, permutation and lookup checks query many polynomials at several rotations of x . The verifier has an ordered list

$$(P_i, t_i, v_i),$$

where each P_i names a particular committed polynomial, t_i is an evaluation point, and v_i is the claimed value. Repeated references to one polynomial use the same identity. Each polynomial–point pair occurs only once in this input list. The verification key and the argument construction determine that list, not an unauthenticated prover-supplied description.

Different polynomials can have different point sets. For example, an advice column might be queried only at x , while a permutation product is queried at $x, \omega x, \omega^\ell x$. Opening each query separately would repeat the inner-product argument. Halo 2 reduces all the queries to one opening using polynomial interpolation and linear commitments.

6.2 First combine equal point sets

Collect the distinct points at which each polynomial is queried. Group polynomials with identical point sets into groups $j = 0, \dots, m-1$. The implementation preserves first occurrence of each polynomial in the query list; groups are ordered by the first polynomial belonging to each group. Point order follows first occurrence in the original list. These rules make both sides derive the same order.

After the original evaluations have entered the transcript, derive x_1 and x_2 . If group j contains polynomials $p_{j,0}, \dots, p_{j,k_j-1}$ in that order, define

$$Q_j(X) = \sum_{i=0}^{k_j-1} x_1^{k_j-1-i} p_{j,i}(X).$$

Use the same Horner combination for their commitments, blinds and claimed values at every point of the shared set T_j . For instance, three polynomials combine as $x_1^2 p_0 + x_1 p_1 + p_2$, not $p_0 + x_1 p_1 + x_1^2 p_2$.

Let r_j be the unique interpolation polynomial of degree less than $|T_j|$ agreeing with these combined claimed values, and let

$$V_j(X) = \prod_{t \in T_j} (X - t).$$

The entire grouped claim is equivalent to

$$V_j(X) \mid Q_j(X) - r_j(X).$$

The *Math Guide*, “Polynomials and evaluation”, constructs both interpolation and division, so this is an explicit polynomial test, not an assumed new commitment primitive.

6.3 Combine the divisibility claims

For honest claims, set

$$f_j(X) = \frac{Q_j(X) - r_j(X)}{V_j(X)}, \quad Q'(X) = \sum_{j=0}^{m-1} x_2^{m-1-j} f_j(X).$$

Every quotient has degree less than n . The prover commits to Q' with a fresh independent blind and sends this one commitment.

An implementation can compute f_j by repeated synthetic division of Q_j by the factors $X - t$, discarding remainders. The resulting quotient is also the quotient of $Q_j - r_j$: the unique remainder upon division by V_j is precisely the interpolation polynomial of Q_j ’s actual values. If a claimed value is wrong, the prover’s actual remainder and the verifier’s claimed interpolation differ. The next check is where that discrepancy must be caught.

After the commitment to Q' , derive x_3 . The prover sends

$$u_j = Q_j(x_3) \quad (0 \leq j < m).$$

Provided $x_3 \notin \bigcup_j T_j$, the verifier computes the expected value

$$v' = \sum_{j=0}^{m-1} x_2^{m-1-j} \frac{u_j - r_j(x_3)}{V_j(x_3)}. \quad (15)$$

The claim is that $Q'(x_3) = v'$. The challenge x_3 is sampled and the claims u_j are transmitted only after the commitment to Q' . Some values can already be predictable, for example for a constant Q_j ; the required order concerns the challenge and messages.

For a singleton set $T_j = \{t\}$, interpolation is just the constant claimed value $r_j = v_j$, and the summand is $(u_j - v_j)/(x_3 - t)$. For two points t_0, t_1 , it is obtained using

$$r_j(X) = v_{j,0} \frac{X - t_1}{t_0 - t_1} + v_{j,1} \frac{X - t_0}{t_1 - t_0}, \quad V_j(X) = (X - t_0)(X - t_1).$$

Thus both the single-point and rotated-query cases use the same construction.

6.4 One final polynomial

Only after the u_j values are in the transcript, derive x_4 . Starting with $P_{\text{final}} = Q'$, perform the Horner update $P_{\text{final}} \leftarrow x_4 P_{\text{final}} + Q_j$ for groups in order. Equivalently,

$$P_{\text{final}}(X) = x_4^m Q'(X) + \sum_{j=0}^{m-1} x_4^{m-1-j} Q_j(X).$$

Its commitment is the same linear combination of known commitments; no extra final commitment is sent. The verifier calculates the claimed evaluation

$$v_{\text{final}} = x_4^m v' + \sum_{j=0}^{m-1} x_4^{m-1-j} u_j.$$

The prover now supplies the masked inner-product opening of P_{final} at x_3 to v_{final} , exactly as in §5. Linear combination of commitment blinds gives the prover's final opening blind.

Each challenge has a specific job. The challenge x_1 prevents fixed errors among polynomials in a group from cancelling deliberately; x_2 combines the quotient identities; x_3 tests their agreement at a fresh point; x_4 combines the last evaluation claims. These explanations rely on earlier objects being fixed and on the eventual opening argument having the necessary soundness guarantee. They do not turn computational binding by itself into knowledge extraction.

Point coincidences, including $x_3 \in T_j$, belong to the exceptional set. Their actual implementation handling is not a licence to divide by zero in the proof of correctness. Nor may an analysis assume that every differently named rotation is distinct when $x = 0$.

7 The transcript and the verifier

7.1 The circuit-specific keys

The public parameters depend on the field, group and domain size. A verification key additionally fixes the circuit: column counts, query rotations, gate and lookup expressions, equality wiring, fixed commitments and degree. A proving key contains that verification key and precomputed polynomial forms needed by the prover. Key generation does not require the private transaction witness. The resulting circuit shape must not change with the witness.

The verifier must obtain the intended verification key through its application's authenticated rules. Accepting a prover-selected key would allow a proof for a different, weaker relation. Fixing the key also fixes the meaning and position of every public input.

In the pinned implementation the verification-key transcript representative is formed by taking the Rust debug-format string of the key’s *pinned* contents, prefixing its UTF-8 byte length as an eight-byte little-endian integer, hashing with 64-byte BLAKE2b under personalisation `Hal02-Verify-Key`, and reducing the 64-byte little-endian result modulo the scalar-field order. The pinned contents include both field moduli, domain parameters, constraint system, fixed commitments and permutation key. This is a version-specific circuit identifier, not a claim that arbitrary Rust debug formatting is a portable consensus serialisation.

The verifier constructs each public instance polynomial by zero-padding the supplied column to n entries, with at most ℓ supplied entries. It computes its commitment with known blind one. The verification-key representative and these instance commitments enter the transcript as common inputs, without being transmitted as proof bytes.

7.2 Hashing messages into challenges

The *Crypto Guide*, “Hash functions and key derivation”, constructs BLAKE2b. Halo 2 initialises its transcript with 64-byte BLAKE2b output and personalisation `Hal02-Transcript`, padded to the hash’s personalisation-field length. The running hash state absorbs the following typed messages:

message	bytes absorbed
scalar a	$0x02 \parallel a^*$
point P	$0x01 \parallel x(P)^* \parallel y(P)^*$
challenge request	$0x00$

Scalar and coordinate encodings here are canonical 32-byte little-endian field encodings. On a challenge request, the state first absorbs the zero byte, then a clone of the state is finalised. Reduce the resulting 64-byte little-endian integer modulo $|\mathbb{F}|$ to obtain the challenge. The original state retains the newly absorbed zero byte; the hash output itself is not appended to it.

Proof bytes and hashed bytes are not identical for points. The proof contains the canonical 32-byte compressed point, whereas the transcript absorbs its two affine coordinates. Decoding must succeed before the coordinates can be hashed. The transcript rejects the identity point, which has no affine coordinates, even though the curve encoding format can represent that identity. A scalar in the proof must also decode canonically; reducing an out-of-range proof encoding would be a different parser.

The point and scalar prefix bytes separate their meanings in the hash input. Circuit and instance binding comes from the common-input prefix. Challenge ordering comes from the prescribed message sequence, not from serialising the names θ , β and so on.

Reduction of a uniformly random 512-bit integer is negligibly biased rather than exactly uniform unless the field order divides 2^{512} . The local algebraic probability calculations use ideal uniform challenges. A concrete security estimate includes this statistical difference as well as the hash model and the number of adversarial hash queries.

7.3 The whole sequence

Before combining constraints with y , the prover additionally samples a uniform degree-less-than- n polynomial R and an independent commitment blind, and sends $C_R = \text{Commit}(R; r_R)$. It later sends $R(x)$. Both R and the collapsed quotient h_x are included among the polynomials opened at x . This masks information in the later multi-opening reduction. The polynomial R is not added to the gate numerator or to the quotient identity.

The transcript order is:

Common inputs or prover messages	Next challenges
Verification-key representative; instance commitments	None
Advice commitments	θ
Permuted lookup commitments (A', S')	β, γ
Permutation products; lookup products; C_R	y
Quotient chunk commitments	x
Column and auxiliary evaluations	x_1, x_2
Combined quotient commitment $C_{Q'}$	x_3
Group evaluations $u_0, \dots, u_{m-1}$	x_4
Opening mask commitment S	ξ, z
Folding pair L_j, R_j , once per round	u_j
Final scalars c, f	Final group check

For several circuit instances proved together, each stage includes all instances before proceeding to the next challenge. Fixed columns, fixed permutation commitments, the random polynomial and the aggregate quotient are shared where the construction specifies; advice and auxiliary products remain per-instance. This shares one argument over several witnesses, not a proof that verifies earlier proofs.

The scalar evaluation messages after x are ordered as follows:

1. Instance-column queries, in configured order for each instance.
2. Advice-column queries, likewise.
3. Shared fixed-column queries.
4. The scalar $R(x)$.
5. The fixed permutation-polynomial values $s_j(x)$.
6. For each instance and each permutation chunk, the values at $x, \omega x$, then at $\omega^\ell x$ unless it is the last chunk.
7. For each instance and lookup, the five values $Z(x), Z(\omega x), A'(x), A'(\omega^{-1}x), S'(x)$.

There is no separate message for the expected quotient value $N_y(x)/(x^n - 1)$.

The multi-opening query list has its own deterministic order. For each instance it lists instance queries, advice queries, permutation queries, then lookup queries; it then appends the shared fixed queries, fixed permutation queries, and finally h_x and R at x . Permutation queries first list $(x, \omega x)$ for each chunk, then the $\omega^\ell x$ queries for the preceding chunks in reverse order. Each lookup lists Z, A', S' at x , then A' at $\omega^{-1}x$, then Z at ωx . This order determines the point-set grouping and Horner weights in §6; it need not equal the scalar transmission order.

7.4 The acceptance algorithm

Given the authenticated verification key, parameters, public instance columns and a proof byte slice, the verifier performs the following logical steps.

1. Check instance column counts and lengths. Construct the common inputs and initialise the transcript.
2. Read canonical proof elements in the fixed sequence, rejecting malformed encodings and read failures. Derive each challenge only after the preceding messages have been absorbed.
3. Evaluate every gate expression at x using the claimed column evaluations. Evaluate every permutation and lookup constraint using its claimed auxiliary evaluations and the public $L_0(x), L_\ell(x), L_{\text{blind}}(x)$.
4. Combine those expressions in Horner order with y . Compute the expected quotient evaluation and the commitment to h_x .
5. Construct the complete opening-query list. Reproduce the point-set groups, interpolants and linear commitment combinations. Compute the expected final evaluation at x_3 .

6. Read and verify the masked inner-product opening. Supply the final challenge-dependent generator contribution and evaluate the entire multiscalar multiplication to $\mathcal{O}$.

The surrounding application supplies proof framing and enforces its specified length or end-of-input rule. The generic transcript reader consumes the elements it needs; it does not by itself establish that an arbitrary enclosing byte stream has no trailing data.

Evaluations are claims until the final opening check succeeds. A vanishing test performed on unauthenticated scalar messages is not already a verified circuit. Conversely, an opening proof authenticates a polynomial claim, not the application's intended semantics: the circuit and public-input binding remain essential.

8 Knowledge soundness and zero knowledge

8.1 What the algebra has established

The preceding derivations establish deterministic identities: interpolation represents columns, vanishing means divisibility, the permutation and lookup constraints enforce their advertised relations outside specified bad events, and every honest inner-product fold preserves the group equation. They therefore explain honest correctness and the structure of the verification test.

A malicious prover need not generate group elements from known coefficient vectors using the honest algorithm. For example, merely observing a point P does not reveal a polynomial and blind opening it. The expression $\text{Commit}(p; r) = P$ has many mathematical solutions. An *extractor* is an algorithm with specified access to the prover which obtains a suitable witness efficiently; saying that a solution exists is not a construction of such an algorithm.

This distinction is why the folding invariant is not, by itself, a knowledge-soundness proof. It checks what happens if the vectors used in the calculation exist with the requisite consistency. Extraction has to establish that consistency against an adversarial transcript.

8.2 A conditional arithmetic-layer statement

It is useful to isolate exactly what a secure commitment-and-opening layer must provide. Consider first an interactive version with independent uniform verifier challenges.

Assumption 8.1 (Consistent polynomial extraction). Except with probability $\varepsilon_{\text{open}}$, an extractor associates every committed polynomial with coefficients of the claimed degree, consistently across all its uses, and accepted openings have the corresponding evaluations. The extraction guarantee respects the challenge chronology: it supplies the relevant fixed polynomial objects before the later challenges to which the algebraic tests are applied.

This is a security requirement on the composed opening protocol, not a property proved by the Pedersen commitment's hiding or binding lemma. It also excludes interpreting a polynomial fitted after learning the test point as a polynomial fixed before that point.

Proposition 8.2 (Recovering a satisfying arithmetic witness). *Under consistent polynomial extraction, and outside polynomial-test, compression and zero-factor bad events, an accepted interactive proof yields advice values satisfying the circuit gates, copy equalities and lookup memberships.*

Proof. Evaluate the extracted advice polynomials on the usable rows of H . Authentication of all queried evaluations makes the verifier's computed numerator the value of the actual combined constraint polynomial. Outside the random-point identity-test event, $N_y = (X^n - 1)h$ as polynomials. Outside the random-combination event, each constituent constraint therefore vanishes on H .

The permutation recurrences telescope. Their initial value is one, their chunk links connect the products, and nonzero factors make the Boolean final value equal one. Outside the labelled-product collision event, the permutation lemma implies all copy equalities.

The lookup products likewise prove that A' and S' have the same multisets as the compressed input and table. The first-row match and run constraints then place every compressed input in the compressed table. Outside tuple-compression collisions, that is membership of the original tuples. The custom gates already vanish at every required row. The extracted advice values are the required circuit witness. $\square$

All algebraic error terms can be bounded using the elementary root bound. For example, for M combined constraints the y test contributes at most $(M-1)/|\mathbb{F}|$. A committed degree-less-than- $(d-1)n$ quotient makes the final difference have degree at most $dn - 1$, giving at most $(dn - 1)/|\mathbb{F}|$ for the x test. With C participating copy cells, a nonzero labelled-product difference has total degree at most C in β, γ . Each zero factor has probability at most $1/|\mathbb{F}|$; count all numerator and denominator factors. Lookup compression must count all distinct input-table tuple pairs, not merely one pair. A union bound combines these contributions and $\varepsilon_{\text{open}}$. Excluded challenges such as $x \in H$ must be included in the failure accounting as well.

These are deliberately modular bounds, not an asserted concrete security level for an entire production proof. The application's circuit-to-relation implication is a further step after extracting a satisfying circuit witness.

8.3 The commitment and Fiat-Shamir boundary

The usual analysis of this family of inner-product commitments uses the discrete-logarithm assumption and a model controlling adversarial group computations. In the *algebraic group model*, an adversary that outputs a group element must also supply, to the analysis, a linear representation of it in previously available group elements. The verifier does not receive these representations. They are an extra restriction on the analysed adversary, not a consequence of canonical point decoding or of the group being cyclic.

Such representations let an extractor compare generator coefficients in suitable related accepting transcripts. Unknown efficient relations between independently generated points prevent inconsistent representations except with the model's failure probability. Constructing the extractor, its rewinding strategy and its time bound is a separate theorem obligation. The *Crypto Guide*, "Knowledge, simulation, and Schnorr", demonstrates these distinctions for a simpler three-message protocol; that demonstration is not automatically a theorem for this longer protocol.

Fiat-Shamir replaces interactive challenges by transcript hashes. Its analysis models the hash as an oracle with consistent random answers on new inputs and counts the adversary's queries. The adversary can try different commitment prefixes and observe their challenges. Consequently, an interactive one-attempt root bound cannot simply be relabelled as the noninteractive security bound. Rewinding or oracle programming, when used by a proof, must be justified within its exact model, including adaptively selected public inputs and correlated openings.

The Halo paper supplies the originating inner-product construction and its modelled security analysis. The pinned implementation includes a masked variant and the concrete reductions described here. We do not assert that a short folding identity or an informal appeal to that paper proves every detail of this implementation's complete composition. The explicit extraction assumption above states the remaining boundary instead of presenting it as a proved consequence of binding. This boundary matters when using the argument as a security premise for the application.

8.4 Why polynomial evaluations need masks

A hiding commitment does not hide a subsequently disclosed evaluation. If a witness polynomial were determined entirely by its active rows, each evaluation could reveal a linear equation in those rows. Halo 2 therefore randomises unused row values as well as commitment blinds.

Fix a set B of s independently randomised domain points and fix all other row values. Let

$$T_B(X) = \prod_{h \in H \setminus B} (X - h).$$

The possible masked column polynomials form the affine space

$$a_0(X) + T_B(X)r(X), \quad \deg r < s.$$

Choosing the s row masks uniformly chooses r uniformly: both parameterisations describe the same s -dimensional affine space bijectively.

For $t \leq s$ fixed distinct query points $z_1, \dots, z_t$ outside H , the evaluation vector of the mask is

$$(T_B(z_1)r(z_1), \dots, T_B(z_t)r(z_t)).$$

Interpolation makes $r \mapsto (r(z_1), \dots, r(z_t))$ surjective, and all the factors $T_B(z_i)$ are nonzero. The *Math Guide*, “Linear algebra and uniform masks”, therefore implies that this vector is uniform on $\mathbb{F}^t$. It hides the corresponding evaluations of a_0 jointly, for these fixed points and this isolated linear observation.

The implementation chooses

$$b = \max(3, q_{\text{adv}}) + 2,$$

where q_{adv} is the maximum number of distinct configured rotations of any advice column, taking one as the fallback when there are no advice columns. The minimum three covers permutation polynomials queried at three rotations. One additional factor covers the multi-opening point x_3 , and one is an extra margin. Advice and permuted lookup columns actually randomise $b + 1$ rows; product polynomials randomise b . This accounts for the concrete row budget rather than merely saying that unspecified blinding is added.

8.5 The quotient mask and the joint transcript

The quotient depends nonlinearly on witness columns and auxiliary products. Its evaluation $h(x)$ is already determined by the constraint evaluations, but an additional value at x_3 could disclose new information. The independent random polynomial R is included with h_x in the group of polynomials queried at x . Their later Horner-combined evaluation at x_3 contains a random contribution from $R(x_3)$.

For fixed distinct x, x_3 and $n \geq 2$, the pair $(R(x), R(x_3))$ is uniform in $\mathbb{F}^2$. After revealing the first coordinate, the second remains uniform. When its batching coefficient is nonzero, it can therefore mask another scalar in that linear combination. The PCS mask s separately masks the final opened polynomial while preserving its required evaluation.

Neither observation proves full zero knowledge in isolation. Different messages can share masks, challenges are derived from earlier commitments, and later openings must remain consistent with those commitments. Two individually uniform messages can jointly reveal a witness, as the example $(r, r + w)$ shows. A complete zero-knowledge proof must construct one joint simulator for the entire view, including commitments, all evaluations, quotient reductions, inner-product messages and hash-derived challenges. If it relies on programming the hash oracle, that ability belongs in the statement of the theorem.

The local affine-mask and folding lemmas explain the mechanisms and their required dimensions. This volume does not replace the joint simulation obligation by those lemmas, nor does it claim that any missing composition theorem has thereby been proved.

8.6 The application-facing contract

To consume Halo 2 correctly, an application must fix the circuit and verification key, define the public-input mapping, frame and parse the proof, run the complete opening verification, and check every

non-circuit rule separately. It must establish that a satisfying circuit witness implies its intended relation, with canonical encodings and integer bounds where needed.

The resulting security claim is conditional on that circuit argument, the commitment and transcript security assumptions, correct parameter and key derivation, and correct implementation. Proof privacy does not hide the public input, the proof’s circuit shape, transaction metadata outside the circuit, or information the wallet discloses through its network activity. These are boundaries of the proved relation, not failures of the masking algebra.

8.7 Primary construction sources

The mechanism and byte order in this volume are checked against the pinned `halo2_proofs` source. Within that source, the relevant files are:

- `plonk/circuit.rs`, `plonk/prover.rs` and `plonk/verifier.rs`: circuit degree, blinding budget, complete message and query order.
- `plonk/permutation/` and `plonk/lookup/`: product recurrences, exact Boolean boundaries and rotations.
- `poly/domain.rs` and `plonk/vanishing/`: extended coset, random polynomial and quotient chunks.
- `poly/commitment.rs`, `poly/commitment/` and `poly/multiopen/`: generator derivation, asymmetric masked folding and Horner-ordered multi-opening reduction.
- `transcript.rs` and `plonk.rs`: challenge bytes and verification-key representative.

The Halo 2 design documentation explains the design; its sketches are not substituted for the concrete source where conventions differ. Bowe, Grigg and Hopwood’s *Halo: Recursive Proof Composition without a Trusted Setup* is the primary paper behind the commitment construction and modelled security analysis. Recursive composition itself is not needed to understand the single proof verified here.

The executable checks in `research/check_foundations.py` and `research/check_halo2.py` recompute the small-field example, folding identities and multi-opening algebra. They test the displayed calculations, not cryptographic hardness, witness extraction or a complete transcript simulation.

Part IV

Consensus Guide: Ledger state, block validation, and the meaning of confirmation

Abstract

This volume of *The Zcash Arboretum* explains how computers agree on an ordered record of payments, check proposed additions, and account for the money those additions create or transfer. It assumes the *Math Guide*, especially “Language, integers, and encodings” and “Probability and computation”, and the *Crypto Guide*, “Hash functions and key derivation” and “Authenticated trees and append-only histories”. The payment’s private computations enter through explicit contracts specifying what their evidence must establish. The exposition does not implement the internals of every historical transaction format. It is non-normative: the protocol specification and the applicable consensus ZIPs determine the rules.

1 Ledger state and transaction acceptance

A payment changes who can spend value. It does not move a physical object between machines. The *ledger* is the accepted record of such changes. A *transaction* is one proposed change, with the data needed to check it. A *node* is a participating computer; a validating node checks the rules itself. Each validating node maintains a public state, receives a proposed change, and checks that the change follows the rules. Privacy changes the evidence used in that check; it does not remove the need for a public decision about whether a particular change has already occurred.

A *transparent output* publicly records an amount and conditions for spending it. A *shielded note* instead describes an amount and receiving capability privately; the chain records a commitment to that description. A *shielded pool* is a separately accounted collection of such notes. Commitment hiding and binding have their security definitions in the *Crypto Guide*, “Groups, commitments, and Sinsemilla”; this volume uses the interface without assuming any one payment construction.

One *zatoshi* is the indivisible accounting unit, and 10^8 zatoshis equal one ZEC. All consensus amounts are integers. A negative public balance will mean a transfer *into* a shielded pool, not a negative-valued private note. The distinction between integer arithmetic and arithmetic modulo a prime is developed in the *Math Guide*, “Prime fields and modular arithmetic”.

1.1 State belongs to a branch

A *block* contains an ordered list of transactions and a *header*, a fixed-format summary naming its parent block by hash. The *genesis block* is the prescribed starting block. Starting there, these parent links form a tree. A *branch* is one path through that tree. Its height counts edges from genesis; competing blocks can therefore have the same height without being the same block.

An *anchor* is a previously accepted note-tree root against which a private existence proof is checked. A *nullifier* is a deterministic public spend tag whose repetition is forbidden within its pool. The note tree is an append-only tree of note commitments; the generic tree and its root are constructed in the *Crypto Guide*, “Authenticated trees and append-only histories”.

For a branch tip b , write its state schematically as

$$S_b = (h, U, \{T_P, A_P, N_P, B_P\}_{P \in \mathcal{P}}, D, \mathcal{H}).$$

Here h is the height; U is the map of unspent transparent outputs; $\mathcal{P}$ is the set of shielded pools recognised by the active rules; T_P is pool P ’s append-only note-commitment tree; A_P records roots eligible as anchors; N_P is its set of revealed nullifiers; and B_P is its public chain value pool balance. The *deferred development pool* is publicly tracked value set aside under prescribed allocation rules; its balance is D . The authenticated block-history state is $\mathcal{H}$.

The state stores commitments and spend tags, not private note amounts or owners. The commitment and nullifier mechanisms must be provided by the payment protocol. The ledger needs their interfaces: append a commitment, recognise an eligible root, and reject a repeated tag.

A state indexed only by height is ambiguous. If blocks b and b' have the same height, their output maps, nullifier sets and note trees can differ. Every state-dependent query must identify a branch, either explicitly by hash or implicitly by a stable branch snapshot.

1.2 A typed payment-validation contract

An *ordinary transaction* transfers existing value; it is not the special transaction permitted to issue the block's subsidy, or newly created money. It supplies transparent input references and outputs, zero or more shielded components, and *authorising data*: proofs and signatures used to validate the proposed effects. A proof's private input is its *witness*; its public inputs are visible to the verifier. Their security contracts and signatures are introduced in the *Crypto Guide*, “Knowledge, simulation, and Schnorr” and “RedPallas signatures”. For each shielded component its public interface contains:

- an ordered commitment list $C_P(t)$ to append;
- a nullifier list $N_P(t)$ to insert;
- any required anchor or anchors;
- a signed public balance $b_P(t)$, positive for net value leaving the pool;
- a proof and signatures establishing the protocol-specific relationship between these public objects and private witnesses.

A list, unlike a set, retains order and multiplicity. Checking only that $N_P(t) \cap N_P = \emptyset$ is insufficient: the list itself must not contain a repeated nullifier.

The payment mechanism must establish that nonzero private inputs existed under the claimed anchors, that their spend tags were computed correctly, that the spender authorised this transaction, and that private input value minus private output value equals $b_P(t)$. A zero-knowledge proof alone need not establish all these claims. A design may divide them between a proof, signatures approving spends and a separate signature enforcing the combined value balance. The ledger invokes the complete specified verifier, not whichever subset is easiest to batch.

The network parameters specify which rules apply at each height. A format's *version* and *version-group identifier* select its byte layout and interpretation. Its *consensus branch identifier* separates cryptographic computations belonging to different upgrade rules; it does not name a block-tree branch. A *verification key* is the public circuit-specific input to proof verification, not a spending key. Field and point encodings use the *Math Guide*, “Prime fields and modular arithmetic” and “Pasta curves and their encodings”. An input is *mature* when its required waiting interval has elapsed. A transaction's lock time limits when it can begin to be accepted; its expiry condition limits how late it can be accepted.

Definition 1.1 (Ordinary transaction acceptance). For network parameters θ , candidate block height h and a branch-local state S , transaction acceptance is the conjunction of:

1. *Well-formedness*: canonical parsing, bounded lengths, valid field and point encodings, supported version and version-group combination, and the branch identifier required at height h .
2. *Context-independent validity*: all applicable proof, signature, range and component-consistency checks.
3. *Context-dependent validity*: unspent and mature transparent inputs, eligible pool-specific an-

chors, unused nullifiers, applicable lock-time and expiry conditions, and upgrade-specific restrictions.

4. *Accounting*: the ordinary fee is nonnegative and all required individual and aggregate money ranges hold.

Acceptance produces a proposed state delta. No part of that delta becomes committed state if any required check fails.

The ordinary fee is determined by public quantities:

$$f(t) = v_{\text{in}}^{\text{transparent}}(t) - v_{\text{out}}^{\text{transparent}}(t) + \sum_{P \in \mathcal{P}} b_P(t) \geq 0. \quad (1)$$

An absent component contributes zero. A *JoinSplit* is an earlier shielded transfer component. Its formats have their own public transfer fields; at a compatibility boundary their net contribution must enter this equation with the specified sign. This formula is not permission to skip their own proofs or signatures.

Example 1.2 (The continuing payment). An existing shielded note holds 80,000 zatoshis. The transaction pays 50,000, creates 20,000 of change and leaves 10,000 as a fee. There are no transparent inputs or outputs. Thus

$$b_P = 80,000 - 50,000 - 20,000 = 10,000 = f.$$

The pool loses 10,000 zatoshis of publicly accounted value. The payment and change amounts remain private. A transaction’s fee is therefore public even when every payment component is shielded.

1.3 Rules, implementations and activation

A *network upgrade* changes the rules at a network-specific height. The rule-selection function $\text{Rules}_\theta(h)$ determines allowed formats, branch identifiers, verification keys, pool restrictions and block-accounting parameters. The branch identifier is a domain-separation value; it is not a block hash or a replacement for the height test.

The normative baseline used here is zips commit 69610984, notably the protocol specification sections “Transaction Encoding and Consensus”, “Block Header Encoding and Consensus”, “Proof of Work”, and “Chain Value Pool Balances”. The implementation cross-check is zebra commit ef6325c0, principally `zebra-consensus/src/block.rs`, `zebra-consensus/src/transaction.rs` and `zebra-state/src/service/check.rs`. References to later rule sets are conditional descriptions of their specified activation rules. The presence of code, a release number, or a ZIP marked Draft or Proposed does not establish activation on a network.

The compatibility boundary is deliberate. A complete node must also validate the active transparent and earlier shielded formats. This volume constructs the common ledger contract; it does not substitute that contract for their complete validation algorithms.

2 Blocks and branch-local validation

A transaction valid by itself may conflict with another transaction in the same block. A block valid on one branch may be invalid on another. These are not failures of a signature or proof. They are consequences of the state to which the transaction proposes to apply.

2.1 Header and body

Producing a block requires finding an *Equihash solution*, a string meeting prescribed collision conditions, and making the whole header’s hash no larger than an integer *target*. These constitute its

proof of work: a publicly checkable result of a computational search. The *nonce* is a header field varied during that search. A *transaction identifier* is its format’s prescribed transaction digest. The first transaction, called the *coinbase*, is the special transaction permitted to claim new issuance and the block’s fees. The modern header retains the following byte layout:

Field	Bytes
Signed version	4
Parent hash	32
Transaction Merkle root	32
Block commitments	32
Timestamp	4
Compact target, nBits	4
Nonce	32
Canonical encoded solution length	3
Equihash solution	1344

Fixed-width integer fields are little-endian. The length is a canonical *CompactSize* integer: values below 253 use one byte; otherwise prefixes 0xfd, 0xfe and 0xff select two-, four- and eight-byte little-endian payloads, respectively, and a longer-than-necessary representation is rejected. Here the payload value is 1344, so its encoded length occupies three bytes.

The block hash is double SHA-256 of the complete header encoding, including the solution length and solution. Display conventions that reverse hash bytes must not leak into internal hash preimages. The transaction Merkle root commits to transaction identifiers in block order. Its binary tree duplicates the final child at a level having an odd number of children. This is not the zero-leaf padding rule of the authorising-data tree in §5.

The header is not the whole block. A header can satisfy proof of work while its body contains an invalid proof, a double spend, a wrong coinbase amount, or no available transactions. A node must obtain and validate the body before calling the block fully valid.

2.2 A validation algorithm

The following sequential algorithm defines a simple correctness model. An implementation may verify independent proofs concurrently, provided its final state and rejection conditions are equivalent.

1. Locate the candidate’s parent and obtain that parent’s validated state. Unknown ancestry is missing context, not evidence of validity.
2. Select the rules at the candidate height. Check header format, target, timestamp constraints and proof of work.
3. Parse the bounded body. Require at least one transaction, with coinbase first and no later coinbase. Check body commitments and all block-wide resource and encoding limits.
4. Evaluate ordinary transactions against a temporary view of the parent state. Consume transparent outputs and reserve nullifiers so that conflicts within the block are visible. Earlier transparent outputs in the block may become available subject to the applicable rules. Do not thereby make a newly computed note-tree root an eligible shielded anchor: modern shielded anchors refer to earlier blocks’ final treestates.
5. Sum ordinary fees, validate the coinbase and prescribed allocations, and check all resulting chain pool balances. Append shielded commitments in transaction order and each component’s output order. Check tree capacities and derive final treestates.
6. Construct the applicable history and authorising-data commitments. Compare every header commitment with its independently derived value.

7. Atomically store—make visible all together or not at all—the accepted block, its state delta and sufficient rollback information. Only then make this branch eligible for chain selection.

This algorithm names two different validation orderings. Ledger effects have a logical order fixed by the transaction sequence. CPU work such as signature checks can be reordered without changing that sequence. Batch verification therefore does not mean applying a transaction before all checks have completed.

A memory pool of unmined transactions is a node’s local proposal set. It may reject transactions for fee, resource or expiry-risk policy beyond consensus validity. Acceptance into that set is neither inclusion in a block nor a promise that another node will relay the transaction.

2.3 Time and resource boundaries

The *median* here means: sort the $n > 0$ observations in increasing order and choose the element at zero-based index $\lfloor n/2 \rfloor$. For even n , this selects the upper of the two middle elements, not their average. This matters during the shorter initial windows.

In the pinned modern Mainnet rules, a header version must be at least 4, and a block is at most 2,000,000 bytes. A non-genesis timestamp must exceed the median of the preceding eleven timestamps, or all preceding timestamps if fewer exist. For the modern range it must also be at most that median plus 90 minutes. These are branch-computable rules. The extra rejection of a block more than two hours ahead of the node’s own clock is time-dependent: the same block may become acceptable as the clock advances.

Transaction expiry and lock-time are independent of the header timestamp bounds. Their precise interpretation depends on format and rules. A wallet must not infer that a recent-looking timestamp makes an expired transaction valid, or that a transaction’s expiry height determines the height at which it will be mined.

3 Work, difficulty, and chain selection

Proof of work gives a publicly checkable cost signal for a block. It does not prove its transactions valid, and it does not make the producer’s identity or motives visible.

3.1 Equihash and the difficulty filter

Zcash uses Equihash with $(n, k) = (200, 9)$. A candidate solution selects $2^9 = 512$ distinct indices from 2^{21} possible indices. Each index selects a 200-bit string derived from the header prefix including its nonce. The selected strings must XOR to zero. Further conditions bind the solution to a binary collision tree: for every consecutive aligned group of 2^r indices, $1 \leq r \leq 8$, its XOR has $20r$ leading zero bits; for every level $1 \leq r \leq 9$, each left index sequence is lexicographically smaller than the corresponding right sequence.

For a zero-based index j , let $a = \lfloor j/2 \rfloor$ and $b = j \bmod 2$. Compute the 50-byte BLAKE2b digest of the 140-byte header prefix followed by the four-byte little-endian encoding of a , personalised by

$$\text{ZcashPoW} \parallel \text{LE}_{32}(200) \parallel \text{LE}_{32}(9).$$

The selected string is bytes $25b$ through $25b + 24$ of that digest. The 512 indices are encoded as consecutive 21-bit big-endian strings, giving $512 \cdot 21/8 = 1344$ bytes. Endianness here differs from the fixed integer header fields. These constructions instantiate the protocol specification’s “Equihash” and “EquihashGen” sections; the equihash crate’s verification algorithm is an implementation reference.

An Equihash solution must additionally pass the difficulty filter:

$$\text{LEInt}(\text{SHA256d}(\text{header})) \leq T.$$

The target T comes from the required `nBits`, not from a value freely chosen by the miner. Under the idealised uniform-hash model, one independent complete-header trial passes this inequality with probability $(T + 1)/2^{256}$. This model motivates the assigned work

$$w(B) = \left\lfloor \frac{2^{256}}{T(B) + 1} \right\rfloor. \quad (2)$$

The actual computational cost of generating Equihash candidates is not measured by this formula; work is the protocol's comparison unit.

3.2 Targets and compact representation

In the 32-bit integer `nBits`, bits 24–31 give the *exponent* e , bit 23 is the sign bit s , and bits 0–22 give the *mantissa* m . These names describe the scale and leading integer digits in the compact representation, not floating-point values. A nonnegative target decodes as $m \cdot 256^{e-3}$, with an integer right shift when $e < 3$. A set sign bit decodes to zero under the specification's conversion; valid proof-of-work targets must satisfy the additional nonzero and limit rules. Target parsing is not floating-point arithmetic.

To encode a positive integer target x , first set

$$e = \left\lceil \frac{\text{bitlength}(x)}{8} \right\rceil, \quad m = \lfloor x \cdot 256^{3-e} \rfloor.$$

If $m \geq 2^{23}$, divide m by 256 with truncation and increment e . Return $m + 2^{24}e$. Lower bits can be discarded, so encoding and then decoding generally rounds a target down. Consensus compares the required compact encoding; retaining extra precision locally must not create a different accepted target.

For modern Mainnet blocks well beyond the initial bootstrap range, let T_i and t_i be previous decoded targets and timestamps. The averaging window is 17 blocks; target spacing after Blossom is 75 seconds. Define

$$\begin{aligned} A &= 17 \cdot 75 = 1275, \\ M(h) &= \text{median}(t_{h-11}, \dots, t_{h-1}), \\ E(h) &= M(h) - M(h - 17), \\ D(h) &= A + \text{trunc}_0((E(h) - A)/4), \\ C(h) &= \max(1071, \min(1683, D(h))), \\ \bar{T}(h) &= \frac{1}{17} \sum_{i=h-17}^{h-1} T_i. \end{aligned}$$

Truncation trunc_0 is towards zero, not towards negative infinity. The bounds are $\lfloor 0.84A \rfloor = 1071$ and $\lfloor 1.32A \rfloor = 1683$. The next unencoded threshold is

$$T_{\text{next}} = \min \left(T_{\text{limit}}, \left\lfloor \frac{\bar{T}(h)}{A} \right\rfloor C(h) \right), \quad T_{\text{limit}} = 2^{243} - 1.$$

Encoding this threshold produces the required `nBits`. Notice the division is rounded *before* multiplication by $C(h)$. Moving the floor can change consensus.

For example, if $\bar{T} = 1,275,000$, an observed interval of 1275 seconds leaves this illustrative unencoded threshold unchanged. Extreme short and long intervals saturate at 1,071,000 and 1,683,000. The numbers illustrate the arithmetic, not a network difficulty estimate. The executable check also exercises negative truncation and compact-target round trips.

At the chain's start, the specification uses available shorter medians, a proof-of-work-limit mean through the averaging-window bootstrap, and a special genesis threshold. Pre-Blossom blocks use their own spacing; Testnet has additional minimum-difficulty exceptions. An independent historical validator must use those branches rather than extrapolating the modern formula backwards.

3.3 Selecting a validated branch

For a chain $C = (B_0, \dots, B_n)$, total work is $W(C) = \sum_i w(B_i)$. A node selects a fully valid visible branch with greatest total work. Height alone is not the comparison: many easier blocks can represent less work than fewer harder blocks. A branch does not become preferable merely because its header declares small targets; its blocks must actually satisfy the targets and the difficulty adjustment rules.

Equal-work ties need not produce instantaneous global agreement. Nodes can temporarily select different visible tips; block propagation and further work can resolve the tie. The rule says what a node does with its information. It does not promise that every node has identical information at every instant.

4 Supply and pool accounting

The proof system checks hidden transfers. Pool accounting checks the public boundary around those transfers. Neither replaces the other.

For a shielded pool initialised at zero, its branch balance is

$$B_P(C) = - \sum_{t \in C} b_P(t). \quad (3)$$

Thus an ordinary transaction updates B_P by $-b_P(t)$. A migration between pools P and Q , with amount v and fee f drawn from P , can have

$$b_P = v + f, \quad b_Q = -v, \quad \sum_R b_R = f.$$

The amounts crossing each pool boundary are public. The accounting does not identify which input notes financed them.

A required nonnegative pool balance prevents withdrawals beyond the pool's publicly recorded total. It does not prove that the remaining private notes are all valid. In particular, an invalid private creation could stay below that aggregate ceiling. Supply integrity still requires sound payment proofs, correct authorisation and uniqueness checks.

4.1 Coinbase and prescribed allocation

The first transaction is the *coinbase*. It claims the permitted block subsidy and ordinary transaction fees; it does not need ordinary spendable inputs to justify new issuance. It is subject to special format, destination and shielded-output rules. The ordinary fee equation must therefore not be applied to it as though it were an ordinary payment.

For the modern post-Blossom subsidy, let h_B be Blossom's activation height and define

$$H(h) = \left\lfloor \frac{h_B - 10,000}{840,000} + \frac{h - h_B}{1,680,000} \right\rfloor, \quad s(h) = \left\lfloor \frac{1,250,000,000}{2 \cdot 2^{H(h)}} \right\rfloor.$$

This is the specification's time-spacing-adjusted halving formula after the slow-start and pre-Blossom intervals. It is a height function, not an estimate obtained from a calendar date.

A funding stream active over $[a, b)$ with prescribed numerator u and denominator v receives $\lfloor s(h)u/v \rfloor$ in that interval and zero outside it. The active rule set supplies the stream list and recipient rules. Amounts directed to the deferred pool are added to D ; an authorised disbursement removes its prescribed amount from D and permits the corresponding coinbase allocation. Such a release is not new issuance. The miner's subsidy share is the block subsidy less required allocations; fees are additionally claimable as specified.

A validator must check both the aggregate amount claim and required payments. Knowing only the total subsidy is not enough to validate a coinbase. The full recipient schedules are compatibility data owned by the applicable funding ZIPs, not an alternative mathematical rule.

4.2 Maturity and spendability

Transparent coinbase outputs have a 100-block maturity rule. That rule should not be carelessly asserted for every kind of shielded coinbase output. The shielded protocols have their own coinbase construction and public-recovery requirements; the commitment format does not itself carry an ordinary transparent-output maturity flag. A wallet’s chosen confirmation delay is another, separate condition. Distinguish node acceptance from the wallet’s choice to risk a spend.

5 Authenticated block history

A header’s parent hash links one block. A commitment to a structured history makes selected statements about earlier blocks efficiently checkable. The generic binary-tree and append-only-forest constructions are in the *Crypto Guide*, “Authenticated trees and append-only histories”. This section fixes Zcash’s concrete metadata, encodings and header commitments. No sampling protocol is needed to construct or validate them.

5.1 An epoch-scoped Merkle mountain range

For a candidate block B_n , let $x < n$ be the most recent network-upgrade activation height strictly below n . ZIP 221’s history tree commits to

$$B_x, B_{x+1}, \dots, B_{n-1}.$$

It excludes B_n itself. The delay avoids asking a block hash to commit to a structure containing that same hash.

The history is a forest of perfect binary trees, one per set bit of the leaf count. Its roots are called *peaks*. Append a new leaf as a height-zero peak; while the two rightmost peaks have equal height, replace them with their parent. Number stored nodes in creation order, placing a leaf before the parents completed by its append. The independent eleven-leaf example produces 19 stored nodes, with peak positions 14, 17, 18 and heights 3, 1, 0.

To obtain one root, combine peaks from left to right:

$$R = \text{Parent}(\text{Parent}(P_0, P_1), P_2)$$

for three peaks. These extra *bagging* combinations need not be stored as ordinary mountain nodes. A right fold would be a different root when there are at least three peaks. For one peak, the root node is that peak; its final root *digest* is still computed as below.

5.2 The complete node record

Every node is a canonical record, not just a hash. The initial record serialises the following fields in order:

1. 32-byte subtree commitment.
2. Earliest and latest timestamps, each unsigned 32-bit little-endian.
3. Earliest and latest compact targets, with the same integer encoding.
4. Earliest and latest final Sapling note-tree roots, each 32 bytes.
5. Total work, an unsigned 256-bit little-endian integer.
6. Earliest height, latest height and number of Sapling-bearing transactions, each canonical CompactSize.

The NU5 record appends earliest and latest final Orchard note-tree roots, then a CompactSize count of transactions with a nonempty Orchard component. The specified NU6.3 extension appends the analogous Ironwood root pair and transaction count. A count is of transactions, not Actions, notes or nonzero payments.

At a leaf, both endpoints of every pair come from that leaf's block. The subtree commitment is the ordinary block hash in internal byte order. Work is (2); each count is computed from the block body. For a parent, earliest fields come from the left child, latest fields from the right child, and work and counts are summed. Earliest means leftmost in chain order, not the numerically smallest timestamp: timestamps need not increase at every block.

For branch identifier β belonging to this history epoch, define

$$H_{\beta}(m) = \text{BLAKE2b}_{256}(\text{ZcashHistory} \parallel \text{LE}_{32}(\beta), m).$$

A parent's subtree commitment is

$$H_{\beta}(\text{Serialize}(L) \parallel \text{Serialize}(R)),$$

using the *entire* child records in the fixed order above. The final chain-history root is

$$H_{\beta}(\text{Serialize}(R_{\text{root}})).$$

Hashing only child digest fields omits the authenticated metadata and produces the wrong tree.

The base record contains 144 fixed bytes and three CompactSize fields, so its length ranges from 147 to 171 bytes. The NU5 record contains 208 fixed bytes and four such fields, giving 212 to 244 bytes. The specified Ironwood extension gives 272 fixed bytes and five such fields, or 277 to 317 bytes. These are encoding bounds, not a claim that every extreme combination can occur in a real chain.

5.3 Upgrade boundaries and authenticated meaning

At the block activating the history-tree rule, Heartwood, the designated header field is the zero sentinel, not a root of an empty ordinary tree. At a later upgrade's activation block, the history commitment completes the preceding epoch. The next block commits to a single-leaf tree beginning with the activation block. The tree's hashing uses the branch identifier of the epoch represented by its nodes, including when the header carrying its completed root belongs to the next epoch. The history implementation and the upgrade selection together determine this boundary; applying a new personalisation to old-epoch nodes would change their committed root.

A valid inclusion path authenticates the supplied records against a chosen root under collision resistance. It does not, by itself, prove that every concealed subtree's declared work sum or count was honestly computed. Fully validating nodes recompute the metadata from accepted blocks. A client trusting only a root needs an additional reason to trust the relevant aggregate semantics. Hash binding is not arithmetic validation or proof of a best chain.

5.4 Authorising-data and block commitments

Transaction identifiers in version 5 commit to *effecting data*, the data specifying the transaction's effects. They exclude proofs and signatures. Each transaction therefore also has an authorising-data digest. ZIP 244 constructs that digest with branch-separated BLAKE2b from the format's transparent, Sapling and Orchard authorising-data component digests. The specified version 6 extension adds Ironwood and moves the affected anchors into authorising data. The payment-format verifier must compute the exact version's digest; an arbitrary hash of serialised transaction bytes is not a replacement.

Build the authorising-data tree in transaction order. A pre-v5 transaction has the prescribed leaf of 32 bytes of 0xff. For versions defining an authorising digest, use that digest. Pad to the next power

of two with 32-byte zero leaves. Since coinbase is mandatory, the leaf count is positive. Internal nodes are

$$H_{\text{auth}}(L, R) = \text{BLAKE2b}_{256}(\text{ZcashAuthDatHash}, L || R).$$

With one transaction there is no internal hash: the root is its leaf. For m transactions the tree height is $\lceil \log_2 m \rceil$.

From the ZIP 244 activation block, the header field is

$$\text{hashBlockCommitments} = \text{BLAKE2b}_{256}(\text{ZcashBlockCommit}, R_{\text{history}} || R_{\text{auth}} || 0^{256}). \quad (4)$$

The final 32 bytes are an extension terminator. A later activated rule may replace that terminator with a further defined commitment structure; a full node must verify the complete active structure. It must not accept an arbitrary terminator because an older light client could treat it as an opaque extension.

There are now three distinct questions. A note-tree anchor can support a private input-existence proof. A transaction digest binds authorised transaction data. A block-history root authenticates selected earlier block records. Sharing a 32-byte representation does not make these objects interchangeable.

6 Reorganisation and confirmation confidence

A transaction's cryptographic validity is stable under unrelated extensions, but its inclusion and contextual validity are branch dependent. Chain selection can therefore change a wallet's apparent balance without breaking a commitment or forging a signature.

6.1 Rollback to a common ancestor

Suppose the selected branch changes from C to C' , with last common ancestor a . Reconstruct S_a by undoing the removed suffix of C or loading its retained snapshot. Apply and validate C' 's suffix in order. Undo data must cover transparent outputs, nullifier insertions, commitment appends, anchor availability, pool balances and history state. A note created only on the removed suffix is no longer an accepted input. A nullifier revealed only there is no longer spent on the new branch.

A transaction from the removed branch may be valid again, invalid because of a conflict, or expired. It is a candidate for re-evaluation, not an automatically accepted member of the new branch or memory pool. Likewise, a proof with an anchor existing only on the removed branch may need to be rebuilt. Persistence must not leave a mixture of old-branch nullifiers and new-branch roots.

Implementations can retain only a bounded rollback window or use checkpoints as safety and storage mechanisms. Such operational bounds are not a theorem that a deeper competing chain is cryptographically impossible. The protocol's most-work criterion and a program's recovery policy are different statements.

6.2 An explicit probabilistic model

Consider an idealised two-producer model with $0 < p < 1$ and a positive integer confirmation count z . Independent block arrivals belong to an honest branch with probability p and an adversarial branch with probability $q = 1 - p$. Both branches have equal per-block work; network delays, changing hash rates and difficulty feedback are ignored. The attacker begins from the same fork point as the payment. Ties count as attacker success. These assumptions define the calculation; they are not observations guaranteed by a real network. The separate endpoint $p = 1, q = 0$ has no adversarial arrivals and reversal probability zero.

For a current honest lead $d > 0$, let u_d be the probability that the attacker ever reaches a tie. First-step conditioning gives

$$u_d = p u_{d+1} + q u_{d-1}, \quad u_0 = 1.$$

When $q < p$, solve first on the finite interval $0, \dots, M$, with failure at M . Substitution verifies

$$u_d^{(M)} = \frac{(q/p)^d - (q/p)^M}{1 - (q/p)^M}.$$

As M increases, the events of reaching zero before M increase to the event of ever reaching zero: any finite successful path has a finite maximum. Taking the limit proves $u_d = (q/p)^d$. For $q = p$, the finite-interval solution is $1 - d/M$ and the limit is one; for $q > p$, the same finite-interval formula tends to one. A strict overtake convention instead requires a different boundary; silently switching between tie and overtake changes the answer.

Waiting for z honest blocks does not give the attacker zero blocks. Let K be their block count when the z th honest block arrives. For $k \geq 0$,

$$\Pr[K = k] = \binom{z+k-1}{k} p^z q^k.$$

Indeed, the last arrival is honest, while the preceding $z+k-1$ arrivals contain $z-1$ honest arrivals and k adversarial arrivals. Their possible orders give the binomial coefficient.

Conditioning on K and using the catch-up probability gives, for $q < p$,

$$P_z = 1 - \sum_{k=0}^{z-1} \binom{z+k-1}{k} p^z q^k (1 - (q/p)^{z-k}). \quad (5)$$

The finite sum combines the $K < z$ cases; $K \geq z$ already reaches a tie. At $q = 0.1$ and $z = 6$, it evaluates exactly to 0.00059141216, approximately 0.059141216%. At $z = 1$ it gives 0.2, not q/p : the latter assumes a fixed known deficit and ignores the attacker's work during the wait. The draft's numerical check uses exact rational arithmetic.

6.3 What a confirmation means

A confirmation policy selects an acceptable risk under assumptions about the adversary and network. It cannot turn (5) into an unconditional finality guarantee. The model also says nothing about a client whose peers control and restrict its entire network view (an *eclipse*), or about a transaction that fails its payment proof.

The complete chain of reasoning is therefore conditional and layered: validated transactions produce valid branch-local state; valid blocks supply admissible work; the most-work rule selects among visible valid branches; a stated network and adversary model supports a probability of later reversal. Each layer answers a different question.

Evidence and executable checks. The concrete history rules are ZIP 221, ZIP 244 and the conditional Ironwood extension in ZIP 258; implementation checks use `zcash_history` and `zebra-chain/src/history_tree.rs`. The public specifications are available at ZIP 221, ZIP 244 and ZIP 258. The companion `check_deployed.py` checks payment signs, compact integers, target rounding, difficulty examples, the confirmation sum, history positions and record-size arithmetic. These checks do not prove hash hardness, honest-majority assumptions, or the correctness of every implementation of the consensus rules.

Part V

Ironwood Guide: Constructing and validating a shielded payment

Abstract

This volume of *The Zcash Arboretum* constructs a private payment: the sender’s preparation, the recipient’s recovery, and the public checks that permit the payment to enter the accepted record. Orchard is the payment construction; Ironwood names a separately accounted use of that construction under the specified upgrade rules. It assumes the *Math Guide*’s field and curve constructions, the *Crypto Guide*’s commitments, signatures and encryption, the *Halo 2 Guide*’s proof contract, and the *Consensus Guide*’s branch-local ledger contract. Each derived object has an explicit type, purpose and consumer. The distinction between a specified upgrade and an activated network rule is maintained throughout. This exposition is non-normative.

1 The shielded payment contract

The public ledger needs an authorised, value-preserving state transition. A ledger, note, pool, anchor and nullifier have the interfaces introduced in the *Consensus Guide*, “Ledger state and transaction acceptance”. For the concrete primitives we use the *Crypto Guide*, “Groups, commitments, and Sin-semilla”, “RedPallas signatures”, and “Key agreement and note encryption”. A proof’s public inputs, private witness and acceptance relation are constructed in the *Halo 2 Guide*, “From a relation to a circuit”. A private payment must supply evidence of that transition without naming the input note or publishing its value and recipient. Four mechanisms divide the work: commitments represent hidden notes; nullifiers identify repeated spends; proofs establish the private arithmetic relation; and signatures authorise the transaction. Encryption delivers the new notes. The node’s branch-local state supplies the final existence and uniqueness context.

This division matters. An accepting proof is not, by itself, a valid transaction. It need not establish possession of a spending secret, correct note delivery, absence of a previous nullifier, or the validity of another pool’s component. The full acceptance contract of the *Consensus Guide*, “Ledger state and transaction acceptance”, remains in force.

1.1 Protocol, pool and rule set

The *Orchard protocol* is the shared cryptographic construction. A *pool* is a separate accounting and state namespace: its own note tree, anchors, nullifier set and public value balance. The original Orchard pool and the specified Ironwood pool use the same key and address family. Sharing addresses does not merge their trees or spend histories.

An *Action* is a record pairing one old note with one new note. Either side can have value zero to fill an otherwise unused slot. A *bundle* collects one pool’s ordered Actions and their shared validation data. These are the units of construction throughout the book. The *lead byte* is the first byte of a delivered note’s plaintext and identifies its format. A transaction *digest* is a hash of prescribed transaction data. A *cross-address restriction* requires the old and new receivers within each Action to be equal.

Two rule profiles will be explicit:

- The Orchard-pool construction uses note plaintext lead byte 0x02 and the version-5 transaction digest structure.
- The specified NU6.3 profile adds Ironwood, lead byte 0x03, version-6 digests and the cross-address restriction used to seal the original Orchard pool.

The latter applies only where and when its defining consensus rules activate. The guide does not infer a network’s activation state from a crate version or a ZIP status.

The principal source pin is `zips commit 69610984`; the implementation references are `orchard 0.15.0`, `librustzcash commit e30517e4`, and the `orchard development tree bef8a27` where explicitly named. The public ZIP 229, ZIP 258 and ZIP 2005 describe the specified Ironwood changes. The version-6 details below use the published ZIP 229 revision consulted on 6 September 2026, including its explicit version group and reserved-bit rules; those later clarifications are not attributed to the older local ZIP commit. Their status is not an activation certificate.

1.2 Types and a continuing example

The two primes and canonical encodings are constructed in the *Math Guide*, “Pasta curves and their encodings”. Let $p = p_{\text{Pallas}}$ and $r = p_{\text{Vesta}}$. Pallas points form a prime-order group of order r , with coordinates in $\mathbb{F}_p$; the Action circuit is over $\mathbb{F}_p$. In particular $p < r$. Converting a canonical integer in $[0, p)$ into a scalar in $\mathbb{F}_r$ is an injective conversion of integer representatives, not a field homomorphism. For example, $(p - 1) + 1 = 0$ in $\mathbb{F}_p$, whereas the same integer representatives sum to $p \neq 0$ in $\mathbb{F}_r$.

The notation $\text{LE}_k(x)$ denotes k little-endian bits. When a byte hash consumes such a string, pack each consecutive eight bits into a byte, least significant bit first. We write $\text{enc}_n(x)$ for an unsigned integer’s n -byte little-endian encoding; for a field or scalar at width 32, encode its canonical representative. Signed encodings use the explicitly marked two’s-complement convention. Write P^* for a compressed 32-byte Pallas point encoding. The coordinate extractor is $\text{Extract}(P) = x(P)$ for an affine point and $\text{Extract}(\mathcal{O}) = 0$. It is not a point encoding: P and $-P$ have the same extraction.

The continuing example spends an 80,000-zatoshi note, pays 50,000, creates 20,000 change and pays a 10,000 fee. It uses two Actions: one pairs the real input with the payment; one pairs a zero-valued dummy input with change. Their private net values are +30,000 and −20,000. These are checked arithmetic examples, not a claim that the text contains a generated production proof.

2 Keys and viewing capabilities

An address lets another party create a payable note. A viewing key discovers and interprets notes. A spending key authorises their consumption. The construction must bind these capabilities together without publishing the secret authorising scalar.

2.1 Expansion and spend authority

For a 32-byte secret s and byte string t , define

$$\text{PRFexpand}_s(t) = \text{BLAKE2b}_{512}(\text{Zcash_ExpandSeed}, s || t).$$

This uses personalised *unkeyed* BLAKE2b with a secret prefix, not BLAKE2b’s native keyed mode. Its required pseudorandom-function property is an assumption discussed in the *Crypto Guide*, “Hash functions and key derivation”. Define ToBase and ToScalar by reading the 64-byte output little-endian and reducing modulo p and r , respectively. The reduction bias of a uniform 512-bit input is bounded by the target modulus divided by 2^{512} ; PRF-generated inputs add the computational PRF distinction.

From spending key sk derive

$$\begin{aligned} a &= \text{ToScalar}(\text{PRFexpand}_{sk}(0x06)), \\ nk &= \text{ToBase}(\text{PRFexpand}_{sk}(0x07)), \\ r_{ivk} &= \text{ToScalar}(\text{PRFexpand}_{sk}(0x08)). \end{aligned}$$

Let $G = \text{GroupHash}^{\text{Pallas}}(z.\text{cash}:\text{Orchard}, G)$, using the group-hash construction in the *Crypto Guide*, “Hash functions and key derivation”. Reject $a = 0$. If $[a]G$ has odd canonical y coordinate, set $\text{ask} = -a$; otherwise set $\text{ask} = a$. Then $\text{ak} = [\text{ask}]G$ has even y and is nonidentity.

The canonical sign permits $\text{Extract}(\text{ak})$ to identify ak in the next construction. It does not mean that every Pallas point can be encoded by its x coordinate without an additional convention.

2.2 Viewing keys and addresses

The full viewing key is $\text{fvk} = (\text{ak}, \text{nk}, r_{\text{ivk}})$. Instantiate Sinsemilla’s short commitment under domain $z.\text{cash}:\text{Orchard-CommitIvk}$:

$$\begin{aligned} \text{ivk} &= \text{Commit}_{r_{\text{ivk}}}^{\text{ivk}}(\text{Extract}(\text{ak}), \text{nk}), \\ &= \text{ShortCommit}_{z.\text{cash}:\text{Orchard-CommitIvk}, r_{\text{ivk}}}(\text{LE}_{255}(\text{Extract}(\text{ak})) \parallel \text{LE}_{255}(\text{nk})). \end{aligned}$$

The construction and its exceptional value $\perp$ are defined in the *Crypto Guide*, “Groups, commitments, and Sinsemilla”. Reject an output of zero or $\perp$ when deriving a receiving key. A successful $\text{ivk} \in [1, p)$ converts to a nonzero Pallas scalar by this canonical-integer convention.

Let

$$\begin{aligned} R &= \text{PRFexpand}_{\text{enc}_{32}(r_{\text{ivk}})}(0x82 \parallel \text{ak}^* \parallel \text{enc}_{32}(\text{nk})), \\ \text{dk} &= R[0 : 32], \quad \text{ovk} = R[32 : 64]. \end{aligned}$$

The byte slicing uses a half-open interval. A single expansion output supplies both keys. The incoming viewing key in the complete wallet sense is (dk, ivk) : ivk performs decryption, while dk generates the wallet’s indexed diversifiers.

A diversifier d is an 11-byte string. Define

$$g_d = \text{DiversifyHash}(d), \quad \text{pk}_d = [\text{ivk}]g_d, \quad \text{addr} = (d, \text{pk}_d).$$

The diversification hash is $\text{GroupHash}^{\text{Pallas}}(z.\text{cash}:\text{Orchard-gd}, d)$, with its identity output replaced by the fixed result for the empty message in the same domain. Thus the protocol’s diversified base is nonidentity. The raw receiver is $d \parallel \text{pk}_d^*$, 43 bytes. Canonical point decoding and nonidentity checks are necessary when accepting an address from another party.

2.3 Internal scope and capability limits

A wallet can separate externally issued addresses from its internal change addresses while retaining the same spend-authorising scalar. Its internal full viewing key keeps ak and nk and replaces the commitment randomness by

$$r_{\text{ivk}_{\text{int}}} = \text{ToScalar}(\text{PRFexpand}_{\text{enc}_{32}(r_{\text{ivk}})}(0x83 \parallel \text{ak}^* \parallel \text{enc}_{32}(\text{nk}))).$$

Apply the preceding incoming and outgoing derivations to that tuple. Standard spending-key validity includes nonzero, nonexceptional incoming keys for both external and internal scopes. This construction is ZIP 32’s Orchard internal key derivation, implemented in `orchard/src/keys.rs`.

Capability	What possession permits
Receiver	Create payments; it is not an authorisation secret.
Incoming key	Discover and decrypt matching incoming notes; derive addresses when the diversifier key is included.
Full viewing key	Additionally compute nullifiers of known notes and derive the specified outgoing recovery keys; it does not sign spends.
Spending authority	Produce spend signatures; with note and state data, construct complete spends.

An outgoing viewing key recovers only outputs whose sender used that key for outgoing recovery. A sender may deliberately omit that recovery capability. A full viewing key therefore does not reveal every possible outgoing payment of a malicious or independently configured spender. Nor does possession of a public receiver prove that its advertised owner knows the spending secret: the sender can validate encodings, but cannot perform a recipient's private key-generation checks.

3 Notes and their commitments

A note is the private description of value that a later proof will consume. Its commitment must bind the amount and receiving capability while concealing them from a chain observer.

3.1 The committed note

Write a note as

$$\mathbf{n} = (d, \text{pk}_d, v, \rho, \psi, \text{rcm}), \quad 0 \leq v < 2^{64}, \quad \rho, \psi \in \mathbb{F}_p, \quad \text{rcm} \in \mathbb{F}_r.$$

The diversified base $\mathbf{g}_d$ is reconstructed from d . The amount's circuit range is 64 bits; transaction and chain money bounds are separate checks, not a reason to omit this range constraint.

Define its 1086-bit message

$$m(\mathbf{n}) = \mathbf{g}_d \star \|\text{pk}_d\| \text{LE}_{64}(v) \\ \|\text{LE}_{255}(\rho)\| \text{LE}_{255}(\psi),$$

where compressed point bytes are interpreted as the corresponding 256 bits. The note commitment and its published extraction are

$$\text{cm} = \text{NoteCommit}_{\text{rcm}}(m(\mathbf{n})), \quad \text{cmx} = \text{Extract}(\text{cm}).$$

The commitment is `SinsemillaCommit` under `z.cash:Orchard-NoteCommit`. The lower primitive pads the final partial ten-bit chunk according to its fixed-length convention; an application must not insert a byte-alignment pad between ρ and ψ . The point commitment is used in the nullifier equation, while only cmx is appended to the note tree.

For genuinely uniform independent rcm , the randomisation component has the hiding property proved for the commitment primitive. In honest notes rcm is derived from a seed and other note data. Their hiding argument is computational and depends on the expansion function's security; it is not a claim of unconditional hiding of the whole transaction.

4 Nullifiers and authenticated existence

A commitment does not disappear when spent. Removing the input leaf would reveal the spend-to-note link. The ledger instead keeps all commitments and records a separate public tag on consumption.

4.1 The nullifier equation

Let

$$K = \text{GroupHash}^{\text{Pallas}}(\text{z.cash:Orchard}, K), \quad F_{\text{nk}}(\rho) = \text{Poseidon}(\text{nk}, \rho).$$

The Poseidon instantiation is the fixed-length-two P128Pow5T3 construction over $\mathbb{F}_p$, with three state words and two input field elements, constructed in the *Crypto Guide*, “Hash functions and key derivation”. For note $\mathbf{n}$ with point commitment cm ,

$$\text{nf} = \text{Extract}([a]K + \text{cm}), \quad a = \text{repr}_{[0,p)}(F_{\text{nk}}(\rho) + \psi). \quad (1)$$

The sum is first reduced in $\mathbb{F}_p$; its canonical integer a then converts to $\mathbb{F}_r$ for scalar multiplication. Adding instead in $\mathbb{F}_r$ would be a different function.

For fixed note and key components this equation is deterministic. With commitment and key-binding security, a valid spend cannot choose a new tag for the same nonzero note. The public node rejects any repeated tag in the relevant pool, including repeats in the same transaction or block. Determinism alone does not prove that arbitrary maliciously chosen notes have collision-resistant nullifiers; the complete argument also uses the commitment construction and how note uniqueness is established.

Every new note uses $\rho_{\text{new}} = \text{nf}_{\text{old}}$ from its own Action. Since accepted nullifiers are unique within a pool, accepted output notes have distinct creation identifiers there. This includes zero-value outputs. Across two pools there are separate nullifier namespaces; no global uniqueness claim follows merely from this rule.

4.2 Reconstructing a note from a seed

The sender draws a fresh 32-byte r_{seed} . Its first required input is ρ : the nullifier of the old side of the same Action, just constructed in §4.1. A dummy old side still produces a nullifier, so this rule applies even when no real note is consumed.

Derive

$$\begin{aligned} \text{esk} &= \text{ToScalar}(\text{PRFexpand}_{r_{\text{seed}}}(\text{0x04} \parallel \text{enc}_{32}(\rho))), \\ \psi &= \text{ToBase}(\text{PRFexpand}_{r_{\text{seed}}}(\text{0x09} \parallel \text{enc}_{32}(\rho))). \end{aligned}$$

Redraw the seed if the ephemeral scalar esk is zero. For an Orchard lead-byte-0x02 note,

$$\text{rcm} = \text{ToScalar}(\text{PRFexpand}_{r_{\text{seed}}}(\text{0x05} \parallel \text{enc}_{32}(\rho))).$$

For a specified Ironwood lead-byte-0x03 note, instead compute

$$\begin{aligned} t_{\text{rcm}} &= \text{0x0b} \parallel g_d^* \parallel pk_d^* \parallel \text{enc}_8(v) \\ &\quad \parallel \text{enc}_{32}(\rho) \parallel \text{enc}_{32}(\psi), \\ \text{rcm} &= \text{ToScalar}(\text{PRFexpand}_{r_{\text{seed}}}(t_{\text{rcm}})). \end{aligned}$$

The preimage after the seed has 137 bytes: $1 + 32 + 32 + 8 + 32 + 32$. Its fields are byte encodings, unlike the 255-bit field encodings inside $m(\mathbf{n})$. The dependency order is forced: derive ψ before the version-3 rcm . Both versions use the same esk and ψ formulas.

The version-3 change binds the seed derivation to the other note fields as specified in ZIP 2005. It does not make the payment proof, signatures or key agreement independent of discrete logarithms. No alternate recovery protocol is needed to understand these bytes.

If note commitment evaluation fails with $\perp$, honest construction retries with fresh randomness. The formal Action relation's treatment of exceptional Sinsemilla cases is different and is stated explicitly in §6.1.

4.3 The note tree

The note tree has depth 32 and leaf capacity 2^{32} . Leaves are published cmx values in transaction and Action order. Unfilled leaves contain the fixed sentinel $2 \in \mathbb{F}_p$.

It is convenient to count hash height from the leaves. For $j = 0, \dots, 31$, first compute

$$h_j(l, r) = \text{ShortHash}_{\text{z.cash:Orchard-MerkleCRH}}(\text{LE}_{10}(j) \parallel \text{LE}_{255}(l) \parallel \text{LE}_{255}(r)).$$

Then set $H_j(l, r) = 0$ if $h_j(l, r) = \perp$, and $H_j(l, r) = h_j(l, r)$ otherwise. This conversion is the concrete MerkleCRH rule, also implemented in `orchard/src/tree.rs`; an exceptional hash is not a separate non-field object stored in the public tree. Each preimage has 520 bits, exactly 52 ten-bit chunks. The

first combine above leaves uses $j = 0$; the root combine uses $j = 31$. The Sinsemilla short-hash construction, including its exceptional semantics, is in the *Crypto Guide*, “Groups, commitments, and Sinsemilla”. The required exceptional-case hardness prevents an efficient adversary from deliberately obtaining such a zero hash.

For a leaf position $i \in [0, 2^{32})$, let i_j be bit j of i and s_j the sibling at height j . Starting with $r_0 = \text{cmx}$, compute

$$r_{j+1} = \begin{cases} H_j(r_j, s_j), & i_j = 0, \\ H_j(s_j, r_j), & i_j = 1. \end{cases} \quad (2)$$

The path has 32 siblings, or 1024 bytes of field encodings, and a position determining left versus right. The final value r_{32} is the root.

The proof keeps the position and siblings private. Its public anchor identifies a candidate tree, not the spent leaf. This does not make every leaf an equally plausible input: publicly identifiable dummy outputs, already explained spends and external metadata can reduce anonymity.

4.4 Anchors and zero-value inputs

All Actions in one bundle use its common public anchor. The node checks that it is eligible in that pool on the candidate branch. Under the modern rule this is an earlier block’s final treestate, not a tree invented by the prover and not an intermediate state of the same block.

In the ordinary nonexceptional path calculation, the circuit gates membership by old value:

$$v_{\text{old}}(r_{32} - \text{anchor}) = 0.$$

For a nonzero 64-bit old amount, the factor is nonzero in $\mathbb{F}_p$, so the root must equal the anchor. A zero-valued old side may have no membership witness to an accepted note. This permits dummy spends and shielding. It does not permit a positive-value invented input. The Action still has other constraints, a nullifier and a spend signature; dummy does not mean an arbitrary unvalidated byte record.

The formal circuit contract has two additional allowances, stated in the protocol specification’s “Merkle Path Validity” and “Action Statement” notes. Intermediate hash inputs can use 255-bit representatives that reduce to the assigned field values without being their canonical representatives. The membership check may also pass if a hash on the path, including the root, is zero. Write PathOK for this permitted predicate, rather than silently identifying it with an ordinary canonical path recomputation. The zero-hash allowance models incomplete Sinsemilla addition. Its security argument needs the exceptional-case hardness assumption; honest public tree computation still uses the canonical function H_j above.

4.5 Action value and authorisation keys

Before encrypting an output, construct two public Action fields that the delivery and proof algorithms will consume. The first hides its net value. Using the commitment construction in the *Crypto Guide*, “Groups, commitments, and Sinsemilla”, define

$$V = \text{GroupHash}^{\text{Pallas}}(\text{z.cash:Orchard-cv}, v), \quad R = \text{GroupHash}^{\text{Pallas}}(\text{z.cash:Orchard-cv}, r).$$

Their required independence means no efficiently known discrete-log relation, not linear independence in a cyclic group. For Action i , let $d_i = v_{\text{old},i} - v_{\text{new},i}$, sample $\text{rcv}_i \in \mathbb{F}_r$, and publish the net value commitment

$$\text{cv}_i^{\text{net}} = [d_i]V + [\text{rcv}_i]R.$$

A negative integer coefficient denotes the negative of the corresponding positive multiple. The values remain integers even though multiplication uses their residues in $\mathbb{F}_r$.

The second field is a randomised spend-authorisation key. Sample $\alpha \in \mathbb{F}_r$ and set

$$\text{rsk} = \text{ask} + \alpha, \quad \text{rk} = \text{ak} + [\alpha]G = [\text{rsk}]G.$$

Resample if $\text{rsk} = 0$, equivalently if $\text{rk} = \mathcal{O}$, since consensus prohibits that public key. The circuit will connect rk to the old note's receiving authority; a later signature will demonstrate approval under rsk . The zero-valued dummy old side uses a fresh valid spending key of the constructor's choice and follows the same construction.

5 Note delivery and recovery

The proof can establish that an output commitment contains a well-formed note without giving its recipient the information needed to spend it. Delivery is a separate protocol. A malicious sender can create an undeliverable output; ordinary transaction validation does not promise otherwise.

5.1 Encrypting the note

For lead byte ℓ , the note plaintext is the fixed 564-byte string

$$\text{np} = \ell || d || \text{enc}_8(v) || \text{rseed} || \text{memo},$$

where the memo occupies exactly 512 bytes. The first 52 bytes contain all note reconstruction data except ρ , which is taken from the Action nullifier. The format supplies the recipient's diversified base through d ; the recipient recovers pk_d from its viewing key.

Using the nonzero ephemeral scalar already derived, compute

$$\begin{aligned} \text{epk} &= [\text{esk}]g_d, & S &= [\text{esk}]pk_d, \\ k_{\text{enc}} &= \text{BLAKE2b}_{256}(\text{Zcash_OrchardKDF}, S^* || \text{epk}^*). \end{aligned}$$

Encrypt np with ChaCha20-Poly1305, nonce 0^{96} and empty associated data. The result C_{enc} has 580 bytes, including the 16-byte authentication tag. The *Crypto Guide*, “Key agreement and note encryption”, constructs the cipher and its security contract.

The fixed nonce is safe only under the construction's key-freshness conditions. Reusing seed and context can reuse a key; the nonce is not a substitute for fresh output randomness. Honest key generation and seed sampling are therefore part of the encryption security assumptions.

The recipient computes the same shared point because

$$[\text{ivk}]epk = [\text{ivk}][\text{esk}]g_d = [\text{esk}]pk_d.$$

This is a group identity. Confidentiality of the resulting ciphertext is a computational claim under key-agreement, key-derivation and cipher assumptions, not a consequence of the identity alone.

5.2 Outgoing recovery

For Action net value commitment cv^{net} , derive

$$k_{\text{out}} = \text{BLAKE2b}_{256}(\text{Zcash_Orchardock}, \text{ovk} || (\text{cv}^{\text{net}})^* || \text{enc}_{32}(\text{cmx}) || \text{epk}^*).$$

The commitment was constructed in §4.5. Encrypt $\text{pk}_d^* || \text{enc}_{32}(\text{esk})$ using the same authenticated-encryption-with-associated-data (AEAD) convention. The 64-byte plaintext yields an 80-byte C_{out} . An auditor with ovk can recover pk_d and esk , derive the incoming key material and recover the note.

The sender can instead choose a fresh random outgoing key and random outgoing plaintext, withholding outgoing recovery. The ciphertext is not proof that a particular ovk was used. Nor does a decryptable memo authenticate a real-world sender identity.

5.3 Recipient acceptance

For each candidate Action and each incoming key, perform the following:

1. Decode the ephemeral point canonically and apply the required nonidentity check. Compute $[\text{ivk}]_{\text{epk}}$ and the incoming KDF.
2. Open the full incoming ciphertext with AEAD verification. Reject failure and a lead byte not allowed for this pool and format.
3. Parse d, v and rseed , and take $\rho = \text{nf}$ from the same Action. Compute g_d and $\text{pk}_d = [\text{ivk}]_{\text{g}_d}$.
4. Recompute esk, ψ and the correct version's rcm . Check $[\text{esk}]_{\text{g}_d} = \text{epk}$.
5. Recompute the note commitment and require its extraction to equal the published cmx . Only then record a detected note.

The ephemeral-key check binds the recovered seed to the transmitted ephemeral key. The commitment check binds the recovered note fields to the published output. A successful AEAD opening alone is insufficient.

Compact scanning omits the memo and authentication tag and therefore uses the stream-cipher prefix plus these reconstruction checks, not a successful full AEAD verification. It discovers candidate notes against supplied chain data; it does not itself validate that chain.

Shielded coinbase has special public recovery rules. Its prescribed outgoing viewing key is zero, allowing validators and any observer to recover the output plaintext and check its required format and values. The ordinary incoming privacy discussion must not be applied to coinbase outputs without this exception.

6 Actions and the proof relation

An *Action* pairs one old side with one new side. The pairing fixes the new note's creation identifier. It does not disclose to a chain observer which old note is used, or whether either side has zero value.

6.1 Public encoding and private witness

The Action wire record is

$$(\text{cv}^{\text{net}}, \text{nf}, \text{rk}, \text{cmx}, \text{epk}, C_{\text{enc}}, C_{\text{out}}).$$

The first five fields are each 32 bytes; the ciphertexts are 580 and 80 bytes. Thus one Action record is 820 bytes. Its spend-authorisation signature is a separate 64-byte field. The bundle adds shared flags, signed value balance, anchor, aggregate proof and a *binding signature*, the signature enforcing its combined value balance.

For the specified cross-address circuit, the public instance vector is

$$(\text{anchor}, x(\text{cv}^{\text{net}}), y(\text{cv}^{\text{net}}), \text{nf}, x(\text{rk}), y(\text{rk}), \text{cmx}, e_s, e_o, \delta).$$

The point-coordinate representation assigns the identity the pair $(0, 0)$ where it is permitted; other points use their affine coordinates. External parsing must match it. Consensus separately prohibits identity rk , while the value commitment can be identity. There are ten field elements. The flags e_s, e_o enable nonzero spends and outputs. The flag δ disables transfers between different receivers. Consensus and the selected verification key determine which flag values apply; they are not free choices of the witness.

The witness contains the old side's $\text{g}_d, \text{pk}_d, v, \rho, \psi, \text{rcm}$ and point commitment, the new side's $\text{g}_d, \text{pk}_d, v, \psi, \text{rcm}$, the old note's membership path and position, $\text{ak}, \text{nk}, r_{\text{ivk}}$, a spend-key randomiser α , and value-commitment randomness rcv . It does *not* contain $d, \text{rseed}, \text{esk}$, the ciphertexts or ask .

This is why proving can be separated from authorising a spend. The proof does not check the honest seed-expansion or diversification-hash procedures; recipient reconstruction checks the delivered note against them. The original circuit’s nine-element instance omits δ and the additional cross-address condition.

Definition 6.1 (Action relation). The typed witness and public inputs satisfy:

- A1.** The old point commitment equals the note commitment computed from its witnessed fields, or that computation returns $\perp$.
- A2.** With $\rho_{\text{new}} = \text{nf}$, the new commitment’s extraction equals public cmx , or that computation returns $\perp$.
- A3.** Either $v_{\text{old}} = 0$ or the old membership witness satisfies PathOK for the public anchor, including its explicitly permitted representation and exceptional-case semantics.
- A4.** Both amounts are in $[0, 2^{64})$ and $\text{cv}^{\text{net}} = [v_{\text{old}} - v_{\text{new}}]V + [\text{rcv}]R$.
- A5.** The old fields and nullifier key give public nf by (1).
- A6.** The randomised spend key satisfies $\text{rk} = \text{ak} + [\alpha]G$.
- A7.** The old receiving key satisfies $\text{pk}_{\text{d}_{\text{old}}} = [\text{ivk}]g_{\text{d}_{\text{old}}}$ with $\text{ivk} = \text{Commit}_{\text{r}_{\text{ivk}}}^{\text{ivk}}(\text{Extract}(\text{ak}), \text{nk})$, or that short-commitment evaluation returns $\perp$.
- A8.** The spend gate satisfies $v_{\text{old}}(1 - e_s) = 0$.
- A9.** The output gate satisfies $v_{\text{new}}(1 - e_o) = 0$.
- A10.** If $\delta = 1$, both $g_{\text{d}_{\text{old}}} = g_{\text{d}_{\text{new}}}$ and $\text{pk}_{\text{d}_{\text{old}}} = \text{pk}_{\text{d}_{\text{new}}}$ hold as full points.

Witness typing requires ak , both diversified bases and both diversified transmission keys to be nonidentity Pallas points. Unlike honest key generation, the Action witness permits either sign of ak . Its incoming-key commitment uses only $\text{Extract}(\text{ak})$; knowledge of the authorising secret up to sign is sufficient. Scalar representatives used only in an isolated multiplication may be non-canonical when reduction gives an equivalent witness. The nullifier scalar and incoming scalar, whose values also participate elsewhere, need the prescribed canonical decompositions. Field, value-range and boolean constraints are part of the relation.

The Sinsemilla $\perp$ alternatives are real weakenings of the simple mathematical equalities, not successful honest note construction. Excluding adversarial exploitation requires the primitive’s exceptional case hardness argument under its generator assumptions. A proof of the weakened relation alone does not justify dropping these alternatives from a security reduction.

The last condition consists of four field equations:

$$\delta(x(P_{\text{old}}) - x(P_{\text{new}})) = 0, \quad \delta(y(P_{\text{old}}) - y(P_{\text{new}})) = 0$$

for $P = g_d, \text{pk}_d$. Equality of extracted coordinates alone would permit opposite points and would not express equality of receivers.

6.2 How the circuit implements the relation

The *Halo 2 Guide*, “From a relation to a circuit”, constructs the constraint language and its connection to a proof. A *gadget* is a reusable group of circuit constraints implementing one operation. A *limb* is a bounded positional digit, such as a ten-bit digit in base 2^{10} . Here the private integer v is represented by bounded limbs. Range lookups and reconstruction equations ensure that the field value is the

intended 64-bit integer. A signed difference can be represented by a bounded magnitude and sign in $\{-1, +1\}$, so fixed-base multiplication commits to the integer difference with its proper sign.

The note-commitment gadget decomposes its assigned points and field values into the constrained bit sequence corresponding to §3.1, subject to the specification's representation allowances and including the 255-bit boundaries. In *fixed-base multiplication* the point is fixed by the circuit; in *variable-base multiplication* it is supplied as part of the instance or witness. Elliptic-curve constraints implement the fixed-base multiplications for commitments and randomised keys, and variable-base multiplication for the old transmission key. The Poseidon gadget implements $F_{nk}(\rho)$; the tree gadget implements the 32 hashes of (2). Copy constraints tie repeated values to the same assigned witness, rather than allowing independently chosen copies.

A field equation only proves its advertised integer statement after the relevant range and canonicity conditions have been established. Similarly, a scalar-multiplication gadget must constrain its base to the intended point. These are obligations of the actual circuit, not facts supplied by the notation $[a]P$.

The source implementation is orchard 0.15.0, `src/circuit.rs`, with note and incoming-key gadgets in `src/circuit/note_commit.rs` and `src/circuit/commit_ivk.rs`. The instance offsets put δ at index 9. Circuit versions choose different verification keys. A verifier must select by the applicable consensus rules, not let the prover select a weaker historical key.

The aggregate proof covers all Actions of a bundle with their ordered instances. It does not verify the blockchain's preceding proofs. The fixed construction uses 2^{11} circuit rows per Action; aggregating m Actions gives the specified proof byte length $2720 + 2272m$. For two Actions this is 7264 bytes. These implementation parameters describe the pinned proof format, not a universal lower bound on zero-knowledge proof sizes.

7 Balance, digests, and authorisation

The Action relation relates each hidden old value to its paired new value. The transaction needs a single public net balance and explicit approval of the transaction data. Two signature roles complete those obligations.

7.1 Value commitments and integer conservation

Using the already constructed cv_i^{net} , d_i , V and R , for the bundle's signed public balance b compute

$$\text{bvk} = \sum_i cv_i^{\text{net}} - [b]V.$$

If $b = \sum_i d_i$ as integers, the value terms cancel and

$$\text{bvk} = [\text{bsk}]R, \quad \text{bsk} = \sum_i rcv_i \pmod{r}.$$

The binding signature uses the RedPallas binding base R and the transaction signature digest. Honest correctness is the displayed group identity plus signature correctness.

Conversely, an unbalanced set of extracted commitment openings leaves

$$\text{bvk} = [D]V + [\sum_i rcv_i]R, \quad D = \sum_i d_i - b.$$

A sound balance argument must connect a valid binding signature to knowledge or unforgeability under this related public key, and then to the inability to know a nontrivial relation between V and R . It is not valid to infer knowledge of bsk merely from commitment binding. The required signature and proof composition assumptions are recorded in §10.

Finally, a group equation yields equality modulo r . The transaction's bounds lift it to integer equality. With fewer than 2^{16} Actions, 64-bit input and output ranges, and $|b| \leq 2.1 \cdot 10^{15}$,

$$|D| \leq m(2^{64} - 1) + |b| < 2^{81} < r.$$

Thus $D = 0 \bmod r$ implies $D = 0$ as an integer. The Action count bound is also far looser than what the block-size limit permits. Omitting this step would leave a modular conservation claim rather than a monetary one.

7.2 Spend authorisation

The randomised key (rsk, rk) was constructed in §4.5. The circuit proves its public-key equation. The spender then signs the transaction digest under rsk using RedPallas SpendAuth. The exact nonce generation, challenge hash, encoding checks and verification equation are constructed in the *Crypto Guide*, “RedPallas signatures”. With nonce point R_s and challenge c , the verification identity has the form $[s]G = R_s + [c]rk$.

Fresh randomisation makes rk appear unrelated to ak under the stated key distribution, but does not hide every other transaction field. The authorising signature says that the party controlling the corresponding secret approved the signed digest. The proof connects that key to the old note's receiving capability. Neither fact alone is the complete spending-authority argument.

A dummy spend also has a randomised key and a signature. Its key can be freshly generated by the transaction constructor; the membership gate permits its zero old value. Its nullifier must nevertheless be unused.

7.3 The complete version-5 byte envelope

Serialization and hashing are different algorithms: their field order need not be the same. Here is the complete serialization for the Orchard-only payment, using the CompactSize construction from the *Consensus Guide*, “Blocks and branch-local validation”.

First emit the five four-byte little-endian header fields:

$$0x80000005, \quad 0x26A7270A, \quad \beta, \quad 0, \quad \eta.$$

The high version bit is set as required; β is the target height's consensus branch identifier. Lock time is zero in this example; η is the chosen valid expiry height, or zero where disabling expiry is permitted. These fields occupy 20 bytes.

Next emit four single zero bytes: transparent input count, transparent output count, Sapling spend count and Sapling output count. Their arrays are empty, and no conditional Sapling balance, anchor or binding signature is present. Version 5 has no Sprout component. For the nonempty Orchard bundle, emit in order:

1. CompactSize Action count m , followed by the m Action records of §6.1, each 820 bytes.
2. One flags byte, $e_s + 2e_o$, with its remaining bits zero under the version-5 encoding rules.
3. Signed eight-byte value balance b , then the 32-byte anchor.
4. CompactSize proof length $2720 + 2272m$, then exactly that many proof bytes.
5. The m spend signatures, each 64 bytes, in Action order.
6. The 64-byte binding signature.

When $m = 0$, only its zero count is present; all those additional Orchard fields are absent. In a general transaction the transparent and Sapling counts instead govern their complete compatibility

components. Their specialized parser must consume and validate those components before returning control at the Orchard count.

For $m = 2$, the proof length 7264 has a three-byte CompactSize encoding. The full ordinary Orchard-only transaction therefore has

$$20 + 4 + 1 + 1640 + 1 + 8 + 32 + 3 + 7264 + 128 + 64 = 9165$$

bytes. This is the actual format’s size calculation, not a fabricated proof or transaction test vector.

7.4 Version-5 effecting and authorising digests

Write $H_t(m) = \text{BLAKE2b}_{256}(t, m)$ for a fixed 16-byte personalisation t . For all Orchard Actions in order, form

$$\begin{aligned} D_c &= H_{\text{ZTxIdOrcActCHash}} \left(\left\| \left(\text{enc}_{32}(\text{nf}_i) \parallel \text{enc}_{32}(\text{cmx}_i) \parallel \text{epk}_i^* \parallel C_{\text{enc},i}[0 : 52] \right) \right\|_i \right), \\ D_m &= H_{\text{ZTxIdOrcActMHash}} \left(\left\| C_{\text{enc},i}[52 : 564] \right\|_i \right), \\ D_n &= H_{\text{ZTxIdOrcActNHash}} \left(\left\| \left((\text{cv}_i^{\text{net}})^* \parallel \text{rk}_i^* \parallel C_{\text{enc},i}[564 : 580] \parallel C_{\text{out},i} \right) \right\|_i \right). \end{aligned}$$

The component digest for a present Orchard bundle is

$$D_O = H_{\text{ZTxIdOrchardHash}}(D_c \parallel D_m \parallel D_n \parallel \text{flags} \parallel \text{enc}_8^{\text{signed}}(b) \parallel \text{enc}_{32}(\text{anchor})).$$

An absent component instead hashes the empty string under its component personalisation. The hashes of an empty component and a nonempty zero-value component are not interchangeable.

The transaction identifier is

$$\text{txid} = H_{\text{ZcashTxHash_}} \parallel \text{enc}_4(\beta)(D_h \parallel D_t \parallel D_S \parallel D_O).$$

Here β is the consensus branch identifier. The header digest D_h hashes, under ZTxIdHeadersHash , the five four-byte fields in order: version header, version-group identifier, branch identifier, lock time and expiry height. The transparent and Sapling component digests are D_t and D_S . For the all-shielded example with neither component present they are the empty hashes under ZTxIdTranspaHash and ZTxIdSaplingHash . When those components are present, their complete ZIP 244 rules must be used; this is a compatibility interface, not a permission to commit only their totals.

For a transaction with no transparent inputs, the shielded signature digest equals this transaction identifier. With transparent inputs it uses ZIP 244’s transparent signature digest, committing additionally to their spent output amounts and scripts and using the required `SIGHASH_ALL` mode for shielded authorisations. In particular, the signer needs authenticated previous-output data; an untrusted coordinator’s claimed input total is not enough.

The Orchard authorising-data component is

$$A_O = H_{\text{ZTxAuthOrchaHash}}(\text{proofs} \parallel \left\| \text{spendSig}_i \parallel \text{bindingSig} \right\|_i).$$

The root authorising digest uses personalisation $\text{ZTxAuthHash_} \parallel \text{enc}_4(\beta)$ and the transparent, Sapling and Orchard authorising digests in order. This digest is committed by the block’s authorising-data tree, constructed in the *Consensus Guide*, “Authenticated block history”.

Excluding signatures and proofs from txid avoids a signing cycle: signatures sign effecting data and subsequently enter a separate digest. Ciphertexts remain in effecting data. The proof need not check their content for the authorising signatures to bind their exact bytes.

7.5 The specified version-6 extension

Version 6 adds an Ironwood component after Orchard. Its root effecting and authorising digests have a fifth and fourth component child, respectively. For the affected shielded components the anchor moves from effecting to authorising data. Thus a changed anchor can require a new proof and authorising-data commitment without changing txid or, for the no-transparent-input case, the signed digest.

Its wire header changes to 0x80000006 and version group 0xD884B698; the latter is also the constant V6_VERSION_GROUP_ID at the pinned library source. After the complete Orchard component, serialize the Ironwood component using the same count, Action, flags, balance, anchor, proof-length, proof and signature ordering as above. Its count is always present; the additional fields are present only when that count is nonzero. Moving anchors between digest categories does *not* move their positions in this wire encoding. The flag interpretation is pool-specific and is fixed in §8.1.

The concrete changed personalisations in ZIP 229 are:

Digest	Version-6 personalisation
Orchard component	ZTxIdOrchardH_v6
Orchard authorising component	ZTxAuthOrchaH_v6
Ironwood component	ZTxIdIronwd_H_v6
Ironwood compact Actions	ZTxIdIrnActCH_v6
Ironwood memos	ZTxIdIrnActMH_v6
Ironwood noncompact Actions	ZTxIdIrnActNH_v6
Ironwood authorising component	ZTxAuthIrnwdH_v6

The Orchard Action-part personalisations remain their version-5 values. For each present version-6 bundle, append its anchor after the existing authorising-component fields and omit it from the effecting-component preimage. An absent bundle hashes empty input under its own domains. Sapling’s changed noncompact-spend and authorising domains are ZTxIdSSpendNH_v6 and ZTxAuthSapliH_v6. Other version-5 components retain their defined structure.

Anchor movement does not detach a proof from its anchor: the anchor remains a proof instance, and the full authorising data remains committed by the block. It changes which commitment identifies that proof-bearing transaction representation.

8 Pool rules and complete validation

A bundle is one pool’s ordered Actions, shared flags, anchor, value balance, aggregate proof and signatures. The same transaction can contain both pool components. Each has its own anchor and binding key, while all authorisations bind the complete transaction digest.

8.1 The conditional seal

Under the specified NU6.3 rules, an Orchard-pool coinbase component must be empty, each Orchard Action disables cross-address transfer, and each transaction has $b_{\text{Orchard}} \geq 0$. These rules apply to the pool regardless of whether the transaction uses version 5 or version 6. They permit value to leave the pool and permit retained value to return to the same receiver; they prohibit net entry and ordinary transfers between different receivers within that pool.

For Ironwood, bit 2 is the specified enableCrossAddress flag, and the circuit input is $\delta = 1 - \text{enableCrossAddress}$. Bits 0 and 1 are spend and output enable flags. Reserved bits must be zero. The Orchard component’s bit 2 remains reserved as zero in the specified transaction rules; the node supplies the required cross-address restriction for that pool and branch. A parser accepting a bit pattern does not establish that the whole consensus verifier accepts it.

For a 70,000-zatoshi migration and a 10,000 fee,

$$b_{\text{Orchard}} = 80,000, \quad b_{\text{Ironwood}} = -70,000.$$

The public fee remains 10,000. Pool balances change in opposite directions, but neither pool's tree acquires the other pool's leaves. Keys and receiver encodings need not change merely because the pool namespace changes.

8.2 The node's checklist

For an ordinary transaction, a validating node:

1. Selects the network rules at the candidate block height and checks transaction version, version-group and branch identifiers, canonical lengths and all resource bounds.
2. Parses all field elements, points, signatures and proof encodings with their specified canonicity and identity conditions. Checks flags and the required nonempty input/output structure.
3. Builds each pool's ordered public instances, including its anchor and the branch-required cross-address semantics. Verifies the aggregate proof under the selected verification key.
4. Recomputes the correct version's transaction signature digest. Verifies every spend signature and each bundle's binding signature under its recomputed binding key.
5. Checks each anchor against eligible earlier final treestates in the correct pool and branch. Rejects repeated nullifiers within the transaction, earlier transactions in the block, or the parent state.
6. Validates every other component under its own rules. Checks transparent input availability and maturity, lock time, expiry, money ranges, fee, pool restrictions and resulting chain balances.
7. Proposes pool-local nullifier insertions and commitment appends in their defined order. Checks capacity and commits them only with the successfully validated whole block.

Coinbase replaces the ordinary spend and fee conditions with its special issuance and recovery rules. A node still verifies its applicable proofs, bindings and public accounting.

The ledger uses explicit lookup for nullifier freshness. In principle a different protocol could prove nonmembership relative to a committed state. It is inaccurate to say that global-history checks can never be represented in a proof; that is simply not how this acceptance contract is divided.

9 One payment, end to end

Alice has a detected, unspent 80,000-zatoshi note and its path to an eligible anchor. Bob supplies a validated receiver. Alice also derives an internal receiver for change. Choose a pool and branch profile that permit both resulting recipients; the sealed original Orchard pool would not permit this ordinary cross-address transfer after its seal.

1. Reconstruct the old note commitment and nullifier from its recorded seed, creation identifier and keys. Confirm locally that its path gives the intended anchor.
2. Construct a fresh zero-valued dummy old note, with its own valid key material and nullifier. The two nullifiers must differ and neither may already be spent in the pool.
3. Pair the real old side with Bob's 50,000 output and the dummy side with Alice's 20,000 output. Each output takes its own paired old nullifier as ρ .

4. Sample fresh output seeds and derive note randomness in the correct version's order. Construct the two commitments. Choose fresh rcv_0 , rcv_1 and α_0, α_1 , retrying forbidden identity randomised keys.
5. Form the net commitments

$$cv_0 = [30,000]V + [rcv_0]R, \quad cv_1 = [-20,000]V + [rcv_1]R.$$

Set the bundle public balance to 10,000.

6. Encrypt both notes and optional outgoing recovery records. Assemble the Action effecting fields, shared flags and anchor. Generate the aggregate proof from the ordered witnesses and instances.
7. Recompute the complete transaction signature digest. Sign both Action keys and the binding key

$$bvk = cv_0 + cv_1 - [10,000]V = [rcv_0 + rcv_1]R.$$

8. Serialize and run the complete local verification checks before submission. Retain the transaction identity and private output metadata needed for recovery.

The public Action count is two; it does not label the dummy spend. The public balance reveals the fee for this all-shielded transaction. Bob detects his note through incoming decryption, not from an explicit recipient field. Alice detects change under internal scope and later recognises consumption of her original note through its nullifier. Once mined, both commitments occupy definite positions in the selected branch's pool tree. A later spend proves membership without disclosing those positions.

10 Security boundaries

The construction supports a precise conditional argument, not a blanket claim that a transaction reveals nothing except validity.

10.1 What is mathematical

The following conclusions are identities or elementary implications: honest key agreement gives the same shared point; the randomised secret and public key match; correctly balanced value commitments cancel to the binding key; a nonzero bounded old value activates membership; bounded modular equality lifts to integer balance; and identical accepted nullifiers are rejected by the ledger's uniqueness rule.

These facts do not establish that a malicious prover cannot violate the premises. The step from an accepting proof to a witness uses knowledge soundness, and the step from a signature to authorised spending uses a signature security argument with the relevant related-key structure.

10.2 What is assumed or imported

The application uses the commitment binding and exceptional-case assumptions of the *Crypto Guide*, "Groups, commitments, and Sinsemilla"; the expansion and nullifier-PRF assumptions of "Hash functions and key derivation"; the signature contract of "RedPallas signatures"; and the proof contract of the *Halo 2 Guide*, "Knowledge soundness and zero knowledge". For extracted malicious witnesses, a balance proof must compose those contracts with the binding-signature argument and the ledger's integer bounds.

Nullifier unlinkability additionally depends on the distribution of honestly generated note data and the appropriate PRF- or group-based argument. The protocol design gives alternative assumption routes; it does not make unlinkability unconditional when viewing keys, note data or addresses are disclosed.

Zero knowledge concerns the full joint proof transcript, with its public instances fixed. It does not hide the public instances, signatures, ciphertext lengths, pool balances or network behaviour. Marginally random-looking columns or commitments are not a substitute for the simulator and composition hypotheses of the proof system. This volume imports that proof-system contract rather than claiming a new complete simulator for deployed Halo 2.

10.3 What remains visible or unavailable

A chain observer sees transaction shape, pool membership, public balances, anchors, nullifiers, randomised keys, output commitments and ciphertexts. Public transfers, timings, repeated external addresses and application disclosures may correlate them. Coinbase plaintexts have the explicit public-recovery exception. A recipient learns its received note and sender-chosen memo, not an authenticated real-world sender identity supplied by the payment protocol.

A malicious sender can withhold delivery or outgoing recovery while creating a consensus-valid output. A malicious wallet server can omit data or present an incomplete branch view unless the client has an independent authentication and availability strategy. A reorganisation can remove a valid payment from the selected branch. None of these events contradicts proof zero knowledge or commitment binding.

Construction evidence. The key, note, nullifier, encryption, tree and circuit implementations are orchard 0.15.0 files `src/keys.rs`, `src/note.rs`, `src/note/nullifier.rs`, `src/note_encryption.rs`, `src/tree.rs` and `src/circuit.rs`. `Cargo.toml` identifies this registry release as version 0.15.0; its `.cargo_vcs_info.json` records source revision 8995ee7e. The development checkout bef8a27 is a separate source, not that release. The versioned digest rules are ZIP 244 and ZIP 229, implemented by `zcash_primitives/src/transaction/txid.rs` and `zcash_primitives/src/transaction/sighash_v5.rs` with the versioned extension paths. The companion check verifies the example's arithmetic, byte and bit widths, personalisation lengths and proof-size formula. It does not generate a production transaction or certify the imported cryptographic assumptions.

Part VI

Wallet Guide: From a seed to an authorised payment

Abstract

This volume of *The Zcash Arboretum* constructs the wallet side of a modern shielded payment, from reproducible secret keys to a transaction whose exact payments its signer has approved. It assumes the *Math Guide*’s encodings, the *Crypto Guide*’s key derivation and wallet permutations, the *Consensus Guide*’s branch-local acceptance rules, and the *Ironwood Guide*’s complete payment construction. The endpoint is an algorithm that turns a selected set of spendable notes and a payment intent into a verifiable transaction, while preserving the distinctions between authority, privacy and chain state. The exposition is non-normative and does not treat wallet policy as consensus.

1 From seed to account

The payment protocol needs a spending key. A wallet additionally needs a recoverable organisation of many keys: distinct accounts, different networks and a sequence of receiving addresses. A deterministic key tree makes those objects reproducible from a single secret seed without making the seed public.

The underlying key and note types come from the *Ironwood Guide*, “Keys and viewing capabilities”, and its complete construction in “One payment, end to end”. This volume’s byte hashes and fixed-width encodings use the *Crypto Guide*, “Hash functions and key derivation”, and the *Math Guide*, “Language, integers, and encodings”. Those constructions precede the derivations below.

A *seed* here is the exact secret byte string accepted by ZIP 32, not the user interface’s written representation of it. *Entropy* quantifies a random choice’s unpredictability. For guessing resistance, k bits of *min-entropy* means no single seed has probability greater than 2^{-k} ; a uniform 32-byte seed meets $k = 256$. The probability language is constructed in the *Math Guide*, “Probability and computation”. ZIP 32 requires a length between 32 and 252 bytes and recommends at least 256 bits of entropy. Length alone does not establish entropy: thirty-two predictable bytes are not a secure seed. If an application represents the seed through words and a passphrase, its exact conversion and normalisation rules are part of recovery. A different passphrase can derive an entirely different wallet without any parsing error.

The source baseline is ZIP 32 at zips commit 69610984, orchard 0.15.0 src/zip32.rs, and librustzcash commit e30517e4. This is a construction pin, not a statement that every wallet implements every newer rule set described below.

1.1 Master and hardened child derivation

A *master* key begins the derivation tree. A *child* is obtained by applying a specified function to a parent and index. *Hardened* means this function consumes the parent’s spending secret, not merely its public or viewing data.

An extended Orchard spending key is (sk, c) : a 32-byte spending key and a 32-byte *chain code*, a separate derivation secret. From seed S , compute

$$I = \text{BLAKE2b}_{512}(\text{ZcashIP32Orchard}, S), \quad sk_m = I[0 : 32], \quad c_m = I[32 : 64].$$

The personalisation is exactly 16 bytes. The chain code is not an additional amount of independently sampled entropy; both halves derive from the same seed.

For a child index i satisfying $2^{31} \leq i < 2^{32}$, set

$$I = \text{PRFexpand}_{c_{\text{par}}}(\text{0x81} \parallel \text{sk}_{\text{par}} \parallel \text{enc}_4(i)), \quad (\text{sk}_i, c_i) = (I[0 : 32], I[32 : 64]).$$

The expansion function is constructed in the *Ironwood Guide*, “Keys and viewing capabilities”. A non-hardened index $i < 2^{31}$ is rejected. The generic ZIP 32 mechanism also has optional tag extensions; ordinary Orchard derivation uses no additional tag or lead-byte payload, which preserves the displayed preimage.

The notation j' means the encoded index $j + 2^{31}$, for $0 \leq j < 2^{31}$. Hardened derivation consumes the parent spending secret, so an exported full viewing key cannot derive sibling or descendant account spending keys. It can still generate diversified receiving addresses within the account as described below.

A derived 32-byte spending key must pass the protocol’s validity tests: a nonzero authorising scalar and valid nonzero incoming-viewing derivations in both external and internal scopes. A failure is not permission to silently invent a different derivation at the same path. The API must return failure or follow a separately specified retry/allocation rule that recovery reproduces.

1.2 Account paths and scopes

The standard account path is

$$m_{\text{Orchard}}/32'/\text{coin_type}'/\text{account}'.$$

For Mainnet, the coin type is 133; test networks use 1. The account index starts at zero and is conventionally allocated sequentially. Account a is an independent bucket of spending authority, not an output position or a diversifier index.

For Mainnet account 9, the three child indices are

$$\text{0x80000020}, \quad \text{0x80000085}, \quad \text{0x80000009}.$$

Each child applies the displayed derivation to the preceding extended key. The resulting sk feeds the complete protocol key hierarchy in the *Ironwood Guide*, “Keys and viewing capabilities”: spend scalar, full viewing key, incoming key, diversifier key and outgoing key.

Internal change scope is not an extra child-path level. It uses the protocol’s 0x83 derivation of $r_{\text{ivk_int}}$ while retaining the spend-authorising key and nullifier key. The resulting incoming and outgoing viewing keys differ. A wallet should record which scope detected a note, since it determines the receiving-key derivation and recovery capabilities for later use.

The *wallet birthday* is the earliest height from which recovery must scan for possible receipts. An account’s derivation path, network and wallet birthday are not secret substitutes for its keys, but can be sensitive metadata. Export only the capability the consumer needs. A view-only wallet should not receive an extended spending key merely to avoid defining a smaller export.

1.3 Key identifiers are not authority

For raw full-viewing-key encoding $F = \text{ak}^* \parallel \text{enc}_{32}(\text{nk}) \parallel \text{enc}_{32}(r_{\text{ivk}})$, the Orchard fingerprint is

$$\text{BLAKE2b}_{256}(\text{Zcash0rcharDFVFP}, F).$$

Its first four bytes form a lookup tag. A short tag can collide and must not be accepted as proof of key identity or as a substitute for comparing the full key. Even a full fingerprint is only a digest of data, not proof that the caller controls the corresponding spending scalar.

The extended-key binary representation stores depth, parent tag, child index, chain code and spending key. Its widths are $1 + 4 + 4 + 32 + 32 = 73$ bytes. The root uses depth zero, zero parent tag and zero child index. The metadata supports navigation and checks; changing it does not perform a new cryptographic child derivation.

2 Addresses and receiver selection

An account can receive many payments without introducing a new spending key for each address. A *receiver* is the protocol-specific payment target. A *Unified Address* is a checksummed container offering one or more receiver types. The container is not itself a shielded note and is not stored as a recipient field in an ordinary Action.

2.1 Indexed diversifiers

The exact FF1 algorithm, its tweak parameter and its AES primitive are constructed in the *Crypto Guide*, “Wallet permutations and address encodings”, with the byte-field arithmetic in the *Math Guide*, “Byte fields for AES”. Given an account’s diversifier key dk and an index $0 \leq j < 2^{88}$, derive

$$d_j = \text{FF1-AES256.Encrypt}(dk, \epsilon, \text{LE}_{88}(j)).$$

The tweak ϵ is empty. The message is an 88-bit string with the specified little-endian bit ordering.

From d_j , compute g_{d_j} and $pk_{d_j} = [\text{ivk}]g_{d_j}$ by the *Ironwood Guide*, “Keys and viewing capabilities”. Every Orchard diversifier index yields a valid diversified base. The default index is zero. Sequential allocation permits a wallet to reconstruct an issued sequence, while the keyed permutation conceals its visible arithmetic structure under its security assumption.

Invertibility has a precise limit. Anyone knowing dk can invert FF1 on *every* 11-byte diversifier, not only ones belonging to the wallet. To recognise ownership of an entire receiver, recover its index and additionally recompute the transmission key with the account’s ivk and compare it to the supplied pk_d . FF1 inversion alone is not an ownership test.

Different addresses from one incoming key do not require one decryption attempt per address. Trial decryption uses the incoming scalar, then recovers the diversifier from the plaintext. More accounts and scopes, by contrast, can require additional trial-decryption work.

2.2 A canonical Unified Address

For the widely used revision-0 format, receiver typecodes are 0x00 for a transparent public-key hash, 0x01 for a transparent script hash, 0x02 for Sapling and 0x03 for Orchard. A container has at most one item of each type, cannot include both transparent alternatives, and must contain at least one shielded item. The transparent hash receivers are 20 bytes; the two shielded raw receivers are 43 bytes.

Serialize items in ascending typecode order:

$$M_0 = \left\| \left(\text{CompactSize}(\text{type}_i) \right) \right\| \left(\text{CompactSize}(|R_i|) \right) \left\| R_i \right\|.$$

The CompactSize encoding is constructed in the *Consensus Guide*, “Blocks and branch-local validation”. Both typecode and length are bounded by 0x2000000, in addition to the total-size bound. No repeated item, noncanonical integer or trailing unparsed byte is permitted.

Let h be the human-readable prefix in ASCII, padded with zero bytes to 16 bytes. For revision-0 Mainnet Unified Addresses it is the byte 0x75, representing u , followed by fifteen zero bytes. Encode

$$\text{Bech32m}_u(\text{F4Jumble}(M_0 \| h)),$$

using ZIP 316’s extended length allowance rather than the ordinary short Bech32m limit. F4Jumble and Bech32m are constructed in the *Crypto Guide*, “Wallet permutations and address encodings”. Jumbling is a public invertible permutation, not encryption of the receiver. Anyone can decode the address.

The smallest Orchard-only revision-0 item begins with 03 2b and contains its 43 receiver bytes. Its raw size is 45 bytes, and its padded jumble input is 61 bytes. For one transparent hash receiver plus

Sapling and Orchard, the three item lengths are 22, 45, 45, giving 128 bytes after the prefix padding. These lengths determine the exact permutation split; dropping the network padding changes the encoded address.

Decode in reverse: check the recognised network prefix and Bech32m checksum; invert F4Jumble within the accepted length range; verify and remove the exact 16-byte prefix padding; parse all canonical items; validate item ordering, uniqueness, allowed combinations and supported receiver encodings. A valid checksum detects many copying errors, but does not authenticate that the address belongs to the intended human.

Later ZIP 316 revisions add distinct prefix and item semantics. A consumer selects the rules from the encoded revision, not from a guess about its contents. Unknown items are handled by that revision's ignore-or-reject rules, including any must-understand category. The revision-0 construction above must not silently accept a newer container under the wrong parser.

2.3 Selecting a receiver and a pool

For the basic receiver list, preference is Orchard, then Sapling, then the supported transparent alternative. A sender chooses the highest priority receiver it supports from those actually present. Serialisation order is ascending typecode; preference order is different. Sorting the container and choosing its first element would therefore select the wrong receiver.

Receiver support is not just a decoding capability. A wallet must be able to construct a valid transaction for the selected network, target height and pool rules. In the specified Ironwood profile, the existing Orchard receiver type also receives Ironwood-protocol outputs. No extra address typecode is required merely to distinguish those pools. The transaction component and applicable rules select the pool; the note's receiver bytes do not.

A wallet may reject a payment it cannot safely perform. It should not silently replace a supported preferred shielded receiver with a transparent receiver to make its current input-selection code succeed. Such a fallback changes the public information disclosed by the payment.

A payment request must be resolved to exact recipient bytes and an integer zatoshi amount before transaction construction. Decimal parsing must be exact: floating-point rounding is not a valid way to interpret a requested monetary amount. A memo and human label remain separate metadata; neither overrides the receiver selected from the address.

3 Fees, inputs, and change

The wallet starts with an intent and a set of candidate notes. It must choose inputs and a transaction shape together because the shape affects the fee, while the fee affects whether change exists.

3.1 What makes a candidate spendable

For each candidate note, the wallet needs its amount, seed and receiving scope; pool and creation identifier; commitment position; nullifier when available; branch-linked mined location; and a path to a selected eligible anchor. It must also know that the note is not already spent on that branch and is not reserved by another pending transaction the wallet is still pursuing.

A displayed balance is not necessarily selectable value. It may include unconfirmed receipts, notes without available witnesses, notes below a chosen confirmation threshold, or notes in a pool whose transition rules restrict the intended transaction. A view-only wallet can know value that it cannot authorise.

Let the observed selected tip have height H and the next target height be $h = H + 1$. A wallet choosing an anchor k blocks behind that target uses height $a = h - k$, provided the branch contains an eligible root there. A note created later than a is not in that anchored tree. The numerical choice of k is wallet policy; consensus eligibility of a historical root does not impose that policy. All inputs of one bundle must be witnessed against its same anchor.

Different pools have separate roots and tree histories. A transaction using two pools can need two independently constructed witness sets. A common height is convenient but does not make their root bytes equal.

3.2 The conventional fee

ZIP 317's basic conventional fee is

$$f_{\text{conv}} = 5000 \max(2, L) \text{ zatoshis},$$

where L is the total number of logical actions. For the component formats used here,

$$\begin{aligned} L_T &= \max\left(\left\lceil \frac{s_{\text{in}}}{150} \right\rceil, \left\lceil \frac{s_{\text{out}}}{34} \right\rceil\right), \\ L_{\text{Sprout}} &= 2n_{\text{JoinSplit}}, \\ L_{\text{Sapling}} &= \max(n_{\text{spend}}, n_{\text{output}}), \\ L_{\text{Orchard}} &= n_{\text{Actions}}. \end{aligned}$$

The transparent quantities $s_{\text{in}}, s_{\text{out}}$ are the serialized input and output field sizes in bytes, not simply counts. The Ironwood fee revision adds its Action count as another contribution. Only apply additional fee-revision terms when their specified format is in use; unrelated memo-bundle proposals are not part of this construction.

The total is the sum of the relevant contributions. A modern Action pairs one input side and one output side, so it counts once, not twice. Padding is visible in the Action count and contributes to the fee even when its private value is zero.

The conventional fee is policy, not a consensus minimum. Ordinary consensus still requires a valid nonnegative fee computed from the public balance equation in the *Consensus Guide*, “Ledger state and transaction acceptance”. Paying that nonnegative amount does not imply a node will relay or a miner will include the transaction. Following a common fee convention also avoids making a wallet's unusual fee choice an unnecessary fingerprint.

3.3 Padding and the fixed-point problem

Here a *fixed point* means a proposed shape, fee and change amount that agree when each is recomputed from the others. For an ordinary nonempty Orchard-protocol bundle with s real input notes and o intended output notes, a common builder padding policy uses

$$m = \max(2, s, o)$$

Actions. Add zero-valued dummy sides until both lists have m entries. The protocol permits the zero-value membership gate; the minimum of two in this policy is not a general consensus minimum of two Actions.

If payment totals P and selected inputs total V , the prospective change is

$$C = V - P - f(m).$$

But adding a change output can increase o , hence m , hence the fee. A correct selection algorithm must evaluate the actual resulting shape, not calculate a fee once before deciding the outputs.

A *dust policy* specifies how to treat a residual too small for the wallet's chosen minimum economical change output. The threshold and treatment are wallet choices, not new consensus amount limits.

A simple terminating algorithm for one supported pool is:

1. Start with a finite ordered candidate list satisfying the anchor and spendability conditions. State the ordering policy, such as oldest first; it is not a consensus rule.

2. Add a candidate input and compute the shape with no change. If its input total is insufficient for payments and that shape's fee, continue.
3. Evaluate the shape including change and its resulting fee. If $C = V - P - f_{\text{with change}}$ is a permitted positive change amount, retain that shape.
4. If there is exact equality for the no-change shape, retain no change. For a small residual, apply the explicitly chosen dust policy: add it to the fee, reject the proposal, or continue selecting inputs.
5. Recompute all totals, padding and fee from the final proposed inputs and outputs. Return failure when candidates are exhausted.

The input list grows on every retry, so the algorithm terminates. It is not guaranteed to minimise the fee or the number of linked notes. More sophisticated selection can improve those objectives without changing the correctness conditions.

A small note may be unhelpful if adding it raises the fee by more than its value. The two-action grace window means this is not equivalent to rejecting every note below 5000 zatoshis: some such notes add no marginal Action cost in a given shape. Dust decisions depend on the whole proposal.

3.4 The shared payment

For the 80,000 input, 50,000 payment and one prospective change output, the final counts are $s = 1, o = 2, m = 2$. Thus

$$f = 5000 \max(2, 2) = 10,000, \quad C = 80,000 - 50,000 - 10,000 = 20,000.$$

The two-Action shape is stable after including change. A 10,000 fee is not an additional hidden note; it is the positive public net withdrawal from the pool, subsequently available to the block's coinbase under its accounting rules.

The input-to-output pairing can use the real input with the payment and the dummy input with change. The corresponding private differences are 30,000 and $-20,000$, summing to the public balance. The builder may randomise pairing and Action order, but must preserve the resulting note-creation identifiers, ciphertext associations, public instances and signature order.

4 Constructing and authorising a transaction

A *proposal* is an approved choice of inputs, outputs, fee, pool routing and intended validity context. It is not yet a transaction proof or a signature. Freezing this semantic object makes the remaining dependencies explicit.

4.1 Construction order

The *Ironwood Guide*, “One payment, end to end”, supplies the exact cryptographic equations. A wallet executes them in the following order:

1. Resolve and validate recipient receivers, zatoshi amounts and memo bytes. Derive change receivers in the intended account and scope.
2. Select target rules, transaction version and branch identifier. Select input notes, pool-specific anchors and witness paths; compute fee, change and padding to a stable final shape.
3. Reconstruct each old note and nullifier. Generate all required dummy old notes and their fresh keys. Choose the final Action order and old/new pairing.
4. Set each new note's ρ to its paired old nullifier. Sample fresh note seeds, derive esk and ψ , then the correct note-version rcm . Compute new commitments.

5. Sample spend-key randomisers and value blinders. Form randomised keys and net commitments. Derive incoming ciphertexts and optional outgoing recovery ciphertexts using those public Action fields.
6. Assemble the exact effecting-data representation and public proof instances. Generate the proof with the selected circuit and witnesses.
7. Recompute the complete signature digest. Obtain spend signatures and the binding signature. Form authorising-data commitments.
8. Serialize canonically. Independently verify proof and signatures, recompute the fee and check the intended outputs before submission. Store the pending record and reserved inputs atomically.

Generating the proof before the signatures is a convenient implementation order, not a cryptographic dependency in every supported format. Both depend on the final effecting data. A changed recipient, amount, ciphertext, flag or public balance can invalidate the proof, signatures or both; a builder must know which dependency it changed.

For the specified version-6 digest, re-anchoring does not change the effecting-data identifier, but it does change proof instances and authorising data. This limited property must not be generalised to a changed output or to version 5 anchors.

4.2 Separated roles and capabilities

A Partially Constructed Zcash Transaction, or *PCZT*, is a container for coordinating construction before all proofs and signatures exist. Its purpose is to separate capabilities, not to change consensus validity. The on-chain product remains an ordinary transaction.

A constructor selects the intended structure and randomness; a prover needs private witnesses; a signer controls spending authority; an extractor assembles and validates the final serialised transaction. One program may perform several roles. A hardware signer or remote prover can perform just one.

Role	Required information and resulting boundary
Planner	Payments, candidate values, pool rules, anchors and fee policy; can propose a transaction but cannot necessarily authorise it.
Prover	Notes, paths, viewing-key components, randomisers and blinders; learns the supplied witness even though chain observers do not.
Spend signer	Spending authority and enough authenticated construction data to approve the entire signed effect. It need not generate Halo 2 proofs.
Finaliser	Complete effecting and authorising data; verifies and extracts the transaction. Missing approval is not repaired by serialisation.

The Action witness does not contain *ask*. A prover knowing the required note, full viewing key and randomiser can construct the relation without spending authority. A remote prover therefore need not receive *ask*, but it does receive sensitive note data unless an additional privacy mechanism is designed. Proof zero knowledge protects the *verifier* from that witness, not the machine computing it.

For a spend randomiser α , the signer uses $rsk = ask + \alpha$ and returns a signature, not *rsk*. Sending *rsk* to a coordinator who knows α reveals $ask = rsk - \alpha$. Algebraic key randomisation is not a reason to export the randomised secret to an untrusted constructor.

The binding signature authorises the aggregate commitment balance. Its secret is the sum of value blinders when the proposal balances. A multi-party construction must arrange that capability deliberately; calling it a separate role does not by itself define a secure multi-party blinder or signing protocol.

4.3 What the signer must check

An untrusted coordinator can supply internally consistent data describing the wrong payment. The signer must bind its decision to user intent:

1. Check the network, transaction version, consensus branch, expiry and any lock-time semantics against its policy and the intended construction context.
2. Identify each claimed wallet input by its actual note data and key derivation. Recompute the nullifier and randomised public key. Do not trust an account label or short fingerprint as proof of ownership.
3. Independently derive every claimed change receiver and scope. Reconstruct its output note and commitment. A coordinator's label *change* does not make an output belong to the signer.
4. Check external recipients, amounts and memos against the approved payment request. Recompute output commitments and, when delivery is promised, the ciphertext construction from the supplied output seed.
5. Compute integer input and output totals, public pool balances and actual fee. Check padding and resource bounds, and authenticate previous transparent output data if such inputs are included.
6. Recompute the precise signature digest from the verified data. Sign only that digest using the correct Action randomised key.

An implementation can replace some local recomputation with a separately justified proof or authenticated attestation. It cannot replace it with unverified coordinator assertions without changing the trust model.

Signer intent checks and node validity checks overlap but are not identical. A node need not know that Bob was the intended payee. A signer need not download and validate the whole blockchain merely to approve Bob's amount, but should know what state assertions it is trusting. A hardware display that shows only an untrusted host-provided amount does not establish a verified authorisation boundary.

4.4 Submission is an uncertain operation

Persist the transaction and input reservations before handing it to a network service. A timeout after submission is ambiguous: the service may have received and relayed the transaction even though the reply was lost. Retrying the same transaction can be safe; constructing a conflicting replacement without recording that ambiguity can cause inconsistent wallet state.

A successful submission response means the service accepted the request under its own behaviour. It does not prove relay to miners, inclusion, or finality. Track the transaction by its exact identity and later branch-linked mined location. Fees are paid through mined transaction effects, not by the act of requesting broadcast.

5 Tracking payment state and recovery

A wallet maintains an interpretation of the selected chain plus its own pending intentions. These two sources of state must remain distinct.

5.1 Pending, mined, conflicted and expired

A useful local lifecycle distinguishes:

- *Constructed*: complete transaction retained, perhaps not yet submitted.

- *Submitted or pending*: the wallet is pursuing inclusion; selected inputs remain reserved.
- *Mined on the selected branch*: inclusion is recorded by block hash, height and transaction position.
- *Conflicted*: another selected-chain transaction consumes a reserved input or nullifier.
- *Expired*: the active expiry rules no longer permit future inclusion at the relevant target height.

These labels are local interpretations, not extra consensus fields. A transaction can move from mined back to pending after a reorganisation. It can subsequently conflict or expire. A dropped network connection does not prove that a pending transaction was never received elsewhere.

Input reservations prevent accidental concurrent construction using the same notes. They do not change consensus spendability. Release them only under a recorded cancellation, expiry, conflict or replacement policy; restore the appropriate reservations after rollback.

5.2 Reorganisation and witness freshness

The *Consensus Guide*, “Reorganisation and confirmation confidence”, constructs the branch transition. The wallet mirrors its relevant effects. A note received only on a removed branch is no longer a mined input. A spend observed only there no longer proves that a note is spent on the replacement branch. A path must be rebuilt for the selected anchor; a path to a removed root does not become valid by relabelling its height.

Store block hashes with mined transaction records and note positions. After finding the common ancestor, roll back the chain-derived records together: received notes, spend observations, tree checkpoints and transaction confirmations. Preserve separately the user’s payment intent and retained signed transaction. Then evaluate whether that transaction can still be pursued.

A wallet can choose not to spend very recent change, or can impose a longer delay on external receipts than internal transfers. Such choices trade availability for its chosen risk tolerance. The choice should be described as policy, not as a cryptographic guarantee attached to the word *confirmed*.

5.3 What a seed backup can recover

A seed with the correct network and account derivation conventions recovers spending and viewing capabilities. Given complete authentic chain data, the wallet can rediscover decryptable incoming notes and derive their nullifiers to distinguish unspent value. It need not know every previously issued diversifier index before trial-decrypting: the incoming scalar is shared across that indexed address family.

Account discovery is a different problem. An implementation that stops after a fixed number of unused accounts assumes that later accounts were not used. That assumption can miss funds if the user previously skipped far ahead. Record account allocation or document the finite discovery policy rather than claiming that one arbitrary gap limit recovers every possible account.

The wallet birthday is an earliest scan height justified by creation or backup history. Setting it too late can omit received notes permanently from that scan. An earlier birthday costs extra work but does not authenticate the server supplying that history.

Seed-only recovery does not reconstruct arbitrary local labels, invoice associations, contact identities or all outgoing memos. Outgoing recovery requires the sender to have used the corresponding outgoing viewing key and the necessary full ciphertext data to remain available. Pending transactions not present in the recovered chain may need a separate backup if their exact signed bytes or intent matter.

5.4 Acceptance invariants

At the end of construction and throughout recovery, the wallet should maintain three invariants:

1. Every selectable note has coherent pool, branch, note data, position, unspent status and witness availability.
2. Every authorisation signs the exact effecting data approved by its signer, with amounts and change independently checked.
3. Every persisted chain-derived update can be associated with a validated or explicitly trusted branch point and rolled back without corrupting unrelated local intent.

These invariants are a constructive endpoint: they specify what an implementation must preserve while choosing its own database and user interface. They do not prescribe a catalogue of RPC calls or schemas.

Evidence and checks. The key and address constructions are ZIP 32 and ZIP 316; the conventional fee is ZIP 317. The implementation references are `orchard/src/zip32.rs`, `zcash_address/src/kind/unified.rs`, `zcash_client_backend/src/fees`, and the transaction-construction interfaces in `zcash_client_backend/src/data_api/wallet.rs` at the stated source pins. Separated roles are represented by `pczt/src/roles`, but their security depends on the explicit checks above, not the container name. The companion arithmetic check covers the shared fee/change example, encoding widths, and published ZIP 32 master and hardened-child vectors. It is not a production wallet implementation or a proof that a particular hardware signer performs these checks.

Part VII

Sync Guide: Discovering notes and maintaining spendable state

Abstract

This volume of *The Zcash Arboretum* constructs the state machine that turns a stream of chain data into a usable shielded wallet. It assumes the *Crypto Guide*’s authenticated trees, the *Consensus Guide*’s branch-local state and reorganisation rules, the *Ironwood Guide*’s note decryption and membership equations, and the *Wallet Guide*’s account and transaction lifecycle. It explains what data to fetch, how to recognise received money, how to maintain the evidence needed to spend it, and how to undo observations when the selected history changes. It then identifies the remaining server-trust and metadata boundaries. It does not equate successful scanning with independent consensus validation.

1 What a wallet must reconstruct

The blockchain publishes commitments, nullifiers and encrypted note data. Their concrete constructions are in the *Ironwood Guide*, “Notes and their commitments”, “Nullifiers and authenticated existence”, and “Note delivery and recovery”. In particular, the private path of sibling hashes used to prove membership is the note’s *membership witness*. A wallet needs a more useful private view: which outputs pay its keys, what values they carry, whether they remain unspent, and which witness will allow each to be spent. These facts arise from different operations. Decryption discovers a note; commitment reconstruction checks its content; ordering determines its position; nullifier matching observes a spend; and chain validation or an explicit trust assumption establishes the branch in which those observations have meaning.

1.1 A complete wallet record

For each detected note, retain enough information to reconstruct

(pool, account, scope, note, position, creation, nullifier, spend observation).

The note includes its recipient, value, creation identifier and seed, or equivalent fully reconstructed components. Its creation record identifies block hash, height, transaction identity and Action/output index. A spend observation is branch-linked and may be absent. A note nullifier may also be unavailable to a wallet given only incoming viewing authority.

Per pool, retain the current commitment-tree size and enough tree data to produce required paths at chosen checkpoints. A *checkpoint* is a retained tree state associated with a particular block boundary. Its root and size must be bound to a branch identity, not just a height.

These records serve the *Wallet Guide*, “Fees, inputs, and change”. A detected note without a usable witness can contribute to a displayed balance while not yet being selectable for a transaction.

1.2 Inputs and authority

A sequential scanner needs:

1. account viewing keys and the external/internal scopes to search;
2. a starting branch point and its pool tree states;
3. complete relevant commitment and nullifier sequences after that point, in their protocol order;

4. output data sufficient for note discovery;
5. a way to decide whether the supplied branch and its data are authentic, or an explicit trust decision about their source.

The last item cannot be supplied by the first four. Matching an encrypted output to one's key does not establish that the output was mined or that the server showed every spend.

The normative and implementation reference baseline is `zips` commit 69610984, especially the note-decryption rules and ZIP 307, and `librustzcash` commit e30517e4, especially `zcash_client_backend/src/scanning.rs`, `src/data_api/chain.rs` within that crate, and `proto/compact_formats.proto`. The compact schema's inclusion of Ironwood fields is capability evidence. It is not proof that their pool rules have activated on a particular network.

2 Compact discovery

A full transaction contains proofs and signatures that a basic key-discovery scan does not need. A *compact block* is a service representation retaining selected discovery and tree-maintenance data. It is not another consensus block format.

2.1 The information retained

A compact Orchard-protocol Action carries:

Field	Bytes	Purpose
Nullifier	32	Spend detection; creation identifier of the paired output.
Extracted commitment	32	Output reconstruction and tree insertion.
Ephemeral key	32	Incoming key agreement.
Ciphertext prefix	52	Lead byte, diversifier, value and note seed.

These fields total 148 payload bytes before service encoding overhead. The omitted incoming ciphertext contains 512 encrypted memo bytes and the 16-byte authentication tag. The 80-byte outgoing ciphertext, net value commitment, randomised key, proof and signatures are also not part of this compact Action.

A compact transaction supplies its original transaction index and identifier, plus the relevant pool lists. A compact block supplies height, hash, parent hash and pool tree-size metadata where supported. Transaction indices can have gaps when the service omits transactions with no requested relevant components. The retained lists still have to preserve the exact order of commitments and nullifiers.

The specified Ironwood extension uses a distinct Action list and tree size. Its Action payload has the same shape. Do not merge its commitments with the original Orchard list: the two pools have separate position spaces and roots even though their receivers and primitive hashes are shared.

The service uses *Protocol Buffers*, a message encoding with numbered fields and prescribed default values. It can represent an absent field by its default zero or empty value. A scanner must interpret presence and metadata semantics according to the service contract; an omitted tree size is not automatically an authenticated assertion that the pool is empty. The compact block's optional header is not guaranteed to be populated. The baseline schema explicitly describes normal compact headers as empty.

2.2 Decrypting without an authentication tag

First parse the Action's field and point encodings canonically and enforce the required ephemeral-point conditions. Only then, for each incoming scalar `ivk`, compute the shared point `[ivk]epk` and the

KDF from the *Ironwood Guide*, “Note delivery and recovery”. The first 52 ciphertext bytes use the same ChaCha20 keystream prefix as the full encrypted note. Invert that prefix to obtain a candidate

$$(\ell, d, v, \text{rseed}).$$

There is no Poly1305 tag in the compact data, so this is not a successful authenticated-encryption-with-associated-data (AEAD) opening. Correctness is tested by reconstructing the note:

1. Require canonical outer field and point encodings and the appropriate ephemeral-point conditions.
2. Check that lead byte ℓ is allowed for the pool’s note format.
3. Set ρ to this Action’s nullifier. Derive g_d from d and $\text{pk}_d = [\text{ivk}]g_d$.
4. Derive esk and ψ , then the correct version’s rcm . Require the derived ephemeral point to match the supplied one.
5. Recompute the commitment and compare its extraction with the Action’s cmx .

Only a candidate satisfying those checks becomes a detected note. The security argument is the note-reconstruction contract, not the missing AEAD tag. When the wallet later retrieves the full transaction, it performs full ciphertext authentication and checks that the retrieved output belongs to the same committed transaction context.

A malformed compact record is not a well-formed record that merely decrypts to somebody else’s key. A production scanner must distinguish structural failure from a harmless no-match result. It must not silently omit a malformed commitment and shift every subsequent note position. If the service does not supply trustworthy complete data, the safe response is to reject or repair the range, not mark it completely scanned.

2.3 Incoming discovery and spend discovery

For k incoming scopes and m output records, straightforward discovery performs km candidate key agreements and reconstruction attempts. Batching and prepared-key operations can reduce cost without changing which key/output pairs must be considered. Deriving many diversified addresses under one scope does not multiply k .

For an account with a full viewing key, a detected note’s nullifier is computed by the *Ironwood Guide*, “Nullifiers and authenticated existence”. Match that value against all observed nullifiers in the same pool. A match means the note was consumed on the supplied branch. Its Action pairing does not tell a recipient which input financed its output: the incoming nullifier also serves as the output’s creation identifier, but the old commitment remains private.

An incoming-only key lacks the nullifier derivation capability. It can discover received value without independently reconstructing ordinary spentness for those notes. Exporting an incoming key and calling the result a complete unspent balance would promise more than that capability permits.

3 Commitment positions and witnesses

The proof needs a path to an eligible root. The encrypted note does not contain that path or its position. They arise from the ordered public commitment history.

3.1 Positions are arithmetic

Suppose a pool contains n commitments before a block, and the block appends c commitments. Their positions are

$$n, n + 1, \dots, n + c - 1, \quad n_{\text{after}} = n + c.$$

Every Action contributes its output commitment, including a zero-valued dummy output. Counting only notes detected by this wallet is incorrect.

For a transaction with several Actions, add the count from all previous transactions' same-pool Actions to n , then add the Action's within-list index. If reliable end-of-block metadata gives n_{after} , one can derive $n = n_{\text{after}} - c$, checking underflow and continuity with known states. This arithmetic locates leaves; it does not authenticate the metadata or prove that c includes every required commitment.

For the shared payment, suppose the pool had 11 leaves before the transaction. Its payment and change occupy positions 11 and 12 in the chosen Action order, even if Bob discovers only the first and Alice only the second. A preceding transaction's commitments would shift both positions by its count. A reorganisation can change these positions.

3.2 Empty roots and the incremental frontier

Use the concrete height-indexed hash H_j from the *Ironwood Guide*, “Nullifiers and authenticated existence”. Let $e_0 = 2$ be the empty-leaf sentinel and define

$$e_{j+1} = H_j(e_j, e_j) \quad (0 \leq j < 32).$$

These are the roots of completely empty subtrees of increasing height.

A *frontier* stores the roots of completed left subtrees needed to continue appending. For $n < 2^{32}$ leaves, its occupied heights are the set bits of n . At height j , the retained value F_j covers the corresponding completed subtree in the ordered prefix. This is binary carrying with subtree hashes instead of digits.

To append a leaf L , first require $n < 2^{32}$. Set $x = L$, $j = 0$ and $t = n$. While t is odd, combine $x \leftarrow H_j(F_j, x)$, remove F_j , set $t \leftarrow \lfloor t/2 \rfloor$ and increment j . When t is even, store $F_j = x$ and increment the original count. At the exact full capacity, retain the resulting height-32 root as a terminal full-tree state; there is no next append.

Example 3.1 (A binary carry). For $n = 11 = 8 + 2 + 1$, frontier heights are 3, 1, 0. Appending the twelfth leaf first joins the old height-zero subtree, then joins the old height-one subtree, producing a new height-two subtree. The frontier heights become 3, 2, matching $12 = 8 + 4$. No earlier leaf is rehashed.

For a nonfull tree, recover the padded depth-32 root without storing empty leaves. Start $x = e_0$. At each height $j = 0, \dots, 31$, set

$$x \leftarrow \begin{cases} H_j(F_j, x), & \text{if bit } j \text{ of } n \text{ is } 1, \\ H_j(x, e_j), & \text{otherwise.} \end{cases}$$

The current x is the partial rightmost subtree of height j . A set bit supplies its completed left sibling; an unset bit supplies an empty right sibling. Induction on height proves that after the final step x is the root of the prefix padded to capacity.

The full frontier needs at most one field element per height and the count. It is sufficient to append and compute roots. It is *not* sufficient to reconstruct an arbitrary old leaf's membership path: hashes deliberately discard the child information. A wallet retaining owned notes needs additional marked subtrees or witness data.

3.3 Retaining paths for marked leaves

Mark a leaf when its note is detected. A simple complete implementation retains all tree nodes; it can answer a path query by retrieving the sibling subtree root at every height. A smaller wallet retains only the nodes needed for marked leaves and checkpoints, collapsing unrelated completed subtrees to their roots.

For a marked position i and a height j , its sibling interval begins at

$$a_j = ((\lfloor i/2^j \rfloor) \text{ xor } 1)2^j$$

and covers 2^j leaves. The sibling value is the root of that interval, padded with the appropriate empty roots where the interval extends beyond the checkpoint's populated prefix. Return these 32 values with i ; the verifier recomputes the root using the bit-directed path equation.

An append at position $p > i$ can change only the sibling subtree whose interval contains p . Its height is the highest set-bit index of $i \text{ xor } p$. Update that subtree's partial root and retain the other siblings. A root-only collapsed subtree entirely outside all future queried paths needs no retained interior.

This retention policy has a limit. If a wallet later imports a new key and discovers an old note inside a subtree whose interior was discarded, that subtree root cannot reveal the missing path. The wallet must retrieve the necessary earlier commitments or a correctly authenticated witness. Small storage is achieved by choosing what future queries to support, not by inverting hashes.

3.4 Shards and checkpoint roots

Partition the tree into subtrees of fixed height s , called *shards*. Shard j covers $[j2^s, (j+1)2^s)$. A shard containing an owned note retains its required interior. A complete shard containing no marked note can be represented by its root. The tree above shard roots retains their ordering and connects them to the full root.

To construct a witness, concatenate the path inside the note's shard with the path of that shard root through the upper tree. The lengths are s and $32 - s$, totalling 32. For example, $s = 16$ divides the path into two sixteen-hash parts; it is a storage choice, not a new consensus tree depth.

A service may provide completed subtree roots to accelerate bootstrap. A correct root for an earlier subtree can replace replay of its interior when that interior is not needed for a marked leaf. The wallet still needs trustworthy placement, completion boundary and branch association. An unverified root supplied by the same untrusted server is a shortcut within that trust model, not an independent security proof.

A checkpoint must also retain enough of the partially filled current shard to reproduce that block's historical root. Keeping only today's root for a mutable partial shard cannot answer every older anchor query. Pruning therefore follows the chosen checkpoint and spend policy: do not remove the only information needed by a still-pending transaction or a supported rollback boundary.

4 Atomic scanning and rollback

A scan updates note records and tree state together. An *atomic* update makes all of its writes visible together or none of them. Their consistency is more important than whether the implementation processes one block or many blocks per database transaction.

4.1 The sequential state machine

Let the current scan checkpoint be $(h, b, \{n_P, F_P\}_P)$, with block hash b . For a next compact block B , perform:

1. Require expected height and parent hash, when these are supplied by the protocol, and validate all structural bounds and encodings. Compare against independently authenticated block information where available. Missing required context is not a completed scan.
2. Work in temporary state. For each pool, determine the starting tree size, checking consistency against prior state and the complete ordered commitment count.

3. Traverse compact transactions in original block order. Match nullifiers against tracked notes and collect spend observations. For every output, run candidate decryption for each relevant scope.
4. Append every commitment in pool order. A detected note receives exactly this append's position; mark that position and compute its nullifier when the supplied key capability permits.
5. Reconcile the block's collected nullifiers with detected notes, verify tree-size metadata, and derive the final roots and checkpoint. If an authenticated final root is available, compare it.
6. Atomically persist detected notes, spend observations, tree changes, transaction metadata and the new branch checkpoint. Mark the range scanned only after those writes succeed.

A rejected block or interrupted transaction leaves the previous checkpoint intact. Replaying an already committed range must be idempotent: unique block/transaction/output identities prevent duplicated notes and duplicated append effects.

The wallet's pool root after a block can be authenticated indirectly by the later history commitment described in the *Consensus Guide*, "Authenticated block history". That path must include the correct epoch, node metadata and header commitment. A root merely written in a compact message is not equivalent evidence.

4.2 Parallel and out-of-order work

Trial decryptions for distinct key/output pairs can run independently. Downloaded ranges can be cached and processed out of order if their tree positions and boundary states are already known reliably. The final database must nevertheless describe one coherent branch.

For a disconnected range $[a, b]$, do not guess its starting tree size from the last locally scanned height. Obtain a boundary state or derive the size from consistent authenticated metadata and exact counts. The scanner may know that it found a note in that range before it can produce the note's complete witness. Record this partial knowledge honestly rather than mark the whole account fully synchronised.

A scan scheduler can prioritise recent blocks to discover recent receipts, then fill older gaps. Its priority is local policy. The highest scanned height is not a sufficient progress measure: an unscanned gap can hide a note or spend. Track completed intervals and their branch associations, and expose whether the contiguous required range is complete.

The source implementation's separation of scan ranges, note discoveries and retained tree data reflects these different kinds of knowledge. The explanatory algorithm does not require a particular RPC inventory or database schema to preserve them.

4.3 Detecting and applying a reorganisation

A mismatching parent hash or a disagreement with an authenticated tip signals that the stored branch may no longer match the selected branch. Locate a common ancestor by comparing retained block identities with the replacement branch. A sequence of server replies indexed only by height must not be mistaken for proof that no reorganisation happened.

Roll back chain-derived records beyond the common ancestor:

- remove or unmine notes created only on the removed suffix;
- remove spend observations that occurred only there;
- restore pool frontiers, marked-path state, checkpoint sets and sizes;
- remove stale scanned-range assertions and mined transaction confirmations for that suffix.

Keep the user’s payment intentions and retained signed transactions separate. Re-evaluate them under the *Wallet Guide*, “Tracking payment state and recovery”. A removed spend may become pending again, but can also conflict or expire on the replacement branch.

Example 4.1 (Positions after rollback). Suppose a common ancestor has 11 commitments. One branch appends the two outputs of the shared payment at positions 11 and 12. The replacement branch first appends three other commitments. If the same payment is then included, its outputs occupy positions 14 and 15. The note data and commitment can be unchanged while the position and path change. Reusing the old path would be incorrect.

If the common ancestor predates retained rollback information, restart from an earlier trusted checkpoint and rescan. A retention window is a resource decision. It does not justify continuing with a mixed or unrecoverable state, and it is not a proof of consensus finality.

4.4 Restoring from a birthday

A birthday says where a required scan begins. If a restored account was created before height b , initialise its starting tree state at the chosen preceding boundary and scan all required data from there. The birthday does not reconstruct that initial root or frontier; the wallet must obtain it by replay, an authenticated checkpoint, or a declared trusted source.

A birthday later than a received note can miss the note entirely. A birthday earlier than necessary costs time and bandwidth. Neither fact reveals whether a server omitted a relevant transaction inside the scanned range. Account enumeration and restored incoming scopes must also match the original wallet; correct block scanning under the wrong keys cannot discover the missing account.

5 Server trust and observable metadata

Running decryption locally prevents a server from learning viewing keys merely to provide compact data. It does not make the entire interaction private or independently validated.

5.1 Four independent checks

Separate these questions:

1. *Content consistency*: does a recovered note match a supplied commitment and ephemeral key?
2. *Inclusion*: do the supplied transaction and state data belong to an authenticated block or tree root?
3. *Chain validity and selection*: does that block belong to the selected valid most-work branch under the client’s model?
4. *Completeness and availability*: did the client receive all data necessary to discover its notes and spends?

A commitment check answers the first. A Merkle path against a trusted root can answer a specified inclusion question. Header work alone does not validate hidden transaction proofs or account for unavailable bodies. Even correct individual inclusion proofs do not prove that the server has revealed every relevant item.

A conventional lightweight scan can trust a service for full validation and completeness while keeping keys local. State that arrangement plainly. A full-node-backed wallet can delegate those same claims to a locally controlled validator. An independently authenticating client needs additional verified data and a chain-selection argument. These are different trust configurations, not different levels of confidence in the same unauthenticated reply.

5.2 What requests reveal

A server can observe source network identifiers, connection times, requested heights, range sizes, pauses and retries. A narrow birthday range can reveal approximate wallet creation or restoration activity. A follow-up request for a full transaction immediately after compact scanning can indicate that the wallet found it relevant. A broadcast request links the submitted transaction to the requesting connection unless another communication layer prevents that observation.

The chain itself also exposes transaction sizes, Action counts, pools, public value balances and timing. Coin selection or unusual change shapes can create additional correlations. Hiding note values from the server's compact scan does not erase those public facts.

Transparent address-specific queries reveal the queried addresses to the service. A design that avoids sending shielded viewing keys but uploads transparent receiver lists has a different privacy boundary for the transparent part of the account.

5.3 Mitigations and their limits

Larger uniform range requests can reduce height-resolution leakage. Batching full-transaction retrieval can obscure which compact match triggered a request. Separating scan and broadcast services can reduce one service's direct view of both activities. Using multiple sources can reveal disagreements or reduce reliance on one operator. These techniques do not automatically provide anonymity: operators may collude, network observers may correlate timing, and all sources may repeat the same omitted data.

Fetching every full transaction avoids relevance-triggered follow-up requests but increases bandwidth. Running one's own validating node changes the server-trust boundary but not public-chain leakage. The correct choice depends on the wallet's explicit bandwidth, storage and adversary model. No mechanism in this chapter proves a universal network privacy theorem.

5.4 The synchronisation contract

A wallet is ready to construct a particular spend when the required account and scope have been scanned over the necessary coherent branch range, the input note has a verified reconstruction and unspent status under the chosen trust model, and its witness is available for the selected anchor. This is more precise than saying that the latest displayed height equals the server's tip.

The core invariant is a correspondence:

recorded note $\longleftrightarrow$ pool and branch position $\longleftrightarrow$ path to the selected anchor.

Spend observations and transaction lifecycle records must refer to that same branch. If any link is missing, retain a partial state or rescan; do not silently convert uncertainty into spendable balance.

Evidence and executable checks. The compact schema is `zcash_client_backend/proto/compact_formats.proto`. The note scan and persistence interfaces in that crate are `src/scanning.rs` and `src/data_api/chain.rs`. The database implementation is `zcash_client_sqlite/src/chain.rs`, at the pinned library commit. The generic retained-tree implementation is the `shardtree` crate, used through those interfaces. The companion `check_deployed.py` checks append occupancy, frontier roots against complete trees, every retained path and its changed sibling through all 32 appends of a depth-five toy tree, position arithmetic and rollback using a clearly labelled toy hash. It does not replace the production Sinsemilla vectors, test a deployed server for omission, or establish an independent light-client security theorem.

Part VIII

FlyClient Guide: Sampling an authenticated proof-of-work history

Abstract

This volume of *The Zcash Arboretum* constructs the sampling argument behind FlyClient. It assumes probability from the *Math Guide*, authenticated histories from the *Crypto Guide*, and work, chain selection and block-history commitments from the *Consensus Guide*. The endpoint is a verifier that checks a committed candidate history by sampling, together with an exact statement of what would still be needed to instantiate that verifier for Zcash. The sampling theorem is proved here. The mining assumption, variable-difficulty transition argument and network availability premise are kept separate from it.

1 The chain-verification contract

The probability language used here—random choices, events, independence, and error bounds—is constructed in the *Math Guide*, §*Probability and computation*. That section also constructs real intervals, integration, probability densities, logarithms, and inverse sampling before they are used below. *Crypto Guide*, §*Security contracts and reductions*, supplies the computational meaning of a negligible failure probability and a security assumption.

A full node checks blocks and their transactions before using their work to select a chain. A resource-limited client would like to learn which history this selects without receiving every header. Receiving a short history commitment solves only half that problem: a hash binds a server to its answer, but does not make the answer valid.

FlyClient adds unpredictable tests of the committed history. A server first fixes a candidate head. The client then asks for selected earlier headers and checks both their work and their connection to that head. A candidate containing enough invalid work should fail a test. The distribution must catch forks near the tip as well as forks near genesis.

Our source is Bünz, Kiffer, Luu and Zamani, *FlyClient: Super-Light Clients for Cryptocurrencies*, revision 2020-08-20, especially Sections 4–6 and Appendix A. We use zero-based history indices consistently and separate the paper’s abstract construction from the Zcash history tree. The deployed tree’s exact encoding belongs to the *Consensus Guide*, §*Authenticated block history*; repeating it here would obscure the different object the sampling theorem assumes.

1.1 Three different questions

Let a candidate consist of headers

$$C = (B_0, \dots, B_n).$$

The head is B_n . The preceding n headers form its committed history. A header’s work is a positive integer w_i ; work and target are not synonyms. The *Consensus Guide*, §*Work, difficulty, and chain selection*, constructs Zcash’s work function and explains why a larger numerical target means an easier proof of work.

The client must distinguish three predicates, meaning conditions that can be true or false:

1. *Membership*: a particular header is at a particular position in the sequence committed by B_n .
2. *Header validity*: the candidate obeys its header-link, proof-of-work and difficulty rules.
3. *Ledger validity*: transactions, proofs, signatures, balances, nullifiers and all other state transitions are valid.

An authenticated path proves the first predicate. Sampling aims at the second, under explicit mining and commitment assumptions. Neither operation independently checks every transaction in the third.

A client accepting a transaction-inclusion proof therefore relies on the underlying consensus model's honest full nodes to reject invalid ledger transitions. The transaction's inclusion path does not itself prove that its spend was authorised. This distinction is the same one that separates a transaction digest from a valid transaction.

Definition 1.1 (Client outcome). The client receives candidate heads and their proofs from servers, at least one of which supplies an honest full node's chain. It rejects malformed candidates and selects a surviving candidate of greatest verified claimed work. Security asks that the selected history agree with the honest chain away from its potentially unstable suffix.

The availability premise is substantive. A completely isolated client can receive a valid but old history and no evidence that a newer one exists. No sampling distribution manufactures a missing honest view. Nor does agreement between two server responses prove that either server is independent or honest. The client checks the claimed objects; it does not replace validation by counting matching responses.

1.2 The assumption paid for by sampling

Suppose a competing history branches from the honest history at some point a . A server claiming at least the honest chain's work cannot ordinarily manufacture all of its fork with valid puzzles. Fly-Client formalises the resulting deficit through a bound on the fraction of post-fork work that can pass the sample verifier.

Assumption 1.2 (Bounded valid fork weight). Fix $0 < c < 1$ and a positive integer short-fork cutoff L . Except with probability $\varepsilon_{\text{mine}}$, every adversarial competing fork outside the short-fork exception has at most a c fraction of its claimed post-fork work in headers that pass the protocol's work, history and difficulty checks. The remainder is detectable invalid work.

This is a property of adversarial histories, not a definition of the adversary's hash rate. In particular, $c = 0.9$ does not mean that 90 percent of total mining power is adversarial. A theorem connecting hash-rate and network parameters to c, L must be established in the chosen chain model.

Short forks are excluded because mining has variance even under honest majority. The verifier checks a trailing region in full. In a constant-difficulty chain its length can be compared directly with L . In a variable-difficulty chain, a block-count cutoff and a work-fraction cutoff are different quantities. A concrete instantiation must make its fully checked suffix cover the short forks exempted by its mining assumption.

2 Work intervals and authenticated samples

The *Crypto Guide*, §*Authenticated trees and append-only histories*, supplies ordered inclusion proofs and append-only commitments. We now specify what their interface must expose to the sampler. This is additional metadata on the authenticated sequence, not a new security assumption about the hash function.

2.1 From positions to work

Define prefix work and total history work by

$$W_0 = 0, \quad W_i = \sum_{j=0}^{i-1} w_j, \quad W = W_n.$$

For $W > 0$, header B_i occupies the half-open interval

$$I_i = [W_i/W, W_{i+1}/W).$$

These intervals are disjoint and cover $[0, 1)$. A point x selects the unique header whose interval contains it. Half-open intervals fix boundary behaviour without double counting.

For equal work per block, $I_i = [i/n, (i+1)/n)$, so work sampling reduces to position sampling. For unequal work, a high-work block covers more of the domain. Sampling header positions uniformly would allow a server to hide its claimed work in a small number of artificially difficult headers.

Construction 2.1 (Authenticated weight lookup). Each tree node commits to its ordered children and their total weights. To locate a work coordinate $u = xW$, start with offset zero at the root. At a parent with left weight W_ℓ , descend left when $u < W_\ell$. Otherwise subtract W_ℓ , add W_ℓ to the offset, and descend right. The proof supplies the sibling node at every step. The verifier recomputes each parent hash and weight, then checks that the final residual coordinate lies in the returned leaf's interval.

The verifier checks lengths, child orientation, integer widths and overflow conditions before using supplied weights. Its final total must equal the total authenticated by the selected head. A server cannot choose a different leaf after seeing the sample without either changing the root or presenting an inconsistent path.

This claim is conditional on the node invariants. A hash commits to false metadata just as effectively as true metadata. Recomputing the visited ancestors checks the additions along that path; it does not recompute every unopened subtree. The chain-validation argument must account for malformed subtrees through the sample predicate.

2.2 A history inside every header

The abstract protocol requires header B_i to commit to the ordered prefix $(B_0, \dots, B_{i-1})$, not merely to its predecessor's hash. Let that commitment be R_i . The head's history root is R_n .

A sample proof has two jobs. First it proves that B_i appears under R_n . Second it reconstructs the commitment to the prefix strictly before B_i , and checks that this equals R_i inside the sampled header. Without the second check, a server could try to insert a previously mined header into a different committed history.

For an append-only forest, a prefix-recovery proof supplies the earlier forest peaks and the left siblings inside the peak containing leaf i . Together these partition the leaves before i into complete subtrees. Their authentication must connect them to the head's root. Ordering these subtrees from left to right and applying the forest's fixed root-folding rule reconstructs the prefix root. The route and leaf count determine their positions; supplied direction bits cannot redefine the leaf position.

The paper uses its recursively rooted MMR and the corresponding prefix-recovery procedure. Zcash uses the different bagging convention constructed in *Consensus Guide*, §*Authenticated block history*. The generic requirement is the same, but a proof for one root convention is not an encoding of a proof for the other.

Proposition 2.2 (History consistency of a sampled header). *Assume the ordered history commitment is binding. If a sampled header passes inclusion under R_n and its own R_i equals the recovered prefix root, then its committed earlier history is the same ordered prefix as that authenticated by the head, except with the commitment’s binding failure probability.*

Proof. Two different ordered prefixes giving the recovered root would violate binding. The equality with R_i therefore identifies one prefix in both commitments. Position binding additionally prevents interpreting the same opening as a different prefix length. $\square$

The proposition says nothing about the transactions inside that prefix. It prevents reuse of a header in an incompatible history; it does not turn the header into a proof of the entire preceding computation.

2.3 Difficulty transitions

Consider the paper’s epoch model. An epoch contains M blocks. A target T is replaced after elapsed time Δ by

$$T' = \text{clamp}(T\Delta f/M, T/\tau, \tau T), \quad \tau > 1,$$

where a round is the model’s unit of time and f is the intended blocks-per-round rate. Here $\text{clamp}(x, a, b) = \max(a, \min(x, b))$. Rounding and the permitted target range are part of a concrete rule. The clamp bounds each target change; it is essential to the model’s security analysis.

The paper’s difficulty tree aggregates a tuple

$$(h, w, t, D_{\text{start}}, D_{\text{next}}, n).$$

Here h authenticates the node, w is total difficulty weight, t is elapsed time, n is leaf count, and D denotes difficulty rather than the oppositely ordered proof-of-work target. A leaf has $n = 1$, its own weight and elapsed time, and the next difficulty prescribed by the rule. Joining left and right nodes adds w, t, n , inherits the left starting difficulty and right next difficulty, and authenticates both full tuples.

At an opened join, the verifier checks $D_{\text{next},\ell} = D_{\text{start},r}$. Within one epoch it checks constant difficulty and consistent aggregate weight. Across epochs it checks that the boundary difficulties, elapsed time and total weight admit legal clamped transitions.

For example, if a sequence of r epoch transitions begins at difficulty D , each transition changes difficulty by at most a factor τ . After j transitions its difficulty lies between $D\tau^{-j}$ and $D\tau^j$. An independently fixed ending difficulty adds the same constraints backwards. The maximum possible intermediate difficulty is bounded by the smaller of the forward and backward upper envelopes; the minimum is bounded by the larger lower envelope. Summing these envelopes bounds aggregate weight. Timing constraints must also be satisfied: a sequence of allowable difficulty levels alone does not show that the retarget formula produces them.

This explains the purpose of the metadata, but is not a complete validator for arbitrary retarget algorithms. The paper’s variable-difficulty argument imports a transition-rewriting lemma: an adversary whose malformed transitions escape some local tests can be replaced, without materially increasing its mining budget, by one respecting the model’s transitions and with the same sample-valid work. The lemma’s hypotheses include a sufficient aggregate-feasibility test. Our sampling theorem below assumes the resulting valid-weight bound; it does not independently prove that lemma, and does not instantiate its feasibility test for Zcash’s averaging-window retarget.

3 Choosing the sampling distribution

The real-number and continuous-probability construction in *Math Guide*, §*Probability and computation*, now gives an exact mathematical problem. Work is normalised to $[0, 1]$, and the final fraction

δ is checked in full. The remaining samples are independent draws from a density on $[0, 1 - \delta]$. A density g assigns an interval $[a, b]$ probability $\int_a^b g(x) dx$; the integral is its area. The probability rules used here are those of *Math Guide*, §Probability and computation.

3.1 Why the tip receives more samples

A fork at a occupies weight $1 - a$. Under Assumption 1.2, at least $(1 - c)(1 - a)$ of this weight is invalid. Uniform sampling over the whole history catches it with probability only $(1 - c)(1 - a)$, which tends to zero as the fork moves towards the tip.

Choose an increasing density. For a fixed amount of invalid work after a , its least detectable placement is at the beginning of the fork, where the density is smallest. Moving any invalid mass from a later, higher-density region to an available earlier region cannot increase the catch probability. Repeating such moves leaves the interval

$$J_a = [a, 1 - c(1 - a)].$$

Its length is exactly $(1 - c)(1 - a)$. Extra invalid work only helps the verifier.

If this interval cannot fit before $1 - \delta$, then any placement avoiding the fully checked suffix is impossible. Otherwise the desired density should assign the same probability to J_a for every possible fork point.

3.2 Normalisation and the inverse sampler

Use the natural logarithm constructed in *Math Guide*, §Probability and computation: for $y > 0$, $\ln y = \int_1^y dt/t$. That construction proves the substitution rule, $\ln(ab) = \ln a + \ln b$, and the inverse exponential function. In particular, $\ln y$ is increasing, is zero at 1, and is negative for $0 < y < 1$.

Take $0 < \delta < 1$ and define

$$g_\delta(x) = \frac{1}{(1 - x) \ln(1/\delta)} \quad (0 \leq x \leq 1 - \delta).$$

The denominator is positive. Substituting $t = 1 - x$ gives

$$\int_0^z g_\delta(x) dx = \frac{-\ln(1 - z)}{\ln(1/\delta)} =: G_\delta(z), \quad G_\delta(1 - \delta) = 1.$$

Thus g_δ is a probability density. It increases towards the tip, but its support stops before the singularity at $x = 1$.

To draw a point, draw an independent uniform $U \in [0, 1]$ and solve $G_\delta(x) = U$:

$$x = 1 - \delta^U.$$

Indeed, G_δ is increasing, so $\Pr[1 - \delta^U \leq z] = G_\delta(z)$. A finite implementation must specify how its random bits approximate this ideal draw; floating-point rounding is not part of the mathematical construction.

For a block occupying $[a, b]$, its probability is the integral over that interval intersected with the sampling support. When $0 \leq a < b \leq 1 - \delta$, this is

$$p_{a,b} = \frac{\ln((1 - a)/(1 - b))}{\ln(1/\delta)}.$$

If the boundary cuts a block, fully checking that entire block is safe. It enlarges the fully checked region, never reducing its protection.

Theorem 3.1 (Single-sample detection). *Suppose a fork after a contains at least $(1 - c)(1 - a)$ invalid work. If no invalid work occurs in the fully checked suffix, a sample from g_δ catches invalid work with probability at least*

$$p = \frac{\ln(1/c)}{\ln(1/\delta)}.$$

The claim applies when $\delta \leq c$. If the invalid work cannot fit before the checked suffix, the suffix itself detects the fork.

Proof. By the increasing-density argument, the least detectable admissible placement is J_a . If it lies in the support, its mass is

$$\int_a^{1-c(1-a)} \frac{dx}{(1-x)\ln(1/\delta)} = \frac{\ln((1-a)/(c(1-a)))}{\ln(1/\delta)} = p.$$

If its right endpoint exceeds $1 - \delta$, there is insufficient uninspected work after a to contain all invalid work. Any admissible placement then intersects the suffix. $\square$

The proof works for whole blocks as well. Their invalid-work regions are finite unions of intervals, a restricted family of the placements allowed to the adversary in the fractional-work argument.

3.3 Optimality and its exact domain

The following optimality question asks for the best guarantee the verifier can achieve against the adversary's worst permitted placement. This best-against-worst criterion is called *minimax*.

Theorem 3.2 (Minimax optimality). *Let k be a positive integer and $\delta = c^k$. Among all one-sample densities supported on $[0, 1 - \delta]$, the largest guaranteed detection probability against the continuum fork model is $1/k$. The density g_δ attains it.*

Proof. For $i = 0, \dots, k - 1$, consider the fork point $a_i = 1 - c^i$. An admissible adversary makes exactly the interval

$$[1 - c^i, 1 - c^{i+1}]$$

invalid and places its valid fraction afterwards. These k intervals partition the sampling support, apart from endpoints of zero probability under a density. At least one interval receives probability at most $1/k$. Thus no density guarantees more than $1/k$. Theorem 3.1 gives $\ln(1/c)/\ln(1/c^k) = 1/k$ for g_δ . $\square$

The integer condition is used by the partition. For non-integral $k = \ln(1/\delta)/\ln(1/c) > 1$, the detection theorem still gives $1/k$, but this partition proof does not establish optimality. For $k = 1$, one sample suffices in the ideal model; expressions involving $\ln(1 - 1/k)$ are then undefined.

3.4 Choosing the sample count

After m independent draws, the probability of missing invalid work is at most

$$\varepsilon_{\text{sample}} = (1 - p)^m.$$

To make this at most $2^{-\lambda}$, where $\lambda > 0$ is the desired error exponent, for $0 < p < 1$ choose

$$m = \left\lceil \frac{\lambda \ln 2}{-\ln(1 - p)} \right\rceil.$$

Taking logarithms preserves the inequality until division by the negative number $\ln(1 - p)$, when its direction reverses.

For constant difficulty, checking L of n history blocks gives $\delta = L/n$. Write $\log_b x = \ln x / \ln b$ for a logarithm to base $b > 0, b \neq 1$. For fixed c ,

$$p = \frac{\ln(1/c)}{\ln(n/L)}, \quad m = O(\lambda \log_{1/c}(n/L)).$$

The order bound follows from $-\ln(1-p) \geq p$, which follows by integrating $1/(1-t) \geq 1$ from zero to p . An authenticated sample costs one header and an $O(\log n)$ -node path. Without sharing duplicate paths, the proof therefore costs

$$O(L + \lambda \log_{1/c}(n/L) \log n)$$

header/node units, assuming bounded-size metadata. Actual bytes depend on header size and metadata encoding, not just this count.

Example 3.3 (A checked parameter calculation). Take $n = 2^{20}$, $L = 2^{10}$, $c = 1/2$, and $\lambda = 64$. Then $\delta = 2^{-10}$, $k = 10$, and $p = 1/10$. The least integer sample count is $m = 422$:

$$(9/10)^{422} \leq 2^{-64} < (9/10)^{421}.$$

This is an arithmetic example under the stated fork model, not a Zcash confirmation recommendation. The accompanying numerical check evaluates both inequalities with exact rational arithmetic.

4 The interactive verifier

The sampler is useful only when each challenge has a uniquely checkable answer. We can now state the complete verifier at the abstract history-interface level.

Construction 4.1 (Candidate verification). The verifier fixes the chain’s genesis, header rules, history commitment scheme, work function, suffix rule, c , and error budget.

1. Receive a candidate head, the authenticated history length and total work, and any enclosing commitment data needed to recover the history root. Check canonical encodings and the head’s own header validity. Freeze this candidate before choosing challenges.
2. Receive and validate the entire required suffix, including its ordering, previous-history roots, work and difficulty transitions. Check its connection to the head and authenticate its boundary in the committed history.
3. Draw m independent sample points. For each point obtain the selected header, its position, sibling metadata, and any adjacent header or epoch-boundary data required by the sample-validity rule.
4. Recompute the inclusion path and total weight. Check the work coordinate selects that leaf; check the header’s own proof of work, target and required difficulty data; recover and compare its prefix root as in Proposition 2.2.
5. Reject the candidate on any failed or missing answer. Otherwise retain it for work comparison with the other checked candidates.

Duplicate samples do not invalidate the bound: the draws remain independent even if two happen to select one header. A prover may compress repeated answers, but the verifier must retain the multiplicity of the original draws when analysing their distribution.

The empty-history and short-history cases use full verification rather than divide by zero or sample an empty support. A malformed proof is not a lower-work candidate. It is a rejected candidate. A timeout gives no positive assertion about chain validity.

Theorem 4.2 (Conditional interactive soundness). *Assume an honest candidate is available; the underlying chain has stable prefix security with failure probability $\varepsilon_{\text{chain}}$; the authenticated sample interface fails with probability at most $\varepsilon_{\text{auth}}$; and Assumption 1.2 holds with failure probability $\varepsilon_{\text{mine}}$. For one fixed competing candidate outside the short-fork exception, the probability that the verifier accepts it despite its invalid work is at most*

$$\varepsilon_{\text{auth}} + \varepsilon_{\text{mine}} + (1 - p)^m.$$

The stable-prefix conclusion additionally pays $\varepsilon_{\text{chain}}$.

Proof. Exclude the authentication and mining failure events. The frozen candidate then has well-defined work intervals and enough invalid work. The suffix either detects it, or Theorem 3.1 gives detection probability at least p for each independent challenge. All m challenges miss with probability at most $(1 - p)^m$. Adding back the excluded events gives the bound. Stable-prefix agreement with other honest nodes uses the separate consensus guarantee. $\square$

With several malicious candidates, apply a union bound using their actual number or a justified bound on it. The server must not be allowed to reset the experiment indefinitely under the same aggregate error budget. Availability of an honest proof and verification of maximal work are parts of the protocol, not consequences of this single-candidate theorem.

5 Fiat–Shamir and persistent verification

Interactive randomness prevents the server from tailoring a committed history to the sample. A transferable proof instead derives that randomness from the candidate. This removes interaction, but introduces the possibility of trying many candidates.

5.1 Binding the challenge

The random-oracle model and transcript programming are defined in *Crypto Guide*, §*Security contracts and reductions* and §*Knowledge, simulation, and Schnorr*. Here the oracle supplies sampling randomness, not a Schnorr challenge or a witness extractor.

A non-interactive instantiation must hash a domain-separated, canonical description fixing the network, protocol version, candidate head, history root, history length, authenticated total work and sampling parameters. If some field is already unambiguously bound by another, its duplicate encoding is unnecessary; its binding is not.

The paper derives challenges from the candidate head. Under that construction, changing the samples entails changing and mining a head. Nevertheless, the cost of trying heads is a mining-model question. It is not enough to say that a hash looks random.

Proposition 5.1 (A finite grinding budget). *Suppose each fresh candidate has conditional sample-evasion probability at most ε given the transcript before its fresh oracle answer. An adversary makes at most Q such attempts. Its probability of finding an evading candidate is at most $Q\varepsilon$.*

Proof. For each attempt, its success event has probability at most ε , by averaging the conditional bound. The probability of the union of these events is at most their sum. The candidates need not be chosen independently. $\square$

Consequently a $2^{-\lambda}$ aggregate sampling target needs

$$(1 - p)^m \leq 2^{-\lambda}/Q.$$

The choice Q must cover all permitted restarts and candidate-grinding opportunities. The paper gives a bound in its own mining model; it is not a universal limit on an implementation's hash queries.

Finite sampling precision creates another explicit error term. Compare the *selected-header indices*, a finite distribution, with the indices induced by the ideal continuous draw. A finite set of numerical coordinates is not close in statistical distance to a continuous distribution on real coordinates: those supports differ completely. If each implemented index distribution is within statistical distance η of its ideal index distribution, independently pair the implemented and ideal draws so that each pair differs with probability at most η . This pairing is called a coupling; its finite construction is given in *Math Guide*, §*Probability and computation*. The probability that any of the m pairs differs is at most $m\eta$, by the union bound. The security budget must include that term, or use a separately proved exact discrete index sampler.

5.2 State retained by a returning client

A returning client can retain a stable authenticated checkpoint rather than begin again from genesis. A new candidate must prove consistency with that checkpoint and satisfy the sampling requirements for its extension. Old accepted work does not have to be retransmitted merely because a new suffix arrived.

A current head is not necessarily stable. A client may replace its unsettled suffix while retaining the earlier common prefix; it must not turn a previously tentative observation into irrevocable finality. The appropriate confirmation rule remains the one in *Consensus Guide*, §*Reorganisation and confirmation confidence*.

After accepting the header history, the client can request a block's membership proof and then a transaction's membership proof under that block's transaction commitment. Effecting-data and authorising-data commitments must be checked separately where the transaction format separates them. A transaction-inclusion response alone must not be treated as evidence of fresh chain selection or transaction execution.

6 Connecting the construction to Zcash

Zcash's history commitment is already part of the core consensus construction. The *Consensus Guide*, §*Authenticated block history*, specifies its canonical metadata, its one-block delay, upgrade-epoch resets, and the outer block commitment. These are inputs to a possible client verifier, not an implementation of the verifier above.

The distinction has concrete consequences:

1. A Zcash history tree covers one upgrade epoch. A proof spanning upgrades must authenticate the transitions and recover the relevant earlier roots. It cannot interpret the latest root as a genesis-rooted tree.
2. The head commits to earlier blocks, not to itself. Its own work, validity and total-score contribution must be checked separately.
3. The root uses Zcash's bagging and node encoding. Paper proofs must be translated and verified under those exact rules.
4. The root is nested inside the block commitment alongside authorising-data commitments. The verifier needs the appropriate enclosing values to reconstruct the header's committed digest.
5. Zcash's retarget rule is not the paper's long, clamped-epoch model. The aggregate-feasibility predicate and a justified valid-fork-weight bound remain necessary work.
6. The wire protocol must specify requests, proofs, integer limits, canonical random-bit sampling, error handling, checkpoint persistence and multi-server comparison. A node's ability to compute a root does not provide those interfaces automatically.

ZIP 221, in its security discussion, explicitly leaves a stronger analysis for rapidly adjusting chains before a FlyClient ecosystem is designed. This is a real boundary of the selected construction, not a historical footnote. The light-client trust contract examined in *Sync Guide*, §*Server trust and observable metadata*, does not become trustless because a block contains a history root.

The result of this volume is therefore precise. The sampling density, its normalisation, its integer-case optimality and its error accounting are complete mathematical constructions. Authenticated prefix and weight checks state the exact interface they consume. Instantiating their security premises and proof transport for Zcash is not completed by ZIP 221, by this text, or by the existence of a valid history commitment.

Part IX

ZSA Guide: Issuance, private transfer, and public supply

Abstract

A shielded asset is not a number with a hidden text label. Its identity must survive issuance, commitments, transfers, splitting, and burning, without letting an adversary invent algebraic exchange rates between unrelated assets. This volume constructs that chain of obligations. It derives the issuer's signature, the public issuance state machine, the private per-asset balance equation, and the membership rule that makes zero-value split inputs safe. The endpoint is the mathematical and validation design in one pinned pair of draft ZIPs. Where those drafts leave their transaction carrier or note-format instantiation unfinished, the boundary remains visible.

1 The asset ledger contract

The integer, field, and point-encoding language used here comes from *Math Guide*, §*Prime fields and modular arithmetic* and §*Pasta curves and their encodings*. Probability, including uniform randomness, is constructed in *Math Guide*, §*Probability and computation*. *Crypto Guide*, §*Security contracts and reductions*, defines computational security and failure probabilities. Its §*Groups, commitments, and Sinsemilla* defines hiding and binding and constructs the commitments used by the inherited note relation; §*Knowledge, simulation, and Schnorr* defines knowledge and extraction.

1.1 What changes when there is more than one asset

In the native payment protocol, every amount uses the same unit: a zatoshi. If one input holds 80 000, outputs hold 50 000 and 20 000, and the fee is 10 000, the equation

$$80\,000 = 50\,000 + 20\,000 + 10\,000$$

has a meaning before any cryptography is applied.

For two assets A and B , the expression $80A = 50B + 30A$ is not a balanced payment. No exchange rate is part of the shielded ledger's addition operation. A valid transaction must conserve each asset separately, except for explicitly authorised creation or public destruction of that same asset.

The design separates three operations.

1. *Issuance* creates custom-asset notes. The asset, recipient, amount, and note randomness are publicly available, and an issuer signature authorises the creation.
2. *Transfer* consumes notes and creates notes. The proof hides their assets and amounts while enforcing per-asset balance.
3. *Burn* removes a public amount of a named custom asset from the shielded ledger. The proof and binding signature account for that removal.

An issuer controls further creation of its assets, not the spend authorisation keys of holders. The native asset retains its existing value base and pays transaction fees. A custom asset cannot be moved into a transparent native-value field by relabelling its units.

1.2 The selected endpoint

The rule set here is ZIP 226 and ZIP 227 at commit 69610984. They are draft specifications, not evidence that these rules are active on a network. Their Orchard definitions incorporate the ZIP 2005 changes at that same source pin.

The asset mechanisms are substantially specified, but the pair still references the withdrawn ZIP 230 transaction carrier and ZIP 246 digest scheme. Its ZSA note lead byte is a placeholder. Consequently, this volume does not claim to define an interoperable, deployable ZSA transaction encoding. It constructs the specified state and proof contracts and identifies exactly which missing interfaces prevent turning them into a complete byte-level validator. The version-six transaction in the core volumes is not silently extended with these fields.

1.3 Dependencies and notation

Ironwood Guide, §*Actions and the proof relation*, supplies the one-input/one-output Action, the note commitment, Merkle membership, nullifier, and ownership relation. Its §*Balance, digests, and authorisation* supplies the binding signature and the distinction between transaction effects and authorising data. We change those constructions only where stated.

Math Guide, §*Pasta curves and their encodings*, supplies Pallas. Write $\mathbb{P}$ for its prime-order group, p for its base-field modulus, and q for its group order. Thus note values such as ρ, ψ, nf lie in $\mathbb{F}_p$, whereas commitment randomness and signature scalars lie in $\mathbb{F}_q$. These two fields must not be interchanged.

Write V for the existing Orchard native-value base and R_v for its value-commitment randomness base. An asset base A is a nonidentity Pallas point. The point's canonical encoding A^* is 32 bytes; when fed to a bit-oriented commitment, these same bytes are unpacked least-significant bit first. $\text{enc}_k(x)$ means exactly k bytes of little-endian integer encoding. The shared notation $\text{LE}_k(x)$ instead denotes exactly k bits, least-significant bit first.

1.4 State is branch-local

A full validator keeps the existing note tree, recognised anchors, and spent-nullifier set. It additionally keeps an issuance map

$$I[A] = (\text{balance}, \text{final}, \text{reference}).$$

The map is part of the state of a particular chain branch. Processing a transaction produces a candidate next state; any failed check discards the entire candidate. A reorganisation restores the state at the common ancestor and replays the replacement branch.

In particular, an issuance finalisation is irreversible *along a fixed accepted branch*, not immune to a reorganisation of that branch. No extra finality assumption is hidden in the word final.

2 Asset identity and issuance authority

2.1 Deriving a separate issuance key

The issuance secret is independent of the protocol role of a note spending key. A holder may authorise a payment without gaining authority to issue more units, and an issuer may sign issuance transactions without learning holders' spend keys.

The derivation reuses the hardened-only machinery in *Wallet Guide*, §*From seed to account*, with a different master domain and path. Let the seed S contain between 32 and 252 bytes; it should contain at least 256 bits of entropy. For the guessing threat here, a conservative meaning is that no one seed occurs with probability greater than 2^{-256} , called at least 256 bits of *min-entropy*. Merely storing 32 bytes does not ensure this unpredictability. Set

$$(s_0, c_0) = \text{split}_{32}(\text{BLAKE2b}_{512}(\text{ZcashSA_Issue_V1}, S)).$$

The first half is a 32-byte secret and the second is a chain code. For a hardened index $i \geq 2^{31}$, define

$$(s', c') = \text{split}_{32}(\text{PRFexpand}_c(0x81 \parallel s \parallel \text{enc}_4(i))).$$

The byte 0x81 is hexadecimal; the empty optional lead and tag arguments of ZIP 32 contribute no bytes here. Apply this step along

$$m_{\text{issuance}}/227'/\text{coin_type}'/0'.$$

On mainnet the coin-type index is 133'. The account index is fixed to zero in this selected design. The final 32-byte secret is isk.

This is derivation of a *byte string*. Interpreting that string as the secp256k1 secret scalar below uses big-endian order. It is not the Pallas little-endian scalar conversion used by RedPallas. If the result is not a valid secp256k1 secret, key derivation fails at the public-key step. ZIP 227 calls for discarding the key and using a different issuance secret; it does not specify an automatic hierarchical-key-derivation retry convention preserving the fixed path.

2.2 The secp256k1 instance

Issuance uses the Schnorr construction from *Crypto Guide*, §*Knowledge, simulation, and Schnorr*, instantiated by BIP 340. This is a different curve from Pallas.

The curve is

$$y^2 = x^3 + 7 \pmod{p_s}, \quad p_s = 2^{256} - 2^{32} - 977,$$

and its group order is the integer whose hexadecimal encoding is

$$\begin{aligned} n_s = & \text{FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF} \\ & \text{BAAEDCE6AF48A03BBFD25E8CD0364141}. \end{aligned}$$

The displayed lines concatenate; they are not an addition. The standard generator G_s has coordinates

$$\begin{aligned} x(G_s) = & 79BE667EF9DCBBAC55A06295CE870B0702 \\ & 9BFCDB2DCE28D959F2815B16F81798, \\ y(G_s) = & 483ADA7726A3C4655DA4FBFC0E1108A8FD \\ & 17B448A68554199C47D08FFB10D4B8. \end{aligned}$$

Again concatenate each coordinate's lines. The arithmetic formulas are those of *Math Guide*, §*Pasta curves and their encodings*, with the curve constant changed from 5 to 7. The curve's prime group order and discrete-logarithm hardness are properties and assumptions of this selected standard curve, not consequences of merely writing its equation.

Let BE_{32} encode an integer in exactly 32 big-endian bytes. A public key contains only the x -coordinate of its point. To decode it, reject $x \geq p_s$, compute

$$c = x^3 + 7 \pmod{p_s}, \quad y = c^{(p_s+1)/4} \pmod{p_s},$$

and reject unless $y^2 = c$. Of y and $-y \pmod{p_s}$, select the even representative. The exponent is a square-root candidate because $p_s \equiv 3 \pmod{4}$; the explicit square check rejects non-residues. This operation is lift_x .

Given $d' = \text{int}_{\text{BE}}(\text{isk})$, reject unless $1 \leq d' < n_s$. The public key is $\text{ik} = \text{BE}_{32}(x([d']G_s))$. For signing, replace d' by $n_s - d'$ if its public point has odd y . The resulting d satisfies $[d]G_s = \text{lift}_x(\text{ik})$.

2.3 Tagged hashing and a complete issuance signature

Crypto Guide, §Hash functions and key derivation, constructs SHA-256. BIP 340 gives each use a tagged wrapper:

$$H_{\text{tag}}(m) = \text{SHA256}(\text{SHA256}(\text{tag}) \parallel \text{SHA256}(\text{tag}) \parallel m),$$

where the tag is its literal UTF-8 byte sequence. For the issuance transaction's 32-byte signature digest M , the selected signing algorithm is as follows.

1. Obtain the normalised scalar d and public key ik . Set

$$t = \text{BE}_{32}(d) \text{ xor } H_{\text{BIP0340/aux}}(0^{32}).$$

Here 0^{32} is 32 zero bytes, as required by ZIP 227.

2. Set

$$k' = \text{int}_{\text{BE}}(H_{\text{BIP0340/nonce}}(t \parallel \text{ik} \parallel M)) \bmod n_s.$$

Reject $k' = 0$. If $[k']G_s$ has odd y , replace k' by $n_s - k'$. Call the result k , and let $R = [k]G_s$.

3. Compute

$$e = \text{int}_{\text{BE}}(H_{\text{BIP0340/challenge}}(\text{BE}_{32}(x(R)) \parallel \text{ik} \parallel M)) \bmod n_s, \quad s = k + ed \bmod n_s.$$

Form $\sigma = \text{BE}_{32}(x(R)) \parallel \text{BE}_{32}(s)$. Verify it with the following algorithm before returning it; abort if this self-check fails.

Verification first checks the public-key and signature lengths. Lift the public key to the even- y point P . Parse the signature as integers r, s , and reject $r \geq p_s$ or $s \geq n_s$. Recompute e from $\text{BE}_{32}(r)$, the public key, and M . Set $R' = [s]G_s - [e]P$. Accept precisely when R' is nonidentity, has even y , and has $x(R') = r$.

Correctness is the identity

$$[k + ed]G_s - [e]([d]G_s) = [k]G_s.$$

The parity normalisations ensure that the verifier uses the same representatives as the signer. Security additionally requires the computational and hash assumptions of Schnorr; correctness alone does not imply unforgeability. Fixed auxiliary bytes do not make the nonce public: its derivation includes the secret d . They do remove the additional fault and side-channel protection that fresh auxiliary randomness can provide.

The scheme signature σ is 64 bytes. At ZIP 227's scheme-tagged interface,

$$\text{issuer} = 0x00 \parallel \text{ik}, \quad \text{issueAuthSig} = 0x00 \parallel \sigma,$$

which are respectively 33 and 65 bytes. This does not specify the outer transaction field framing; that is part of the carrier boundary discussed later.

2.4 From a description to an asset base

An asset description D is a nonempty byte sequence, preferably well-formed UTF-8. Consensus stores its digest, not a requirement to publish a particular human-readable description. Define

$$\begin{aligned} h_D &= \text{BLAKE2b}_{256}(\text{ZSA-AssetDescCRH}, D), \\ \text{AssetId} &= (\text{issuer}, h_D), \\ B_{\text{id}} &= 0x00 \parallel \text{issuer} \parallel h_D, \\ h_A &= \text{BLAKE2b}_{512}(\text{ZSA-Asset-Digest}, B_{\text{id}}), \\ A &= \text{GroupHash}^{\mathbb{P}}(\text{z.cash:OrchardZSA}, h_A). \end{aligned}$$

The BLAKE2b personalisation strings are zero-padded to their 16-byte parameter fields, as in *Crypto Guide*, §*Hash functions and key derivation*, which also constructs the exact Pallas GroupHash.

The first byte of B_{id} is an asset-identifier version. The next byte is the signature-scheme byte inside the issuer. They happen both to be zero, but perform different jobs. The total length is $1 + 33 + 32 = 66$ bytes. h_A has 64 bytes, and A^* has 32 bytes.

An asset name is not an identity. Two issuers may hash the same text and still obtain different identifiers because their issuer fields differ. Conversely, two bitwise different descriptions chosen to collide in h_D would define the same identifier under one issuer. Avoiding that attack is a collision-resistance assumption, not a type-system guarantee. Wallets must identify assets by authenticated identifiers or user-established names bound to them, not by an untrusted description alone.

2.5 The algebraic requirement on asset generation

All nonidentity Pallas points generate the same cyclic group. There is therefore some scalar a such that $A = [a]V$, and some scalar relation between any two asset bases. The construction does not make those relations cease to exist. It must make useful nontrivial relations computationally infeasible to find.

In particular, issuers must not choose a scalar a themselves and declare $[a]V$ to be their asset base. Their choice is an identifier preimage; the independently domain-separated group hash determines the base. The later conservation argument assumes that an efficient adversary cannot find a nonzero relation among these derived bases and R_D , within its permitted choices and queries. Modelling the hash-to-group outputs as independent random group points gives the usual discrete-logarithm-based justification. The concrete hash is an instantiation of that model, not a proof of its own hardness.

The map from 512-bit digests to a roughly 255-bit group cannot be injective. Nor is the group hash's output type, by itself, a proof that it avoids the identity or the native base. The note type requires a nonidentity point, and native V has its own branch. A complete deployment must resolve exceptional generated points and asset-identity collisions consistently. It must not treat a negligible-probability edge case as an unspecified parser choice.

3 Issuance state and issue notes

3.1 The public creation record

An issue note contains

$$(d, \text{pk}_d, v, A, \rho, \text{rseed}).$$

Here d is an 11-byte diversifier, pk_d a nonidentity diversified transmission key, $v \in [0, 2^{64} - 1]$ an integer amount, A the asset base, $\rho \in \mathbb{F}_p$, and rseed 32 bytes of note randomness. The recipient construction is inherited from *Ironwood Guide*, §*Keys and viewing capabilities*, and *Wallet Guide*, §*Addresses and receiver selection*.

Unlike an encrypted transfer output, these issue-note fields are available to validators and chain observers. Anyone can reconstruct its commitment once the note-format derivations are instantiated. The issuer cannot conceal a larger minted value inside the commitment while publicly accounting for a smaller one: the validator recomputes the commitment from the public amount.

The recipient's spending authority is still private for an ordinary issue note. Knowing its commitment opening is not knowing its nullifier key or spending key. Later unlinkability nevertheless depends on the inherited nullifier, proof, and transaction-privacy assumptions; it is not an unconditional consequence of putting the note into a common tree.

3.2 Freshness derived from the transaction

An *Action Group* is an ordered nonempty list of transfer Actions sharing a public anchor, spend/output/custom-asset enable flags, an expiry bound, a public burn list, and a proof cover-

ing the group; each Action also has its spend-authorisation signature. This is the logical grouping required by the selected design’s referenced carrier, not a claim that the withdrawn carrier supplies a deployable encoding. The selected issuance rules allow at most one such group.

Every issuance bundle accompanies a transfer bundle with an Action from which a nullifier can be taken. Let n_0 be the nullifier of the first Action in the first Action Group. For issuance Action index i and note index j , both zero-based 32-bit integers, the issued note uses

$$\rho_{i,j} = \text{ToBase}\left(\text{PRFexpand}_{\text{enc}_{32}(n_0)}(0x84 \parallel \text{enc}_4(i) \parallel \text{enc}_4(j))\right).$$

Here ToBase interprets the 64 output bytes little-endian and reduces modulo p . The first byte is $0x84$.

This gives each note a transaction-dependent origin, rather than letting an issuer reuse an arbitrary ρ in several transactions. The input-nullifier uniqueness rules and the transaction signature bind the construction to the containing transaction. The PRF’s key is public in this use; secrecy is not its purpose here. Collision avoidance for derived ρ values is a computational hash requirement, not literal injectivity of reduction modulo p .

An `IssueAction` contains a description hash h_D , a finalisation flag, and an ordered list of issue notes. An issuance bundle contains one issuer identifier, an ordered list of those Actions, and the issuer’s signature over the *whole transaction* signature digest. It is not merely a signature over the issuance amounts.

3.3 The reference note

For the first issuance of an asset, the first note must have value zero and the default external diversified address derived from the all-zero 32-byte Orchard spending key, at diversifier index zero. Its address bytes are the following concatenation:

```
cc36601959213b6b0cdb96a75c17c3a668a97f0d6a
8c5ce164a518ea9ba9a50ea75191fd861b0ff10e62b0.
```

These 43 bytes are $d \parallel \text{pk}_d^*$. The derivation, rather than a visual resemblance to a human-readable address, gives the meaning of this constant.

The reference is a real committed leaf but holds no value. Its note data and its deliberately public spending key are known. After its inclusion and an accepted anchor are available, anyone can use the corresponding witness material to demonstrate the existence of the asset in the split construction below. This explains how that construction can obtain a publicly usable membership witness without publishing a value-bearing spend key.

The reference’s publicly known keys make it an exception to the ordinary issue note’s private-key argument. It must never be used as a recipient for funds that are meant to be owned exclusively by one person. Copying its opening does not mint units: a split input contributes zero to the value equation.

3.4 The state transition

For a missing asset entry, read

$$I[A] = (0, \text{false}, \perp), \quad \text{MAX_ISSUE} = 2^{64} - 1.$$

The balance records *issued minus burnt* units. It is a cap on circulating ledger supply, not on the sum of all issuance events over the asset’s lifetime.

Start transaction validation with a working copy $J \leftarrow I$. The burn set contains distinct pairs (A, b_A) , with

$$A \neq V, \quad A \neq \mathcal{O}, \quad 1 \leq b_A \leq 2^{63} - 1.$$

Every point must decode canonically. There may not be two burn entries for the same base.

Process all burns before any issuance:

1. For each (A, b_A) , require $J[A].\text{balance} \geq b_A$.
2. Replace that balance by $J[A].\text{balance} - b_A$.

The transaction's proof and binding signature, not this public subtraction alone, establish that the sender actually supplies the burnt units.

If there is an issuance bundle, require the accompanying Action Group and a valid issuer signature over the transaction digest. Process the issuance Actions in order:

1. Derive A from the bundle's issuer and the Action's h_D . Require $J[A].\text{final} = \text{false}$.
2. If $J[A].\text{reference} = \perp$, require a first note and check that it is the zero-valued reference note just specified. Store it as $J[A].\text{reference}$.
3. For each note in order, validate its recipient and field encodings, compute its required $\rho_{i,j}$, and check its equality with the supplied ρ . Construct its note commitment from its public fields and the selected note derivations, rejecting any undefined commitment operation.
4. Before adding v , require $J[A].\text{balance} + v \leq \text{MAX_ISSUE}$ as an integer inequality. Add v with checked arithmetic.
5. If the Action finalises issuance, set $J[A].\text{final} \leftarrow \text{true}$.

Two issuance Actions for one asset are not made independent by sharing a bundle. The second observes the first's state changes. If the first finalises, a later Action for that asset fails. An empty list cannot supply the mandatory first reference for a new asset.

Only if the entire transaction passes all its proof, signature, encoding, nullifier, anchor, and state checks does J become the new issuance state. At activation the map is empty. Within a block, each transaction sees the previous transaction's output map; the first sees the parent's final map.

3.5 Supply and finalisation invariants

For one asset and any accepted branch prefix, let M_A be the sum of public issue-note values and B_A the sum of public burn values in that prefix. By induction on the ordered operations,

$$I[A].\text{balance} = M_A - B_A, \quad 0 \leq M_A - B_A \leq 2^{64} - 1.$$

The base case is the empty map. A burn subtracts the same amount from both expressions, with its lower-bound check preserving nonnegativity. An issuance adds the same amount, with its checked upper bound preserving the cap. Transfer-only transactions alter neither side. This proves the public accounting invariant exactly.

It does *not* yet prove that privately spendable notes have that total value. That stronger statement also requires the transfer conservation argument, proof knowledge soundness, note binding, and the nullifier uniqueness rules.

For example, issue 1000 units, burn 100, then issue 50. The state balances are 1000, 900, 950, while lifetime issuance is 1050. At a full supply cap, burning b and then issuing b in one transaction is compatible with the cap, provided the asset has not been finalised. Reversing the specified operation order could change acceptance at the cap.

Finalisation is a monotone Boolean along a branch. Once true, no later issuance Action for that asset is permitted. Burning remains possible and does not clear the flag. Thus finalisation limits future creation, not holders' transfers, and the outstanding supply can still decrease.

3.6 Appending the commitments

The new tree leaves are ordered independently of the order in which public supply changes are evaluated:

1. Append transfer-output commitments in Action Group order and Action order.
2. Append issue-note commitments in issuance Action order and note order.

The reference note is included in the second part like any other issue note. Commitment positions, paths, and anchors must agree with this order. A public issue-note description is not a substitute for a Merkle path against a recognised anchor.

4 Asset-bearing notes and conservation

4.1 Binding the asset into the note

The conceptual note relation now opens

$$(d, \text{pk}_d, v, A, \rho, \psi, \text{rcm}).$$

For the native base $A = V$, its commitment is the inherited Orchard commitment, without an appended asset field.

For a custom asset define the bit string

$$m_A = g_d^* \parallel \text{pk}_d^* \parallel \text{LE}_{64}(v) \parallel \text{LE}_{255}(\rho) \parallel \text{LE}_{255}(\psi) \parallel A^*,$$

where point encodings in this expression are their 256-bit forms. Then

$$\begin{aligned} \text{cm}_A &= \text{SinsemillaHashToPoint}(\text{z.cash:ZSA-NoteCommit-M}, m_A) + [\text{rcm}]R_c, \\ R_c &= \text{GroupHash}^{\mathbb{P}}(\text{z.cash:Orchard-NoteCommit-r}, \epsilon). \end{aligned}$$

The empty string is ϵ . *Crypto Guide*, §Groups, commitments, and Sinsemilla, gives the bit chunking, incomplete-operation rejection, and the distinction between the full point and its extracted x -coordinate. The leaf is the inherited extracted commitment cm_x .

For a custom note, the input has $256 + 256 + 64 + 255 + 255 + 256 = 1342$ bits before Sinsemilla padding. Changing the asset changes committed input bits as well as the hash domain relative to a native note. The randomness base R_c is shared; the native branch remains byte-for-byte the native commitment construction.

The proof witnesses A once and uses it for the old note, the new note, and the value commitment. Three unconstrained copies merely bearing the same name would not enforce this requirement.

4.2 Randomness and the remaining note-format parameter

The note's ψ is derived by the inherited ZIP 2005 function

$$\psi = \text{ToBase}(\text{PRFexpand}_{\text{rseed}}(0x09 \parallel \text{enc}_{32}(\rho))).$$

The split-only randomness below deliberately uses $0x0A$, not this $0x09$ domain.

The selected ZIP pair calls the ZIP 2005 note-randomness derivation with its placeholder ZSA lead byte. The core rule distinguishes legacy $0x02$ randomness from recoverable $0x03$ randomness, but neither fact assigns a ZSA format byte. In particular, one cannot replace the placeholder by $0x03$ just because it once appeared in a withdrawn carrier.

Accordingly, the relation here takes a valid note opening (ρ, ψ, rcm) as witness data. Deriving rcm from the public rseed of an *issue note*, or constructing an interoperable encrypted transfer plaintext, additionally needs the missing ZSA note-format instantiation. This is a concrete boundary of the selected draft, not permission for validators to choose their preferred derivation.

4.3 A private multibase value commitment

For an ordinary Action with old and new values v_o, v_n , set

$$C = [v_o - v_n]A + [r]R_v.$$

The amount difference is a signed integer, subsequently interpreted modulo q for scalar multiplication. The circuit's range checks must establish that the note values are actual 64-bit integers before this scalar interpretation.

There is one randomness scalar r for each Action. If r is uniform in $\mathbb{F}_q$, then for any fixed A and value difference, C is uniform in $\mathbb{P}$: translation by the fixed amount term permutes the group. Thus this one commitment perfectly hides its amount term under an independently uniform mask.

That marginal statement is not a proof of joint transaction privacy. The transaction reveals a linear combination of commitments through its binding key, contains note commitments and ciphertexts, and may publish issuance or burn information. Security for all those correlated objects needs the inherited proof and encryption properties and appropriately joint randomness assumptions.

4.4 The transaction equation

Let $\Delta_i = v'_{o,i} - v_{n,i}$, where $v'_{o,i}$ is the effective input value: the real value for a normal input and zero for a split input. Then

$$C_i = [\Delta_i]A_i + [r_i]R_v.$$

Let b_{nat} be the native value leaving this bundle, including its contribution towards the transaction fee, and let (A, b_A) range over the public burn set. The binding public key is

$$K_{\text{bind}} = \sum_i C_i - [b_{\text{nat}}]V - \sum_A [b_A]A.$$

For an honest balanced transaction,

$$K_{\text{bind}} = [k_{\text{bind}}]R_v, \quad k_{\text{bind}} = \sum_i r_i.$$

The binding signature uses the inherited binding-signature generator R_v , not the spend-authorisation generator or an issuer key, and signs the complete transaction signature digest.

Collecting terms by asset gives

$$K_{\text{bind}} = [\sum_i r_i]R_v + \sum_A [\delta_A]A,$$

where native V is included among the A 's, and

$$\delta_A = \begin{cases} \sum_{i:A_i=V} \Delta_i - b_{\text{nat}}, & A = V, \\ \sum_{i:A_i=A} \Delta_i - b_A, & A \neq V, \end{cases}$$

with $b_A = 0$ when no burn entry exists.

The equality is algebra. The claim that a successful adversary cannot leave some $\delta_A \neq 0$ is computational. If the Action proofs permit extraction of their values, bases, and masks, and the signature argument permits extraction of a scalar k for K_{bind} , an imbalance gives the known relation

$$\sum_A [\delta_A]A + [\sum_i r_i - k]R_v = \mathcal{O}.$$

This contradicts the no-efficient-relation assumption when at least one coefficient is nonzero modulo q .

Binding alone is insufficient for this argument: a binding commitment does not automatically reveal its opening to a reduction. Nor does ordinary fixed-key signature unforgeability by itself establish extraction for an adversarially constructed binding key. The required joint knowledge and composition argument is the same kind of obligation identified in *Ironwood Guide*, §*Balance, digests, and authorisation*, now with multiple asset bases. This volume derives the relation but does not claim a new proof of the complete Halo/signature composition.

4.5 From field balance back to integer balance

The relation argument establishes $\delta_A = 0 \bmod q$. To conclude $\delta_A = 0$ as an integer, prove an absolute bound $|\delta_A| < q$.

If there are at most N Actions, each effective input and output lies in $[0, 2^{64} - 1]$. Therefore

$$\left| \sum_{i:A_i=A} \Delta_i \right| \leq N(2^{64} - 1).$$

For a custom asset, a sufficient bound is

$$N(2^{64} - 1) + (2^{63} - 1) < q.$$

For the native asset use the permitted absolute native balance instead of the custom burn maximum. For example, even the very loose $N \leq 2^{32} - 1$ would make these quantities below 2^{97} , far below the Pallas group order. This is a sufficient arithmetic condition, not a claim that the unspecified carrier actually allows billions of Actions.

A complete transaction format must supply an explicit count or size bound from which this condition follows. Range checks on individual notes do not, by themselves, prevent arbitrarily many field-wrapped terms from summing to zero.

4.6 A two-asset check

Suppose an Action spends 80 units of asset A and creates 50, a second contributes a zero effective input and creates 20 units of A , and the burn set contains $(A, 10)$. The asset equation is

$$(80 - 50) + (0 - 20) - 10 = 0.$$

A separate native Action may pay a fee. Its native balance does not alter this equation.

Replacing the second output with 20 units of another asset B leaves nonzero per-asset coefficients. Even if the ordinary integers appear to add up, the group equation contains different bases. Acceptance would require a useful relation between those bases and the randomness base, unless a proof or range constraint has already failed.

5 Split inputs and the counterfeiting boundary

5.1 Why ordinary zero-value padding is insufficient

An Orchard Action has one input position and one output position. To turn one asset note into two output notes, the transaction needs a second input position. For the native asset, a zero-value dummy input can omit Merkle membership: the native base is a fixed protocol constant and the dummy contributes no money.

If that exception were extended to an arbitrary custom base, the prover could introduce a point with a known relation to V . Consider the hypothetical invalid choice $A_{\text{evil}} = -V$. Create one native output of value w and one custom output of value w , both with dummy zero inputs. Their amount terms are

$$[-w]V + [-w]A_{\text{evil}} = [-w]V + [w]V = \mathcal{O}.$$

The binding-key equation could then hold although the transaction created native value from nothing. All values can be small and in range. The error would be admitting an attacker-chosen base, not overflowing a machine integer.

This is an attack on the *hypothetical weakened relation*, not on the complete split construction. It shows why merely asserting that each Action uses the same base on its two sides is not enough.

5.2 Authenticate the base without consuming value

A split input carries the opening and membership path of an existing custom-asset note, with a private Boolean $s = 1$. The old commitment is opened with its *actual* old value v_o . Only its contribution to the value commitment is zeroed:

$$v'_o = (1 - s)v_o, \quad C = [(1 - s)v_o - v_n]A + [r]R_v.$$

When $s = 0$, the ordinary value equation is recovered. When $s = 1$, changing a large stored v_o cannot create spendable input value because the multiplier is exactly zero.

Let ν be the private native-asset indicator. The relation enforces

$$\nu \in \{0, 1\}, \quad \nu = 1 \iff A = V, \quad s \in \{0, 1\}, \quad s\nu = 0.$$

Thus split inputs are custom-asset inputs, not another native dummy-note encoding.

Write PathOK for the inherited permitted path predicate from *Ironwood Guide*, §*Actions and the proof relation*. It permits noncanonical 255-bit intermediate representatives that reduce to the assigned field values, and the zero-hash alternatives arising from exceptional Sinsemilla addition. It is not merely a canonical path recomputation and root comparison. The old-note membership condition is

$$(v_o = 0 \wedge \nu = 1) \vee \text{PathOK}(\text{cmx}_o, \text{path}, \text{position}, \text{anchor}).$$

All custom inputs require membership, including a zero-valued reference note and a split copy of a previously spent note. Whether the old note's ordinary nullifier is already spent is irrelevant to its use as a split witness; the split creates a different nullifier and supplies no old value.

The inherited key-consistency and spend-authorisation obligations must also be discharged. A Merkle path and a guessed opening of someone else's private note are not sufficient. The selected ZIP's informal freedom to reuse an old note does not remove those inherited proof equations. One can use one's own available witnesses, obtain the required cooperation, or use the deliberately public reference-note keys. The last option provides a universal usable source without assuming access to arbitrary holders' keys.

5.3 A fresh nullifier for the non-spending copy

A normal spend uses the note's fixed nullifier. Reusing that nullifier for every split would either expose repeated use of the same source or cause the spent-nullifier set to reject later splits. The split therefore introduces independent randomness.

Sample a fresh 32-byte seed u and compute

$$\psi_{\text{split}} = \text{ToBase}(\text{PRFexpand}_u(0x0A \parallel \text{enc}_{32}(\rho_o))).$$

Let

$$L = \text{GroupHash}^{\mathbb{P}}(\text{z.cash:Orchard}, L),$$

and let K be the inherited Orchard nullifier base. The relation uses a witnessed ψ_{nf} constrained by

$$(1 - s)(\psi_{\text{nf}} - \psi_o) = 0.$$

For ordinary inputs this forces the original note randomness; for a split the honest prover uses ψ_{split} . The nullifier equation is

$$\text{nf} = \text{Extract}([(\text{Poseidon}(\text{nk}, \rho_o) + \psi_{\text{nf}}) \bmod p]K + \text{cm}_o + [s]L).$$

The sum inside the brackets is first computed in the Pallas base field, then its canonical integer is used for scalar multiplication. It is not initially an addition modulo the group order q . The Poseidon nullifier PRF and Extract are the inherited functions described in *Ironwood Guide*, §*Nullifiers and authenticated existence*.

The old commitment still uses ψ_o , not ψ_{split} . Otherwise a prover would change the leaf whose membership it had supposedly demonstrated. For the new note, the inherited Action linkage uses $\rho_n = \text{nf}$.

The L offset is a group-level domain separation. It does not reserve a visibly separate range of nullifier bytes: extraction can map distinct points to the same field value. Independence and collision resistance here are computational properties, not an assertion that normal and split nullifiers occupy disjoint sets.

The seed-to- ψ_{split} expansion is an honest-prover randomness algorithm, not an additional BLAKE2b computation proved inside the Action circuit. The circuit permits a witnessed split ψ_{nf} subject to the stated relation. Consensus still checks every emitted nullifier, including split nullifiers, for duplicates within the transaction and against the branch's spent-nullifier set.

5.4 Provenance is an inductive invariant

Assume note-commitment binding across the native and custom branches, key-commitment binding for the inherited ownership relation, authenticated tree membership under the permitted representations, and knowledge soundness of the Action proofs. Also assume the Sinsemilla exceptional-case hardness from *Crypto Guide*, §Groups, commitments, and Sinsemilla: efficiently causing the permitted failure output $\perp$, rather than an ordinary commitment or hash, yields an infeasible generator relation. This assumption is needed both for note/key commitment alternatives and for zero-hash path acceptance. A proof of the weakened relation alone does not remove those alternatives. Outside these explicitly bounded failure events, consider accepted custom-asset notes in their order of creation.

An issue note's base is publicly derived from the issuer and description hash and checked by the issuance validator. Every custom transfer output shares its base with an input note whose membership is proved. That input therefore supplies an earlier committed occurrence of the base. By induction, the base traces to a publicly validated issuance. A split input preserves this argument because its membership condition is unchanged.

The induction concerns *asset origin*, not ownership of all previous notes or preservation of their values. The latter is discharged by the effective-value equation and the binding signature. It also concerns the accepted branch and recognised anchors: a path into an unrelated or rejected tree does not establish provenance.

This separates two obligations that are easy to conflate. Membership prevents introduction of an arbitrary relational base; balance prevents creation of extra units of a legitimate base. Neither condition alone implies the other.

6 The Action relation and transaction validation

6.1 Public statement and private witnesses

The public statement contains the inherited anchor, value commitment, input nullifier, randomised spend-authorisation key, output commitment, and spend/output enable flags, together with the custom-asset enable flag e_A . The asset base and split/native indicators remain private.

The witness contains the old and new note openings, the old-note path and position, the inherited ownership and key-randomisation witnesses, the value-commitment mask, and the additional $A, \nu, s, \psi_{\text{nf}}$. The new circuit must connect these objects, rather than checking disconnected subrelations:

1. Validate each point and scalar in its proper type, and constrain both note values to 64-bit integers.
2. Constrain the inherited diversified-key, incoming-viewing-key, and randomised spend-authorisation-key equations. A split is not a blanket exception to key consistency.

3. Open the old and new note commitments using the one shared asset base and the appropriate native/custom branch, with the inherited exceptional Sinsemilla alternatives.
4. Enforce $\nu = 1$ exactly when $A = V$, require Boolean s, ν , and require $s\nu = 0$.
5. Require PathOK unless it is a native zero-value dummy. Check the conditional normal/split nullifier and the new-note ρ linkage.
6. Enforce the value commitment with effective old value $(1 - s)v_o$.
7. Enforce the inherited spend/output enable conditions and require $e_A = 0 \Rightarrow \nu = 1$.

The last rule prevents a prover from hiding custom-asset use in a proof whose public enable flag disables it. Selecting the flag for a transaction or upgrade is a consensus policy obligation, not a choice that can safely be delegated to an unconstrained private witness.

6.2 How the Boolean conditions become field equations

Halo 2 Guide, §*From a relation to a circuit*, explains selectors, constraints, and range checks. Here a Boolean s is enforced by $s(s - 1) = 0$ in the prime field. Similarly enforce $\nu(\nu - 1) = 0$.

Let a_x, a_y be the canonical affine coordinates of the nonidentity A , and v_x, v_y those of V . For the native direction use

$$\nu(a_x - v_x) = 0, \quad \nu(a_y - v_y) = 0.$$

For the non-native direction, witness u_x, u_y and enforce

$$(1 - \nu)((a_x - v_x)u_x - 1)((a_y - v_y)u_y - 1) = 0.$$

When $\nu = 0$, at least one coordinate difference must have an inverse. If both are zero, both parenthesised factors are -1 , so the equation fails. If one is nonzero, its inverse makes the product zero. This enforces inequality of points, not just inequality of their x -coordinates.

In the ordinary nonexceptional path calculation, let r_{path} be the computed root. The membership gate is

$$(v_o + 1 - \nu)(r_{\text{path}} - \text{anchor}) = 0.$$

Because v_o is range-constrained and $2^{64} < p$, the first factor is zero exactly when $v_o = 0, \nu = 1$. Without the range check it could also vanish through an unintended field value. This gate explains the native-dummy bypass; the complete inherited PathOK predicate retains the representation and zero-hash allowances stated above. It is not strengthened to canonical root equality by this illustrative equation.

The inherited enable conditions are

$$v_o(1 - e_{\text{spend}}) = 0, \quad v_n(1 - e_{\text{output}}) = 0,$$

and the new condition is

$$(1 - e_A)(1 - \nu) = 0.$$

Notice that the spend-enable equation uses the *old note value*, not its zeroed split contribution. Copying a nonzero old note still requires spends enabled under the inherited condition. A zero-valued public reference avoids that particular nonzero-value requirement.

These equations explain the new gates. They do not replace the elliptic-curve, Sinsemilla, Merkle, and key-consistency gadgets they connect, or prove that a particular compiled circuit has wired them all correctly.

6.3 Complete validation order at the defined interface

A validator must combine the following checks, using the generic atomic state-transition contract in Consensus and the inherited payment checks in *Ironwood Guide*, §*Pool rules and complete validation*.

1. Select the rules for the containing branch and activation context. Parse canonical transaction and field encodings, reject inconsistent counts, and enforce resource bounds. Under the selected ZIP 227 rule, the Action Group count is zero or one and its group expiry height is zero.
2. Check the bundle flags, recognised anchor, and every nullifier's uniqueness within the transaction and against the branch state. A split nullifier is not exempt.
3. Reconstruct the exact public proof instances from the transaction fields and verify the proof under the matching circuit/verifying key.
4. Compute the transaction signature digest and verify every spend-authorisation signature under its randomised key. Recompute the multibase binding key and verify its binding signature.
5. Check the distinct public burn entries and run the burn-first issuance-state transition. If issuance is present, verify its issuer signature, reconstruct every issue note, and check the reference, supply, and finalisation rules.
6. Check the inherited native transaction balance and fee conditions, as well as all remaining contextual transaction rules.
7. Commit all state changes together: spent nullifiers, transfer and issuance commitment leaves in the specified order, and the new issuance map. Otherwise change none of them.

The expensive cryptographic checks can be batched or reordered operationally when that preserves acceptance semantics. The state effects cannot be partially applied merely because an earlier check succeeded.

6.4 What is still needed for a deployed endpoint

Three unresolved interfaces in the selected pin prevent the above from being a complete network implementation.

1. *Carrier and digests*. A non-withdrawn transaction encoding must frame all transfer, issuance, and burn fields, fix count/size limits, assign branch/version identifiers, and specify which exact bytes every signature and authorising commitment covers.
2. *Note-format instantiation*. The ZSA plaintext lead byte, note-randomness derivation, encrypted plaintext lengths, and incoming/outgoing decryption behaviour must agree. In particular, validators must deterministically reconstruct issue-note commitments from the public fields.
3. *Circuit and activation*. The complete relation must be implemented and its verifying key selected by explicit consensus rules, including the meaning of the enable flag and exceptional invalid asset-generation outcomes.

The new group equations do not fill these gaps automatically. Nor do they supply a new proof of the complete multibase knowledge-soundness and authorisation composition.

Within the specified interface, the critical path is now explicit: issuer identity determines a hashed asset base; public issuance places that base in authenticated notes; every custom transfer preserves such provenance; constrained value commitments and authorisation enforce conservation; public burns and issuance update a branch-local supply map. Every step has a distinct cryptographic or state-machine obligation, and no step can be replaced by trusting a displayed asset name.

Part X**Crosslink Guide: Two confirmation guarantees from one transaction chain****Abstract**

A proof-of-work chain can keep accepting blocks when a voting committee cannot communicate. A voting protocol can refuse to make conflicting decisions even during a partition. Crosslink combines these different guarantees without changing the order of transactions inside the underlying chain. This volume constructs the exact Crosslink 2 state machine at one formal source pin: its two kinds of blocks, cross-links, validity rules, persistent finalised view, and bounded-available view. It proves the elementary safety implications of those rules, while keeping the assumptions about the two component protocols and their progress explicit.

1 Two ledgers and their guarantees**1.1 The payment question**

A recipient who sees a payment in a recently mined block faces two questions: is the block valid, and will the network later replace its branch? *Consensus Guide*, §*Work, difficulty, and chain selection*, constructs the proof-of-work rule and cumulative-work comparison. §*Reorganisation and confirmation confidence* explains why a finite confirmation depth gives conditional confidence, not an unconditional promise against every possible future execution.

Crosslink supplies two views of the *same kind of transaction chain*. A *finalised view* advances only when the construction's finalisation condition is met. A *bounded-available view* may include a confirmed tail beyond that point. If finalisation does not advance for too many blocks, the design restricts which new blocks may carry ordinary user activity.

The name does not mean that every request is always answered within a fixed time. The bound is on a specified block-height gap. Voting can remain stalled indefinitely, and restricted blocks can still continue to be produced.

1.2 The selected construction

The endpoint is the Crosslink 2 formal construction at `fe6e1d6`. It combines a best-chain protocol, denoted Π_b , and a voting agreement protocol, denoted Π_f . These are parameterised interfaces with explicit requirements; the source does not specify one production-ready byte encoding, staking system, and voting implementation for all of them.

The elementary implications proved here concern the *modified* component protocols actually participating in Crosslink. Transferring assumptions from an unmodified protocol or from a different prototype requires an additional argument. In particular, the separately identified v13 implementation in the last section is not silently treated as this formal state machine.

1.3 Executions, honesty, safety, and progress

An *execution* is a history of message sends, message deliveries, local computations, and decisions, starting at protocol initiation. We index events by times $t \geq 0$. A complete history fixes which events occurred, even if the protocol originally chose some of them randomly.

A node is *honest at time t* when it has followed its specified rules throughout the history up to and including t . It is therefore also honest at every earlier time. This convention is essential when a proof refers to an earlier event at which a node stored its current finalisation point. Merely behaving correctly for one recent instant would not satisfy the definition.

A *safety property* says that a forbidden outcome never occurs; for example, two honest nodes never finalise conflicting chains. A *liveness property* says that progress eventually occurs under stated conditions; for example, a transaction is eventually included or a finalisation point eventually advances. These are different assertions. A protocol that makes no decisions can preserve agreement while failing to make progress.

The default communication model permits arbitrary message delays and losses. Required messages are authenticated, meaning their content is attributed to the actual sender under the signature assumptions in *Crypto Guide*, §*Security contracts and reductions*. A *network partition* is an interval during which some groups of nodes cannot exchange the relevant messages. No bounded-delivery assumption is smuggled into a safety proof.

Progress arguments may require intervals in which messages arrive within a delay Δ , sufficiently many honest producers participate, and sufficiently much voting weight is online. Those conditions must cover the actual interval and state in which progress is claimed. A timing argument for one successful interval does not automatically cover every later interruption and recovery.

1.4 Chains, prefixes, and compatibility

A chain is a sequence of blocks beginning with a distinguished genesis block. Each later block contains a collision-resistant hash reference to its parent. Subject to the hash-binding assumption from *Crypto Guide*, §*Authenticated trees and append-only histories*, its tip determines a unique ancestor sequence. We therefore use the same symbol for a tip and its chain.

Write $A \leq_b B$ when the best-chain block sequence A is a prefix of B , including equality. Write

$$A \sim_b B \iff A \leq_b B \text{ or } B \leq_b A$$

for *compatibility*. They conflict when neither is a prefix of the other. Use subscripts f for voting-chain blocks. Subscripts may be omitted when the chain type is unambiguous.

Two prefixes of one chain are compatible: compare their lengths; the shorter occupies the same initial positions of that chain. We will repeatedly use this simple fact.

A sequence $A_0, A_1, \dots$ is *monotone by extension* if $A_j \leq A_k$ whenever $j \leq k$. Pairwise compatibility is weaker. The sequence

$$[g, a, b], \quad [g, a]$$

contains compatible chains but retracts the last block. Thus agreement between a node's views at two times does not, on its own, prove that it never rolls its own view back.

The height of genesis is zero. H^{-k} means remove k blocks from the tip, stopping at genesis if necessary. The tip has depth zero; its parent has depth one. A node's current best-chain tip is C_i^t , with i the node and t the time. This function of node and time is its *view*. A view is local knowledge, not an omniscient global list of all blocks.

2 Chain and agreement interfaces

2.1 The best-chain component

A best-chain block is valid only if its parent is valid and it satisfies the inherited block and transaction rules. Its header summarises and commits to the block. Header validity is necessary but not sufficient for full block validity: a header with valid proof of work may still commit to an invalid transaction.

Each valid chain has a score in a totally ordered set: any two scores can be compared, and the comparisons are consistent. Extending a chain by one valid block strictly increases its score. For proof of work, the score is cumulative work, not merely height. The node selects a maximum-score valid chain it knows, with a specified tie-break rule.

The Crosslink validity additions restrict the set of valid chains. They do not replace the score comparison with a preference for whichever branch has the most attractive voting certificate. Any claimed

security of this modified process must account for the validity restrictions as well as the unchanged score function.

2.2 Two chain-security properties

Fix a positive integer confirmation depth σ . *Prefix Agreement at depth σ* means that for all honest observations, at possibly different times,

$$(C_i^t)^{-\sigma} \sim_b (C_j^u)^{-\sigma}.$$

The entire execution is quantified over. Agreement only at the same instant would allow every node to switch together between two incompatible histories on alternating days.

Prefix Consistency at depth σ is the directional condition

$$t \leq u \implies (C_i^t)^{-\sigma} \leq_b C_j^u$$

for nodes honest at their respective observation times. The earlier confirmed prefix must still be inside the later *full* best chain.

Prefix Consistency implies Prefix Agreement at the same depth. Order the two times, say $t \leq u$. By consistency the earlier truncated chain is a prefix of C_j^u ; the later truncated chain is also a prefix of C_j^u . The two are therefore compatible. This argument does not say that the later truncated chain must be at least as long as the earlier one.

These are properties of an execution, not automatic facts about every proof-of-work network. One can subsequently assign a failure probability under a mining and communication model. Long partitions, censorship of a node's block information, or an adversarial work advantage can invalidate the premises.

2.3 Voting epochs, proposals, and certificates

The agreement component proceeds through numbered *epochs*. An epoch is a voting opportunity with a designated proposer and a fixed assignment of voting units to participants. A participant may control several units. Let the total number of units in epoch e be $n_e > 0$. Weights can equivalently be represented by nonnegative amounts and summed; the counting argument below uses the integer-unit model.

A *proposal* identifies its epoch, its parent voting block, and its payload. The proposer signs all of this content. A *ballot* is an authenticated vote for that precise proposal. A *certificate*, called a notarisation proof by the formal construction, demonstrates that distinct voting units of total weight at least $2n_e/3$ supported it, together with any extra conditions imposed by the component protocol.

A direct certificate implementation could list each participating identifier and its valid signature and add its known weight, rejecting duplicate identifiers or units. Compact certificate schemes may replace that list only if their verified statement establishes the same property. A count asserted inside a payload is not such a proof.

The resulting voting block consists of the proposal and its certificate, signed again by the proposer. The parent hash of the next proposal commits to the *whole* parent voting block, including its certificate and outer signature. The formal model requests strong signature unforgeability: without the key an adversary cannot create a new accepted signature even for a message already signed, as defined in *Crypto Guide*, §*Security contracts and reductions*.

Proposal validity and block validity are distinct. A valid proposal has no certificate yet. A valid voting block also requires its certificate, its outer signature, and a valid parent chain. An honest proposer emits at most one proposal and one corresponding signed block for its epoch. A dishonest proposer may produce different signed representations of the same certified proposal; uniqueness of the proposal does not make the entire block encoding unique.

2.4 The one-third condition and what it proves

A voting unit is cast non-honestly if it is used without its holder's authorisation, used for two distinct proposals in an epoch, or used contrary to the honest-voting rules. The selected condition is:

for each epoch e , fewer than $n_e/3$ units are ever cast non-honestly.

The word *ever* includes later use of an old key to forge historical ballots. Deleting an expired signing key can therefore be part of meeting the assumption. A majority of currently honest machines does not imply that all their historical keys remain safe.

Suppose two distinct proposals for epoch e each have sound certificates, supported by sets S, S' . Then

$$|S \cap S'| = |S| + |S'| - |S \cup S'| \geq \frac{2n_e}{3} + \frac{2n_e}{3} - n_e = \frac{n_e}{3}.$$

Under the strict one-third bound, the intersection contains an honestly cast unit. Such a unit cannot vote for two distinct proposals, a contradiction. Hence at most one proposal in that epoch can obtain a valid certificate.

For $n_e = 10$, a certificate needs at least seven units and two certificates intersect in at least four. The strict bound permits at most three non-honest units. Rounding strengthens the intersection, but does not change the need for the strict bound in the general statement.

This proves *same-epoch proposal uniqueness*. It does not prove agreement between decisions formed in different epochs. That needs the component's parent-selection, honest-voting, and finality rules. Nor does the argument prove that any certificate will ever be assembled.

2.5 Finality is evaluated in a context

The voting component supplies an efficiently computable function $F(B)$, the last voting block final in the context of B . It has the following contract:

$$\begin{aligned} F(B) = \perp &\iff B \text{ is invalid,} \\ F(B) \leq_f B &\text{ for valid } B, \\ B \leq_f B' &\implies F(B) \leq_f F(B'), \\ F(g_f) &= g_f. \end{aligned}$$

Here g_f is the voting genesis and $\perp$ denotes failure. The third rule says that extending a valid voting chain does not retract its contextually final prefix.

For a concrete illustration of such a function, the adapted Streamlet finality rule examines a valid voting chain for three adjacent blocks with consecutive epoch numbers. The middle block of each such triple is final in that context; $F(B)$ returns the latest such middle block, or genesis if no triple exists. Extending the chain preserves old triples, so this function has the required monotonicity. This illustrates the function, not an imported proof that every Crosslink instantiation of Streamlet satisfies all the remaining voting and timing assumptions.

Final Agreement is the component security premise: for all valid voting blocks B, B' observed at times when their observers are honest,

$$F(B) \sim_f F(B').$$

A block is not final merely because its payload calls it final. The context and the verified function F supply the evidence.

3 Cross-links and the state machine

3.1 Parameters and cross-linked objects

Choose $\sigma \geq 1$ for finalisation confirmation depth and a finality-gap bound L substantially larger than σ ; the formal design recommends $L \geq 2\sigma$. For the bounded-available view a client chooses $1 \leq \mu \leq \sigma$, normally $\mu = \sigma$. These are block counts, not time durations.

Each best-chain header H gains a field $\text{ctx}(H)$ committing to a voting block. Each non-genesis voting proposal and block B carries a list of exactly σ best-chain headers, in ancestor-to-descendant order:

$$\text{tail}(B) = [H_{h-\sigma+1}, \dots, H_h].$$

Thus the first list entry has greatest depth relative to its tip. The snapshot is its parent:

$$S(B) = \text{parent}(\text{tail}(B)[0]) = H_{h-\sigma}.$$

It is neither the first supplied header nor the last one.

Set $S(g_f) = g_b$, where g_b is best-chain genesis, and $\text{ctx}(g_b) = g_f$. The voting genesis carries a special empty tail, not a fabricated σ -header list. The explicit genesis convention avoids trying to manufacture two ordinary cryptographic hashes that refer cyclically to each other.

For any valid best-chain block H , define

$$\text{LF}(H) = F(\text{ctx}(H)), \quad Q(H) = S(\text{LF}(H)).$$

The first is a voting block; the second is a best-chain block. Keeping these types distinct prevents a height in one chain from being compared accidentally with a height in the other.

3.2 Voting-chain validity

A non-genesis voting proposal or voting block must satisfy its inherited proposal or block rules, including parent validity. It additionally satisfies:

1. *Snapshot linearity*: $S(\text{parent}(B)) \leq_b S(B)$.
2. *Tail confirmation*: its supplied σ headers form the tail of a fully valid best-chain block sequence.

Full validity is important. Checking only header hashes, targets, and proof of work does not validate the transactions underneath. The header list tells a validator what additional blocks it must obtain; it does not excuse obtaining and validating them.

Snapshot equality with the parent is allowed. A voting chain can therefore keep making valid proposals about the same snapshot while the best-chain snapshot is temporarily unable to advance. Along any valid voting chain, snapshot linearity implies

$$B \leq_f B' \implies S(B) \leq_b S(B')$$

by induction over the intervening parent links.

These are objective validity rules: every observer with the same objects and ancestor data obtains the same result. A certificate with all voting weight behind it cannot make an objectively invalid snapshot chain valid.

3.3 Best-chain validity

Each non-genesis best-chain block H , with parent P , must satisfy its inherited block and transaction rules and:

1. $\text{ctx}(H)$ is a valid voting block.

2. $\text{LF}(P) \leq_f \text{LF}(H)$.
3. $Q(H) \leq_b H$.
4. With

$$d_{\text{fin}}(H) = \text{height}(H) - \text{height}(Q(H)),$$

either $d_{\text{fin}}(H) \leq L$, or $\text{Stalled}(H)$ is true.

The predicate Stalled specifies the permitted restricted blocks during a long finalisation gap. It is an application-level parameter of this formal construction; it must be fixed as an objectively checkable block-validity rule.

The third rule makes the height difference meaningful: the snapshot is an ancestor, not an unrelated block at a smaller height. The gap depends on the block's committed context, not on whichever finalisation point one observer currently stores.

The second rule is about the *last final ancestor* of the contexts. It does not require the raw contexts themselves to form one monotonically extending voting chain along every best-chain branch.

3.4 Honest proposal, voting, and production

An honest voting proposer waits until it knows a best chain with at least $\sigma + 1$ blocks including genesis. It tries to use that chain's last σ headers. If the resulting snapshot would violate snapshot linearity relative to the proposal's parent, it repeats the parent's header list. It otherwise follows the component's proposer and parent-selection rules.

An honest voter first validates the offered header tail and its full blocks, updating its views of both components. Each best-chain block may refer to a voting context, whose ancestors and snapshot blocks must in turn be validated. Already validated objects can be cached by their hashes. The resulting dependency graph must be resolved to known valid ancestors; an unresolved cycle is not validation evidence.

After updating its view, the voter votes only if the proposal is objectively valid *and*

$$S(B) \leq_b (C_i^t)^{-\sigma}.$$

It also obeys every inherited voting condition. The offered tail need not be identical to the voter's eventual best-chain tail: both can extend the same snapshot on different short branches. The snapshot itself must be σ -confirmed in the voter's updated best chain.

An honest best-chain producer retains the maximum-score rule for choosing its best-chain view. For a proposed new block with parent P , the formal context-selection rule considers valid voting-chain tips T satisfying

$$\text{LF}(P) \leq_f F(T).$$

It selects a longest such voting chain; ties maximise the best-chain score of $S(F(T))$, then choose the smallest hash. The resulting block must still satisfy *all* best-chain validity rules, including snapshot ancestry.

In particular, this ranking does not authorise attaching a foreign-fork snapshot to an incompatible best-chain parent. Proving that the ranking and the complete validity conditions always permit the desired progress is a liveness obligation. The simple existence of some valid context, shown later, is not by itself a proof about this ranking in every reachable state.

4 Ledger extraction and safety

4.1 Computing a candidate

For two chains, their *last common ancestor* is the longest prefix they share. It can be found by following parent links until the histories meet. Define

$$N(H) = \text{LCA}_b(Q(H), H^{-\sigma}).$$

By construction,

$$N(H) \leq_b Q(H), \quad N(H) \leq_b H^{-\sigma}.$$

Because block validity already gives $Q(H) \leq_b H$, the two arguments lie on one chain. Their last common ancestor is simply the shorter one. Writing the operation explicitly keeps the ancestor requirement visible.

Why not use $Q(H)$ directly? The snapshot in a committed voting context need not be σ blocks deep relative to this particular best-chain block H . The candidate must carry both forms of evidence: it lies in the final snapshot and in the σ -confirmed prefix of the local best chain.

4.2 The persistent finalised view

Node i stores f_i , initially best-chain genesis. Whenever its best-chain view changes to H , it performs:

1. Compute $N = N(H)$.
2. If $f_i \leq_b N$, set $f_i \leftarrow N$.
3. Otherwise retain f_i . If $N \not\leq_b f_i$, also record a finalisation safety hazard, with the conflicting chains and the relevant history of stored updates.

The superscript f_i^t denotes the stored value after the events through time t .

A candidate that is merely an ancestor of the stored point causes no rollback and no fork-conflict report. An incompatible candidate causes a report but still no rollback of the stored finalised view. This is not a rule to ignore the evidence or continue ordinary application operations after a hazard.

Local monotonicity. Every change to f_i is an extension. Induction on updates therefore proves

$$t \leq u \implies f_i^t \leq_b f_i^u$$

while the node follows the update algorithm. This is an exact state-machine property, with no voting threshold or honest-work assumption.

It does not prove that another node stored a compatible chain. Two disconnected nodes can each extend different chains monotonically. Global agreement needs a separate premise.

4.3 The bounded-available view

For a selected depth $1 \leq \mu \leq \sigma$, define

$$a_{i,\mu}^t = \begin{cases} (C_i^t)^{-\mu}, & f_i^t \leq_b (C_i^t)^{-\mu}, \\ f_i^t, & \text{otherwise.} \end{cases}$$

This immediately gives the local ledger-prefix property

$$f_i^t \leq_b a_{i,\mu}^t.$$

The otherwise branch matters. A maximum-work reorganisation can yield a shorter chain in block count, or a context whose candidate temporarily retreats along the same history. The application must not see its bounded-available ledger retract behind its already stored finalised prefix.

The bounded-available view is not generally monotone. Its unfinalised tail can be replaced. Reducing μ can reveal more recent blocks, but it also changes the chain-security assumption needed to protect that tail. It does not change f_i , whose candidate always uses the fixed σ .

4.4 A local update example

Take $\sigma = \mu = 2$. Suppose the validated local inputs have the following ancestry relations. The prime marks a new best-chain branch; all listed snapshots remain on their corresponding best chains, and both branches agree through height seven.

	C_i	$Q(C_i)$	$(C_i)^{-2}$	$N(C_i)$	$f_i ; a_{i,2}$
t_1	H_8	H_5	H_6	H_5	$H_5 ; H_6$
t_2	H'_9	H_4	H_7	H_4	$H_5 ; H_7$
t_3	H'_{10}	H_7	H'_8	H_7	$H_7 ; H'_8$

At t_2 , the candidate drops from height five to height four, but the stored finalised point stays at five. At t_3 it advances to seven. This table illustrates the local algorithm on already validated inputs; it is not a complete generated voting-and-mining execution.

Now suppose a later candidate lies on a fork that diverges before H_5 . Neither it nor H_5 is a prefix of the other. The algorithm retains its stored finality and records a hazard. The agreement theorems below identify which component premises must have failed for honest nodes to reach such incompatible finalisation evidence.

4.5 Two historical-witness lemmas

For every honest observation of f_i^t , there is an earlier time $r \leq t$ such that

$$f_i^t \leq_b (C_i^r)^{-\sigma}.$$

Choose the last time when the stored value changed. At that time it was assigned a candidate, and every candidate is a prefix of the then-confirmed best chain. If it never changed, choose time zero and the genesis chain.

The same argument also supplies a voting context observed honestly at some such earlier time, say B_i , with

$$f_i^t \leq_b S(F(B_i)).$$

At the assignment time $B_i = \text{ctx}(C_i^r)$, and the candidate was a prefix of $Q(C_i^r) = S(F(B_i))$. Again genesis handles the no-update case.

These lemmas are different. The first gives best-chain confirmation evidence. The second gives voting-finality evidence. Replacing one by the other without its corresponding premise would not establish the desired safety theorem.

The historical times are legitimate honest observations because honesty at t includes following the rules at all earlier times. The stored value may now be deeper than a current truncated view; the lemma does not falsely insist that its witness time is t .

4.6 Safety from the best-chain component

Theorem. If the modified best-chain component has Prefix Agreement at depth σ , then every pair of honest finalised views, at arbitrary times, is compatible:

$$f_i^t \sim_b f_j^u.$$

Proof. Use the first historical-witness lemma to obtain

$$f_i^t \leq_b (C_i^r)^{-\sigma}, \quad f_j^u \leq_b (C_j^s)^{-\sigma}.$$

Prefix Agreement makes the two right-hand chains compatible. Choose the longer of them. Both finalised views are prefixes of that one chain, so they are compatible. No Final Agreement assumption about the voting component was used.

4.7 Safety from the voting component

Theorem. If the modified voting component has Final Agreement, then every pair of honest finalised views is compatible.

Proof. Use the second historical-witness lemma:

$$f_i^t \leq_b S(F(B_i)), \quad f_j^u \leq_b S(F(B_j)).$$

Final Agreement gives $F(B_i) \sim_f F(B_j)$. Snapshot linearity along a valid voting chain therefore makes their snapshots compatible. Choose the longer snapshot. Both finalised views are prefixes of it, proving compatibility. No Prefix Agreement assumption about the best-chain component was used.

These are proofs about the defined state machine and its stated component premises. They do not fill missing proofs that the modified components meet those premises under a particular resource distribution, network schedule, or implementation.

4.8 What the disjunction means

Let P be the event that the modified best-chain execution has Prefix Agreement at depth σ , and F the event that the modified voting execution has Final Agreement. Let E mean a conflict between honest finalised views. The two theorems give

$$E \subseteq P^c \cap F^c.$$

A conflict therefore requires both safety premises to fail. Their failure is necessary, not sufficient: violating an assumption does not force a successful attack.

For any probability model on executions,

$$\Pr[E] \leq \Pr[P^c \cap F^c] \leq \min\{\Pr[P^c], \Pr[F^c]\}.$$

Multiplying the two failure probabilities would additionally require independence or another justified joint bound. The protocols interact and may share adversaries, so independence is not supplied by their different names. If cryptographic validity can also fail outside the model, its failure event must be included separately.

The property is often called assured finality. Here it means the precisely quantified compatibility statement above. Combined with the independent local monotonicity proof, it gives both no local retraction and no cross-node conflict under either component safety premise.

4.9 The bounded-available tail has a different guarantee

For every honest observation and $\mu \leq \sigma$, some earlier time $r \leq t$ satisfies

$$a_{i,\mu}^t \leq_b (C_i^r)^{-\mu}.$$

If the available view uses the current truncated chain, choose $r = t$. Otherwise it equals f_i^t , and the first historical lemma applies because a σ -truncated chain is a prefix of its μ -truncation.

It follows, by the same two-prefix argument, that Prefix Agreement of the best-chain component at depth μ implies agreement between all bounded-available views at that depth.

Under the stronger directional Prefix Consistency at depth μ , one can additionally prove, for $t \leq u$,

$$a_{i,\mu}^t \leq_b C_j^u.$$

Apply consistency to its historical witness at $r \leq t \leq u$. This says the earlier available view is in the later full best chain. It does not say it is contained in every later μ -truncated view.

Voting Final Agreement alone protects the finalised prefix, not every block in the available tail. Nevertheless, each local available view always extends its local finalised prefix. When two finalised views are compatible, their shorter prefix is therefore inside both nodes' available views. No stronger tail guarantee follows from the voting theorem alone.

4.10 Why transaction order is preserved

Both outputs f_i^t and $a_{i,\mu}^t$ are actual best-chain block sequences. Extract the transaction ledger by reading blocks from genesis and transactions in each block's committed order. A prefix of blocks gives a prefix of that transaction sequence.

There is no operation that takes transactions from incompatible branches, reorders them, and filters out the ones that have become invalid. Block heights, note-tree positions, nullifier sets, transaction expiry, and branch-local state evolution retain their inherited meaning on the chosen chain.

This is why the block-level proofs transfer cleanly to transaction prefixes. They still require full validation of those blocks. An authenticated reference to a snapshot without its valid underlying ledger is not enough.

5 Progress, stake, and settlement

5.1 Restricted blocks do not mean a stopped block chain

A stalled-block policy is a predicate on block content. One possible policy permits only a coinbase transaction, which creates and distributes that block's authorised subsidy without spending previous outputs. The formal construction leaves the exact policy to the transaction protocol. It is not correct to assume that every transaction type is automatically harmless merely because it is not an ordinary payment.

Assume that for every valid best-chain parent P , a producer can construct a valid coinbase-only child satisfying Stalled. Keep the parent's voting context:

$$\text{ctx}(H) = \text{ctx}(P).$$

Then context validity is inherited, $\text{LF}(H) = \text{LF}(P)$, and the extension rule holds. Also $Q(H) = Q(P) \leq_b P \leq_b H$. If the gap exceeds L , the stalled predicate permits the child.

This proves the existence of content satisfying the extra Crosslink validity rules. It does not manufacture proof of work: the inherited mining process still has to find a header meeting the required target. Under its usual progress conditions, that process can keep increasing the chain's work even though user spends are restricted.

The distinction matters. If block production itself stopped, there would be no further work accumulating above the tail. Stalled Mode is designed to restrict new economic exposure while allowing the chain's work process to continue.

5.2 What the gap bound actually bounds

Suppose along a valid branch the contextually final snapshot stays fixed at a block Q of height h_Q . For every later block that is *not* a stalled block,

$$\text{height}(H) - h_Q \leq L.$$

There are therefore at most L such block positions after Q while that snapshot remains fixed. Blocks above height $h_Q + L$ must satisfy the restricted policy.

This is an exact consequence of validity. It does not bound the number of later restricted blocks, the wall-clock duration of a stall, or the number of transactions that users may keep constructing and broadcasting off-chain. It also does not guarantee detection of every attack on voting. An attacker who can keep final snapshots advancing may avoid triggering the gap condition.

A crude time estimate might multiply a block count by an expected inter-block interval. An expectation is an average under a probability model, not a deterministic deadline. The selected construction supplies no universal formula turning L into a maximum number of seconds before recovery.

5.3 Why voting can still progress without a new snapshot

Snapshot equality is valid, so a proposer can reuse its parent's valid header tail. This supplies a proposal satisfying the two new objective snapshot rules even when no advancing snapshot is currently available.

It does not ensure that enough honest voters will support it. They must still see the snapshot as σ -confirmed in their updated best-chain views, meet the inherited voting rules, and exchange the required messages. A valid but permanently withheld proposal does not help liveness.

Advancing finalised snapshots needs more:

1. The best-chain component continues growing on a history that honest participants can learn and validate.
2. Enough voting weight sees an eligible, sufficiently confirmed snapshot, and the proposer/ballot/certificate schedule makes progress under the selected voting protocol.
3. A resulting final voting context is incorporated into valid best-chain blocks and becomes visible through their selected maximum-score chains.
4. The candidate extends the stored local finalised prefix and obtains its required best-chain confirmation depth.

If any stage stalls, the later stages cannot simply declare the missing evidence to exist.

The formal source's general liveness discussion does not supply a complete quantitative theorem for these interacting conditions. In particular, a proof that L is almost never reached under specified healthy conditions needs bounds on message delay, certificate production, honest voting weight, honest block production, and their interaction. No single inherited confirmation estimate substitutes for that proof.

5.4 What a stake-based instantiation must supply

The formal voting interface assumes that the voting-unit distribution for an epoch is fixed and known. A *stake-based* instantiation would derive that distribution from funds assigned to participation. Its *roster* is the resulting list of voting identities and weights.

That word does not fill in the required state machine. A concrete instantiation must specify:

1. Which accepted ledger state fixes the roster for each epoch, and how every node authenticates the same state.
2. Which transactions register an identity, assign or remove weight, and transfer any associated spending authority.
3. When a change becomes active and when previously assigned funds can be withdrawn.
4. Which exact epoch and roster identifiers are signed in proposals and ballots, and how signatures are checked against that roster.

A weight cannot be counted twice merely because its owner appears under two identifiers. A newly observed stake transaction on an unsettled branch cannot silently redefine the denominator n_e for an already running epoch.

A delayed withdrawal rule may support an economic penalty for provable misbehaviour. Such a penalty is not identical to cryptographic safety: unavailable funds, uncaught misconduct, or rewards do not change the set-intersection proof by themselves. The formal premise concerns which voting units are ever used non-honestly, including use of old keys.

The selected construction leaves these details as an instantiation boundary. Importing a prototype's bond amount, reward schedule, or delay constant would select an additional protocol whose security and ledger rules would also need to be constructed. Those numbers are therefore not presented as parameters of the formal safety theorems.

5.5 Starting a node and exposing its ledgers

The local variables initially contain genesis, but a newly started node must not represent that as a current settled view. The source recommends a voting checkpoint B_{cp} whose provenance the node trusts, and requires before exposing the ledgers:

$$\begin{aligned} B_{cp} &\leq_f LF(C_i^t), \\ S(B_{cp}) &\leq_b f_i^t, \end{aligned}$$

the timestamp of f_i^t is within a configured recency threshold.

A checkpoint is a trusted anchor in history, not a proof that its distribution was honest. A timestamp recency test also depends on the chosen clock and threshold policy. These checks address a startup interface; they do not remove the component security assumptions or prove that an isolated node has found the freshest chain.

The node must durably preserve its finalisation history if applications rely on local monotonicity across restarts. Resetting f_i and exposing a different branch as though no earlier promise had been made would not implement the same continuous application contract. A deployment must specify restoration, checkpoint upgrades, and hazard handling accordingly.

5.6 What a wallet may safely infer

A wallet first needs a valid transaction and a recognised note-tree anchor, as constructed in *Wallet Guide*, §*Constructing and authorising a transaction*. Crosslink changes the confidence attached to a branch prefix; it does not replace note membership or transaction authorisation.

An anchor carried by a block inside a settled prefix remains part of that prefix under the finality premises. But a full validator checks anchor validity against the ancestry of the *candidate block* it is validating, not against an unrelated displayed ledger or a height number alone. If the current best-chain view is transiently inconsistent with a retained finalised prefix, the application must examine that situation; it cannot infer immediate transaction acceptance merely from local finality storage.

Transaction expiry is another independent condition. A transaction may expire while ordinary inclusion is restricted. If the protocol allows non-expiring transactions, or users keep broadcasting fresh transactions during a stall, one cannot conclude that a fixed delay automatically clears every pending transaction.

For a concrete payment, the arithmetic stays the core example: an existing 80 000-zatoshi note funds 50 000 to the recipient, 20 000 change, and 10 000 in fees. If its containing block is only in the bounded-available tail, the recipient is using that tail's chain-security guarantee. When the block enters the finalised prefix, the disjunctive finality guarantee applies. Its proof relation, signatures, amounts, and ciphertexts are unchanged.

6 The implementation boundary

6.1 A pin is not an equivalence proof

The independently pinned implementation reference is `crosslink_monolith v13` at 807b995. Its own scope is a prototype. That description alone neither proves nor disproves a particular mechanism, but it prevents using the repository's existence as evidence of a deployed consensus rule set.

Several concrete interfaces show why the formal construction must not be identified with the code merely by matching names. The implementation has a voting payload with a parent reference and a header vector in `librustzcash/zcash_primitives/src/bft.rs`. Its `finalization_candidate` accessor returns the first header. In the formal construction, the snapshot is the *parent* of the first header. These are different objects.

The integration's proposal path selects a candidate height, fetches following headers, and limits how far a candidate may advance in one step. Its finalisation callback takes the first header's block

hash, stores a finalised height/hash pair, and requests a state finalisation update. Those operations live in `zebra-crosslink/zebra-crosslink/src/lib.rs`. They are not a literal execution of

$$N(H) = \text{LCA}_b(S(F(\text{ctx}(H))), H^{-\sigma})$$

followed by the persistent extension-only updater.

The voting engine lives in `tenderlink/src/lib.rs` and has its own proposal and multi-stage voting state machine. Its correctness must be analysed against its actual messages, weight accounting, state transitions, and integration callbacks, not inferred from the illustrative three-block finality function given earlier.

These are boundary examples, not a catalogue of historical implementations. This volume's theorems quantify over the formal objects and conditions already defined. Applying them to v13 would require a demonstrated mapping of the concrete execution into that model, preserving the relevant validity, ancestry, honesty, finality, and update properties.

6.2 The obligations of a complete instantiation

A complete implementation of the selected formal endpoint needs the following connected contracts.

1. Canonical encodings, hash domains, signatures, proposal identities, and certificate statements for both block types. All references must bind the intended full object.
2. An objectively validated graph of parent and cross-link dependencies, including full transaction validity and resource limits for fetching and verifying that graph.
3. A voting protocol with a concrete finality function and stated agreement and progress theorems under its actual corruption, message, and roster model.
4. A best-chain validity and production implementation matching the formal rules, together with an argument that its required confirmed-prefix properties hold.
5. Durable ledger extraction, startup criteria, and safety hazard handling that preserve the application's finality contract.

The requirement to validate full blocks is a data-availability requirement in a literal sense: the validator must be able to obtain the data it is asked to validate. A compact hash or certificate is not that data. Caching and validating shared ancestors can save repeated work; they cannot turn unavailable blocks into valid known state.

The safety proofs in this volume are deliberately explicit about their endpoint. They establish that the formal local algorithm converts either of two component agreement properties into compatible finalised transaction histories. They do not claim that a named voting crate, a branch tagged v13, or a particular stake threshold automatically meets every premise.

The construction's central dependency is now visible: valid snapshots move monotonically along voting chains; best-chain blocks commit to a contextually final snapshot; the candidate intersects that evidence with local work confirmation; and the persistent update exposes only extensions. The bounded-available view adds a separately protected tail. Each stage supplies a different guarantee, and the interface between them is part of the protocol.

Part XI

FROST Guide: Threshold custody for randomised RedPallas keys

Abstract

This volume of *The Zcash Arboretum* constructs threshold spend authorisation: several holders of key shares produce one ordinary RedPallas signature without reconstructing the spending key during signing. It assumes interpolation from the *Math Guide*, Schnorr and RedPallas from the *Crypto Guide*, Action authorisation from the *Ironwood Guide*, and separated transaction-building roles from the *Wallet Guide*. The construction includes share provisioning, transcript encodings, nonce state, randomisation and the final signing workflow. Algebraic correctness, imported unforgeability results and unproved composition requirements are distinguished explicitly.

1 Threshold custody and its contract

Math Guide, §Groups and scalar multiplication and §Prime fields and modular arithmetic, supplies the group and scalar language used here. *Crypto Guide*, §Security contracts and reductions, defines signature unforgeability and the adversarial experiments used to state it.

A spending key normally belongs to one signer. Threshold custody replaces that signer by n participants with a threshold t : any authorised set of at least t participants can cooperate, while fewer than t should not be able to forge a signature. The chain still sees one public key and one signature. It does not learn the threshold or the selected participants from an additional on-chain list of signatures.

This volume selects the signing mechanism of RFC 9591, Sections 4–5, and its Pallas integration in `frost` at 2016e44b and `reddsa` at 3792daa9. The exact randomizer construction is the seed-and-commitments API in `frost-rerandomized/src/lib.rs`; the group and hash functions are in `reddsa/src/frost/redpallas.rs`. ZIP 312 supplies the Zcash compatibility requirements, but its direct-randomizer helper is not silently substituted for this selected implementation.

The endpoint is threshold authorisation of an Orchard-style Action. No Crosslink, voting or future recovery protocol is needed to understand that endpoint.

1.1 The selected group, encodings, and hashes

A *ciphersuite* is a fixed choice of group, encodings, and hash functions. *Crypto Guide*, §Hash functions and key derivation, constructs the BLAKE2b operation used in this suite. We fix the functions before using them in the protocol equations.

The group is Pallas, with order $q = p_{\text{Vesta}}$ and base $G = G^{\text{SpendAuth}}$, as constructed in *Crypto Guide*, §RedPallas signatures. Canonical compressed points occupy 32 bytes. Scalar encoding is a 32-byte little-endian integer strictly less than q ; decoding rejects an out-of-range integer rather than reducing it. Participant identifiers additionally reject zero.

The group decoder rejects malformed points and the identity at FROST input positions that require non-identity elements. Algebraic identities below still hold when a group sum is the identity, but the concrete ciphersuite’s serialisation and validation errors govern whether that rare session can be completed. It must fail cleanly, not reinterpret an invalid encoding.

For a 64-byte string b , write $\text{Wide}(b) = \text{LEInteger}(b) \bmod q$. The scalar-valued hashes apply this map to BLAKE2b-512’s output. The complete personalised functions are:

Function	Output	BLAKE2b personalisation
H_1	scalar	FROST_RedPallasR
H_2	scalar	Zcash_RedPallasH
H_3	scalar	FROST_RedPallasN
H_4	64 bytes	FROST_RedPallasM
H_5	64 bytes	FROST_RedPallasC
H_R	scalar	FROST_RedPallasA
H_D	scalar	FROST_RedPallasD

The hash H_D supports participants jointly generating shares of a key without giving any participant the complete secret. It is an additional library function, not an RFC signing hash. Identifiers may simply be assigned as distinct nonzero scalars; no identifier hash is needed for that choice. We write P^* for the canonical encoding of a point P , and $\text{enc}(x)$ for a scalar or identifier encoding. The selected key-generation API takes 16-bit participant counts and requires $2 \leq t \leq n$, although the mathematical sharing construction also makes sense at $t = 1$.

1.2 Public and secret objects

Work in an additive prime-order group $\mathbb{G} = \langle G \rangle$ of order q . Scalars belong to $\mathbb{F}_q$. The Pallas instantiation will set $q = p_{\text{Vesta}}$. The *Crypto Guide*, §*Knowledge, simulation, and Schnorr*, constructs the verification equation

$$[z]G = R + [c]Y, \quad c = H_2(R^* || Y^* || m).$$

Here $Y = [x]G$ is the group public key, m is the approved byte-string message, R is the nonce commitment and z the response scalar. FROST distributes the calculation of z ; it does not change this outside verification rule.

Each participant has a distinct nonzero identifier $i \in \mathbb{F}_q$, a secret signing share x_i , and a public verification share $Y_i = [x_i]G$. A public key package fixes Y , all identifiers and their Y_i , and the threshold. A signer does not accept a coordinator's replacement verification-share map as a new definition of its key.

The coordinator collects nonce commitments, distributes one signing package and aggregates response shares. Its ability to schedule or interrupt the protocol does not give it spending authority. Conversely, unforgeability does not oblige a malicious coordinator or selected signer to finish a session.

Definition 1.1 (Threshold authorisation requirement). Fix an honestly provisioned key and an adversary controlling fewer than t participants. Honest participants approve the complete application message before answering a signing request. A successful forgery is an accepting signature outside the signing interactions permitted by the security game's freshness rule: the rule specifying which earlier signing interactions disqualify a proposed forgery. The game must state when participants may be corrupted, whether sessions may overlap, who chooses each key randomizer, and the exact disqualifying interactions.

These qualifications matter. A new signature on a previously signed message is excluded by an ordinary new-message game but included by a strong-unforgeability game. A theorem for corruption choices fixed before key generation does not automatically cover secrets exposed adaptively during signing. We identify the imported guarantees in §4.

2 Shares and verifiable key generation

The interpolation theorem is proved in *Math Guide*, §*Polynomials and evaluation*. We use it to construct a secret sharing and then to explain precisely what the public share checks add. That same

section proves the hidden-constant-coefficient lemma used for the privacy statement below.

2.1 A polynomial with the key at zero

Choose $1 \leq t \leq n < q$, a secret $x \in \mathbb{F}_q$, and independent uniform coefficients $a_1, \dots, a_{t-1} \in \mathbb{F}_q$. Define

$$f(X) = x + \sum_{j=1}^{t-1} a_j X^j, \quad x_i = f(i).$$

The degree is at most $t - 1$; a highest coefficient may be zero. Identifiers cannot be zero because $f(0)$ is the secret itself.

For a signing set S of distinct identifiers with $|S| \geq t$, define

$$\lambda_i^S = \prod_{\substack{j \in S \\ j \neq i}} \frac{j}{j - i}.$$

All denominators are invertible because identifiers are distinct. Evaluation of the interpolation formula at zero gives

$$\sum_{i \in S} \lambda_i^S x_i = x, \quad \sum_{i \in S} \lambda_i^S = 1.$$

The second equality is interpolation of the constant polynomial 1. It will be essential when all shares receive the same randomizer.

This construction is called *Shamir secret sharing*. Any fixed set of $r < t$ private shares is jointly uniform in $\mathbb{F}_q^r$, independently of the fixed secret x : apply the lemma “A hidden constant coefficient” in *Math Guide*, §*Polynomials and evaluation*, with its constant $s = x$ and evaluation points equal to the participant identifiers. Its requirements are exactly the nonzero distinct identifiers and independent uniform nonconstant coefficients enforced here.

Publishing $Y = [x]G$ changes the information-theoretic statement: an unbounded observer can recover x from Y . Threshold key secrecy with public verification data is computational. The private-share statement must not be quoted as perfect privacy of the entire public-key protocol.

2.2 Checking a dealer's shares

A trusted dealer can distribute the shares over confidential, authenticated channels and publish coefficient commitments

$$C_j = [a_j]G \quad (0 \leq j < t), \quad a_0 = x.$$

Each recipient checks

$$[x_i]G = \sum_{j=0}^{t-1} [i^j]C_j.$$

The right side equals $[f(i)]G$. Since scalar multiplication by a generator is injective over $\mathbb{F}_q$, the equality verifies that the received scalar is the evaluation described by those public commitments.

All participants must receive the same commitment list. Otherwise a dealer could give individually consistent but mutually incompatible views. An authenticated private channel identifies the sender to one recipient; it does not by itself establish agreement between recipients.

The dealer knows x . Splitting an existing key is therefore compatible with a seed-derived account, but its threshold guarantee requires the original full key and dealer randomness to be erased or otherwise accounted for. Dividing a key does not erase previously made backups.

2.3 Generating shares without a single dealer

The selected library also implements distributed key generation (DKG) in `frost-core/src/keys/dkg.rs`. It runs a dealer contribution from every participant and adds the results. We state its essential messages, including the checks needed to obtain one common key.

Each participant ℓ samples a polynomial $f_\ell(X) = \sum_{j=0}^{t-1} a_{\ell j} X^j$. In the selected implementation the constant coefficient is sampled nonzero, as a signing key; the other coefficients are independent uniform scalars. It publishes $C_{\ell 0} = [a_{\ell 0}]G$. It proves knowledge of its constant coefficient using a Schnorr proof, with a fresh nonzero nonce u_ℓ :

$$U_\ell = [u_\ell]G, \quad e_\ell = H_D(\text{enc}(\ell) \| C_{\ell 0}^* \| U_\ell^*), \quad v_\ell = u_\ell + e_\ell a_{\ell 0}.$$

The verifier recomputes e_ℓ and checks $[v_\ell]G = U_\ell + [e_\ell]C_{\ell 0}$. The selected implementation hashes exactly these three fields. Its hash function separates the DKG role; an identifier for this one key-generation run, called a ceremony identifier, and membership agreement remain responsibilities of the enclosing transport protocol, not extra inputs we invent for this library call.

After agreeing on all first-round packages, participant ℓ sends $f_\ell(i)$ privately to recipient i . Each recipient verifies

$$[f_\ell(i)]G = \sum_{j=0}^{t-1} [i^j]C_{\ell j}$$

for every contributor. It rejects missing, duplicate, unauthenticated or inconsistent packages. On success it forms

$$x_i = \sum_{\ell=1}^n f_\ell(i), \quad Y_i = \sum_{\ell=1}^n \sum_{j=0}^{t-1} [i^j]C_{\ell j}, \quad Y = \sum_{\ell=1}^n C_{\ell 0}.$$

The sum $F(X) = \sum_{\ell} f_\ell(X)$ has degree at most $t-1$, $x_i = F(i)$, and $Y = [F(0)]G$. Thus the shares have exactly the interpolation relation signing needs.

The private contributions and polynomial coefficients are erased after their required checks and aggregation. The enduring private object is the participant's resulting key share. The enduring public object is the agreed key package.

This is an aborting DKG, not a proof that malicious participants cannot block key generation. A proof of knowledge of each contribution also does not by itself prove unbiased output when an adversary aborts unfavourable runs and allows only favourable runs to complete. The signing theorem below does not silently acquire a DKG-composition guarantee merely because the final interpolation identities are correct.

2.4 Normalising an Orchard authorisation key

Orchard's long-term spend-authorisation key uses the even- y representative; *Ironwood Guide*, §*Keys and viewing capabilities*, defines the convention. If the generated Y has odd y , negate every secret share and corresponding public share, the group key, and any retained polynomial commitments:

$$x'_i = -x_i, \quad Y'_i = -Y_i, \quad Y' = -Y.$$

Then $\sum_i \lambda_i^S x'_i = -x$, so the new shares interpolate to the normalised group key. Negating only the group key would leave an inconsistent key package.

The selected RedPallas `post_generate` and `post_dkg` hooks apply this transformation. The later per-Action randomised key has no even- y requirement. Applying the normalisation again to that key would change the transaction's authorisation equation.

3 Two-round signing

We first construct signing for one fixed group key Y . The later randomisation step changes the key inputs but preserves the equations.

3.1 Nonce generation and first-round commitments

For each signing session, participant i creates two nonce scalars d_i, e_i , called the hiding and binding nonces. The RFC's hedged nonce generator combines fresh randomness with the secret share:

$$\text{Nonce}(x_i; b) = H_3(b \parallel \text{enc}(x_i)), \quad b \leftarrow \{0, 1\}^{256}.$$

Invoke it independently twice, using two fresh 32-byte strings, to obtain d_i, e_i . The participant retains the scalars and sends only

$$D_i = [d_i]G, \quad E_i = [e_i]G$$

to the coordinator. Hashing the secret share into nonce generation does not make repeated random inputs safe. Identical inputs repeat the nonce.

The participant associates the pair with durable single-use state. Publishing a commitment is not permission to reuse its secret in several responses. Preprocessing may prepare pairs before the message is known; each pair is still consumed by at most one signing operation.

3.2 The canonical signing package

The coordinator selects S , with $t \leq |S| \leq n$, and constructs

$$B = ((i, D_i, E_i))_{i \in S}$$

in ascending numerical identifier order. The group encoding is

$$\text{Enc}(B) = \big\|_{i \in S} (\text{enc}(i) \parallel D_i^* \parallel E_i^*).$$

Within one ciphersuite all three encodings have fixed lengths; this makes the concatenation unambiguous. Ascending scalar order is not lexicographic order of little-endian byte strings: that order compares the first differing byte, not the integers represented by all bytes.

Each signer receives m, B , validates every point and identifier, checks uniqueness and ordering, checks that the selected participants belong to the stored key package, and verifies that its own identifier and exact commitments occur once. It must also approve m according to the application; a syntactically valid byte string is not necessarily an authorised payment.

Define the message and commitment-list prehashes and binding factors:

$$\begin{aligned} M &= H_4(m), & C &= H_5(\text{Enc}(B)), \\ \rho_i &= H_1(Y^* \parallel M \parallel C \parallel \text{enc}(i)). \end{aligned}$$

These are RFC 9591's ordered inputs. The public key, message, complete commitment list and identifier all participate. The symbol ρ_i here denotes a FROST binding factor, not a note's uniqueness seed.

Each participant computes

$$R_i = D_i + [\rho_i]E_i, \quad R = \sum_{i \in S} R_i, \quad c = H_2(R^* \parallel Y^* \parallel m).$$

No secret nonce is needed to recompute these public values.

3.3 Responses and aggregation

Participant i calculates the Lagrange coefficient for this exact S , and returns

$$z_i = d_i + \rho_i e_i + c \lambda_i^S x_i.$$

The coordinator checks scalar encodings, the response-to-identifier mapping and completeness of the selected response set. It computes

$$z = \sum_{i \in S} z_i, \quad \sigma = R^* \parallel \text{enc}(z).$$

The public verification equation is the ordinary Schnorr equation. A coordinator may first verify the aggregate and verify individual shares only after aggregate failure. To check a share, it tests

$$[z_i]G = R_i + [c\lambda_i^S]Y_i.$$

These checks use the stored Y_i , not a public share supplied alongside an allegedly bad response. Authenticated messages make a failed check attributable to its sender; the equation alone does not prove who sent an arbitrary byte string.

Theorem 3.1 (Signing correctness). *If all selected shares belong to the same key polynomial, all participants use the same valid signing package, and all computed values satisfy the ciphersuite's encoding and nonidentity checks, the aggregate verifies under Y . Freshness of the nonce pairs is additionally required for security, not for this algebraic identity.*

Proof. Summing the response equations and using interpolation gives

$$z = \sum_{i \in S} (d_i + \rho_i e_i) + c \sum_{i \in S} \lambda_i^S x_i = r + cx, \quad r = \sum_{i \in S} (d_i + \rho_i e_i).$$

The group commitment is $R = [r]G$. Multiplying the scalar equality by G yields $[z]G = R + [c]Y$. $\square$

A signer may leave before answering. The coordinator cannot simply drop it and sum the others: the participant set is inside the binding factors and interpolation weights, and the dropped commitments are inside R . A retry selects fresh nonce pairs and constructs a fresh signing package.

3.4 A scalar-field walkthrough

For an arithmetic illustration only, take $q = 101$, $f(X) = 42 + 17X + 29X^2$, and $S = \{1, 3, 5\}$. The shares and coefficients are

$$(x_1, x_3, x_5) = (88, 51, 44), \quad (\lambda_1, \lambda_3, \lambda_5) = (65, 24, 13).$$

Both $\sum_i \lambda_i = 1$ and $\sum_i \lambda_i x_i = 42$ hold modulo 101. Choose illustrative nonce pairs

$$(d_1, d_3, d_5) = (11, 13, 17), \quad (e_1, e_3, e_5) = (19, 23, 29).$$

For illustrative hash outputs $(\rho_1, \rho_3, \rho_5) = (2, 4, 6)$ and $c = 7$, the effective nonces are $(49, 4, 90)$, whose sum is 42. The response shares are $(93, 88, 54)$; their sum is 33. Finally,

$$33 = 42 + 7 \cdot 42 \pmod{101}.$$

The retained numerical check recomputes every entry. These invented hash outputs and small scalars test the algebra; they are not a ciphersuite test vector or an instance with cryptographic security.

4 Security and nonce state

Correctness says honest responses add to a valid signature. Security must explain why a malicious coordinator interacting with several sessions cannot obtain an unapproved one. The equations alone do not establish that conclusion. The random-oracle model, which treats each distinct hash input as receiving one independently random answer consistently reused on repetition, is constructed in *Crypto Guide*, §Security contracts and reductions.

4.1 What the binding factor prevents

A response is linear in the secret quantities:

$$z_i = d_i + \rho_i e_i + c \lambda_i^S x_i.$$

If the coordinator could select the coefficients of this equation independently after collecting nonce commitments, it could try to combine responses from separate sessions into an equation it wanted. The binding factor makes the effective nonce depend on the entire signing package, including the eventual message and public key. Changing another participant's commitment therefore changes the honest participant's coefficient too.

The domain-separated hashes are essential here. Calling two different functions the same random oracle, or omitting the participant list from one side's calculation, changes the protocol analysed by the security proof. The fact that the resulting aggregate still resembles a Schnorr signature is not a repair.

4.2 The imported signing guarantee

The signing-layer result we rely on is the analysis of FROST1 in Bellare, Crites, Komlo, Maller, Tessaro and Zhu, *Better than Advertised Security for Non-interactive Threshold Signatures*, CRYPTO 2022; the FROST analysis is retained in ePrint 2022/833, Theorem 5.4. Its setting uses random oracles, trusted-dealer key distribution and static corruption, meaning the corrupted set is fixed before key generation, of fewer than t participants. Its strong threshold-forgery notion also accounts for which participants contributed to a session; an ordinary single-signer theorem against new-message forgery after chosen signing queries is not that result.

The hardness assumption is *one-more discrete logarithm*. In its game, an adversary obtains independent random challenge group elements and may ask a discrete-log oracle for selected logarithms. It must not be able to output the logarithms of more challenge elements than the number of oracle queries it used, with non-negligible probability. This assumption permits oracle access that the ordinary discrete-log game does not. We do not equate the two assumptions.

The imported result covers the matching signing construction under its stated key-generation and corruption hypotheses. It does not prove that the implemented DKG composes with signing, that arbitrary adaptive corruptions are safe, or that backup software preserves the nonce requirements. Those are separate obligations. The purpose of importing a theorem is to state a precise guarantee, not to replace the unforgeability question with an adjective.

4.3 Why a nonce pair has only one response

If one nonce pair is answered under three packages, the attacker obtains equations

$$z_j = d + \rho_j e + \beta_j x_i, \quad \beta_j = c \lambda_i^{S_j}, \quad j = 1, 2, 3.$$

Subtract the first equation from the other two. This leaves two linear equations in e, x_i , solved by the elimination procedure in *Math Guide*, §*Linear algebra and uniform masks*. Whenever

$$(\rho_2 - \rho_1)(\beta_3 - \beta_1) - (\rho_3 - \rho_1)(\beta_2 - \beta_1) \neq 0,$$

the coefficient matrix is invertible and reveals x_i , then d . Fewer observations can suffice in special cases or with extra leaked information. A failed aggregate does not make an emitted response harmless: the attacker still possesses its linear equation.

Nonce generation from a deterministic function of just the share and message is therefore unsuitable. Signing the same message in another coordinator-chosen package would repeat the secret pair while changing its public coefficients. Fresh randomness and single-use storage solve different parts of this problem; neither substitutes for the other.

Construction 4.1 (Application nonce state). Associate each generated pair with a unique local record.

1. Store the private pair as available before advertising its public commitments.
2. Bind that record to one complete accepted signing request.
3. Irreversibly mark it consumed before releasing any response derived from it. A crash must not roll that transition back.
4. Erase the private pair when its response is computed or the session is abandoned. A restart obtains new commitments.

This state machine is an application requirement, not a persistence service furnished by a scalar-signing function. The selected Rust API takes references to nonce objects; the surrounding application must not call it again with a consumed pair. Restoring a backup, cloning a device or racing two signing requests must not duplicate available nonce state. Ordinary durable key backups should not restore an old pool of preprocessed nonces.

A malicious participant can withhold a response. Share verification can identify inconsistent responses from authenticated senders; it cannot force participation or prove that a sender intended to disrupt the session. Availability policy and retry policy remain outside the unforgeability theorem.

5 Randomised RedPallas signing

The *Ironwood Guide*, §*Balance, digests, and authorisation*, publishes a per-Action key

$$\text{rk} = Y + [\alpha]G$$

rather than the long-term $Y = \text{ak}$. The Action proof establishes the relation to the authorised spend key. The outside signature must verify under rk , not under Y .

5.1 Shifting all shares

For a common scalar α , define

$$\bar{x}_i = x_i + \alpha, \quad \bar{Y}_i = Y_i + [\alpha]G, \quad \bar{Y} = Y + [\alpha]G.$$

All signers use $\bar{Y}$ in the binding factors and signature challenge and $\bar{x}_i$ in their response equation. The coordinator uses $\bar{Y}_i$ for share verification and $\bar{Y}$ for final verification.

Proposition 5.1 (Randomisation commutes with sharing). *The shifted shares interpolate to $x + \alpha$ for every permitted signing set S . Their FROST aggregate is an ordinary signature under $\bar{Y}$.*

Proof. The two interpolation identities give

$$\sum_{i \in S} \lambda_i^S (x_i + \alpha) = x + \alpha \sum_{i \in S} \lambda_i^S = x + \alpha.$$

Equivalently, the shifted polynomial is $f(X) + \alpha$, whose degree remains at most $t - 1$. Theorem 3.1 then applies to the shifted inputs. $\square$

Adding the same scalar to every share does not add $|S|\alpha$ to the group key, because shares are interpolated rather than simply summed. This is why no separate sharing ceremony for the randomizer is needed.

5.2 The selected randomizer construction

Let B be the exact first-round commitment list for the Action. The coordinator draws a 32-byte seed s and computes

$$\alpha = H_R(s \parallel \text{Enc}(B)).$$

It sends s, B to each participant. Each participant regenerates α locally and constructs the shifted key package.

This is the mechanism of `Randomizer::new_from_commitments` and `regenerate_from_seed_and_commitments` at the selected frost commit. The message is *not* inside this randomizer hash. It is inside H_4 , every binding factor and the signature challenge. That distinction permits the builder to calculate rk before the transaction digest containing rk is known.

Fresh commitments contribute input variation even if the coordinator's seed generator malfunctions. This is hedging, not a theorem that every maliciously selected seed produces an unbiased randomizer. A coordinator can examine candidate seeds and can learn α . Its privacy role must be accounted for explicitly.

5.3 Signature compatibility

Every list entry is therefore $32 + 32 + 32 = 96$ bytes before transport framing. The final signature is $32 + 32 = 64$ bytes. Neither number depends on whether the selected signing set has two participants or twenty; only the intermediate transcript grows with its size.

The essential compatibility condition is H_2 . Its personalisation, input order, scalar conversion and point/scalar encodings are exactly RedPallas's. A verifier does not care whether the valid response was computed from one secret or from interpolated shares. Other FROST ciphersuites are not interchangeable merely because they also output 64-byte signatures.

5.4 Privacy and the remaining theorem boundary

For an independent uniform α , the map $\alpha \mapsto Y + [\alpha]G$ is a bijection from $\mathbb{F}_q$ to $\mathbb{G}$. Thus the randomised public key alone is uniform, independently of Y . With wide reduction of a random-oracle output, the distribution is statistically close to uniform, rather than exactly uniform.

A person learning α can compute $Y = rk - [\alpha]G$. The coordinator, signers and transaction prover therefore are not covered by a claim that their own signing sessions are unlinkable to the long-term key. The seed must travel over a confidential, authenticated channel: s and public B reveal α . Transaction timing, signer identity and network traffic may reveal additional links.

Gouvêa and Komlo's *Re-Randomized FROST* provides the algebraic construction and a claimed unforgeability analysis. Its stated freshness condition concerns new messages; it does not establish the stronger fresh-message-signature-pair property requested by the Zcash specification. The selected seed-and-commitments API also needs its precise correspondence to an analysed randomizer model. We do not present the preprint's claimed concrete bound as a proved bound for this full implementation and DKG composition.

The identity in Proposition 5.1, consensus verification compatibility and the public-key masking calculation are proved here. A full threshold strong-unforgeability and joint privacy theorem for the complete selected workflow remains a separate security obligation.

6 Transaction construction with separated signers

A signer receiving only a 32-byte digest cannot determine what payment it authorises. Threshold cryptography does not fix a missing transaction review. The complete signer contract is the one in *Wallet Guide*, §*Constructing and authorising a transaction*; this section inserts the threshold rounds at the necessary points.

6.1 The order that avoids a circular digest

For each real spend assigned to a threshold key:

1. Select the participants and obtain their unused first-round commitments. At this point the final transaction message need not exist.
2. Freeze B , generate s , derive α , and construct $rk = Y + [\alpha]G$.
3. Insert that rk into the Action. Complete the transaction effects, including outputs, public balances, fee and all other fields covered by the applicable signature digest.
4. Compute the transaction's signature digest m . Give each signer the transaction information needed to verify its authorisation policy, along with m, B, s and the identification of the Action it is authorising.
5. Each signer checks that its regenerated rk equals the Action's key, recomputes the required digest, approves the transaction, and produces one response using its consumed nonce pair.
6. Aggregate, verify under the Action's rk , and insert the resulting spend-authorisation signature. Complete the proof and remaining authorising data in the order their own interfaces permit.

The proof relation uses the same α that defines the signature key. The prover must possess the required proof-authorisation data; threshold spend-key shares alone are not the entire Orchard witness. Likewise, withholding the spend key from the prover does not make a malicious builder's proposed outputs safe to approve.

The final signature digest may exclude some authorising data by design. The signer follows the exact digest definition for that transaction version; it must not hash an arbitrary serialisation or replace the digest by the transaction identifier. A partially created transaction container transports roles and fields; it is not itself proof that the signer checked them.

6.2 Applying the shared payment example

The *Wallet Guide*'s running transaction spends an 80,000-zatoshi note, pays 50,000, returns 20,000 in change and pays a 10,000 fee. Its two padded Actions may have net values +30,000 and -20,000. FROST changes none of that arithmetic.

The holders of the real note's threshold spend authority jointly sign its Action. A dummy spend still follows its own permitted authorisation construction; it does not represent another threshold-owned 80,000 zatoshi note. Before approving, the real signers check recipient, amount, change ownership, fee and the relevant Action/digest bindings. The aggregate signature does not reveal which subset approved, but the participants and coordinator know their own session.

The binding signature is a different role. It supplies the authorisation component of the balance argument, whose opening-knowledge premises, proof relations and integer bounds are stated in *Ironwood Guide*, §*Balance, digests, and authorisation*. Threshold spend authorisation does not replace that proof of balance or distribute its key automatically.

6.3 What a completed endpoint guarantees

A complete honest run outputs the same signature format the existing Action verifier accepts, with no reconstruction of the spend key during signing. The off-chain system must still provide an agreed key package, participant authentication, private share distribution, confidential randomizer transport, application-level transaction approval and durable nonce consumption.

Those requirements are not optional implementation polish: each closes a specific step in the construction. The threshold-signing endpoint is complete only when the surrounding custody system implements them. Nothing in this construction turns a proposed threshold application for some other protocol into an implemented one.